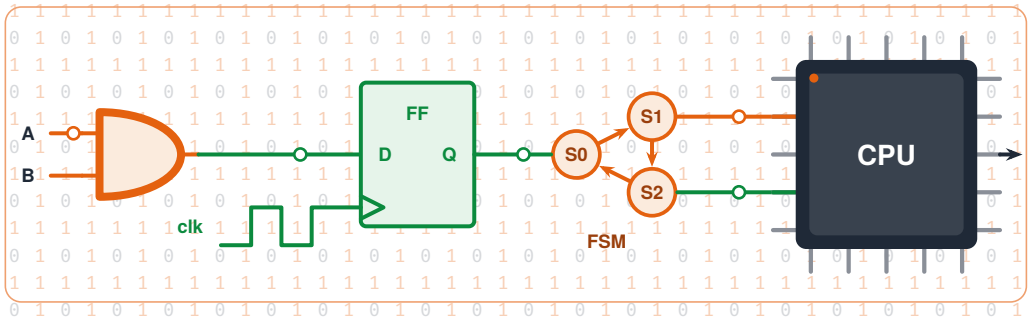
TEXTBOOK | COURSES 010113025 & 010153101

# Digital Circuits and Microprocessor Fundamentals

From Boolean Algebra to Basic General-Purpose Microprocessors with VHDL




**Dr. Ruslee Sutthaweekul**

Department of Electrical and Computer Engineering
Faculty of Engineering
King Mongkut's University of Technology North Bangkok

First Edition • 2026

**Digital Circuits and Microprocessor Fundamentals: From Boolean Algebra to Basic General-Purpose Microprocessors with VHDL**
First Edition, 2026

Written for the courses 010113025 *Digital Circuit and Logic Design* and
010153101 *Digital Circuit and Microprocessor Fundamental*,
Department of Electrical and Computer Engineering, Faculty of Engineering,
King Mongkut's University of Technology North Bangkok.

**Author** Dr. Ruslee Sutthaweekul

**Published by** *Publisher name and address to be added*

**ISBN** (***to be assigned by the publisher***)

**Cataloguing-in-publication data**
(*to be completed by the publisher/library*)
Ruslee Sutthaweekul: Digital Circuits and Microprocessor Fundamentals: From Boolean Algebra to Basic General-Purpose Microprocessors with VHDL. – First Edition. – Bangkok: King Mongkut's University of Technology North Bangkok, 2026.
Includes bibliographical references and index.
1. Digital electronics. 2. Logic circuits. 3. VHDL (Computer hardware description language).

**Copyright and permissions**
© 2026 Ruslee Sutthaweekul. All rights reserved. Except for brief quotations with proper acknowledgement, no part of this book may be reproduced or distributed without the author's written permission.

All circuit diagrams, state diagrams, tables, timing diagrams and VHDL listings in this book were created by the author. Screenshots of simulation and synthesis tools were produced by the author with Intel® Quartus® Prime and ModelSim; product names are trademarks of their respective owners. Facts, standards and historical data drawn from other works are cited where they appear and listed in the Bibliography.

**Typeset** with XeLaTeX. Source files, TikZ/circuitikz figures and VHDL designs are kept alongside the book so that every figure and listing can be regenerated.

*To my students, whose questions*
*taught me how to explain*

*and to my family, who gave me the time to write it down.*

# Preface

Every phone, car, medical instrument and industrial controller you will work with as an electrical or computer engineer contains digital logic. Digital design is therefore not a specialist topic: it is the common language in which hardware is described, verified and built. This book teaches that language from the ground up, starting with a single switch and finishing with a small general-purpose processor that you will design, simulate and run yourself.

### Why this book

This text grew out of the lectures I give for the courses *Digital Circuit and Logic Design* (010113025) and *Digital Circuit and Microprocessor Fundamental* (010153101) at the Department of Electrical and Computer Engineering, King Mongkut's University of Technology North Bangkok. For several semesters my students asked for one document that follows the lectures in order, explains the reasoning that slides cannot carry, and offers enough worked examples and exercises to practise on. This book is that document. It does not replace the classic references listed in the Bibliography; it condenses them into the path that I have found works for a single semester, and it adds the thing I consider most important in a modern course: every technique is shown both on paper *and* in VHDL, so that you see the same circuit as a truth table, a schematic, a state diagram and a hardware description.

### What is in the book

The ten chapters follow the course syllabus.

- **Chapters 1–3** introduce digital systems, Boolean algebra, logic gates, logic synthesis and Karnaugh-map optimisation.
- **Chapters 4–5** cover number representation, arithmetic circuits and the standard combinational building blocks (decoders, multiplexers, comparators, ALU).
- **Chapters 6–8** move to sequential logic: latches and flip-flops, registers and counters, and finite state machines (FSMs) in the Moore and Mealy styles.
- **Chapters 9–10** combine everything into processors: first *dedicated* microprocessors (a datapath controlled by an FSM), then a small *general-purpose* processor with an instruction set, an instruction cycle and a program in memory.

Four appendices provide a VHDL quick reference, number-system and Boolean-algebra

reference tables, the complete listings of the processor designs, and answers to selected exercises. A glossary and an index are included at the end.

### How the book is organised

Each chapter opens with learning objectives and closes with a summary, exercises and suggestions for further reading. Worked examples are boxed and numbered; definitions and key design rules are set apart so that they are easy to find again when you revise. Figures are numbered by chapter, and the VHDL listings can be downloaded together with the book source so that you can run them.

### A note on originality and sources

Digital design rests on well-established results by many people. I have tried to give credit wherever a result, a historical fact or a data table originates elsewhere, and the Bibliography lists the books and papers I relied on. The explanations, examples, diagrams, tables and VHDL designs are my own, written for this course.

### Feedback

No book of this size is free of errors. If you find one, or a passage that confused you, please tell me at ruslee.s@eng.kmutnb.ac.th; corrections will be collected for the next edition.

Dr. Ruslee Sutthaweekul
Department of Electrical and Computer Engineering
King Mongkut's University of Technology North Bangkok
Bangkok

# Acknowledgements

I thank my colleagues in the Department of Electrical and Computer Engineering for sharing the teaching of this course over many semesters and for the discussions that shaped its structure. The students of courses 010113025 and 010153101 found countless typographical errors and unclear explanations in earlier handouts and slides; their feedback is visible on nearly every page.

The design of this course owes a great deal to the textbooks of Brown and Vranesic, of Hwang, and of Mano and Ciletti (see the Bibliography), from which I learned the subject and which I recommend for deeper study. Any errors that remain are mine alone.

Finally, I thank the open-source communities behind TEX, TikZ, circuitikz and the many free tools used to prepare this book, and the makers of the simulation and synthesis tools that let students run real hardware designs on an ordinary laptop.

Ruslee Sutthaweekul

# About the Author

I received the B.Eng. degree in electrical engineering from King Mongkut's Institute of Technology North Bangkok, the M.Eng. degree in electrical engineering and information technology from the University of Applied Sciences Rosenheim in Germany, and the Ph.D. degree in electrical and electronic engineering from Newcastle University in the United Kingdom. I have been a lecturer in the Department of Electrical and Computer Engineering at King Mongkut's University of Technology North Bangkok (KMUTNB) since November 2003, where I teach digital circuit and logic design and related courses.

My research interests include non-destructive testing and evaluation, electromagnetic and RF sensors, wireless sensor networks, machine learning, and signal and image processing. My work has been recognised with the second-place award in the Sensors and Measurements Student Contest at IEEE Sensors 2017 and the Best Poster Award at the International SoC Design Conference (ISOCC) 2024.

Comments and corrections are welcome at ruslee.s@eng.kmutnb.ac.th.

Dr. Ruslee Sutthaweekul

# Contents

**Preface** v

**Acknowledgements** vii

**About the Author** ix

**Introduction: How to Use This Book** xxvii

**1 Digital Systems** 1

1.1 Why Study Digital Systems? . . . 1
1.2 Analog and Digital Signals . . . 2
1.3 Binary Representation of Information . . . 3
1.3.1 Positional notation . . . 3
1.4 Analog-to-Digital Conversion . . . 4
1.5 From Switches to Transistors . . . 6
1.6 A Short History of Digital Hardware . . . 6
1.6.1 Relays and vacuum tubes . . . 6
1.6.2 The transistor and the integrated circuit . . . 7
1.6.3 The microprocessor . . . 7
1.7 Moore's Law . . . 8
1.8 Types of Logic Chips . . . 9
1.9 The Digital Design Flow . . . 10
Exercises . . . 12
Further Reading . . . 13

**2 Logic Circuits** 15

2.1 Logic Circuits . . . 15
2.2 Boolean Algebra . . . 16
2.2.1 Variables, operations and axioms . . . 16
2.2.2 Properties of two and three variables . . . 17
2.2.3 Duality . . . 17
2.2.4 De Morgan's theorem . . . 17
2.3 From Switches to Logic Functions . . . 18
2.3.1 Truth tables . . . 18

2.4 Logic Gates and Networks . . . 19
2.4.1 A first look at VHDL . . . 20
2.5 Logic Synthesis . . . 20
2.5.1 Sum of products and minterms . . . 21
2.5.2 Product of sums and maxterms . . . 21
2.5.3 Minimality and simplification . . . 21
2.6 Design Examples . . . 23
2.6.1 Three-way light control . . . 23
2.6.2 Two-to-one multiplexer . . . 24
2.6.3 Car safety alarm . . . 25
2.6.4 Half adder . . . 26
2.7 The VHDL Testbench . . . 27
Exercises . . . 30
Further Reading . . . 31

**3 Logic Optimisation with Karnaugh Maps 33**
3.1 The Need for Optimisation . . . 33
3.2 The Karnaugh Map . . . 35
3.2.1 The two-variable map . . . 35
3.2.2 The three-variable map . . . 35
3.2.3 Rules for simplification . . . 37
3.3 Prime Implicants . . . 38
3.4 The Four-Variable Map . . . 39
3.5 Don't-Care Conditions . . . 40
3.6 Five- and Six-Variable Maps . . . 42
3.7 Product-of-Sums Simplification . . . 44
3.8 NAND and NOR as Universal Gates . . . 44
3.8.1 Two-level NAND–NAND implementation of an SOP . . . 45
3.8.2 Two-level NOR–NOR implementation of a POS . . . 46
3.9 Exclusive-OR and Exclusive-NOR . . . 46
3.9.1 Odd and even functions . . . 47
3.10 Application: Parity Generation and Checking . . . 49
Exercises . . . 51
Further Reading . . . 52

**4 Number Representation and Arithmetic Circuits 53**
4.1 Positional Number Systems . . . 53
4.1.1 Converting from decimal . . . 54
4.2 Signed Integer Representations . . . 54
4.3 Arithmetic and Overflow . . . 56
4.4 Adders . . . 57
4.4.1 Half adder and full adder . . . 57

4.4.2 Ripple-carry adder . . . 59
4.4.3 Adder/subtractor . . . 61
4.5 The Carry Delay and the Carry-Lookahead Adder . . . 64
4.5.1 Delay of the ripple-carry adder . . . 65
4.5.2 Generate and propagate . . . 65
4.5.3 Hierarchical lookahead . . . 68
4.6 Multiplication . . . 69
4.7 Fractions and Floating Point . . . 73
4.7.1 Fixed-point numbers . . . 73
4.7.2 Floating-point numbers (IEEE 754) . . . 75
4.8 Codes for Decimal Digits and Characters . . . 75
Exercises . . . 78
Further Reading . . . 79

**5 Combinational Circuit Building Blocks 81**
5.1 Combinational and Sequential Circuits . . . 81
5.2 Analysis of a Combinational Circuit . . . 82
5.3 Design Procedure . . . 83
5.3.1 Example: BCD to excess-3 converter . . . 84
5.4 Decimal (BCD) Adder . . . 87
5.5 Decoders . . . 89
5.5.1 Demultiplexers . . . 91
5.6 Encoders . . . 91
5.6.1 Priority encoders . . . 93
5.7 Multiplexers . . . 96
5.7.1 Implementing Boolean functions with multiplexers . . . 98
5.7.2 Shannon's expansion theorem . . . 99
5.8 The Arithmetic Logic Unit . . . 100
5.9 Three-State Gates and Buses . . . 102
5.10 Magnitude Comparator . . . 102
Exercises . . . 105
Further Reading . . . 106

**6 Latches, Flip-Flops and Synchronous Sequential Circuits 107**
6.1 Why Sequential Circuits? . . . 107
6.2 Structure of a Sequential Circuit . . . 108
6.3 Latches . . . 108
6.3.1 The SR latch . . . 108
6.3.2 The $\overline{SR}$ latch . . . 110
6.3.3 Gated SR latch and D latch . . . 110
6.4 Flip-Flops . . . 112
6.4.1 The D flip-flop . . . 112

6.4.2 The JK flip-flop . . . 112
6.4.3 The T flip-flop . . . 114
6.4.4 Characteristic tables . . . 114
6.4.5 VHDL descriptions . . . 114
6.4.6 Asynchronous and synchronous reset . . . 117
6.5 Analysis of Clocked Sequential Circuits . . . 118
6.6 Mealy and Moore Models . . . 122
6.7 Design of Sequential Circuits . . . 122
6.7.1 Example: a sequence detector . . . 124
6.7.2 Example: a binary counter with T flip-flops . . . 128
Exercises . . . 129
Further Reading . . . 130

**7 Registers and Counters 131**
7.1 Registers . . . 131
7.1.1 Register with parallel load . . . 132
7.2 Shift Registers . . . 132
7.2.1 Serial transfer . . . 132
7.2.2 Serial adder . . . 132
7.2.3 Universal shift register . . . 135
7.3 Counters . . . 139
7.3.1 Ripple counters . . . 139
7.3.2 BCD ripple counter . . . 140
7.3.3 Synchronous binary counters . . . 142
7.3.4 Up/down counter . . . 142
7.3.5 Counter with parallel load . . . 142
7.3.6 Modulo-$N$ counters . . . 147
7.3.7 Self-correcting counters . . . 147
7.3.8 Ring counters . . . 150
7.3.9 Application: generating timing signals . . . 151
Exercises . . . 152
Further Reading . . . 153

**8 Finite State Machines 155**
8.1 Finite State Machines . . . 155
8.2 Moore and Mealy Machines . . . 156
8.3 Analysis of an FSM . . . 156
8.4 Synthesis of an FSM . . . 159
8.5 Design Examples . . . 159
8.5.1 A 2-bit up/down counter . . . 159
8.5.2 A car security system (Moore) . . . 161
8.5.3 A modulo-6 counter with enable . . . 163

8.5.4 A one-shot circuit . . . 167
8.5.5 A control unit . . . 176
8.5.6 An elevator controller: Moore and Mealy . . . 177
8.6 Moore versus Mealy Timing . . . 182
8.7 Timing of a Synchronous FSM . . . 182
8.8 A Template for VHDL State Machines . . . 182
Exercises . . . 186
Further Reading . . . 187

**9 Dedicated Microprocessors 189**
9.1 The Von Neumann Model . . . 189
9.2 From Transistors to Microprocessors . . . 190
9.3 Dedicated Microprocessors . . . 190
9.4 Datapath and Control Unit . . . 190
9.5 Register Transfer Level . . . 192
9.6 Datapath Design . . . 192
9.6.1 Sharing a functional unit with multiplexers . . . 192
9.6.2 Status signals and conditional operations . . . 194
9.6.3 An if-then-else datapath . . . 196
9.6.4 A datapath with a loop: summing the numbers 1 to 10 . . . 197
9.7 Control Unit Design . . . 197
9.7.1 Example 1: counting from 1 to 10 . . . 197
9.7.2 Example 2: the if-then-else control unit . . . 199
9.7.3 Example 3: the GCD processor . . . 205
9.8 FSM+D versus FSMD . . . 209
Exercises . . . 213
Further Reading . . . 213

**10 General-Purpose Microprocessors 215**
10.1 General-Purpose Microprocessors . . . 215
10.2 Overview of CPU Design . . . 216
10.3 The EC-1 Microprocessor . . . 216
10.3.1 Overview . . . 216
10.3.2 Instruction set . . . 216
10.3.3 Datapath . . . 217
10.3.4 ROM implementation . . . 217
10.3.5 Intel HEX format . . . 217
10.3.6 Control unit . . . 219
10.3.7 Programming example: countdown from 10 to 1 . . . 224
10.4 The EC-2 Microprocessor . . . 224
10.4.1 Overview . . . 224
10.4.2 Instruction set . . . 224

10.4.3 Datapath . . . 229
10.4.4 Control unit . . . 229
10.4.5 Memory wait states and timing . . . 232
10.4.6 Programming example: GCD . . . 233
10.4.7 Program loading and self-modifying code . . . 233
10.5 EC-1 versus EC-2 . . . 239
Exercises . . . 240
Further Reading . . . 241

**A VHDL Quick Reference** **243**

A.1 Structure of a design . . . 243
A.2 Data types . . . 243
A.3 Operators . . . 244
A.4 Concurrent statements . . . 244
A.5 Processes . . . 244
A.6 Type conversions (package `numeric_std`) . . . 245
A.7 Testbench constructs . . . 245
A.8 Common mistakes . . . 246

**B Reference Tables** **247**

B.1 Logic gates . . . 247
B.2 Number systems . . . 247
B.3 Seven-segment display . . . 247
B.4 ASCII printable characters . . . 247

**C Complete VHDL Listings** **251**

C.1 GCD processor (Chapter 9) . . . 251
C.2 EC-1 microprocessor (Section 10.3) . . . 258
C.3 EC-2 microprocessor (Section 10.4) . . . 264

**D Answers to Selected Exercises** **277**

**Glossary** **285**

**Bibliography** **289**

**Index** **291**

# List of Figures

1.1 An analog signal has continuous values; a digital signal has discrete, step-like values that are defined only at the sampling instants. . . . . 2
1.2 A switch as a one-bit memory element: open = 0, closed = 1. . . . . 3
1.3 Binary representation of the decimal number 79. Each bit position has a weight that is a power of two. . . . . 4
1.4 A 4 × 4 matrix keypad. Scanning identifies the connected row and column; the pair is encoded in four bits. . . . . 5
1.5 Sampling and quantisation. The dots are the samples taken every $T_s$; the stair-step is the quantised signal; the red marks are the quantisation errors. . . . . 5
1.6 (a) An ideal switch connecting a battery to a lamp. (b) An n-channel MOS transistor used as a switch: a high $V_{in}$ closes the path from drain to source and pulls $V_{out}$ low. . . . . 6
1.7 Milestones in digital hardware (the date of the Intel 4004 is the year of its commercial introduction). . . . . 7
1.8 Transistor count of Intel processors from Table 1.1, with a doubling time fitted between the first and last points. . . . . 9
1.9 A development flow for a digital circuit. The design phase (left) produces a verified behavioural description; the implementation phase (right) turns it into hardware. . . . . 11

2.1 A switching network: discrete inputs are transformed into discrete outputs. . . . . 16
2.2 Venn-diagram illustration of De Morgan's laws: the region shaded for the left-hand side equals that of the right-hand side. . . . . 18
2.3 Switching circuits for the three basic functions. . . . . 19
2.4 Graphical symbols of the basic gates. . . . . 20
2.5 A logic network for $f = (x_1 + x_2) \cdot x_3$: an OR gate feeding an AND gate. 20
2.6 Two circuits for the same function of Example 2.2. . . . . 22
2.7 A 2-to-1 multiplexer. The multiplexer is one of the most useful blocks in digital design and is studied further in Chapter 5. . . . . 25
2.8 The car safety alarm of Example 2.5. . . . . 26
2.9 Half adder. . . . . 27

2.10 The relationship between a testbench and the unit under test (UUT). . 28
2.11 Simulation of the three-way light in Intel Questa (author's screenshot). 29
2.12 Segment labels of the seven-segment display used in Exercise 2.4. . . . 31

3.1 Design flow of a combinational circuit: specification, truth table, Boolean expression, optimisation and implementation. . . . . 34
3.2 A Karnaugh map is a truth table rearranged so that neighbouring squares differ in one variable. . . . . 35
3.3 Two-variable K-maps: the layout of minterms and variables, and two plotted functions. . . . . 36
3.4 Structure of the three-variable K-map. Columns follow a Gray code and wrap around. . . . . 36
3.5 K-maps for Examples 3.1 to 3.4. . . . . 38
3.6 Structure of the four-variable K-map. . . . . 39
3.7 (a) Example 3.5. (b–d) Example 3.6: the map and its two minimal covers. 41
3.8 K-map with don't-care conditions for Example 3.7. . . . . 42
3.9 K-maps for five and six variables. . . . . 42
3.10 Grouping the 0s of Example 3.8. . . . . 44
3.11 Universal gates: NOT, AND and OR built from NAND gates only (top row) or from NOR gates only (bottom row). . . . . 45
3.12 Two-level circuits: an SOP as AND–OR and as NAND–NAND (a, b); a POS as OR–AND and as NOR–NOR (c, d). . . . . 46
3.13 Symbols and truth tables of XOR and XNOR. . . . . 47
3.14 Odd and even functions of three variables: K-maps and logic diagrams. 48
3.15 A parity system: the generator appends a parity bit; the checker at the receiver signals an error. . . . . 50
3.16 Even-parity generator and checker for a 3-bit message. . . . . 51

4.1 An 8-bit signed number: the MSB is the sign bit, the LSB is bit 0. . . . . 55
4.2 Two implementations of the full adder. . . . . 58
4.3 A 4-bit ripple-carry adder built from four full adders. . . . . 59
4.4 Ripple-carry adder: ModelSim simulation of the testbench (top) and the RTL viewer schematic produced by Quartus (bottom), author's screenshots. . . . . 62
4.5 A 4-bit adder/subtractor. The XOR gates conditionally complement $B$, and the control input is also the initial carry. . . . . 63
4.6 ModelSim simulation of the adder/subtractor: $5+3=8$, $5-3=2$, $3-5=-2$ (`1110`), $7+2$ (overflow, `1001`) and $-8-1$ (overflow, `0111`). Author's screenshot. . . . . 64
4.7 The carry must ripple through every stage in the worst case. . . . . 65
4.8 Block diagram of a 4-bit carry-lookahead adder: a P/G generator for each bit, the lookahead logic, and sum XOR gates. . . . . 66

4.9 A 16-bit adder with four 4-bit carry-lookahead blocks and a second-level lookahead unit. . . . 70
4.10 16-bit carry-lookahead adder: RTL viewer (top) and ModelSim simulation (bottom). Author's screenshots. The VHDL files `cla_4bit_block`, `cla_group_logic` and `cla_16bit_adder` accompany the book. . . . 71
4.11 A 4 × 4 array multiplier: partial products are formed by AND gates and summed by rows of adders. . . . 72
4.12 4 × 4 multiplier: RTL viewer (top) and ModelSim simulation (bottom). Author's screenshots. . . . 74
4.13 The IEEE 754 single-precision format. . . . 76
4.14 BCD digits drive the displays of a digital clock. . . . 77

5.1 A combinational circuit has $n$ input variables and $m$ output variables. . . 82
5.2 The circuit analysed in Example 5.1. . . . 83
5.3 BCD to excess-3 converter of Example 5.2. . . . 85
5.4 K-maps for the outputs of the BCD to excess-3 converter; X marks don't-care inputs. . . . 86
5.5 A one-digit BCD adder: a binary adder, the correction-detection logic and a second adder that adds 6. . . . 88
5.6 A 3-to-8-line decoder. . . . 90
5.7 Decoder with enable input and cascading. . . . 92
5.8 A 1-to-4 demultiplexer. . . . 94
5.9 An octal-to-binary (8-to-3) encoder. . . . 94
5.10 A 4-to-2 priority encoder. . . . 95
5.11 Logic diagram of a 4-to-1 multiplexer. . . . 97
5.12 Example 5.4: a four-variable function with an 8-to-1 multiplexer. . . . 99
5.13 Multiplexer tree for a function of three variables. . . . 100
5.14 The 74381 ALU: symbol and one bit slice. . . . 101
5.15 A three-state buffer with enable input $C$. . . . 102
5.16 A 4-bit magnitude comparator. . . . 103

6.1 Block diagram of a sequential circuit: combinational logic, memory elements and the state feedback. . . . 109
6.2 Basic latches built from cross-coupled gates. . . . 110
6.3 Gated latches. . . . 111
6.4 Graphical symbols of the SR, $\overline{SR}$ and D latches. . . . 111
6.5 Level triggering, positive-edge triggering and negative-edge triggering. 113
6.6 Edge-triggered D flip-flop. . . . 113
6.7 JK flip-flop. . . . 114
6.8 T flip-flop. . . . 115
6.9 D flip-flops with asynchronous preset and clear inputs. . . . 118
6.10 The circuit analysed in Example 6.1. . . . 120

6.11 The circuit analysed in Example 6.2. . . . 121
6.12 The circuit analysed in Example 6.3. . . . 123
6.13 State diagram of the 101 sequence detector (Mealy). . . . 124
6.14 Logic diagram of the 101 detector with D flip-flops. . . . 125
6.15 A 3-bit binary counter with T flip-flops. . . . 129

7.1 A 4-bit register built from D flip-flops. . . . 132
7.2 A 4-bit register with parallel load. When $Load = 0$ the flip-flops recirculate their own outputs. . . . 133
7.3 A 4-bit shift register: serial input at the left, serial output at the right. . . . 134
7.4 Serial transfer from register $A$ to register $B$. . . . 134
7.5 A serial adder: one full adder and a carry flip-flop. . . . 135
7.6 A 4-bit universal shift register. . . . 136
7.7 General $n$-bit counter. . . . 139
7.8 A 4-bit binary ripple counter built from T flip-flops with $T = 1$. . . . 139
7.9 Timing diagram of a ripple counter: each flip-flop changes a delay after the previous one. . . . 140
7.10 The BCD counter. . . . 141
7.11 A 4-bit synchronous binary counter with T flip-flops. . . . 143
7.12 Up/down counter. . . . 144
7.13 A 4-bit counter with parallel load. . . . 146
7.14 Two ways of making a BCD counter from a binary counter,. . . . 148
7.15 Block diagram of a modulo-$N$ counter. . . . 148
7.16 State diagram of the self-correcting modulo-6 counter. . . . 149
7.17 Ring counter and the timing signals it generates. . . . 150
7.18 Two timing-signal generators. . . . 151

8.1 General structure of an FSM: next-state logic, state register and output logic. . . . 157
8.2 Block diagrams of the Moore and Mealy models; the only difference is whether the inputs feed the output logic. . . . 158
8.3 The circuit analysed in Example 8.1. . . . 158
8.4 Synthesis of the 2-bit up/down counter. . . . 160
8.5 Car security controller. . . . 162
8.6 Modulo-6 counter with enable. . . . 164
8.7 Conversions among `std_logic_vector`, `unsigned`, `signed` and `integer` in `numeric_std`. . . . 168
8.8 The Mealy one-shot circuit. . . . 169
8.9 The Moore one-shot and the comparison of the two outputs. . . . 174
8.10 State diagram of the control unit of Example 8.6. . . . 176
8.11 Two-floor elevator controller, Moore version. . . . 177
8.12 Two-floor elevator controller, Mealy version. . . . 181

8.13 Moore and Mealy output timing. . . . 183
8.14 K-map for Exercise 8.8. . . . 187

9.1 The von Neumann model of a computer. . . . 190
9.2 Layers of abstraction: from transistors to microprocessors. . . . 191
9.3 A dedicated microprocessor: the control unit sends control signals to the datapath and receives status signals from it. . . . 191
9.4 The RTL model: registers, combinational logic between them, and a common clock. . . . 192
9.5 Two dedicated datapaths. . . . 193
9.6 A shared-adder datapath that performs $A = A + 3$ and $A = B + C$. Both multiplexers are driven by the same control signal *Mux* (input 1 selects $A$ and the constant 3, input 0 selects $B$ and $C$); the bus width is marked with a slash, and the clock is common to all registers. . . . 194
9.7 Alternative datapath for $A = B + C$ and $A = A + 3$: one multiplexer *Amux* in front of register $A$. . . . 195
9.8 Generating the status signal $(A = 0)$: (a) comparator with the constant 0; (b) 8-input NOR gate. . . . 195
9.9 Datapath for the if-then-else algorithm. . . . 196
9.10 A datapath for summing the numbers from 1 to 10 with one adder and three multiplexers. The status signal $(i \neq 11)$ is a NAND gate whose inputs for the zero bits of $11 = 00001011_2$ are inverted; the output *Sum* is enabled by *OE*. . . . 198
9.11 The counter processor of Example 9.3. . . . 200
9.12 Control-unit state diagrams for the if-then-else algorithm. . . . 202
9.13 Simulation waveforms of the if-then-else processor (author's screenshots). . . . 203
9.14 The if-then-else control unit connected to its datapath. . . . 204
9.15 The GCD processor. . . . 206
9.16 Simulation of the GCD processor (author's screenshot). . . . 210
9.17 The FSMD GCD: logic diagram from the Quartus RTL viewer (top) and simulation (bottom). Author's screenshots. . . . 212

10.1 Datapath of the EC-1 microprocessor. Multiplexer inputs are numbered 1 (left) and 0 (right); a slash on a bus gives its width in bits. The clock and the reset are common to all registers, and the control signals (left) come from the control unit. . . . 218
10.2 State diagram of the EC-1 control unit. . . . 220
10.3 Simulation of the EC-1 running the countdown program (author's screenshots). . . . 225
10.4 RTL viewer (Quartus) of the top level of the EC-1: the control unit and the datapath (author's screenshot). . . . 228

10.5 The von Neumann architecture: a single memory holds instructions and data, connected to the processor by a common bus. . . . . 228
10.6 Datapath of the EC-2 microprocessor. The memory-address multiplexer *MemInst* selects the PC (0) in the fetch state and the address field of the IR (1) when a memory operand is accessed; the accumulator multiplexer is selected by $Asel_{1..0}$ (00 RAM, 01 ALU, 10 input port, 11 constant 0). $(A = 0)$ and $(A \geq 0)$ are the status signals for `JZ` and `JPOS`. . . . . 230
10.7 State diagram of the EC-2 control unit. . . . . 231
10.8 Multi-cycle memory access in the EC-2. . . . . 232
10.9 Simulation of the EC-2 computing the GCD of 25 and 15: (a)–(d) successive parts of the run: $Y$ becomes 10, then $X$ becomes 5, then $Y$ becomes 5, and finally $A = X - Y = 0$, `JZ` is taken and the processor halts. Author's screenshots. . . . . 234
10.10 RTL viewer (Quartus) of the EC-2 top level (author's screenshot). . . . 239

# List of Tables

1.1 Selected Intel processors. Values are approximate and are those quoted in the manufacturer's literature for the original product of each family; transistor counts of different sources vary slightly. . . . . 8
1.2 Comparison of the three implementation technologies. . . . . 10

2.1 Axioms and single-variable theorems of Boolean algebra. Each line on the left is the *dual* of the line on the right. . . . . 16
2.2 Properties of Boolean algebra. . . . . 17
2.3 Truth tables of AND, OR and NOT. . . . . 19
2.4 Truth table of Example 2.1 with minterms and maxterms. . . . . 22
2.5 Truth table for the three-way light. . . . . 24
2.6 Truth table of a 2-to-1 multiplexer. . . . . 25
2.7 Half adder. . . . . 27

4.1 The numbers 0 to 16 in four radices. . . . . 54
4.2 Four-bit signed numbers in the three representations. . . . . 55
4.3 Truth table of the full adder. . . . . 57
4.4 Example ASCII codes. . . . . 77

5.1 Truth table obtained in Example 5.1. . . . . 83

5.2 BCD and excess-3 codes. . . . 84
5.3 Truth table of a 3-to-8-line decoder (active-high outputs). . . . 89
5.4 3-to-8 decoder with active-low enable $E$ and active-low outputs. . . . 91
5.5 1-to-4 demultiplexer. . . . 91
5.6 4-to-2 priority encoder ($D_3$ has the highest priority). . . . 95
5.7 4-to-1 multiplexer. . . . 96
5.8 Implementation table for Example 5.4. . . . 98
5.9 Function table of the 74381 ALU. . . . 100

6.1 SR latch with NOR gates. . . . 110
6.2 $\overline{SR}$ latch with NAND gates. . . . 110
6.3 D latch. . . . 111
6.4 Characteristic tables and equations of the flip-flops. . . . 115
6.5 State table of Example 6.1. . . . 119
6.6 State table of Example 6.2. . . . 119
6.7 State table of Example 6.3. . . . 122
6.8 State table of the 101 detector, $S_0 = 00$, $S_1 = 01$, $S_2 = 10$. . . . 124
6.9 Excitation tables of the JK and T flip-flops. . . . 125
6.10 Excitation table for the 101 detector with JK flip-flops. . . . 126
6.11 State table and T inputs of the 3-bit counter. . . . 128

7.1 Serial transfer of $A = 1011$ into $B$ ($A$ is rotated). . . . 134
7.2 Trace of the serial addition 1001 + 0011. . . . 135
7.3 Function table of a 4-bit universal shift register ($A_i^+$ is the next value of $A_i$). . . . 138
7.4 Function table of a 4-bit counter with parallel load (the clear is asynchronous and active-high in the VHDL of Listing 7.2). . . . 142
7.5 State table of the self-correcting modulo-6 counter. . . . 149
7.6 Sequence of a 4-bit ring counter. . . . 150

8.1 State table of Example 8.1. . . . 156
8.2 Excitation table of the 2-bit up/down counter. . . . 160
8.3 State table of the car security controller. . . . 161
8.4 State table of the modulo-6 counter with enable $C$. . . . 164
8.5 State table of the Mealy one-shot. . . . 168
8.6 VHDL coding differences between Moore and Mealy machines. . . . 176

9.1 Control words of the shared-adder datapath. . . . 194
9.2 Control words for the if-then-else algorithm. . . . 196
9.3 Control words for the summation algorithm. . . . 197
9.4 Tables of the counter control unit ($x = (i = 10)$). . . . 199
9.5 Next-state and output tables of the if-then-else control unit with wait state ($I = (A = 5)$). . . . 202

10.1 The EC-1 instruction set. . . . 217
10.2 Control signals of the EC-1 in each state ("–" = don't care). . . . 219
10.3 EC-1 program: count down from the input value. . . . 224
10.4 The EC-2 instruction set. . . . 229
10.5 Control signals of the EC-2 in each state ($Z$: $A = 0$; $P$: $A \geq 0$). . . . 231
10.6 EC-2 program that computes the GCD of the numbers in locations 30 and 31 (addresses in decimal). . . . 233
10.7 A bootloader for the EC-2 using self-modifying code; the loaded program starts at address 10. . . . 239
10.8 Comparison of EC-1 and EC-2. . . . 240

A.1 Common VHDL types. . . . 244
A.2 Conversion functions. . . . 245

B.1 Two-input gate functions. . . . 247
B.2 Expressions and VHDL operators of the gates. . . . 247
B.3 Powers of two. . . . 248
B.4 Four-bit binary, Gray and excess-3 codes (excess-3 is used for the decimal digits 0–9 only). . . . 248
B.5 Ranges of $n$-bit integers. . . . 248
B.6 Segments that are *lit* for each decimal digit (1 = lit; segment letters as in Figure 2.12). For a common-anode display the control signal is the complement. . . . 249
B.7 The printable ASCII characters (codes 32–126) [1]. . . . 249

# Listings

2.1 VHDL description of $f = (x_1 + x_2) \cdot x_3$. . . . 19
2.2 One entity with two architectures: canonical and simplified. . . . 22
2.3 Three-way light: two architectures. . . . 24
2.4 Car alarm logic. . . . 26
2.5 Half adder in VHDL (dataflow style). . . . 27
2.6 A testbench for the three-way light that applies all eight input combinations. . . . 28
3.1 Even-parity generator and checker in VHDL. . . . 49
4.1 Structural VHDL of the full adder, built from two half-adder components. 57
4.2 Structural VHDL of a 4-bit ripple-carry adder. . . . 59

4.3 VHDL of the 4-bit adder/subtractor, reusing the ripple-carry adder. . . 61
4.4 Carry-lookahead logic block of the 4-bit CLA. . . . . 66
4.5 Top level of the 4-bit carry-lookahead adder. . . . . 67
4.6 VHDL of the 4×4 array multiplier (first part: declarations, partial products and the first adder row). The remaining rows follow the same pattern. . . . . 69
5.1 VHDL of the BCD to excess-3 converter. . . . . 84
5.2 Behavioural VHDL of a one-digit BCD adder. The package `numeric_std` provides the type `unsigned` and arithmetic on it. . . . . 87
5.3 A 3-to-8 decoder in VHDL. . . . . 89
5.4 4-to-1 multiplexer, behavioural architecture. . . . . 96
5.5 4-to-1 multiplexer written as a Boolean expression ("structural" in the lecture notes). . . . . 96
5.6 A three-state buffer. . . . . 102
5.7 Behavioural description of the comparator using the relational operators. 103
5.8 Gate-level description of the comparator. . . . . 104
6.1 A D latch. The process is sensitive to both `D` and `C`; since no value is assigned when `C` is 0, the synthesis tool infers a latch. . . . . 110
6.2 A positive-edge-triggered D flip-flop. . . . . 114
6.3 A JK flip-flop. The internal signal `q_int` lets the value of `Q` be read back for the toggle operation. . . . . 116
6.4 A T flip-flop. . . . . 116
6.5 Process with asynchronous reset. . . . . 117
6.6 Process with synchronous reset. . . . . 117
6.7 The 101 detector as a state machine with an enumerated state type. . . 126
6.8 The 101 detector as two D flip-flops and gates (structural description). 127
7.1 Behavioural VHDL of a 4-bit universal shift register with asynchronous clear. . . . . 135
7.2 VHDL of a 4-bit counter with asynchronous clear, parallel load and count enable. . . . . 142
8.1 VHDL of the car security controller: an enumerated state type, a state-register process and a combinational process (the two-process method). 161
8.2 Behavioural VHDL of the modulo-6 counter with enable. . . . . 164
8.3 VHDL of the Mealy one-shot. . . . . 167
8.4 VHDL of the Moore one-shot (exact one-cycle pulse). . . . . 173
8.5 VHDL of the Moore elevator controller (four states). . . . . 177
8.6 VHDL of the Mealy elevator controller (two states). . . . . 180
8.7 Template of the two-process FSM description (the state register is process 1 and the combinational logic process 2). . . . . 185

9.1 Structural VHDL of the if-then-else control unit (state register of D flip-flops and the synthesised equations). The component `d_ff_r` is the flip-flop with asynchronous reset of Listing 6.8. . . . . 201
9.2 VHDL of the GCD control unit (FSM with five states). . . . . 205
9.3 The GCD calculator as an FSMD: datapath operations inside a single behavioural state machine. . . . . 209
10.1 VHDL of the EC-1 control unit (structural, from the state equations). . 221
10.2 VHDL of the EC-1 datapath. . . . . 222
A.1 Skeleton of a VHDL design unit. . . . . 243
A.2 Concurrent signal assignments. . . . . 244
A.3 Processes for combinational logic and flip-flops. . . . . 245
A.4 Clock generation, stimulus and a self-checking assertion. . . . . 245
C.1 GCD processor: library of components (registers, multiplexers, subtractor, comparator, ...). . . . . 251
C.2 GCD processor: datapath. . . . . 253
C.3 GCD processor: top level connecting control unit and datapath. . . . . 255
C.4 GCD processor: testbench. . . . . 257
C.5 EC-1: components package. . . . . 258
C.6 EC-1: top level. . . . . 261
C.7 EC-1: testbench (input value 10). . . . . 262
C.8 EC-2: control unit. . . . . 264
C.9 EC-2: datapath. . . . . 266
C.10 EC-2: components package, including the RAM whose initial contents are the GCD program of Table 10.6. . . . . 268
C.11 EC-2: top level. . . . . 272
C.12 EC-2: testbench. . . . . 273

# Introduction: How to Use This Book

## Who this book is for

This book is written for first-year students of electrical and computer engineering, who take this course in their second semester, and for anyone who wants a rigorous but practical first course in digital design. No prior course in electronics or programming is required. You should be comfortable with high-school algebra; the few additional mathematical ideas (binary numbers, sets of minterms, modular arithmetic) are introduced as they are needed.

## Courses

This book accompanies the lectures of 010113025 *Digital Circuit and Logic Design* and 010153101 *Digital Circuit and Microprocessor Fundamental*, which follow the same ten lectures; the chapters correspond one to one to those lectures.

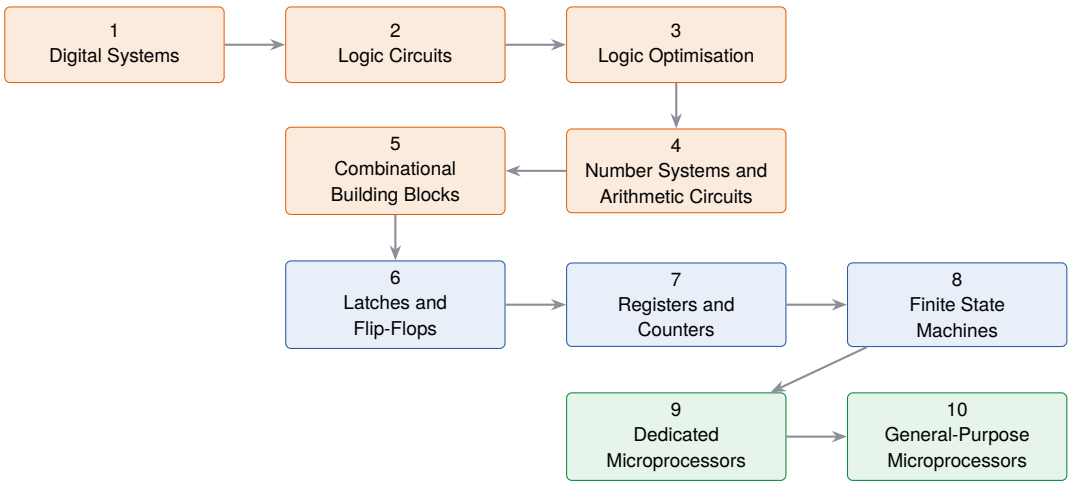
## What you will be able to do

After studying the whole book you should be able to:

1. convert between number systems and represent signed numbers, fractions and characters in binary;
2. write a Boolean function from a truth table, simplify it, and draw the resulting logic network with AND, OR, NOT, NAND, NOR and XOR gates;
3. analyse and design adders, decoders, encoders, multiplexers, comparators and arithmetic logic units;
4. explain the behaviour of SR, D, JK and T latches and flip-flops from their symbols, characteristic tables and timing diagrams;
5. analyse and design synchronous sequential circuits (registers, counters and finite state machines), using Moore and Mealy models;
6. design a dedicated microprocessor as a datapath plus a control FSM, and describe each part in VHDL; and
7. read, run and extend a small general-purpose processor with its own instruction set.

## Roadmap

The chapters build on each other, as shown in the diagram below. Chapters 1–3 and 4–5 form the combinational half of the book; Chapters 6–8 add memory and time; Chapters 9–10 combine both halves into processors.




Reading order of the chapters. Orange: combinational logic. Blue: sequential logic. Green: processors.

## How each chapter is organised

**Learning objectives** say what you should be able to do after the chapter.

**Sections** explain a concept, give the reasoning behind it, and then work through at least one example. Technical terms appear in *italics* at the place where they are defined, and each such term is listed in the index.

**Worked examples** are boxed and numbered; try to solve them yourself before reading the solution.

**Chapter summary** lists the ideas and procedures you must remember.

**Exercises** are graded from routine drills to design problems. Answers to selected exercises are in Chapter D.

**Further reading** points to books and papers for readers who want more depth.

## Conventions

- **Boolean operators.** Variables are written $x, y, z$ or $A, B, C$. AND is written by juxtaposition $(xy)$ or $x \cdot y$, OR as $x + y$, XOR as $x \oplus y$, and NOT as $x'$ or $\overline{x}$; the two forms of complement are interchangeable.
- **Binary numbers** are set in typewriter type, e.g. `1011`; a subscript gives the base where it might be ambiguous, e.g. $1011_2 = 11_{10}$.

- **Minterms and maxterms** are written $m_i$ and $M_i$, where $i$ is the row number of the truth table.
- **VHDL** keywords and signal names appear in `typewriter type`; listings are numbered by chapter and can be downloaded from the book repository.
- **Register transfers** are written $R1 \leftarrow R2$: the contents of register R2 are copied into R1 on the next active clock edge.

## Tools

You can work through all of the examples with a free VHDL simulator and a schematic editor. The screenshots in this book were produced with Intel Quartus Prime (including its RTL viewer) and ModelSim; any standards-compliant VHDL simulator, for example GHDL, will run the listings. For small schematics, browser-based tools such as CircuitVerse or the desktop program Logisim are convenient. No physical hardware is required, but Chapter 10 explains how the processors can be loaded onto an FPGA board if one is available.

## Study advice

Digital design is learned by doing. Do not just read a K-map or a state diagram: redraw it. Write the VHDL for each design, simulate it, and compare the waveform with your expectation from the truth table or state diagram. When a simulation disagrees with your calculation, you have found the most useful thing to learn from. Work the exercises in the order given, and return to earlier chapters when a prerequisite feels shaky.

# 1

# Digital Systems

**Learning Objectives**

After studying this chapter you will be able to:

- distinguish *analog* from *digital* signals and explain why most modern systems are digital;
- explain how a binary number represents a quantity, and compute how many bits are needed to represent $N$ distinct items;
- describe the two steps of analog-to-digital conversion (sampling and quantisation) and estimate the resolution of a converter;
- outline the key inventions—relay, vacuum tube, transistor, integrated circuit, microprocessor—that led to modern digital hardware;
- state Moore's law and use it to estimate how transistor counts grow; and
- classify logic chips as standard (74xx), programmable (PLD/FPGA) or custom (ASIC), and describe the modern digital design flow.

## 1.1 Why Study Digital Systems?

Digital circuits are the foundation of almost every electronic product of the modern world. A short list of where you will meet them as an engineer gives an idea of the breadth of the subject:

- **Smart devices:** phones, watches and home assistants;
- **Computing:** laptops, servers and supercomputers;
- **Communication:** cellular (5G) networks, Wi-Fi and the Internet;
- **Automotive:** engine control, safety systems such as ABS and airbags, and infotain-

ment;

- **Aerospace and defence:** avionics, guidance systems and radar.

All of these systems store, move and transform information that has been encoded as numbers. A *digital system* is one that represents information with a finite set of discrete values and processes it with logic circuits. In this book we shall learn how such circuits are described, simplified, built and verified, and how the same ideas, scaled up, produce a processor.

## 1.2 Analog and Digital Signals

A *signal* is a quantity that varies and carries information, for example a voltage that changes with time. An *analog signal* is continuous in both time and amplitude: between any two instants there are infinitely many instants, and at each instant the voltage may take any value in a continuous range. A *digital signal* is discrete: it is defined at specific instants and takes one of a finite number of values. Figure 1.1 contrasts the two.

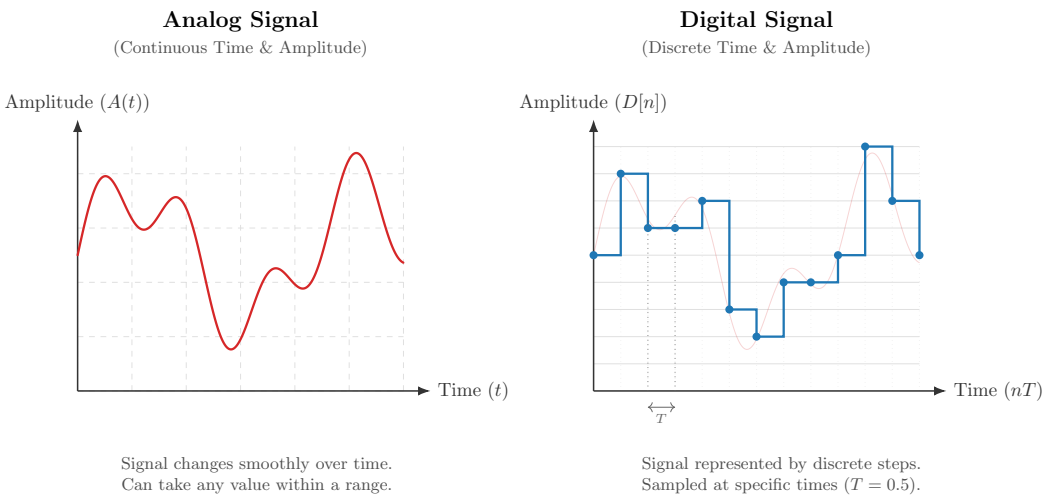



**Figure 1.1.** An analog signal has continuous values; a digital signal has discrete, step-like values that are defined only at the sampling instants.

Why do engineers prefer digital representations? There are three main reasons.

1. **Noise immunity.** A digital receiver need only decide whether a voltage is closer to a "low" or a "high" level, so moderate noise does not change the information. An analog signal, in contrast, is degraded irreversibly by every disturbance it picks up. This is why a digital copy of a recording is identical to its source, whereas the hiss of an analog tape accumulates with every copy.
2. **Exact and flexible processing.** Once information is a number, it can be stored in-

definitely, copied without loss, compressed, encrypted and processed by a general-purpose machine.

3. **Richer information.** A digital sensor can send the measurement together with diagnostic data, calibration constants and its own identity over a single digital link, whereas an analog sensor (for instance a gauge producing a 4–20 mA current) carries just one value.

## 1.3 Binary Representation of Information

The simplest digital signal has just two values, usually called *0* and *1*. Such a signal can be produced by any device that has two stable states: a switch that is open or closed, a lamp that is off or on, a voltage that is low or high. One such value is a *bit* (binary digit). Figure 1.2 shows the correspondence between a switch and a bit.

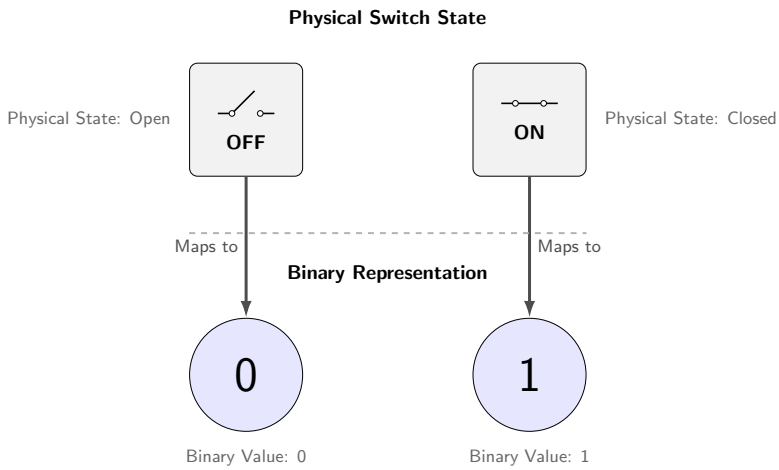



**Figure 1.2.** A switch as a one-bit memory element: open = 0, closed = 1.

### 1.3.1 Positional notation

A single bit can represent only two things, but a group of bits can represent many more. In a *positional number system* with base (radix) $r$, a number is a string of digits $d_{n-1} \ldots d_1 d_0$ whose value is

$$V = \sum_{i=0}^{n-1} d_i \, r^i. \tag{1.1}$$

In the *binary* system $r = 2$ and each $d_i \in \{0, 1\}$. For instance, the binary number `1001111` has the value shown in Figure 1.3:

$$1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 4 + 2 + 1 = 79.$$

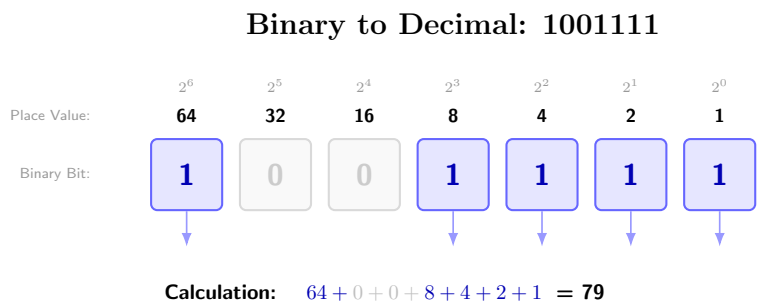



**Figure 1.3.** Binary representation of the decimal number 79. Each bit position has a weight that is a power of two.

**Definition 1.1: Number of codes**

An $n$-bit string can take $2^n$ different values. To represent $N$ distinct items by a binary code we need

$$n = \lceil \log_2 N \rceil$$

bits, where $\lceil \cdot \rceil$ rounds up to the next integer.

**Example 1.1: Encoding the keys of a keypad**

A keypad is arranged as a matrix of row wires and column wires, as in Figure 1.4. Pressing a key connects one row to one column. A microcontroller *scans* the keypad: it drives one row at a time and reads the columns to find which, if any, is connected. The unique (row, column) pair is then *encoded* as a binary number.

**Question.** How many bits are needed to encode the keys of (a) a $4 \times 4$ keypad and (b) a telephone keypad with 12 keys?

**Solution.** (a) There are $N = 16$ keys, so $n = \lceil \log_2 16 \rceil = 4$ bits (key '1' could be `0000`, key '2' `0001`, and so on). (b) For $N = 12$, $\log_2 12 \approx 3.58$, so $n = 4$ bits again; four of the 16 possible codes are unused.

## 1.4 Analog-to-Digital Conversion

The physical world is analog—temperature, pressure, sound and light vary continuously—so an interface is needed between it and a digital system. An *analog-to-digital converter* (ADC) performs two operations, shown in Figure 1.5.

1. *Sampling*: the value of the analog signal is measured at regular instants separated by the *sampling period* $T_s$. The *sampling frequency* is $f_s = 1/T_s$.
2. *Quantisation*: every sampled value is replaced by the nearest of a finite set of levels. The distance between two adjacent levels is the *step size* $Q$. The difference between the true sample and the level chosen is the *quantisation error*, which is at most $Q/2$.

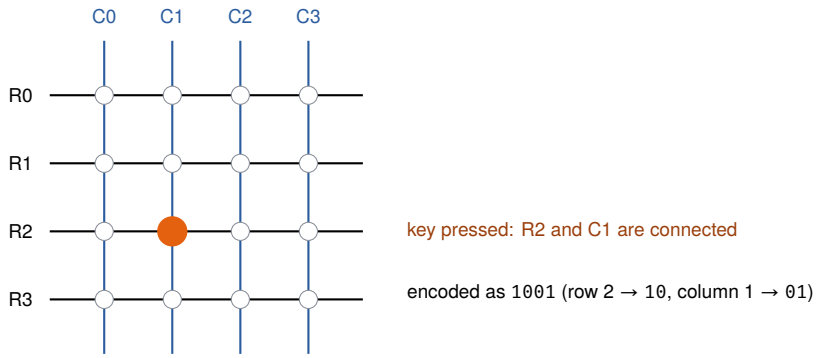



**Figure 1.4.** A 4 × 4 matrix keypad. Scanning identifies the connected row and column; the pair is encoded in four bits.

The level number is then written in binary. An $n$-bit ADC with full-scale range $V_{\mathrm{FS}}$ has $2^n$ levels and

$$Q = \frac{V_{\mathrm{FS}}}{2^n}. \tag{1.2}$$

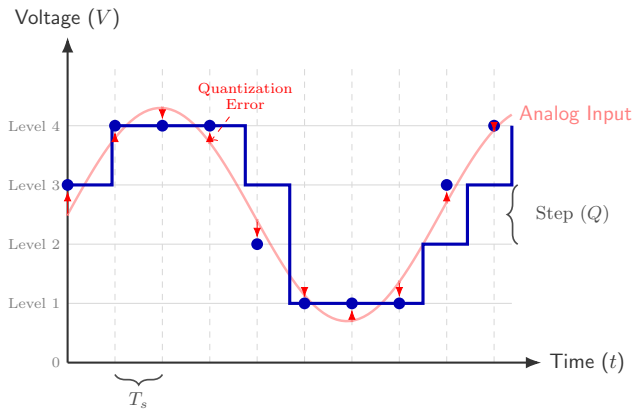



**Figure 1.5.** Sampling and quantisation. The dots are the samples taken every $T_s$; the stair-step is the quantised signal; the red marks are the quantisation errors.

**Example 1.2: Resolution of an ADC**

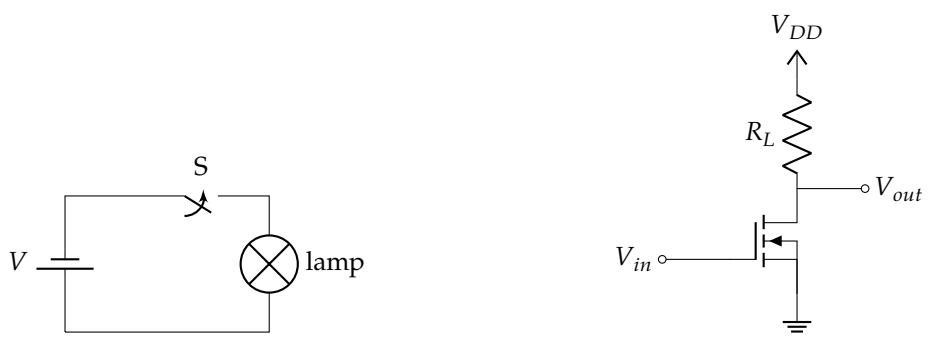
A sensor produces a voltage between 0 and 5 V. It is digitised by a 10-bit ADC.
**Question.** What are the step size and the largest quantisation error? How many bits would be needed to make the step smaller than 1 mV?
**Solution.** With $n = 10$ the number of levels is $2^{10} = 1024$, so $Q = 5/1024 \approx 4.88$ mV, and the largest error is $Q/2 \approx 2.44$ mV. For $Q < 1$ mV we need $2^n > 5000$, so $n \geq \lceil \log_2 5000 \rceil = 13$ bits.

**Sampling rate**

If the signal contains frequencies up to $f_{\max}$, the sampling frequency must exceed $2f_{\max}$ (the Nyquist criterion); otherwise different signals become indistinguishable after sampling. Treatment of this theorem belongs to a course on signals and systems; here it is enough to remember that sampling too slowly loses information.

## 1.5 From Switches to Transistors

Every digital circuit is built from devices that act as electrically controlled switches. An *ideal switch* (Figure 1.6a) changes state instantaneously, has zero resistance when closed, infinite resistance when open and dissipates no power. A real device falls short of this in every respect. The transistor, which is today's switch of choice, takes a finite time to change state (rise and fall times), has a small non-zero on-resistance and a very large but finite off-resistance, and dissipates energy while switching. Much of digital design is concerned with the consequences of these imperfections, including propagation delay and power consumption.




**(a)** ideal switch **(b)** transistor used as a switch

**Figure 1.6.** (a) An ideal switch connecting a battery to a lamp. (b) An n-channel MOS transistor used as a switch: a high $V_{in}$ closes the path from drain to source and pulls $V_{out}$ low.

## 1.6 A Short History of Digital Hardware

The history of digital hardware is a history of better switches, summarised in Figure 1.7.

### 1.6.1 Relays and vacuum tubes

In the 1930s digital logic was built from electromechanical *relays*: an electromagnet moves a metal contact. Relays are slow (milliseconds) and wear out. In the 1940s the *vacuum tube* replaced them. With no moving parts it could switch in microseconds,

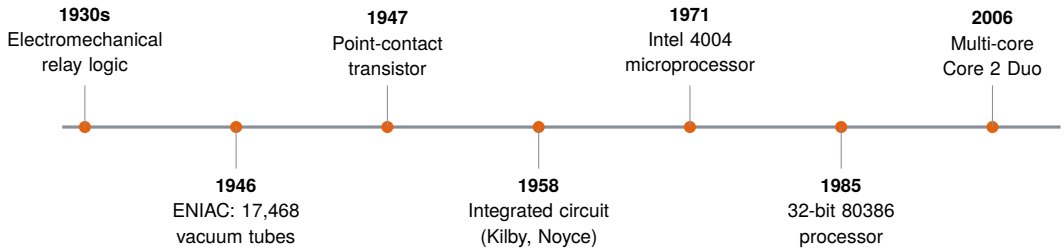



**Figure 1.7.** Milestones in digital hardware (the date of the Intel 4004 is the year of its commercial introduction).

which made electronic computing practical. The *ENIAC* (Electronic Numerical Integrator and Computer, 1946) used 17,468 vacuum tubes, about 70,000 resistors, 10,000 capacitors, 1,500 relays and 6,000 manual switches; it weighed about 30 tons, occupied roughly 167 $m^2$ and consumed about 160 kW. It could perform about 5,000 additions per second, some 200,000 times faster than a human with a desk calculator.

**Origin of the word "bug"**

> The word *bug* for a fault predates computers, but a famous entry in the logbook of the Harvard Mark II relay calculator, dated 9 September 1947, records a moth found trapped between the contacts of Relay 70, Panel F, which was causing errors. The operators taped the moth into the log as "first actual case of bug being found"; the story is credited to the team working with Grace Murray Hopper and is the origin of the term *debugging*.

### 1.6.2 The transistor and the integrated circuit

In 1947 John Bardeen, Walter Brattain and William Shockley at Bell Laboratories demonstrated the first *transistor*, a solid-state device able to amplify and to switch electrical signals. It was smaller, faster, cooler and far more reliable than a vacuum tube, and the three inventors shared the 1956 Nobel Prize in Physics. In 1958 Jack Kilby (Texas Instruments) built the first *integrated circuit* (IC), several components on one piece of germanium, and in 1959 Robert Noyce (Fairchild Semiconductor) developed the monolithic silicon IC that is the basis of manufacturing today. Kilby was awarded the Nobel Prize in Physics in 2000. By 1961 the first *logic* ICs (flip-flops with a handful of transistors) were on the market, and by 1967 a semi-custom chip with about 150 logic gates was available.

### 1.6.3 The microprocessor

Intel was founded in 1968 by Robert Noyce and Gordon Moore, soon joined by Andy Grove. The *microprocessor*, a complete central processing unit on a single chip, appeared in 1971 as the Intel 4004, designed for a calculator. The Intel 8088, a variant of the 8086, was chosen for the original IBM Personal Computer in 1981 and established

the x86 architecture that remains in use. The 80386 (1985) was the first 32-bit x86 processor and made modern multitasking operating systems practical. By 2006 designers had turned from raising the clock rate, which was limited by heat, to placing several processor *cores* on one chip. Table 1.1 summarises the growth of the Intel line.

**Table 1.1.** Selected Intel processors. Values are approximate and are those quoted in the manufacturer's literature for the original product of each family; transistor counts of different sources vary slightly.

| Processor | Year | Feature size ($\mu$m) | Transistors | Clock (MHz) | Word (bits) |
|---|---|---|---|---|---|
| 4004 | 1971 | 10 | 2.3 k | 0.75 | 4 |
| 8008 | 1972 | 10 | 3.5 k | 0.5–0.8 | 8 |
| 8080 | 1974 | 6 | 6 k | 2 | 8 |
| 8086 | 1978 | 3 | 29 k | 5–10 | 16 |
| 80286 | 1982 | 1.5 | 134 k | 6–12 | 16 |
| 80386 | 1985 | 1.5–1.0 | 275 k | 16–25 | 32 |
| 80486 | 1989 | 1–0.6 | 1.2 M | 25–100 | 32 |
| Pentium | 1993 | 0.8–0.35 | 3.2–4.5 M | 60–300 | 32 |
| Pentium Pro | 1995 | 0.6–0.35 | 5.5 M | 166–200 | 32 |
| Pentium II | 1997 | 0.35–0.25 | 7.5 M | 233–450 | 32 |
| Pentium III | 1999 | 0.25–0.18 | 9.5–28 M | 450–1000 | 32 |
| Pentium 4 | 2001 | 0.18–0.13 | 42–55 M | 1400–3200 | 32 |
| Core 2 Duo | 2006 | 0.065 | 291 M | 1800–3330 | 64 |
| Core i7 | 2008 | 0.045 | 731 M | 2660–3330 | 64 |
| Core i7 (Sandy Bridge) | 2011 | 0.032 | 995 M | 3400–3800 | 64 |
| Core i7 (Skylake) | 2015 | 0.014 | 1.75 B | 4000–4200 | 64 |

## 1.7 Moore's Law

In 1965 Gordon Moore, then at Fairchild and later a co-founder of Intel, observed that the number of components on an integrated circuit had been doubling every year, and predicted that the trend would continue [2]. He revised the doubling period to about two years in 1975, and the statement that the number of transistors per chip doubles every one to two years became known as *Moore's law*. It is an empirical observation about an industry, not a law of nature, but it has described the evolution of chips for half a century. Figure 1.8 plots the transistor counts of Table 1.1 on a logarithmic axis, on which an exponential trend appears as a straight line.

**Example 1.3: Doubling time**

The Intel 4004 (1971) had about 2,300 transistors and the Skylake Core i7 (2015) about $1.75 \times 10^{9}$.

**Question.** How many doublings does this represent, and what is the average doubling time?

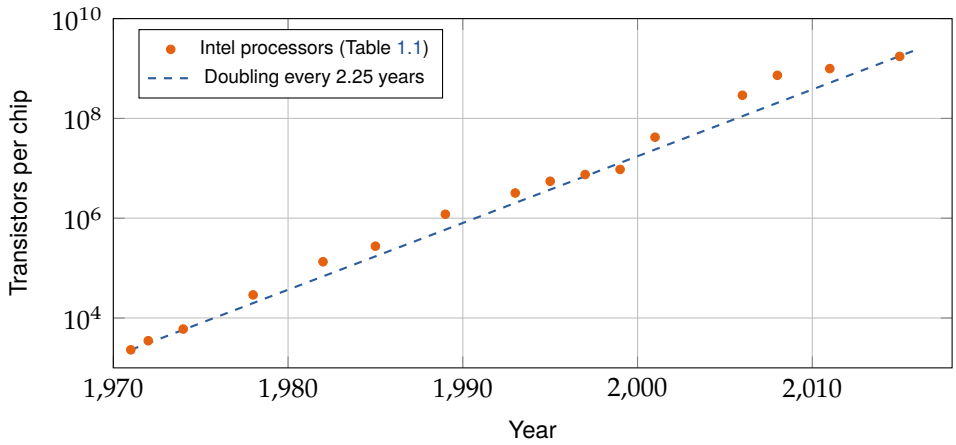



**Figure 1.8.** Transistor count of Intel processors from Table 1.1, with a doubling time fitted between the first and last points.

**Solution.** The ratio is $1.75 \times 10^9 / 2300 \approx 7.6 \times 10^5$, and $\log_2(7.6 \times 10^5) \approx 19.5$ doublings. The period is $2015 - 1971 = 44$ years, so the average doubling time is $44/19.5 \approx 2.3$ years, consistent with Moore's revised estimate of two years.

## 1.8 Types of Logic Chips

The hardware that implements a digital design can take one of three broad forms, summarised in Table 1.2.

**Standard chips (74xx).** A standard chip contains a small, fixed amount of logic, typically fewer than 100 transistors: a few gates, a flip-flop or a counter. The *74xx series* (for example the 7400, which holds four 2-input NAND gates) is the best-known family. These chips are not programmable, since their function is fixed at manufacture, and today they are used mostly as "glue logic" between larger chips, in teaching laboratories and for very simple functions.

**Programmable logic (PLD and FPGA).** A *programmable logic device* (PLD) is a chip containing a collection of logic elements whose interconnections can be configured by the user *after* manufacture. A *field-programmable gate array* (FPGA) is a very large PLD with thousands to millions of logic cells, memory blocks and arithmetic units. The designer describes the circuit in a hardware description language, such as VHDL or Verilog, and software tools translate it into a configuration file. FPGAs are excellent for prototypes, low-volume products and systems that must be updated in the field.

**Custom chips (ASIC).** An *application-specific integrated circuit* (ASIC) is designed for one purpose. It offers the best performance, the smallest area and the lowest power consumption, but it cannot be changed after manufacture, and the cost of design and fabrication masks (the non-recurring engineering cost) is very high. ASICs are economical only for high-volume products such as mobile phones and graphics processors.

**Table 1.2.** Comparison of the three implementation technologies.

| | **Standard (74xx)** | **PLD / FPGA** | **ASIC** |
|---|---|---|---|
| Function fixed at | manufacture | configuration (field) | manufacture |
| Typical size | < 100 transistors | up to millions of gates | up to billions of transistors |
| Up-front cost | very low | low | very high |
| Unit cost at high volume | moderate | high | lowest |
| Performance / power | modest | good | best |
| Typical use | glue logic, teaching | prototypes, small volumes | mass-market products |

## 1.9 The Digital Design Flow

A modern digital circuit is designed in two phases, shown in Figure 1.9.

**Design phase.** The problem specification states *what* the circuit must do. The designer enters the design either as a schematic or, more commonly, as an RTL description in a *hardware description language* (HDL) such as VHDL or Verilog. *Functional simulation* then checks, using a test bench, that the description behaves as specified. No hardware details such as delays are involved yet.

**Implementation phase.** *Synthesis* translates the HDL description into a netlist of gates and flip-flops. *Post-synthesis simulation* checks the result; *place and route* maps the netlist onto the cells of the chosen FPGA or ASIC; *timing simulation* then includes the real delays, and finally the circuit is verified in hardware.

The rest of this book follows the design phase. We start at the lowest level, the logic gates, and learn how to describe and simplify circuits on paper (Chapters 2–3). We then build the arithmetic and combinational blocks that every processor contains (Chapters 4–5), add memory in the form of flip-flops (Chapters 6–7), organise it with finite state machines (Chapter 8), and finally combine all of these into dedicated and general-purpose microprocessors (Chapters 9–10). Throughout, designs are written in VHDL, which is introduced gradually and summarised in Chapter A.

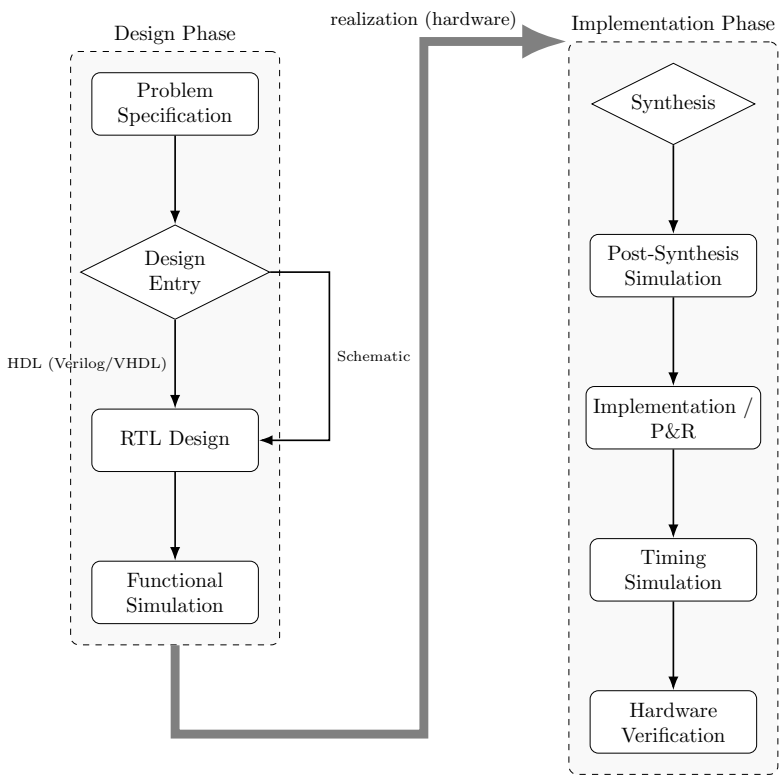



**Figure 1.9.** A development flow for a digital circuit. The design phase (left) produces a verified behavioural description; the implementation phase (right) turns it into hardware.

**Chapter Summary**

- An **analog** signal is continuous in time and amplitude; a **digital** signal takes a finite set of values, most often two (0 and 1). Digital signals tolerate noise and can be processed exactly.
- An $n$-bit binary number has $2^n$ values; encoding $N$ items needs $\lceil \log_2 N \rceil$ bits. The value of a binary number is $\sum d_i 2^i$.
- **Analog-to-digital conversion** consists of *sampling* (in time) and *quantisation* (in amplitude); an $n$-bit ADC has step size $V_{FS}/2^n$.
- Digital hardware evolved from relays to vacuum tubes (ENIAC, 1946), transistors (1947), integrated circuits (1958) and microprocessors (Intel 4004, 1971).
- **Moore's law**: transistor counts per chip double about every two years.
- Logic is implemented in **standard chips**, **programmable logic (PLD/FPGA)** or **ASICs**, trading flexibility against performance and volume cost.
- The design flow has a **design phase** (specification, HDL entry, functional simulation) and an **implementation phase** (synthesis, place and route, timing simulation, verification).

## Exercises

**1.1** Classify each of the following signals as analog or digital, and explain: (a) the output of a thermometer using a liquid column; (b) the voltage on a wire carrying the bits of a text message; (c) the position of a light switch; (d) the reading of a mercury barometer.

**1.2** Convert the following binary numbers to decimal: (a) `1011`; (b) `110010`; (c) `10000001`.

**1.3** Convert the following decimal numbers to binary: (a) 37; (b) 100; (c) 255.

**1.4** How many bits are required to encode (a) the 26 letters of the English alphabet; (b) the 10 decimal digits; (c) 1,000,000 distinct items?

**1.5** An 8-bit ADC has a full-scale range of 0–3.3 V. Compute the step size and the maximum quantisation error. What is the binary output for an input of 1.00 V, if the code is obtained by rounding down, i.e. $\lfloor V/Q \rfloor$?

**1.6** A temperature sensor must be read with a resolution of at least 0.1 K over a range of 0–500 K. What is the smallest number of ADC bits that suffices?

**1.7** Explain why a digital signal is more tolerant of noise than an analog signal. Sketch a noisy digital waveform and show how a receiver with a single threshold recovers the original bits.

**1.8** The 80386 (1985) had about 275,000 transistors and the 80486 (1989) about 1.2 million. Calculate the doubling time implied by these two points, and compare it with the average of Example 1.3.

**1.9** List one advantage and one disadvantage of using (a) a 74xx chip, (b) an FPGA

and (c) an ASIC for a product that will be sold in a quantity of 50 units.

**1.10** Describe in your own words the difference between functional simulation and timing simulation. At which stage of the design flow is each performed, and why does the timing simulation need the result of place and route?

## Further Reading

Brown and Vranesic [3] give a concise treatment of digital hardware and the design flow in their first chapter; Hwang [4] places the same ideas in the context of microprocessor design. Moore's original 1965 article [2] is short and still worth reading, and the history of the relay-to-chip transition is told in many popular accounts of the computer.

# 2

# Logic Circuits

**Learning Objectives**

After studying this chapter you will be able to:

- state the axioms, theorems and properties of Boolean algebra, including duality and De Morgan's theorem, and prove them with truth tables;
- relate switching circuits to the AND, OR and NOT functions and draw the standard gate symbols;
- derive a canonical sum-of-products (SOP) or product-of-sums (POS) expression from a truth table using minterms and maxterms;
- simplify a Boolean expression algebraically and compare the cost of the original and simplified circuits;
- translate a word problem into a truth table, a Boolean expression, a logic network and a VHDL description; and
- verify a VHDL design with a simple testbench.

## 2.1 Logic Circuits

A *logic circuit* performs operations on digital signals. In a *binary logic circuit* each signal takes one of only two values, 0 or 1, which in an electronic circuit correspond to two voltage ranges ("low" and "high"). The most general form of a logic circuit is the *switching network* of Figure 2.1: a block that accepts a set of discrete inputs $x_1, \ldots, x_n$ and produces a set of discrete outputs $f_1, \ldots, f_m$ that depend on those inputs.

If the outputs depend only on the *present* values of the inputs the network is *combinational*; if they also depend on past inputs, the network is *sequential* and must contain

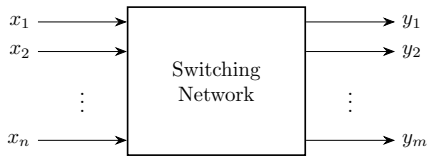



**Figure 2.1.** A switching network: discrete inputs are transformed into discrete outputs.

memory. This chapter and the next four deal with combinational networks; sequential circuits begin in Chapter 6.

## 2.2 Boolean Algebra

The mathematical tool used to describe and manipulate logic circuits is *Boolean algebra*. It was introduced by the English mathematician George Boole in *An Investigation of the Laws of Thought* (1854) [5], where it was meant as an algebra of human reasoning rather than of circuits. The link to hardware was made by Claude Shannon in his 1938 master's thesis [6], which showed that the algebra describes networks of relay switches exactly: a closed switch is 1 and an open switch is 0. Shannon's insight means that *we can design digital circuits by doing mathematics*.

### 2.2.1 Variables, operations and axioms

A *Boolean variable* ($x$, $y$, $A$, ...) may have only the value 0 or 1. These two symbols are *not* ordinary numbers; they stand for two states, for example a low or a high voltage. Three operations are defined:

- *AND* (logical product), written $x \cdot y$ or $xy$;
- *OR* (logical sum), written $x + y$; and
- *NOT* (complement or inversion), written $x'$ or $\overline{x}$.

They are defined by the *axioms* of Table 2.1. Note that $1 + 1 = 1$: the "sum" is a logical OR, not an arithmetic sum.

**Table 2.1.** Axioms and single-variable theorems of Boolean algebra. Each line on the left is the *dual* of the line on the right.

| **Axioms** | | **Single-variable theorems** | |
|---|---|---|---|
| $0 \cdot 0 = 0$ | $1 + 1 = 1$ | $x \cdot 0 = 0$ | $x + 1 = 1$ |
| $1 \cdot 1 = 1$ | $0 + 0 = 0$ | $x \cdot 1 = x$ | $x + 0 = x$ |
| $0 \cdot 1 = 1 \cdot 0 = 0$ | $1 + 0 = 0 + 1 = 1$ | $x \cdot x = x$ | $x + x = x$ |
| $x = 0 \Rightarrow x' = 1$ | $x = 1 \Rightarrow x' = 0$ | $x \cdot x' = 0$ | $x + x' = 1$ |
| | | $(x')' = x$ | (involution) |

### 2.2.2 Properties of two and three variables

The identities of Table 2.2 are used continually to manipulate and simplify expressions. As in Table 2.1, they come in dual pairs. Operator precedence follows ordinary algebra: NOT first, then AND, then OR, so $x + yz'$ means $x + (y \cdot (z'))$.

**Table 2.2.** Properties of Boolean algebra.

| Name | AND form | OR form |
|---|---|---|
| Commutative | $xy = yx$ | $x + y = y + x$ |
| Associative | $x(yz) = (xy)z$ | $x + (y + z) = (x + y) + z$ |
| Distributive | $x(y + z) = xy + xz$ | $x + yz = (x + y)(x + z)$ |
| Absorption | $x(x + y) = x$ | $x + xy = x$ |
| Combining (adjacency) | $(x + y)(x + y') = x$ | $xy + xy' = x$ |
| De Morgan | $(x + y)' = x'y'$ | $(xy)' = x' + y'$ |
| Consensus | $(x + y)(y + z)(x' + z) = (x + y)(x' + z)$ | $xy + yz + x'z = xy + x'z$ |

### 2.2.3 Duality

**Definition 2.1: Dual of an expression**

The *dual* of a Boolean expression is obtained by interchanging AND with OR and 0 with 1, leaving all variables (and complements) unchanged.

*Principle of duality:* if an equation is true, its dual is also true. For example, the dual of $x + 0 = x$ is $x \cdot 1 = x$. This halves the work of learning the theorems, because every identity of Tables 2.1 and 2.2 automatically yields a second one.

### 2.2.4 De Morgan's theorem

*De Morgan's theorem* gives the complement of a product or a sum:

$$(xy)' = x' + y', \qquad (x + y)' = x'y'. \tag{2.1}$$

In words: the complement of an AND is the OR of the complements, and vice versa. The theorem extends to any number of variables, $(x_1 x_2 \cdots x_n)' = x_1' + x_2' + \cdots + x_n'$. It is the key to converting between SOP and POS forms and to implementing circuits with NAND or NOR gates (Chapter 3).

**Proof by perfect induction.** Because each variable takes only two values, an identity in $n$ variables can be proved by checking all $2^n$ combinations. For $(xy)' = x' + y'$:

| $x$ | $y$ | $xy$ | $(xy)'$ | $x'$ | $y'$ | $x'+y'$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | **1** | 1 | 1 | **1** |
| 0 | 1 | 0 | **1** | 1 | 0 | **1** |
| 1 | 0 | 0 | **1** | 0 | 1 | **1** |
| 1 | 1 | 1 | **0** | 0 | 0 | **0** |

The bold columns agree in every row, so the two sides are equal. The same result can be seen in the Venn diagrams of Figure 2.2, where the shaded areas representing the two sides of the identities coincide.

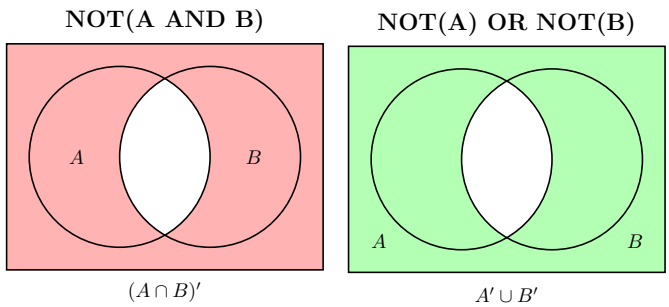



**Figure 2.2.** Venn-diagram illustration of De Morgan's laws: the region shaded for the left-hand side equals that of the right-hand side.

## 2.3 From Switches to Logic Functions

The simplest binary element is a switch, and combinations of switches realise the basic functions.

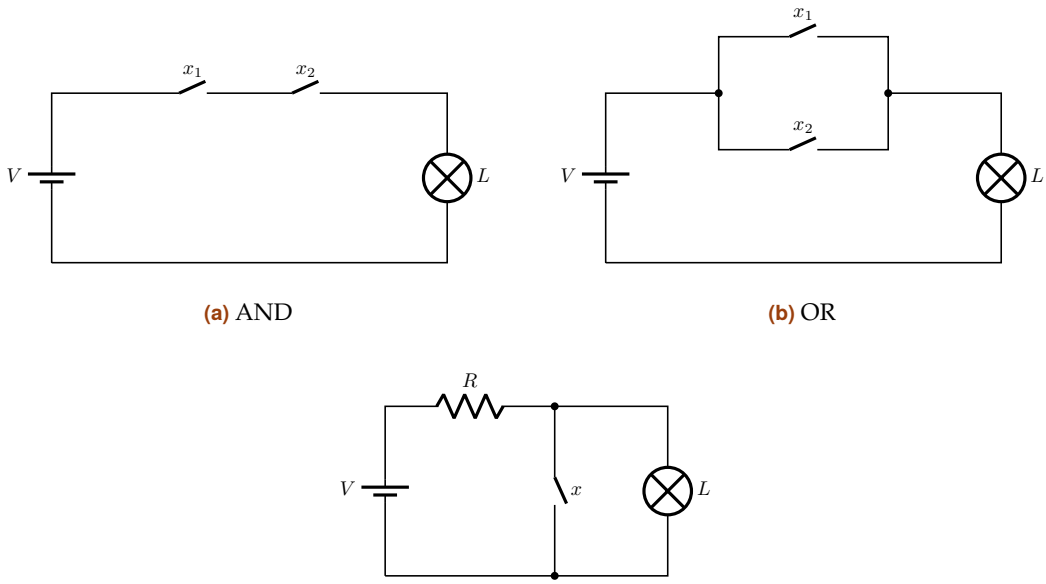
**AND: switches in series.** The lamp $L$ in Figure 2.3a lights only if $x_1$ *and* $x_2$ are closed: $L(x_1, x_2) = x_1 \cdot x_2$.

**OR: switches in parallel.** In Figure 2.3b the lamp lights if $x_1$ *or* $x_2$ (or both) is closed: $L(x_1, x_2) = x_1 + x_2$.

**NOT: a switch that diverts current.** Suppose the lamp should be on when the switch is *open*. In Figure 2.3c current flows through the resistor $R$ and the lamp when switch $S$ is open ($x = 0$); closing $S$ ($x = 1$) short-circuits the lamp to ground. Hence $L = x'$.

### 2.3.1 Truth tables

A *truth table* lists the value of a function for every combination of its inputs; a function of $n$ variables has $2^n$ rows. The truth tables of the basic functions are in Table 2.3. A




**(a)** AND **(b)** OR

**(c)** NOT

**Figure 2.3.** Switching circuits for the three basic functions.

truth table is the most complete and unambiguous specification of a combinational function, and it is the usual starting point of a design.

**Table 2.3.** Truth tables of AND, OR and NOT.

| $x_1$ | $x_2$ | $x_1x_2$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

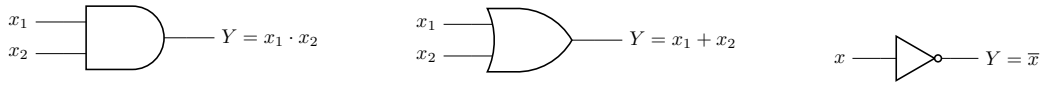
| $x_1$ | $x_2$ | $x_1 + x_2$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| $x$ | $x'$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

## 2.4 Logic Gates and Networks

Each basic operation is implemented by an electronic circuit called a *logic gate*, drawn with the standard symbols of Figure 2.4. A gate has one or more inputs and one output. Gates are connected into a *logic network* (or logic circuit) that implements a larger function. For example, Figure 2.5 shows the function $f = (x_1 + x_2) \cdot x_3$ realised by an OR gate feeding an AND gate.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity logic_function is
5   port (x1, x2, x3 : in  std_logic;
```




**(a)** AND **(b)** OR **(c)** NOT (inverter)

**Figure 2.4.** Graphical symbols of the basic gates.

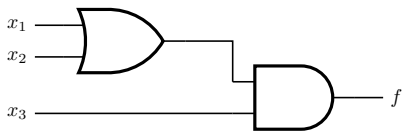



**Figure 2.5.** A logic network for $f = (x_1 + x_2) \cdot x_3$: an OR gate feeding an AND gate.

```
6         f          : out std_logic);
7 end logic_function;
8
9 architecture behavioral of logic_function is
10 begin
11   f <= (x1 or x2) and x3;
12 end behavioral;
```

**Listing 2.1.** VHDL description of $f = (x_1 + x_2) \cdot x_3$.

### 2.4.1 A first look at VHDL

A hardware description language lets us write a circuit as text. A VHDL design unit has two parts, illustrated by Listing 2.1.

- The `entity` declares the *interface*: the name of the circuit and its `ports`, each with a direction (`in` or `out`) and a type. We use `std_logic`, a one-bit type from the library `IEEE.std_logic_1164`.
- The `architecture` describes the *behaviour* or structure. The statement `f <= (x1 or x2) and x3;` is a *concurrent signal assignment*: it describes a gate network that evaluates continuously, not a step executed once. Operators `and`, `or`, `not`, `xor`, ... correspond directly to gates.

Appendix A summarises the language elements used in this book.

## 2.5 Logic Synthesis

*Logic synthesis* is the process of producing a logic circuit that implements a desired function, usually given as a truth table or a Boolean expression. The primary goal is correctness; the secondary goal is optimisation: a circuit that is smaller, faster or uses less power. A function can be synthesised by looking at the rows where it is 1 or at the

rows where it is 0, which leads to two standard forms.

### 2.5.1 Sum of products and minterms

A *minterm* of $n$ variables is a product (AND) term in which every variable appears exactly once, either complemented or not. Each minterm is 1 for exactly one row of the truth table, and the row number $i$ (read as a binary number) names the minterm $m_i$: a variable is complemented if its value in that row is 0 and uncomplemented if it is 1.

**Synthesis of a sum of products**

(1) Mark the rows in which $f = 1$. (2) Write the minterm of each marked row. (3) OR the minterms together. The result is the *canonical sum of products* (canonical SOP), abbreviated $f = \sum m(\dots)$.

### 2.5.2 Product of sums and maxterms

Dually, a *maxterm* is a sum (OR) term containing every variable once; it is 0 for exactly one row, and the maxterm $M_i$ is formed with a variable uncomplemented if its value in the row is 0 and complemented if it is 1. The *canonical product of sums* (canonical POS) ANDs the maxterms of all rows in which $f = 0$: $f = \prod M(\dots)$. Note that $m_i' = M_i$.

**Example 2.1: SOP and POS of one function**

The function $f(a, b, c)$ is 1 whenever $c = 1$ (Table 2.4).
**SOP:** the rows with $f = 1$ are 1, 3, 5, 7, hence

$$f = a'b'c + a'bc + ab'c + abc = m_1 + m_3 + m_5 + m_7 = \sum m(1,3,5,7).$$

**POS:** the rows with $f = 0$ are 0, 2, 4, 6, hence

$$f = (a+b+c)(a+b'+c)(a'+b+c)(a'+b'+c) = M_0M_2M_4M_6 = \prod M(0,2,4,6).$$

Both are correct and equal; in this case each simplifies to $f = c$.

### 2.5.3 Minimality and simplification

A canonical expression is seldom the cheapest. The cost of a circuit is roughly the number of gates and gate inputs, so we try to find an equivalent expression with fewer literals.

**Example 2.2: Algebraic simplification**

Simplify the canonical SOP $f(x_1, x_2) = x_1'x_2' + x_1'x_2 + x_1x_2$, whose truth table is 1, 1, 0, 1 for $x_1x_2 = 00, 01, 10, 11$.

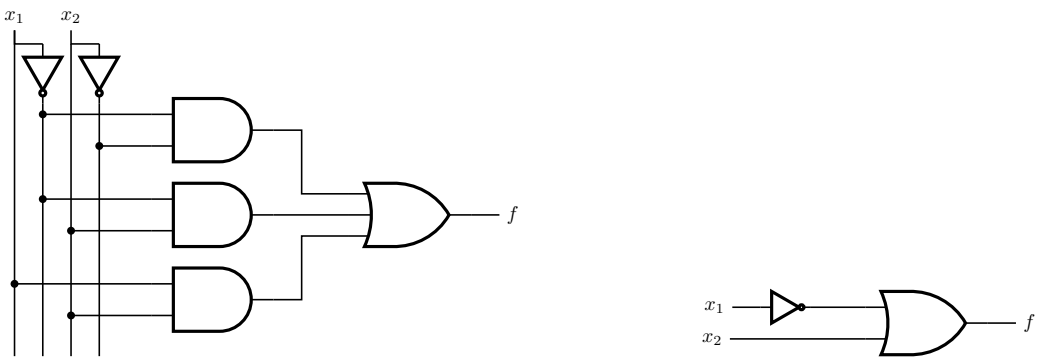
**Table 2.4.** Truth table of Example 2.1 with minterms and maxterms.

| $a$ | $b$ | $c$ | $f$ | Minterm | Maxterm |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | $m_0 = a'b'c'$ | $\mathbf{M_0 = a + b + c}$ |
| 0 | 0 | 1 | 1 | $\mathbf{m_1 = a'b'c}$ | $M_1 = a + b + c'$ |
| 0 | 1 | 0 | 0 | $m_2 = a'bc'$ | $\mathbf{M_2 = a + b' + c}$ |
| 0 | 1 | 1 | 1 | $\mathbf{m_3 = a'bc}$ | $M_3 = a + b' + c'$ |
| 1 | 0 | 0 | 0 | $m_4 = ab'c'$ | $\mathbf{M_4 = a' + b + c}$ |
| 1 | 0 | 1 | 1 | $\mathbf{m_5 = ab'c}$ | $M_5 = a' + b + c'$ |
| 1 | 1 | 0 | 0 | $m_6 = abc'$ | $\mathbf{M_6 = a' + b' + c}$ |
| 1 | 1 | 1 | 1 | $\mathbf{m_7 = abc}$ | $M_7 = a' + b' + c'$ |

**Solution.**

$$
\begin{aligned}
f &= x_1'(x_2' + x_2) + x_1x_2 && \text{(distributive)}\\
&= x_1' \cdot 1 + x_1x_2 = x_1' + x_1x_2 && (x + x' = 1)\\
&= (x_1' + x_1)(x_1' + x_2) && \text{(distributive, OR form)}\\
&= 1 \cdot (x_1' + x_2) = x_1' + x_2.
\end{aligned}
$$

The canonical form needs three 2-input AND gates, two inverters and a 3-input OR; the simplified form needs one inverter and one 2-input OR (Figure 2.6).




**(a)** canonical form **(b)** simplified form $x_1' + x_2$

**Figure 2.6.** Two circuits for the same function of Example 2.2.

Listing 2.2 shows an advantage of VHDL: one entity can have several architectures, so the unsimplified and simplified descriptions can be kept side by side and checked against each other in simulation. A particular architecture is selected by *direct entity instantiation*, `UUT : entity work.Logic_Function(RTL_Simplified) port map (\dots );`, or by a `configuration` declaration.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity Logic_Function is
  port (x1, x2 : in  std_logic;
        f      : out std_logic);
end entity Logic_Function;

-- canonical sum of products:  f = x1'x2' + x1'x2 + x1x2
architecture RTL_Canonical of Logic_Function is
begin
  f <= (not x1 and not x2) or (not x1 and x2) or (x1 and x2);
end architecture RTL_Canonical;

-- simplified:  f = x1' + x2
architecture RTL_Simplified of Logic_Function is
begin
  f <= not x1 or x2;
end architecture RTL_Simplified;
```

**Listing 2.2.** One entity with two architectures: canonical and simplified.

## 2.6 Design Examples

A systematic design proceeds from a word description to a truth table, to a Boolean expression, to a simplified expression and finally to a circuit and its VHDL description. We illustrate the flow with four examples.

### 2.6.1 Three-way light control

**Example 2.3: Light controlled from three doors**

A room has three doors, each with a switch $x_1, x_2, x_3$, that control a single light. Flipping any switch must toggle the light. When all switches are open $(0)$ the light is off. Design the circuit.

**Truth table.** If one switch is closed the light is on; if two are closed it is off (flipping one switch from the one-closed state switched it off); if all three are closed it is on again. So $f = 1$ in rows 1, 2, 4 and 7 (Table 2.5).

**Expression.** The canonical SOP is

$$f = x_1'x_2'x_3 + x_1'x_2x_3' + x_1x_2'x_3' + x_1x_2x_3.$$

This is already the simplest SOP, but it has a compact form: $f$ is 1 when an *odd* number of inputs is 1, which is the *exclusive OR* (XOR) of the three variables, $f = x_1 \oplus x_2 \oplus x_3$, where $x \oplus y = x'y + xy'$.

**Table 2.5.** Truth table for the three-way light.

| $x_1$ | $x_2$ | $x_3$ | $f$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | **1** |
| 0 | 1 | 0 | **1** |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | **1** |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | **1** |

Listing 2.3 gives both descriptions and a testbench. The testbench is explained in Section 2.7.

```
library ieee;
use ieee.std_logic_1164.all;

entity three_way_light is
  port (x1, x2, x3 : in  std_logic;
        f          : out std_logic);
end three_way_light;

-- direct sum of products
architecture arch_sop of three_way_light is
begin
  f <= (not x1 and not x2 and x3) or (not x1 and x2 and not x3) or
       (x1 and not x2 and not x3) or (x1 and x2 and x3);
end arch_sop;

-- XOR form: same function, simpler description
architecture arch_simplified of three_way_light is
begin
  f <= x1 xor x2 xor x3;
end arch_simplified;
```

**Listing 2.3.** Three-way light: two architectures.

### 2.6.2 Two-to-one multiplexer

**Example 2.4: Multiplexer**

Design a circuit that selects between two data inputs $x_1$ and $x_2$ using a control input $s$: $f = x_1$ if $s = 0$ and $f = x_2$ if $s = 1$.
**Truth table.** Rows are ordered $s\,x_1x_2$ (Table 2.6). $f = 1$ in rows 2, 3, 5 and 7.

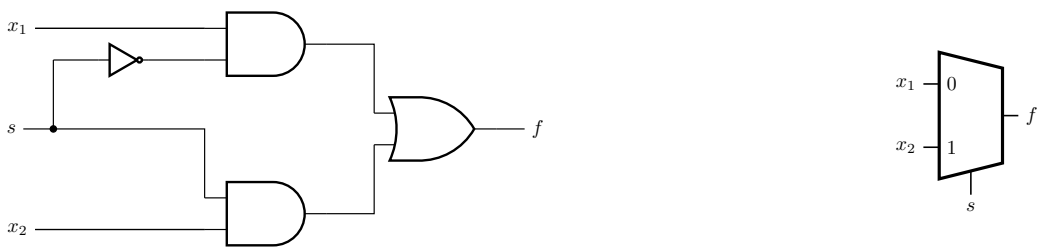
**Expression and simplification.**

$$\begin{aligned} f &= s'x_1x_2' + s'x_1x_2 + sx_1'x_2 + sx_1x_2 \\ &= s'x_1(x_2' + x_2) + sx_2(x_1' + x_1) = s'x_1 + sx_2. \end{aligned}$$

The circuit is two AND gates, one inverter and one OR gate (Figure 2.7).

**Table 2.6.** Truth table of a 2-to-1 multiplexer.

| $s$ | $x_1$ | $x_2$ | $f$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | **1** |
| 0 | 1 | 1 | **1** |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | **1** |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | **1** |




**(a)** circuit for $f = s'x_1 + sx_2$

**(b)** graphical symbol

**Figure 2.7.** A 2-to-1 multiplexer. The multiplexer is one of the most useful blocks in digital design and is studied further in Chapter 5.

### 2.6.3 Car safety alarm

**Example 2.5: Car alarm**

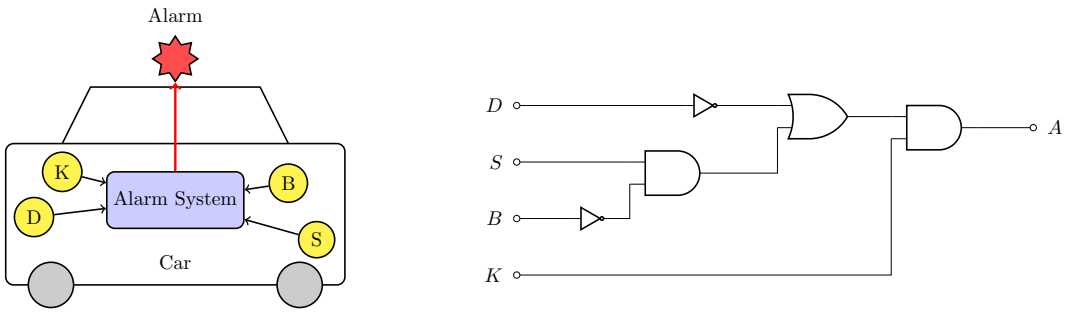
A car alarm $A$ must sound if (1) the key is in the ignition ($K = 1$) and the door is not closed ($D = 0$), or (2) the door is closed ($D = 1$), the key is in ($K = 1$), the driver is in the seat ($S = 1$) and the seat belt is not fastened ($B = 0$). Derive a minimal expression.

**Solution.** Translating the conditions, $A = KD' + DKSB'$. Factor out $K$: $A = K(D' + DSB')$. Using the identity $X + X'Y = X + Y$ (proved by distributing:

$X + X'Y = (X + X')(X + Y) = X + Y$) with $X = D'$,

$$A = K\,(D' + SB').$$

This needs one inverter for $D$, one for $B$, one AND, one OR and one more AND (Figure 2.8), far fewer than the original expression, and the VHDL is a single line, Listing 2.4.




**(a)** the system **(b)** circuit for $A = K(D' + SB')$

**Figure 2.8.** The car safety alarm of Example 2.5.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity car_alarm_logic is
  port (K, D, S, B : in  std_logic;
        A          : out std_logic);
end entity car_alarm_logic;

architecture rtl of car_alarm_logic is
begin
  A <= K and (not D or (S and not B));      -- A = K (D' + S B')
end architecture rtl;
```

**Listing 2.4.** Car alarm logic.

### 2.6.4 Half adder

**Example 2.6: Half adder**

Design a circuit that adds two bits $x$ and $y$ and produces a *sum* bit $s$ and a *carry* bit $c$.

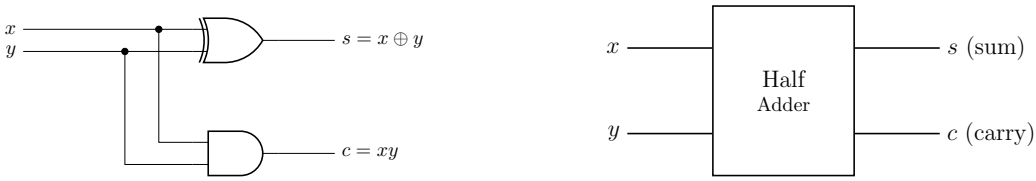
**Truth table.** The arithmetic 0+0 = 00, 0+1 = 01, 1+0 = 01, 1+1 = 10 gives Table 2.7. The sum is 1 only when the inputs differ, so $s = x'y + xy' = x \oplus y$; the carry is 1 only when both inputs are 1, so $c = xy$.

The circuit is an XOR gate and an AND gate (Figure 2.9). The half adder is the starting

**Table 2.7.** Half adder.

| $x$ | $y$ | sum $s$ | carry $c$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

point of the arithmetic circuits of Chapter 4.




**(a)** gate-level circuit

**(b)** block diagram

**Figure 2.9.** Half adder.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
  port (x, y : in  std_logic;     -- the two bits to add
        s, c : out std_logic);    -- sum and carry-out
end entity half_adder;

architecture dataflow of half_adder is
begin
  s <= x xor y;                   -- sum
  c <= x and y;                   -- carry
end architecture dataflow;
```

**Listing 2.5.** Half adder in VHDL (dataflow style).

## 2.7 The VHDL Testbench

A *testbench* is a VHDL entity used to simulate and check another entity, the *unit under test* (UUT). A testbench has no ports because it is closed: it generates its own stimulus and observes the UUT's response (Figure 2.10). Writing a testbench takes four steps.

1. **Instantiate** the UUT as a `component` (or by direct entity instantiation) and connect its ports to local signals.
2. **Generate stimulus.** A `process` assigns values to the input signals and uses `wait for` statements to advance simulated time.

3. **Observe** the outputs in the simulator's waveform window.
4. **Verify.** Compare the waveform with the truth table, either by inspection or automatically with `assert` statements that report an error if an output is wrong.

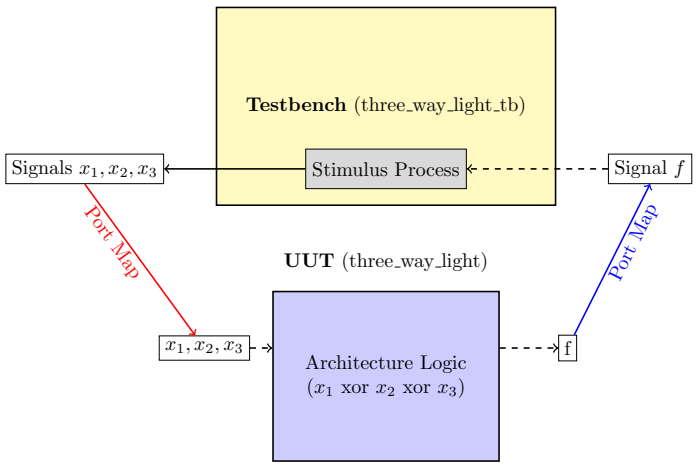



**Figure 2.10.** The relationship between a testbench and the unit under test (UUT).

```
library ieee;
use ieee.std_logic_1164.all;

entity three_way_light_tb is end entity;          -- no ports

architecture behavior of three_way_light_tb is
  signal x1, x2, x3 : std_logic := '0';            -- inputs to the UUT
  signal f          : std_logic;                   -- output of the UUT
begin
  uut : entity work.three_way_light(arch_simplified)
        port map (x1 => x1, x2 => x2, x3 => x3, f => f);

  stim_proc : process
  begin
    wait for 100 ns;                               -- let the simulator settle
    x1<='0'; x2<='0'; x3<='0'; wait for 10 ns;     -- expect f = 0
    x1<='0'; x2<='0'; x3<='1'; wait for 10 ns;     -- expect f = 1
    x1<='0'; x2<='1'; x3<='0'; wait for 10 ns;     -- expect f = 1
    x1<='0'; x2<='1'; x3<='1'; wait for 10 ns;     -- expect f = 0
    x1<='1'; x2<='0'; x3<='0'; wait for 10 ns;     -- expect f = 1
    x1<='1'; x2<='0'; x3<='1'; wait for 10 ns;     -- expect f = 0
    x1<='1'; x2<='1'; x3<='0'; wait for 10 ns;     -- expect f = 0
    x1<='1'; x2<='1'; x3<='1'; wait for 10 ns;     -- expect f = 1
    wait;                                          -- stop the process
  end process;
end architecture behavior;
```

**Listing 2.6.** A testbench for the three-way light that applies all eight input combinations.

Figure 2.11 shows the waveform obtained by simulating Listing 2.6 in the Intel® Questa simulator. Each of the eight 10 ns intervals corresponds to one row of Table 2.5; the output $f$ is high in rows 1, 2, 4 and 7, as predicted.




**Figure 2.11.** Simulation of the three-way light in Intel Questa (author's screenshot).

**Common Mistake**

> Writing the expected value of $f$ only by looking at the simulation output proves nothing: the simulator will always give you the answer your design computes. Work out the expected outputs from the truth table *first*, then compare.

**Chapter Summary**

- **Boolean algebra** has two values and three operations (AND, OR, NOT). Its theorems appear in dual pairs; **De Morgan's theorem** gives the complement of a sum or a product.
- An identity can be proved by **perfect induction**: check every row of the truth table.
- A **truth table** fully specifies a combinational function. **Gates** implement the basic operations and are connected into **logic networks**.
- A function with $f = 1$ in rows $i_1, i_2, \ldots$ has the canonical **SOP** $\sum m(i_1, i_2, \ldots)$; with $f = 0$ in rows $j_1, \ldots$ it has the canonical **POS** $\prod M(j_1, \ldots)$.
- **Simplification** reduces the number of gates and inputs; the same function may have many equivalent expressions.
- The design flow is: *word problem* → *truth table* → *expression* → *simplification* → *circuit* → *VHDL* → *testbench*.
- A VHDL **entity** defines the interface and an **architecture** the behaviour; a **testbench** stimulates the unit under test and lets us check it against the truth table.

## Exercises

**2.1 Majority function.** A majority function is true when more than half of its inputs are true. (a) Write the truth table of a 3-input majority function $f(x, y, z)$. (b) Write its canonical SOP. (c) Simplify it algebraically. (d) Draw the circuit using AND, OR and NOT gates. (e) Write VHDL and a testbench that applies all eight input combinations, and show the waveform.

**2.2 Home alarm.** The alarm $A$ sounds if the system is armed ($S = 1$) *and* either the front door ($D = 1$) or the window ($W = 1$) is open. It never sounds if the system is not armed. Give the truth table, the Boolean expression, the circuit, and a simulation of the case $S = 1, D = 1$.

**2.3 Line-following robot.** A robot has three sensors $L, C, R$ (value 0 over the black line, 1 over the white surface) and two motors $M_L, M_R$. It must *go forward* ($M_L = M_R = 1$) when centred ($L = 1, C = 0, R = 1$); *turn right* ($M_L = 1, M_R = 0$) when it drifts left ($L = 1, C = 1, R = 0$); *turn left* ($M_L = 0, M_R = 1$) when it drifts right ($L = 0, C = 1, R = 1$); and *stop* ($M_L = M_R = 0$) in every other case. (a) Write the truth table. (b) Write SOP expressions for $M_L$ and $M_R$ and simplify them. (c) Draw the circuits. (d) Simulate all four behaviours.

**2.4 Seven-segment decoder.** A seven-segment display has segments $a, b, c, d, e, f, g$ (Figure 2.12). Design a decoder with a 4-bit BCD input $b_3 b_2 b_1 b_0$ that lights the segments to show the digits 0–9. Assume a *common anode* display, in which a segment lights when its control signal is *low*. (a) Give the truth table, treating the input values 10–15 as don't-cares. (b) Find the simplified SOP for segment

$a$ (you may postpone the other segments until you have learnt Karnaugh maps in Chapter 3). (c) Draw the circuit for segment $a$.

**2.5** Use truth tables to prove: (a) $x + xy = x$; (b) $x + yz = (x + y)(x + z)$; (c) $xy + x'z + yz = xy + x'z$.

**2.6** Write the dual of each identity: (a) $x + x'y = x + y$; (b) $(x + y)(x + z) = x + yz$; (c) $xy + x'z + yz = xy + x'z$.

**2.7** Use De Morgan's theorem to find the complement of each function and express it as a sum of products: (a) $F = x + yz'$; (b) $F = x'y + xy'$; (c) $F = (a + b)(c + d)$.

**2.8** Simplify algebraically, stating the identity used in each step: (a) $F = x'y + xy + x'y'$; (b) $F = (x + y)(x + y')(x' + z)$; (c) $F = xy + x'z + yz$.

**2.9** A function is defined by $f = \sum m(0, 3, 5, 6)$ of three variables $x, y, z$. (a) Write the canonical SOP and the canonical POS. (b) Describe in words when $f = 1$. (c) Give the complement $f'$ in $\sum m$ notation.

**2.10** Design a circuit with inputs $x_1, x_2, x_3$ whose output is 1 if and only if *exactly two* of the inputs are 1. Give the canonical SOP and draw the circuit.

**2.11** Design a circuit with two 2-bit inputs $A = A_1A_0$ and $B = B_1B_0$ whose output $E$ is 1 when $A = B$. *Hint:* $A = B$ if and only if each pair of corresponding bits is equal, and two bits are equal when their XOR is 0.

**2.12** How many different Boolean functions exist of (a) two variables; (b) three variables; (c) $n$ variables? *Hint:* each function is defined by a column of the truth table.

**2.13** The three-way light of Example 2.3 is extended to a corridor with four switches $x_1, \ldots, x_4$, each of which toggles the light. (a) Write the function using XOR. (b) How many minterms does the canonical SOP contain?

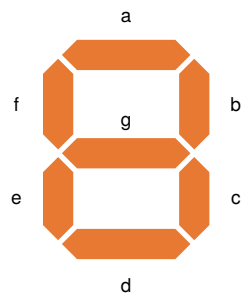



**Figure 2.12.** Segment labels of the seven-segment display used in Exercise 2.4.

## Further Reading

The reader who wants a systematic treatment of Boolean algebra and switching functions will find it in Brown and Vranesic [3] and in Mano and Ciletti [7]; Wakerly [8] gives a particularly careful account of the relation between gates, expressions and timing. Shannon's thesis [6] remains very readable. For VHDL, see Ashenden [9] and the language standard [10].

# 3

# Logic Optimisation with Karnaugh Maps

**Learning Objectives**

After studying this chapter you will be able to:

- draw two-, three- and four-variable Karnaugh maps and plot a function on them;
- find all prime implicants, identify the essential ones, and derive a minimal sum-of-products (SOP) expression;
- exploit don't-care conditions to obtain simpler circuits;
- derive a minimal product-of-sums (POS) expression by grouping the 0s;
- implement SOP and POS expressions with two-level NAND–NAND and NOR–NOR circuits, and explain why NAND and NOR are universal gates;
- use XOR and XNOR gates, recognise odd and even functions, and design parity generators and checkers.

## 3.1 The Need for Optimisation

In the design flow of Figure 3.1, the step after deriving a Boolean expression from a truth table (for example with minterms, Section 2.5) is *optimisation*. A simpler expression gives a circuit with fewer gates, which is cheaper, usually faster and consumes less power. Algebraic manipulation, as in Example 2.2, can do the job, but it follows no fixed rule: it is not obvious which identity to apply next, nor when to stop. The *Karnaugh map* turns simplification into a pattern-recognition task that a person can perform reliably for up to about five variables.

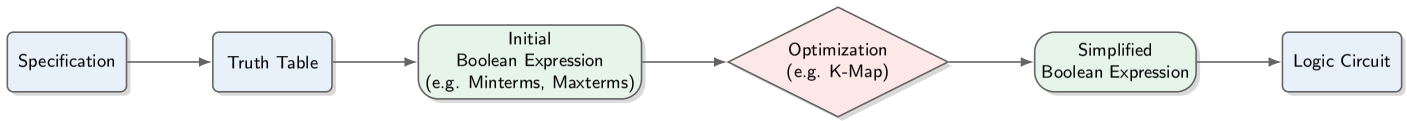



**Figure 3.1.** Design flow of a combinational circuit: specification, truth table, Boolean expression, optimisation and implementation.

## 3.2 The Karnaugh Map

The Karnaugh map (K-map), introduced by Maurice Karnaugh in 1953 [11], is a pictorial form of the truth table. It is a diagram made up of squares, *each square representing one minterm*. It is arranged so that squares that are *adjacent*, horizontally or vertically, represent minterms that differ in exactly one variable. Two such minterms can be combined by the combining identity $xy + xy' = x$, which removes the variable that changes. The map thus displays, at a glance, all the pairs of minterms that can be merged.

| | | $yz$ = 00 | 01 | 11 | 10 |
|---|---|---|---|---|---|
| $x$ | 0 | $x'y'z'$ | $x'y'z$ | $x'yz$ | $x'yz'$ |
| | 1 | $xy'z'$ | $xy'z$ | $xyz$ | $xyz'$ |

**Figure 3.2.** A Karnaugh map is a truth table rearranged so that neighbouring squares differ in one variable.

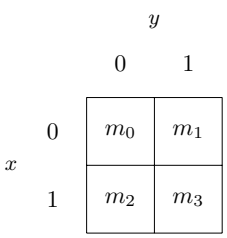
### 3.2.1 The two-variable map

A function of two variables has $2^2 = 4$ minterms, so its map has four squares (Figure 3.3). The rows correspond to the values of $x$ and the columns to the values of $y$. To plot a function, write a 1 in the square of every minterm of the function. For instance $f = xy$ is the single minterm $m_3$, while $f = x + y = m_1 + m_2 + m_3$ has three 1s.

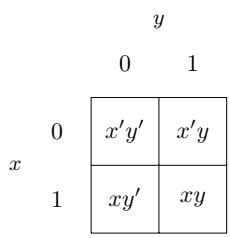
### 3.2.2 The three-variable map

Three variables give $2^3 = 8$ squares, arranged in two rows and four columns (Figure 3.4). The key point is that the columns are ordered in *Gray code*, $00, 01, 11, 10$, not in binary order: in a Gray code only one bit changes between consecutive code words. Because of this, horizontally adjacent squares differ in exactly one variable. The ordering also makes the *leftmost and rightmost columns adjacent*, as if the map were wrapped around a cylinder (column 10 is adjacent to column 00).

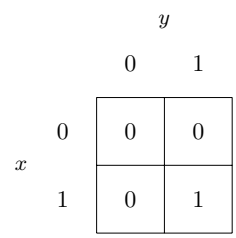
Two adjacent 1s can be merged to remove one variable. For example $m_5$ and $m_7$ are adjacent: $xy'z + xyz = xz(y' + y) = xz$.



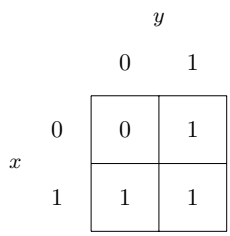

**(a)** minterms




**(b)** variables




**(c)** $f = xy$




**(d)** $f = x + y$

**Figure 3.3.** Two-variable K-maps: the layout of minterms and variables, and two plotted functions.

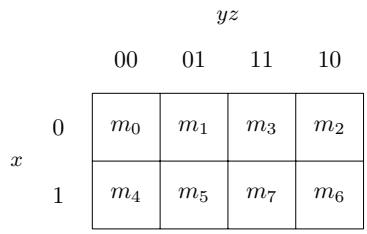



**Figure 3.4.** Structure of the three-variable K-map. Columns follow a Gray code and wrap around.

### 3.2.3 Rules for simplification

**Rules for grouping on a K-map**

The aim is to cover all the 1s of the map with as few and as large groups as possible.

1. Groups must be **rectangles** containing $1, 2, 4, 8, \ldots$ squares (a power of two).
2. Make each group **as large as possible**; a group of four is better than two groups of two.
3. **Every 1 must be covered** by at least one group.
4. Groups **may overlap** if this allows larger groups.
5. **Wrap-around** adjacency is allowed: the left and right edges, and the top and bottom edges, are neighbours.
6. Use the **fewest groups** that cover all the 1s.

Each group gives one product term; the variables that take a constant value over the whole group appear in the term (uncomplemented if the constant is 1, complemented if it is 0) and the variables that change are eliminated. The simplified function is the OR of the product terms. A group of $2^k$ squares eliminates $k$ variables.

**Example 3.1: Two groups of two**

Simplify $F(x, y, z) = \sum m(2, 3, 4, 5)$.
**Solution.** Plot 1s in squares 2, 3, 4 and 5 (Figure 3.5a). Minterms $m_2, m_3$ ($x'yz'$ and $x'yz$) are adjacent and combine to $x'y$; $m_4, m_5$ ($xy'z'$ and $xy'z$) combine to $xy'$. Hence

$$F = x'y + xy' = x \oplus y.$$

**Example 3.2: Groups with wrap-around**

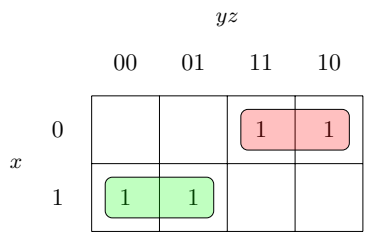
Simplify $F(x, y, z) = \sum m(3, 4, 6, 7)$.
**Solution.** $m_3$ and $m_7$ ($x'yz$, $xyz$) combine to $yz$. $m_4$ and $m_6$ ($xy'z'$, $xyz'$) are adjacent *through the wrap-around* (column 00 to column 10) and combine to $xz'$. Hence $F = yz + xz'$ (Figure 3.5b).

**Example 3.3: A group of four**

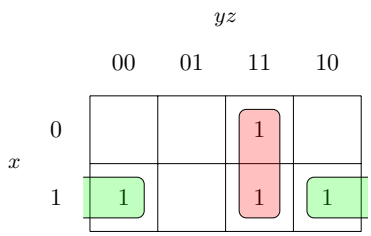
Simplify $F(x, y, z) = \sum m(0, 2, 4, 6)$.
**Solution.** All four 1s form a single group (the two outer columns, joined by wrap-around). Over this group $x$ takes both values and so does $y$, so both are eliminated; $z$ is 0 in all four squares, so $z'$ remains: $F = z'$ (Figure 3.5c). A group of four eliminates two variables.

**Example 3.4: Overlapping groups**

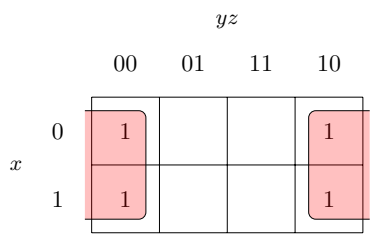
Simplify $F(x,y,z) = \sum m(0,2,4,5,6)$.
**Solution.** The largest group covers $m_0, m_2, m_4, m_6$ and gives $z'$. The 1 in $m_5$ is not yet covered; the largest group containing it is $m_4, m_5$, which gives $xy'$. Note that $m_4$ belongs to both groups, which is allowed and gives the simplest result: $F = z' + xy'$ (Figure 3.5d).



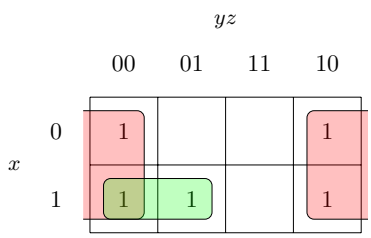

(a) Example 3.1




(b) Example 3.2




(c) Example 3.3




(d) Example 3.4

**Figure 3.5.** K-maps for Examples 3.1 to 3.4.

## 3.3 Prime Implicants

The grouping procedure can be stated more precisely with three definitions.

**Definition 3.1: Implicants**

An *implicant* of a function is a product term that covers one or more minterms of the function (every group of 1s on the map is an implicant). A *prime implicant* (PI) is an implicant that cannot be combined with another to form a larger implicant, that is, a group of 1s that is as large as it can be. An *essential prime implicant* (EPI) is a prime implicant that covers at least one minterm covered by no other prime implicant.

**Minimisation procedure**

1. Find all prime implicants.
2. Include all essential prime implicants.
3. Choose the smallest set of the remaining prime implicants that covers all the 1s not yet covered.

## 3.4 The Four-Variable Map

A function of four variables has 16 minterms and a $4 \times 4$ map (Figure 3.6). Both the rows (variables $w, x$) and the columns (variables $y, z$) are in Gray-code order $00, 01, 11, 10$. Wrap-around applies in both directions: the top row is adjacent to the bottom row, and the left column to the right column; in effect the map is a torus. The four corners are therefore adjacent to one another.

| $wx$ \ $yz$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | $m_0$ | $m_1$ | $m_3$ | $m_2$ |
| 01 | $m_4$ | $m_5$ | $m_7$ | $m_6$ |
| 11 | $m_{12}$ | $m_{13}$ | $m_{15}$ | $m_{14}$ |
| 10 | $m_8$ | $m_9$ | $m_{11}$ | $m_{10}$ |

**Figure 3.6.** Structure of the four-variable K-map.

**Example 3.5: Four-variable simplification**

Simplify $F(w, x, y, z) = \sum m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$.

**Solution.** The map is Figure 3.7a. Look for the largest groups first.

- The eight squares in the two columns where $y = 0$ (minterms 0, 1, 4, 5, 8, 9, 12, 13) form a group of eight: the term is $y'$.
- $m_0, m_2, m_4, m_6$ form a group of four (using the wrap-around between columns 00 and 10): $w'z'$.
- $m_4, m_6, m_{12}, m_{14}$ form a group of four (again by wrap-around): $xz'$.

All eleven 1s are covered, and each group is needed ($y'$ is the only group containing $m_1, m_9$; $w'z'$ is the only one containing $m_2$; $xz'$ is the only one containing $m_{14}$).

Hence

$$F = y' + w'z' + xz'.$$

**Example 3.6: More than one minimal expression**

Simplify $F(A,B,C,D) = \sum m(0,1,2,5,7,8,9,10,13,15)$.

**Solution.** The prime implicants are $B'C'$ $(m_0, m_1, m_8, m_9)$, $B'D'$ $(m_0, m_2, m_8, m_{10})$, $BD$ $(m_5, m_7, m_{13}, m_{15})$ and $C'D$ $(m_1, m_5, m_9, m_{13})$. $B'D'$ is essential because it alone covers $m_2$ and $m_{10}$, and $BD$ is essential because it alone covers $m_7$ and $m_{15}$. The essential prime implicants cover all the 1s except $m_1$ and $m_9$, and either of the remaining prime implicants covers both. There are therefore *two* minimal expressions (Figure 3.7):

$$F = BD + B'D' + B'C' \qquad \text{or} \qquad F = BD + B'D' + C'D.$$

They have the same cost; the choice may depend on which signals are already available.

## 3.5 Don't-Care Conditions

In many designs the output for some input combinations is not specified. These are *don't-care conditions*. They arise because (a) the input combination can never occur (for example, the six invalid codes 1010 to 1111 of a 4-bit BCD input), or (b) the output for that input does not matter. On the map a don't-care is marked with an **X** (sometimes "d"). The rules are:

- treat an X as a 1 if this lets you form a larger group;
- otherwise treat it as a 0 (ignore it);
- you need *not* cover all the X squares.

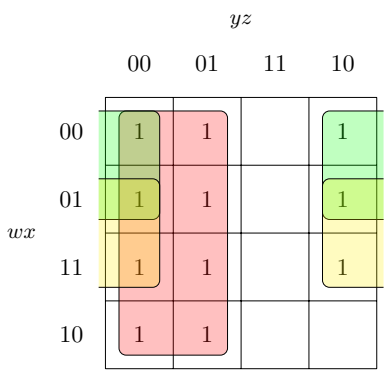
**Example 3.7: Using don't-cares**

Simplify $F(w,x,y,z) = \sum m(1,3,7,11,15) + d(0,2,5)$.

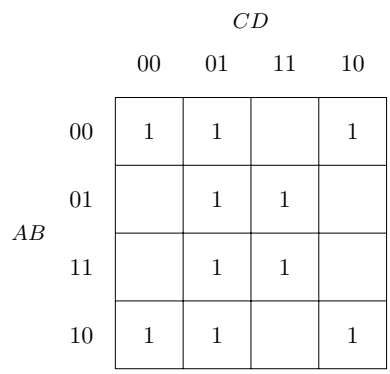
**Solution.** See Figure 3.8. The group $m_3, m_7, m_{11}, m_{15}$ gives $yz$. The remaining 1, $m_1$, can be grouped with the don't-cares in two ways: $m_1, m_3, m_5, m_7$ gives $w'z$, and $m_0, m_1, m_2, m_3$ gives $w'x'$. Either is acceptable:

$$F = yz + w'z \qquad \text{or} \qquad F = yz + w'x'.$$

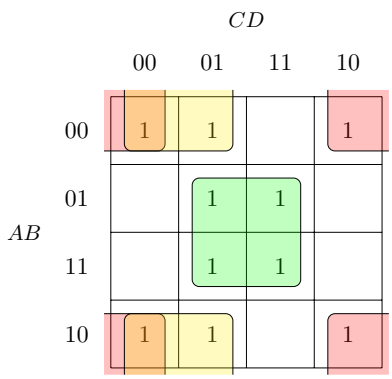
Without don't-cares, $m_1$ could be combined only with $m_3$, giving a less simple term $w'x'z$.



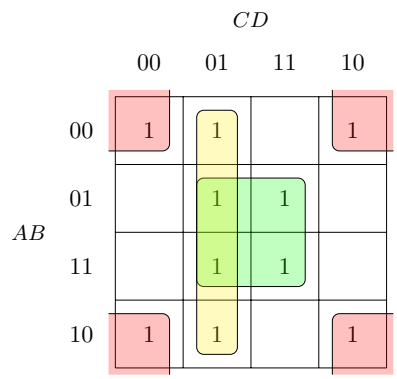

**(a)** Example 3.5




**(b)** plotted $F$




**(c)** solution 1




**(d)** solution 2

**Figure 3.7.** (a) Example 3.5. (b–d) Example 3.6: the map and its two minimal covers.

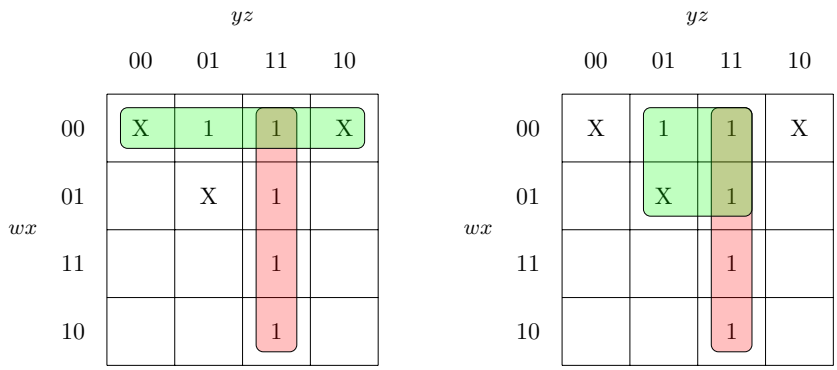
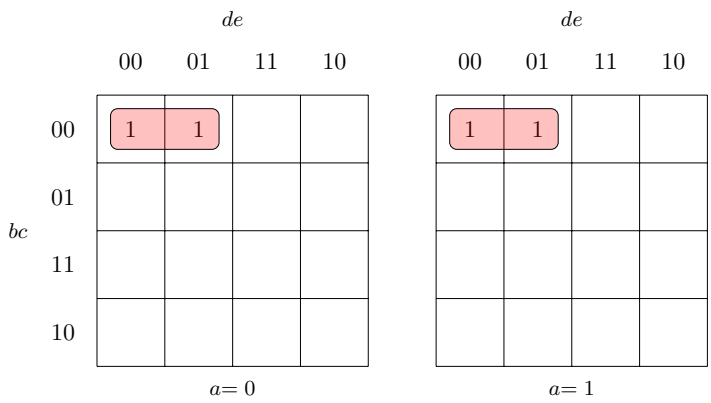



**Figure 3.8.** K-map with don't-care conditions for Example 3.7.

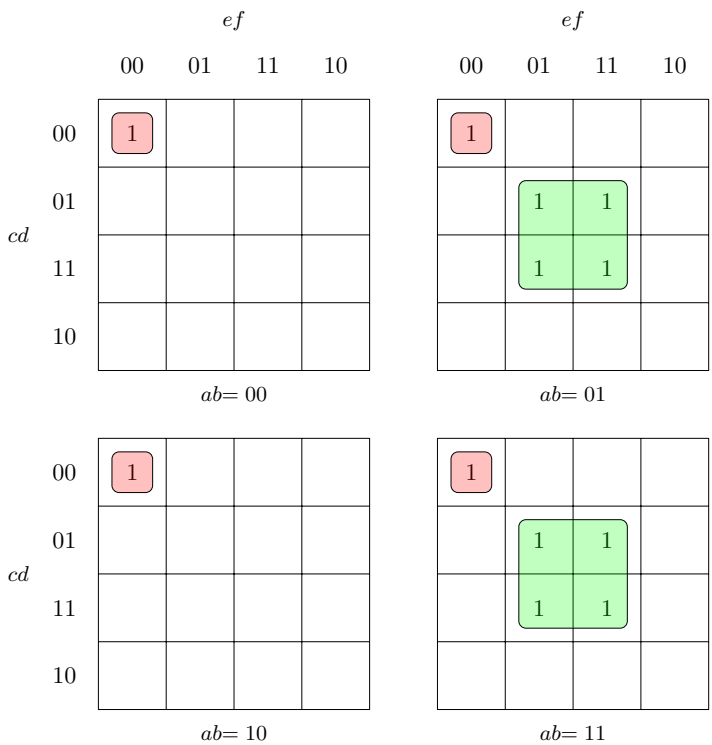
## 3.6 Five- and Six-Variable Maps

For more than four variables the method still works but becomes harder to apply. A **five-variable** map consists of two four-variable maps, one for $a = 0$ and one for $a = 1$, imagined to be stacked (Figure 3.9a). Squares are adjacent if they are adjacent within one four-variable map *or* are in the same position in the two maps; thus $m_0$ and $m_{16}$ are adjacent. A **six-variable** map has four such maps, arranged in a $2 \times 2$ grid according to $ab$ (Figure 3.9b). Beyond six variables, a K-map is no longer practical, and tabular methods such as the *Quine–McCluskey algorithm* [12], which can be programmed, are used; the logic-synthesis tools used in practice are based on more efficient heuristics of the same kind.




**(a)** five variables

**Figure 3.9.** K-maps for five and six variables.




**(b)** six variables

**Figure 3.9.** K-maps for five and six variables.

## 3.7 Product-of-Sums Simplification

A K-map also yields a minimal POS expression: *group the 0s* instead of the 1s. There are two equivalent methods.

**Method 1: simplify $F'$ and complement.** Group the 0s of $F$ (the 1s of $F'$) to obtain a minimal SOP for $F'$, then apply De Morgan's theorem: if $F' = AB + C'D$, then $F = (AB + C'D)' = (A' + B')(C + D')$.

**Method 2: direct.** For each group of 0s write a *sum* term: a variable that is 0 over the group appears uncomplemented, a variable that is 1 appears complemented, and a variable that changes is eliminated. The function is the product of the sum terms.

**Example 3.8: POS from a K-map**

Find the minimal POS of $F(A,B,C,D) = \sum m(0,1,2,5,8,9,10)$.
**Solution.** The 0s are at $m_3, m_4, m_6, m_7, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}$ (Figure 3.10). Group them as follows.

- $m_{12}, m_{13}, m_{14}, m_{15}$: $A = 1, B = 1$, giving the sum $(A' + B')$.
- $m_3, m_7, m_{11}, m_{15}$: $C = 1, D = 1$, giving $(C' + D')$.
- $m_4, m_6, m_{12}, m_{14}$: $B = 1, D = 0$, giving $(B' + D)$.

Hence $F = (A' + B')(C' + D')(B' + D)$.

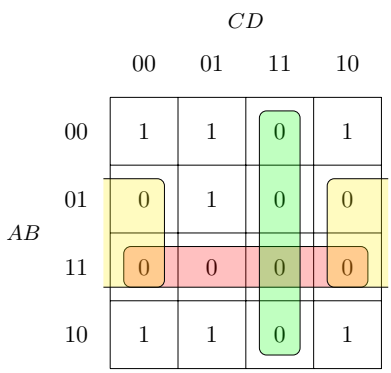



**Figure 3.10.** Grouping the 0s of Example 3.8.

## 3.8 NAND and NOR as Universal Gates

The NAND gate $((xy)')$ and the NOR gate $((x + y)')$ are called *universal gates*: every other gate can be built from NANDs alone, or from NORs alone, so any digital circuit

can be built from just one type of gate. This is useful because in many technologies NAND and NOR gates are smaller and faster than AND and OR gates. Figure 3.11 shows the constructions; each follows from De Morgan's theorem. A NAND with its inputs tied together is a NOT: $(xx)' = x'$. An AND is a NAND followed by a NAND used as an inverter. An OR is built from inverted inputs: $x + y = (x'y')'$. The NOR constructions are the duals.

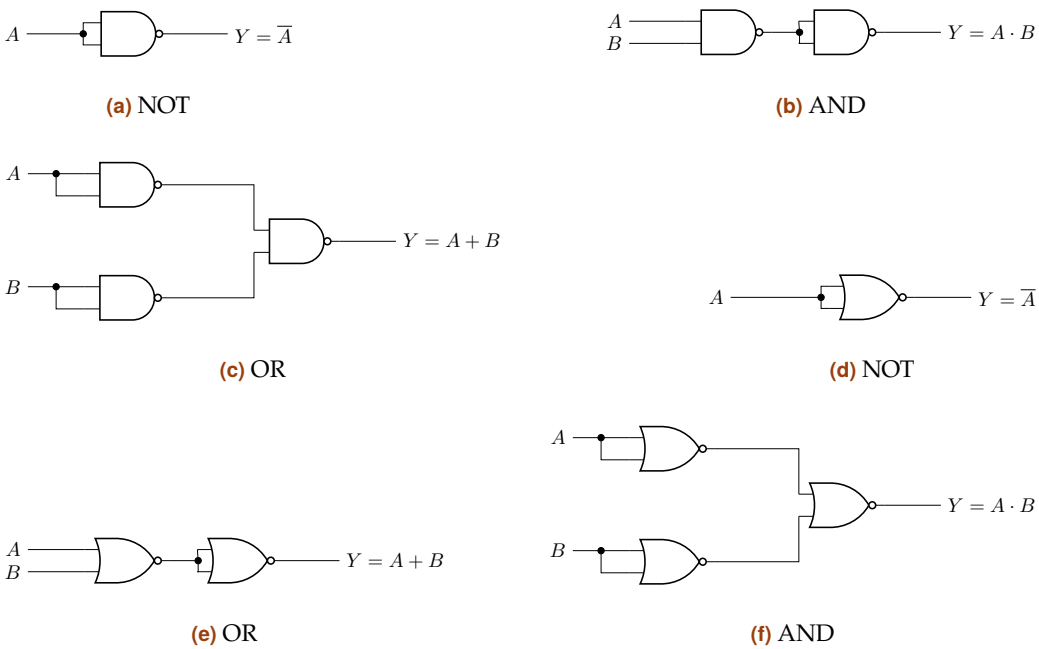



**Figure 3.11.** Universal gates: NOT, AND and OR built from NAND gates only (top row) or from NOR gates only (bottom row).

### 3.8.1 Two-level NAND–NAND implementation of an SOP

A minimal SOP expression is normally drawn as a two-level AND–OR circuit. Applying De Morgan's theorem to the double complement converts it directly to *two levels of NAND gates*:

$$F = AB + CD + E = \big((AB + CD + E)'\big)' = \big((AB)' \cdot (CD)' \cdot E'\big)'.$$

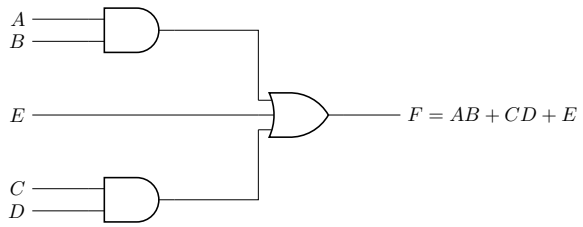
The first level of NAND gates forms the complemented product terms $(AB)'$, $(CD)'$ and $E'$ (a single literal needs only an inverter); the second-level NAND gate then ORs the products. The circuits are shown in Figure 3.12a and b. This is a very common implementation of the SOP obtained from a K-map.

### 3.8.2 Two-level NOR–NOR implementation of a POS

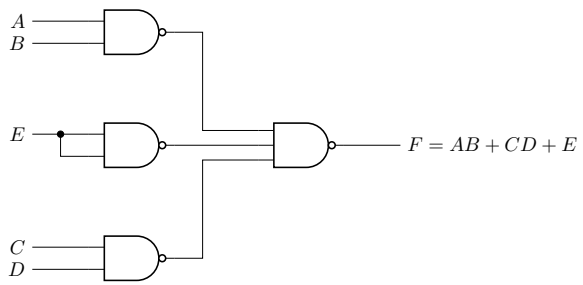
Dually, a POS expression, drawn as OR–AND, converts to *two levels of NOR gates*:

$$F = (A + B)(C + D)E = (((A + B)(C + D)E)')' = ((A + B)' + (C + D)' + E')'.$$

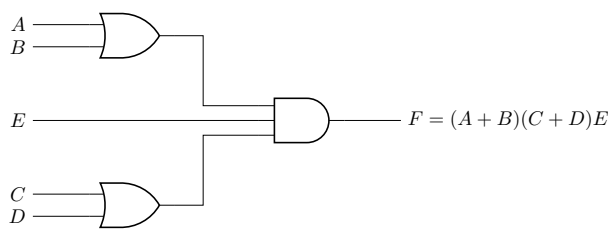
This is the standard implementation when the expression was found by grouping 0s (Figure 3.12c and d).




**(a)** AND–OR




**(b)** NAND–NAND

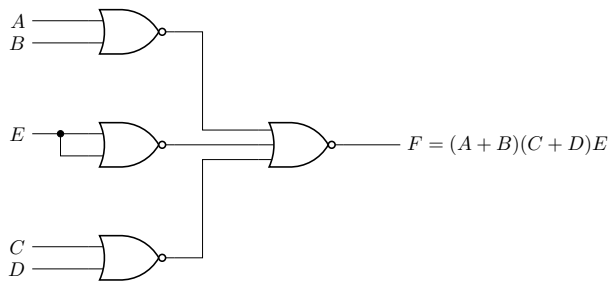



**(c)** OR–AND

**Figure 3.12.** Two-level circuits: an SOP as AND–OR and as NAND–NAND (a, b); a POS as OR–AND and as NOR–NOR (c, d).

## 3.9 Exclusive-OR and Exclusive-NOR

The *exclusive-OR* (XOR) function is 1 when its inputs *differ*: $x \oplus y = x'y + xy'$. The *exclusive-NOR* (XNOR, or *equivalence*) is 1 when the inputs are *equal*: $(x \oplus y)' = xy + x'y'$.



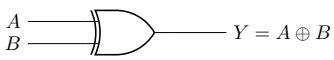

**(d)** NOR–NOR

**Figure 3.12.** Two-level circuits: an SOP as AND–OR and as NAND–NAND (a, b); a POS as OR–AND and as NOR–NOR (c, d).

Both are fundamental in arithmetic and comparison circuits. Useful identities are

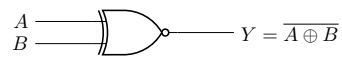
$$x \oplus 0 = x, \qquad x \oplus 1 = x', \qquad x \oplus x = 0, \qquad x \oplus x' = 1,$$

and XOR is commutative and associative, $A \oplus B = B \oplus A$ and $(A \oplus B) \oplus C = A \oplus (B \oplus C)$, so that XOR of more than two variables is unambiguous. The symbols are shown in Figure 3.13.




| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**(a)** XOR




| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

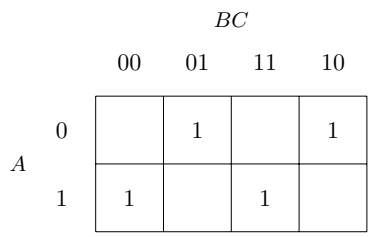
**(b)** XNOR

**Figure 3.13.** Symbols and truth tables of XOR and XNOR.

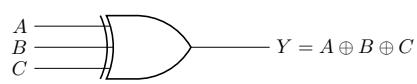
### 3.9.1 Odd and even functions

The XOR of several variables is 1 when an *odd* number of its inputs are 1; it is called the *odd function*. Its complement, the XNOR of several variables, is 1 when an *even* number of the inputs are 1, and is called the *even function*. On a K-map these functions have a checkerboard pattern in which no two 1s are adjacent, so no simplification is possible (Figure 3.14). For three variables

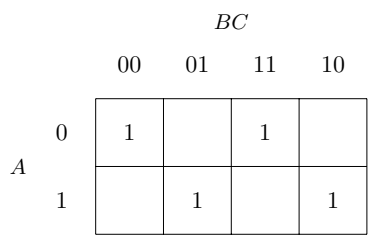
$$A \oplus B \oplus C = \sum m(1,2,4,7), \qquad (A \oplus B \oplus C)' = \sum m(0,3,5,6).$$



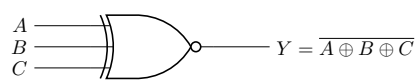

**(a)** K-map of the odd function $A \oplus B \oplus C$




**(b)** logic diagram of the odd function




**(c)** K-map of the even function $(A \oplus B \oplus C)'$




**(d)** logic diagram of the even function

**Figure 3.14.** Odd and even functions of three variables: K-maps and logic diagrams.

## 3.10 Application: Parity Generation and Checking

XOR gates are ideal for detecting errors in data transmission. A *parity bit* is an extra bit appended to a message so that the total number of 1s is always even (*even parity*) or always odd (*odd parity*). Figure 3.15 shows the system: a parity generator at the transmitter, the channel, and a parity checker at the receiver.

**Even-parity generator.** For a 3-bit message $x, y, z$ the parity bit $P$ must make the number of 1s in $(x, y, z, P)$ even. This is true if and only if $P$ is the odd function of the message: $P = x \oplus y \oplus z$.

**Even-parity checker.** The receiver computes $C = x \oplus y \oplus z \oplus P$ from the four received bits; $C = 0$ means that the number of 1s is even and no error is detected, whereas $C = 1$ signals an error. A single-bit error always changes the parity and is detected; a double-bit error is not.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity parity_generator is
  port (x, y, z : in  std_logic;
        p_even  : out std_logic);          -- parity bit for the even-parity scheme
end parity_generator;

architecture behavioral of parity_generator is
begin
  p_even <= x xor y xor z;                 -- 1 if the message has an odd number of 1s
end behavioral;

library ieee;
use ieee.std_logic_1164.all;

entity parity_checker is
  port (x, y, z, p_in : in  std_logic;     -- message and received parity bit
        err           : out std_logic);    -- 0 = no error detected, 1 = error detected
end parity_checker;

architecture behavioral of parity_checker is
begin
  err <= x xor y xor z xor p_in;           -- 1 if the total number of 1s is odd
end behavioral;
```

**Listing 3.1.** Even-parity generator and checker in VHDL.

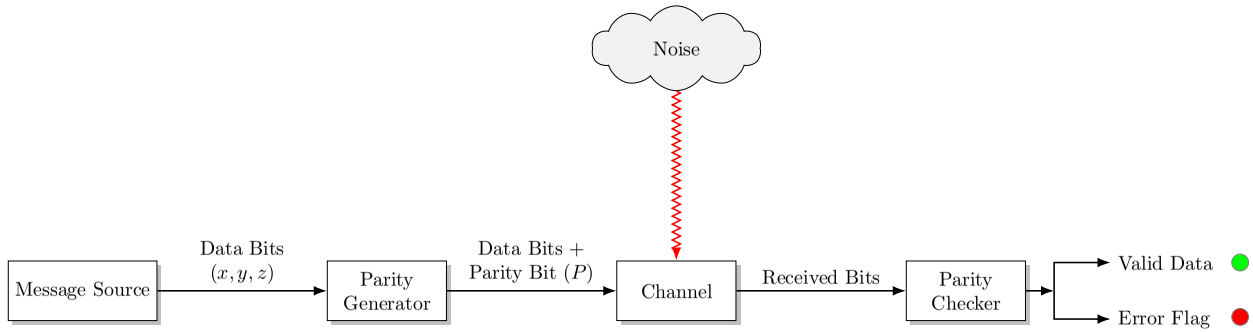
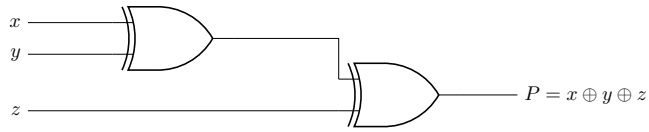
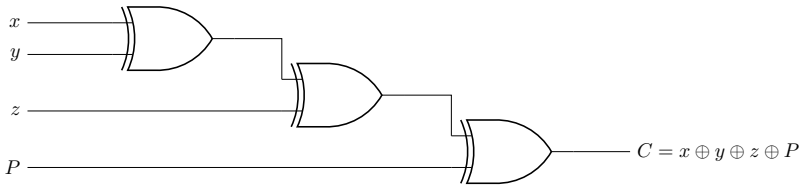



**Figure 3.15.** A parity system: the generator appends a parity bit; the checker at the receiver signals an error.




**(a)** generator $P = x \oplus y \oplus z$




**(b)** checker $C = x \oplus y \oplus z \oplus P$

**Figure 3.16.** Even-parity generator and checker for a 3-bit message.

**Chapter Summary**

- A **Karnaugh map** is a truth table arranged in Gray-code order so that adjacent squares differ in one variable. Groups of $2^k$ adjacent 1s eliminate $k$ variables.
- Group the 1s with **rectangles of size** $2^k$, as large as possible, allowing overlap and wrap-around; find all **prime implicants**, take the **essential** ones, then complete the cover with as few others as possible. A minimal SOP is not always unique.
- **Don't-cares** may be treated as 1 or as 0 to make larger groups.
- For a **minimal POS**, group the 0s and write sum terms (or simplify $F'$ and apply De Morgan).
- **NAND** and **NOR** are universal; an SOP maps to a two-level **NAND–NAND** circuit, a POS to a two-level **NOR–NOR** circuit.
- **XOR** is the odd function and **XNOR** the even function; they form a checkerboard on the K-map and are the basis of **parity** generation and checking.

## Exercises

**3.1** Simplify $F(A,B,C) = \sum m(0,2,3,4,6)$. (a) Construct the truth table. (b) Draw the K-map and the groups. (c) Write the minimal SOP. (d) Draw the circuit with AND, OR and NOT gates.

**3.2** Simplify $F(w,x,y,z) = \sum m(0,1,2,7,8,9,10,15)$ with don't-cares $d(5,13)$. (a) Draw the K-map, marking X squares. (b) Find the minimal SOP, stating which don't-cares were used as 1s. (c) Draw the circuit.

**3.3** Simplify $F(x,y,z) = \sum m(0,1,2,5,7)$ to a minimal *POS*. (a) Draw the K-map and plot the 0s. (b) Find the minimal POS. (c) Draw the two-level NOR–NOR

circuit.

**3.4** **Prime and parity detector.** Design a 4-input circuit with inputs $A, B, C, D$ (binary number $ABCD$, $A$ the most significant bit) and two outputs. $F = 1$ only when the input is a *prime number* (0 and 1 are not prime). $G = 1$ only when the number of 1s in the input is *even*. Inputs 10–15 are not valid BCD and are don't-cares. (a) Make one truth table for both outputs. (b) Find the minimal SOP for $F$ and the minimal POS for $G$. (c) Draw $F$ using only NAND gates and $G$ using only NOR gates.

**3.5** Find all prime implicants and all essential prime implicants of $F(w, x, y, z) = \sum m(0, 2, 3, 5, 7, 8, 10, 11, 13, 15)$. Give all minimal SOP expressions.

**3.6** Simplify the following four-variable functions to minimal SOP: (a) $F = \sum m(0, 2, 8, 10)$; (b) $F = \sum m(0, 1, 2, 3, 12, 13, 14, 15)$; (c) $F = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$.

**3.7** Convert the expression $F = AB + CD' + A'C'$ into (a) a circuit with only NAND gates and (b) a circuit with only NOR gates. Count the gates in each case.

**3.8** Show that the XOR operation is associative, $(A \oplus B) \oplus C = A \oplus (B \oplus C)$, by means of a truth table. How many 1s does the K-map of the 4-variable odd function $A \oplus B \oplus C \oplus D$ contain?

**3.9** Design an *odd*-parity generator for a 4-bit message $w, x, y, z$. Write the VHDL entity and architecture, and give the output for the messages `0110`, `0111` and `1111`.

**3.10** The 7-bit ASCII code of the letter 'A' is `1000001`. What parity bit must be appended for (a) even parity and (b) odd parity? A receiver using even parity gets `11000001` (parity bit first): is an error detected? Can the receiver tell which bit is wrong?

**3.11** Complete the 4-bit BCD to seven-segment decoder of Exercise 2.4 by finding minimal SOP expressions for segments $b$ and $c$ with the help of K-maps (outputs for inputs 10–15 are don't-cares).

## Further Reading

Karnaugh's original paper [11] is short and clear. McCluskey's tabular method [12] is the basis for computer-aided minimisation, and is covered in most digital-design textbooks, including Mano and Ciletti [7] and Wakerly [8], the latter with an excellent discussion of timing hazards in two-level logic. Brown and Vranesic [3] treat minimisation, NAND/NOR implementation and multilevel synthesis in depth.

# 4

# Number Representation and Arithmetic Circuits

**Learning Objectives**

After studying this chapter you will be able to:

- convert numbers among decimal, binary, octal and hexadecimal;
- represent signed integers in sign-magnitude, one's-complement and two's-complement form, and explain why two's complement is used in practice;
- add and subtract two's-complement numbers and detect arithmetic overflow;
- design half adders, full adders, ripple-carry adders and adder/subtractor circuits, and describe them in VHDL;
- explain the delay problem of the ripple-carry adder and design a carry-lookahead adder using generate and propagate signals;
- multiply unsigned numbers with an array multiplier;
- represent fractions in fixed-point and IEEE 754 floating-point formats, and decimal digits and characters in BCD and ASCII.

## 4.1 Positional Number Systems

Section 1.4 showed that a binary string can represent a quantity. In a positional system with radix $r$ the value of the digit string $d_{n-1}\cdots d_1 d_0 . d_{-1} d_{-2}\cdots$ is $\sum_i d_i r^i$, with negative $i$ for the fraction. Four radices matter in digital design:

- **Binary** ($r = 2$), digits 0 and 1; each digit is a *bit*, and a group of eight bits is a *byte*.
- **Octal** ($r = 8$), digits 0–7; one octal digit stands for exactly three bits.

- **Decimal** ($r = 10$), the system used by people.
- **Hexadecimal** ($r = 16$), digits 0–9 and A–F (A= 10, ..., F= 15); one hex digit stands for exactly four bits. It is the usual compact way of writing binary values.

Table 4.1 lists the first values. The binary–octal–hex conversions are trivial because $8 = 2^3$ and $16 = 2^4$: group the bits in threes or fours starting at the binary point, and replace each group by its digit.

**Table 4.1.** The numbers 0 to 16 in four radices.

| Dec | Binary | Oct | Hex | Dec | Binary | Oct | Hex |
|---|---|---|---|---|---|---|---|
| 0 | 00000 | 00 | 0 | 9 | 01001 | 11 | 9 |
| 1 | 00001 | 01 | 1 | 10 | 01010 | 12 | A |
| 2 | 00010 | 02 | 2 | 11 | 01011 | 13 | B |
| 3 | 00011 | 03 | 3 | 12 | 01100 | 14 | C |
| 4 | 00100 | 04 | 4 | 13 | 01101 | 15 | D |
| 5 | 00101 | 05 | 5 | 14 | 01110 | 16 | E |
| 6 | 00110 | 06 | 6 | 15 | 01111 | 17 | F |
| 7 | 00111 | 07 | 7 | 16 | 10000 | 20 | 10 |
| 8 | 01000 | 10 | 8 | | | | |

### 4.1.1 Converting from decimal

To convert the *integer part* to radix $r$, divide repeatedly by $r$ and collect the remainders; the first remainder is the least significant digit. To convert the *fractional part*, multiply repeatedly by $r$ and collect the integer parts; the first is the most significant fractional digit.

**Example 4.1: Radix conversion**

Convert $156_{10}$ to binary, hexadecimal and octal, and convert $0.6875_{10}$ to binary.
**Solution.** Dividing 156 by 2 repeatedly gives remainders $0, 0, 1, 1, 1, 0, 0, 1$ (least significant first), so $156 = \texttt{10011100}_2$. Grouping in fours gives $\texttt{1001 1100} = 9\text{C}_{16}$, and grouping in threes ($\texttt{010 011 100}$) gives $234_8$. Check: $2 \cdot 64 + 3 \cdot 8 + 4 = 156$. For the fraction: $0.6875 \times 2 = 1.375 \to 1$; $0.375 \times 2 = 0.75 \to 0$; $0.75 \times 2 = 1.5 \to 1$; $0.5 \times 2 = 1.0 \to 1$, so $0.6875 = \texttt{0.1011}_2$.

## 4.2 Signed Integer Representations

To represent negative numbers in binary, the most significant bit (MSB) is used as a *sign bit*: 0 means positive and 1 means negative (Figure 4.1). Three schemes differ in how the remaining bits represent a negative value.

**Sign-magnitude.** The remaining $n-1$ bits hold the magnitude. The range is $\pm(2^{n-1}-1)$ and there are *two zeros*, `0000` ($+0$) and `1000` ($-0$). Arithmetic is awkward, since the

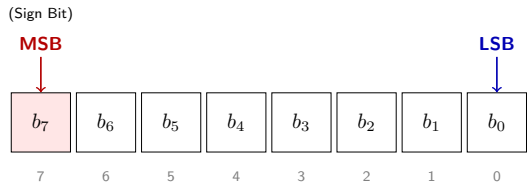



**Figure 4.1.** An 8-bit signed number: the MSB is the sign bit, the LSB is bit 0.

signs and magnitudes must be compared first.

**One's complement.** Positive numbers are as in sign-magnitude; a negative number is obtained by *inverting every bit* of the positive number: $+5 = \texttt{0101}$, $-5 = \texttt{1010}$. There are again two zeros (`0000` and `1111`). Addition requires an *end-around carry*: the carry out of the MSB is added back to the result.

**Two's complement.** A negative number is obtained by inverting all bits and *adding 1*: $+5 = \texttt{0101} \to \texttt{1010} \to \texttt{1011} = -5$. There is only one zero, and the range is asymmetric: $-2^{n-1}$ to $2^{n-1}-1$. The value of an $n$-bit two's-complement number is

$$V = -d_{n-1}\,2^{n-1} + \sum_{i=0}^{n-2} d_i\,2^i. \tag{4.1}$$

Hence $\texttt{10110101} = -128+32+16+4+1 = -75$. Table 4.2 compares the three schemes for 4-bit numbers.

**Table 4.2.** Four-bit signed numbers in the three representations.

| Dec | Sign-mag. | 1's comp. | 2's comp. | Dec | Sign-mag. | 1's comp. | 2's comp. |
|---|---|---|---|---|---|---|---|
| +7 | 0111 | 0111 | 0111 | −0 | 1000 | 1111 | — |
| +6 | 0110 | 0110 | 0110 | −1 | 1001 | 1110 | 1111 |
| +5 | 0101 | 0101 | 0101 | −2 | 1010 | 1101 | 1110 |
| +4 | 0100 | 0100 | 0100 | −3 | 1011 | 1100 | 1101 |
| +3 | 0011 | 0011 | 0011 | −4 | 1100 | 1011 | 1100 |
| +2 | 0010 | 0010 | 0010 | −5 | 1101 | 1010 | 1011 |
| +1 | 0001 | 0001 | 0001 | −6 | 1110 | 1001 | 1010 |
| +0 | 0000 | 0000 | 0000 | −7 | 1111 | 1000 | 1001 |
| | | | | −8 | — | — | 1000 |

**Example 4.2: Two's-complement conversions**

(a) Represent $-75$ as an 8-bit two's-complement number, in binary and hexadecimal. (b) Find the decimal value of `1011 0101`. (c) Extend `1011` ($-5$) to 8 bits.

**Solution.** (a) $+75 = \texttt{0100 1011}$. Invert: $\texttt{1011 0100}$. Add 1: $\texttt{1011 0101} = \mathrm{B5}_{16}$. (b) Using Equation (4.1), $-128+32+16+4+1 = -75$; this is indeed the number found in (a). (c) To widen a two's-complement number, *sign-extend* it by copying the sign bit into the new upper bits: $\texttt{1111 1011}$, which is again $-5$ ($-128+64+32+16+8+2+1=-5$).

**Why two's complement?**

Two's complement has a single zero, a simple negation (invert and add 1), and, above all, *the same adder circuit adds signed and unsigned numbers*: subtraction is just addition of the negated operand. This is why every modern processor uses it.

## 4.3 Arithmetic and Overflow

Example 4.3 shows the calculation $4 + (-3)$ in the three representations.

**Example 4.3: $4 + (-3)$ in the three systems**

**Sign-magnitude.** $+4 = \texttt{0100}$, $-3 = \texttt{1011}$. The signs differ, so subtract the smaller magnitude from the larger: $\texttt{100}-\texttt{011} = \texttt{001}$, with the sign of the larger magnitude (positive): $\texttt{0001} = +1$.
**One's complement.** $-3 = \texttt{1100}$. $\texttt{0100} + \texttt{1100} = \texttt{1 0000}$. The end-around carry gives $\texttt{0000} + 1 = \texttt{0001} = +1$.
**Two's complement.** $-3 = \texttt{1101}$. $\texttt{0100} + \texttt{1101} = \texttt{1 0001}$. Discarding the carry-out gives $\texttt{0001} = +1$.

If the true result of an operation lies outside the range of the representation, *arithmetic overflow* occurs and the result is wrong. With $n$ bits the range for signed numbers is $-2^{n-1}$ to $2^{n-1}-1$. For example, in 4 bits $(+7)+(+2) = +9$ computes as $\texttt{0111}+\texttt{0010} = \texttt{1001}$, which reads as $-7$. Similarly $(-7)+(-2) = \texttt{1001}+\texttt{1110} = \texttt{1 0111}$, which after discarding the carry reads as $+7$. Two facts make detection easy. First, overflow *cannot occur when the operands have different signs*. Second, when it does occur, the carry into the sign bit differs from the carry out of it.

**Overflow rule for two's complement**

Let $c_{n-1}$ be the carry into the sign-bit position and $c_n$ the carry out of it. Overflow occurs if and only if

$$\text{overflow} = c_{n-1} \oplus c_n .$$

In the first example, $c_4 = 0$ and $c_3 = 1$; in the second, $c_4 = 1$ and $c_3 = 0$. In both cases the two carries differ, so overflow is signalled.

## 4.4 Adders

### 4.4.1 Half adder and full adder

The half adder of Example 2.6 adds two bits. To add multi-bit numbers we need to include the carry arriving from the lower position: a *full adder* (FA) adds three bits, $A$, $B$ and $C_{in}$, and produces a sum $S$ and carry-out $C_{out}$ (Table 4.3). From the truth table

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + C_{in}(A \oplus B). \tag{4.2}$$

The carry output is the *majority* function of the three inputs, so it also equals $AB + AC_{in} + BC_{in}$.

**Table 4.3.** Truth table of the full adder.

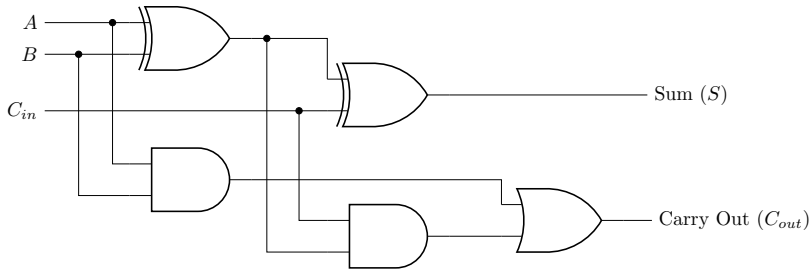
| $A$ | $B$ | $C_{in}$ | $C_{out}$ | $S$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Equation (4.2) shows that a full adder can be built from *two half adders and an OR gate*: the first half adder adds $A$ and $B$ giving $S_1 = A \oplus B$ and $C_1 = AB$; the second adds $S_1$ and $C_{in}$ giving $S = S_1 \oplus C_{in}$ and $C_2 = S_1 C_{in}$; and $C_{out} = C_1 + C_2$ (Figure 4.2). Listing 4.1 is the corresponding *structural* VHDL description, in which a design is written as a set of component instances connected by signals; it uses the half-adder entity of Listing 2.5.

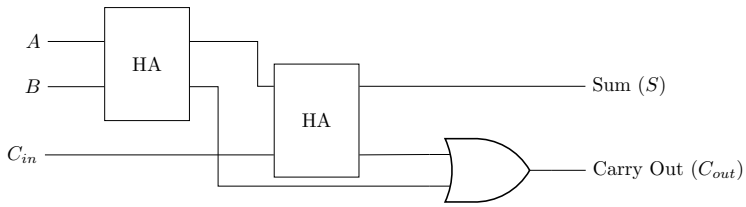
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-------------------------------------------------------------------------
-- ENTITY: Full Adder Interface
-------------------------------------------------------------------------
ENTITY full_adder IS
    PORT (
        A, B, Cin : IN  STD_LOGIC;  -- Three inputs
        S, Cout   : OUT STD_LOGIC   -- Two outputs
    );
END ENTITY full_adder;

-------------------------------------------------------------------------
-- ARCHITECTURE: Structural Implementation
```




**(a)** gate-level circuit




**(b)** from two half adders

**Figure 4.2.** Two implementations of the full adder.

```
16 ----------------------------------------------------------------------
17 ARCHITECTURE structural OF full_adder IS
18
19     -- 1. Declare the component used (Half Adder)
20     COMPONENT half_adder
21         PORT (
22             X, Y : IN  STD_LOGIC;
23             S, C : OUT STD_LOGIC
24         );
25     END COMPONENT;
26
27     -- 2. Define internal signals to connect the components
28     SIGNAL S1, C1, C2 : STD_LOGIC;
29     -- S1: Sum from HA1 (A XOR B)
30     -- C1: Carry from HA1 (A AND B)
31     -- C2: Carry from HA2 ((A XOR B) AND Cin)
32
33 BEGIN
34
35     -- 3. Component Instantiation 1 (HA1)
36     HA_1: half_adder
37         PORT MAP (
38             X => A,
39             Y => B,
40             S => S1, -- Output S1 feeds into HA2
41             C => C1  -- Output C1 feeds into the final OR gate
42         );
```

```
43
44      -- 4. Component Instantiation 2 (HA2)
45      HA_2: half_adder
46          PORT MAP (
47              X => S1,  -- Input X is the sum from HA1
48              Y => Cin, -- Input Y is the Carry-in
49              S => S,   -- Output S is the final Sum of the Full Adder
50              C => C2   -- Output C2 feeds into the final OR gate
51          );
52
53      -- 5. Final OR Gate (The Sum of the carries C1 and C2)
54      -- Cout = C1 OR C2
55      Cout <= C1 OR C2;
56
57  END ARCHITECTURE structural;
```

**Listing 4.1.** Structural VHDL of the full adder, built from two half-adder components.

### 4.4.2 Ripple-carry adder

To add two $n$-bit numbers we cascade $n$ full adders: the carry-out of stage $i$ becomes the carry-in of stage $i+1$. The first carry-in $C_0$ is 0 for ordinary addition. Figure 4.3 shows a 4-bit *ripple-carry adder*, so called because the carry "ripples" from the least significant to the most significant stage. Listing 4.2 describes it in VHDL using a `std_logic_vector`, which is an array of bits written with an index range such as `(3 downto 0)`, and an internal vector of carries.

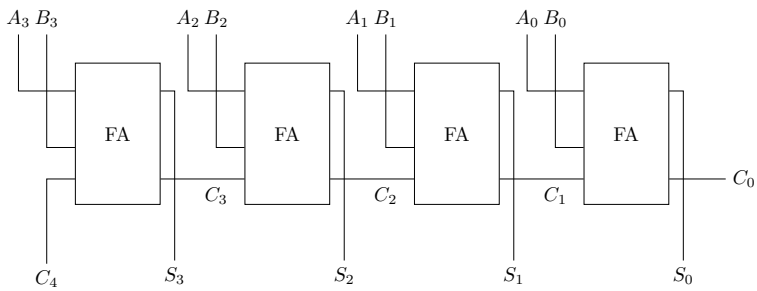



**Figure 4.3.** A 4-bit ripple-carry adder built from four full adders.

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ----------------------------------------------------------------------
5   -- ENTITY: 4-bit Ripple Adder Interface
6   ----------------------------------------------------------------------
7   ENTITY ripple_adder_4bit IS
8       PORT (
9           A, B    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); -- Two 4-bit inputs
10          Cin     : IN  STD_LOGIC;                    -- Global Carry-in
```

```
        S       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0); -- 4-bit Sum output
        Cout    : OUT STD_LOGIC                     -- Final Carry-out (MSB)
    );
END ENTITY ripple_adder_4bit;

-------------------------------------------------------------------------
-- ARCHITECTURE: Structural Implementation
-------------------------------------------------------------------------
ARCHITECTURE structural OF ripple_adder_4bit IS

    -- 1. Declare the Full Adder Component
    COMPONENT full_adder
        PORT (
            A, B, Cin : IN  STD_LOGIC;
            S, Cout   : OUT STD_LOGIC
        );
    END COMPONENT;

    -- 2. Define Internal Signals for the Carry Chain
    -- We need 3 internal carry signals (C1, C2, C3) to link the 4 FAs.
    -- Cin from the entity is C0. Cout from the last FA is Cout of the entity.
    SIGNAL C_internal : STD_LOGIC_VECTOR(3 DOWNTO 1);

BEGIN

    -- FA0: LSB (Bit 0)
    -- Inputs: A(0), B(0), Entity Cin
    -- Outputs: S(0), Internal Carry C_internal(1)
    FA_0: full_adder
        PORT MAP (
            A   => A(0),
            B   => B(0),
            Cin => Cin,
            S   => S(0),
            Cout => C_internal(1) -- C1
        );

    -- FA1: Bit 1
    -- Inputs: A(1), B(1), Carry from FA0 (C_internal(1))
    -- Outputs: S(1), Internal Carry C_internal(2)
    FA_1: full_adder
        PORT MAP (
            A   => A(1),
            B   => B(1),
            Cin => C_internal(1),
            S   => S(1),
            Cout => C_internal(2) -- C2
        );

    -- FA2: Bit 2
    -- Inputs: A(2), B(2), Carry from FA1 (C_internal(2))
```

```
    -- Outputs: S(2), Internal Carry C_internal(3)
    FA_2: full_adder
        PORT MAP (
            A   => A(2),
            B   => B(2),
            Cin => C_internal(2),
            S   => S(2),
            Cout => C_internal(3) -- C3
        );

    -- FA3: MSB (Bit 3)
    -- Inputs: A(3), B(3), Carry from FA2 (C_internal(3))
    -- Outputs: S(3), Entity Cout
    FA_3: full_adder
        PORT MAP (
            A   => A(3),
            B   => B(3),
            Cin => C_internal(3),
            S   => S(3),
            Cout => Cout -- Final Cout of the 4-bit adder
        );

END ARCHITECTURE structural;
```

**Listing 4.2.** Structural VHDL of a 4-bit ripple-carry adder.

Figure 4.4 shows a ModelSim simulation of this adder using a testbench with assertions: $0+0$, $1+1$, $15+1$ (carry out, sum 0000), $15+15+1$ (sum 1111, carry out 1) and $5+3$ (sum 1000). The testbenches of this chapter, in the style of Listing 2.6, are provided with the book's source files.

### 4.4.3 Adder/subtractor

Since $A - B = A + (-B) = A + \overline{B} + 1$ in two's complement, the same adder performs subtraction if the bits of $B$ are inverted and the initial carry-in is 1. An XOR gate works as a *controllable inverter*: $B_i \oplus 0 = B_i$ and $B_i \oplus 1 = B_i'$. With a control input $sub$ ($0 =$ add, $1 =$ subtract) connected both to the XOR gates and to $C_0$ we obtain the adder/subtractor of Figure 4.5. When $sub = 0$ the circuit computes $A + B$, and when $sub = 1$ it computes $A + \overline{B} + 1 = A - B$. The overflow signal is $c_{n-1} \oplus c_n$.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Top-Level Entity
entity adder_subtractor_4bit is
    port (
        A       : in  std_logic_vector(3 downto 0); -- Input operand A
        B       : in  std_logic_vector(3 downto 0); -- Input operand B
        add_sub : in  std_logic;                    -- Mode control: '0' = Add,
```

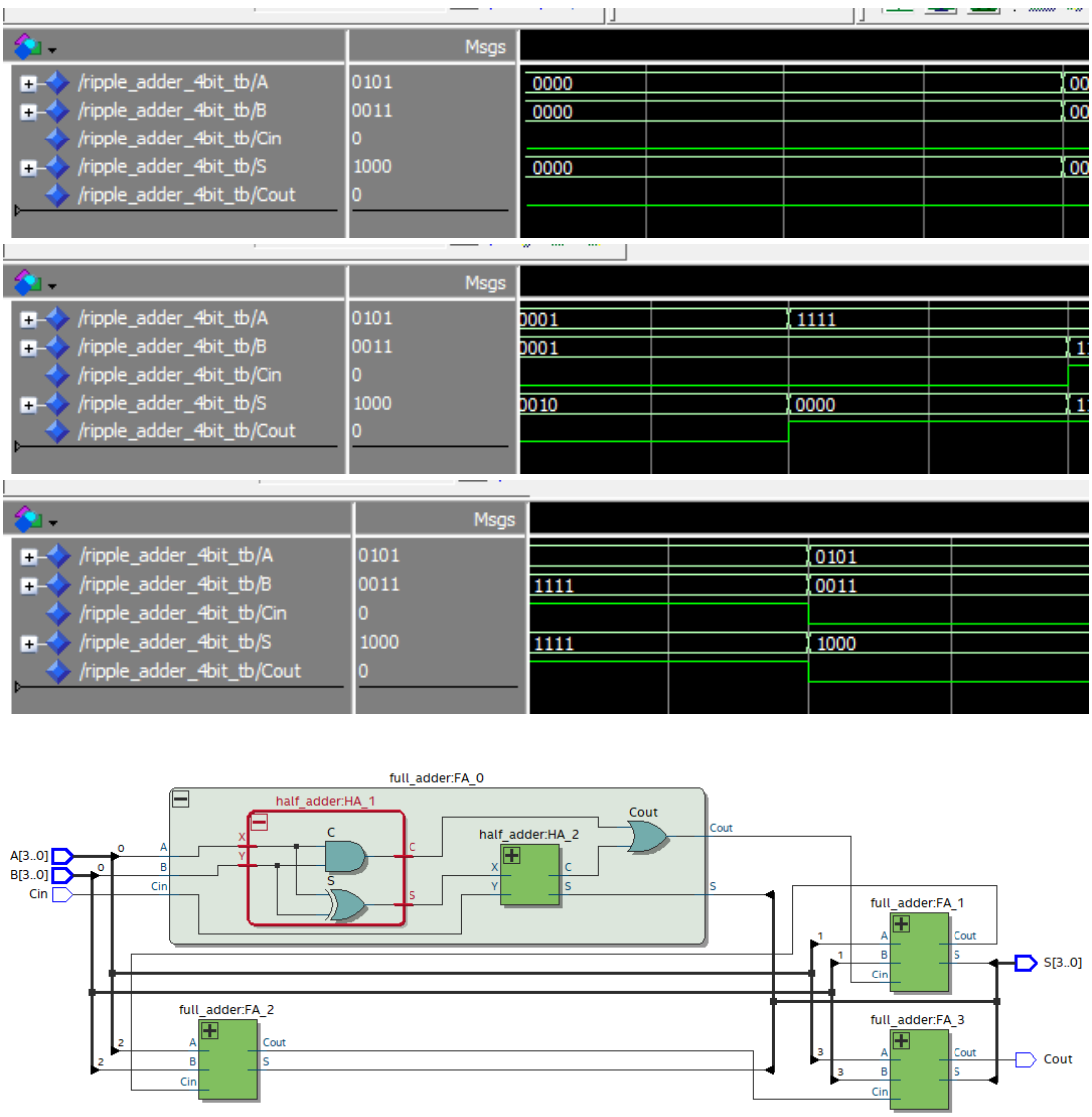



**Figure 4.4.** Ripple-carry adder: ModelSim simulation of the testbench (top) and the RTL viewer schematic produced by Quartus (bottom), author's screenshots.

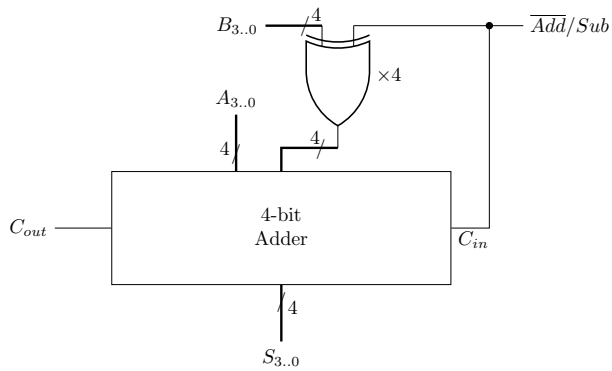



**Figure 4.5.** A 4-bit adder/subtractor. The XOR gates conditionally complement $B$, and the control input is also the initial carry.

```
    '1' = Subtract
        S       : out std_logic_vector(3 downto 0); -- Sum / Difference output
        Cout    : out std_logic                     -- Carry out / Borrow out
    );
end entity adder_subtractor_4bit;

architecture structural of adder_subtractor_4bit is

    -- 1. Component Declaration (Matches ripple_adder_4bit port list)
    component ripple_adder_4bit
        port (
            A       : in  std_logic_vector(3 downto 0);
            B       : in  std_logic_vector(3 downto 0);
            Cin     : in  std_logic;
            S       : out std_logic_vector(3 downto 0);
            Cout    : out std_logic
        );
    end component;

    -- 2. Internal Signals
    -- Signal to hold the modified B input (B XOR add_sub)
    signal B_comp : std_logic_vector(3 downto 0);

begin
    -- ** Logic for Subtraction (2's Complement) **

    -- 1. Bitwise XOR Operation (Equivalent to 4 XOR gates)
    -- add_sub='0' -> B_comp = B (Addition)
    -- add_sub='1' -> B_comp = not B (1's Complement)
    B_comp <= B xor (add_sub & add_sub & add_sub & add_sub);

    -- ** Component Instantiation **

    -- 2. Instantiate the 4-bit Ripple Adder
```

```
    Adder_Instance : ripple_adder_4bit
        port map (
            A    => A,
            B    => B_comp,        -- Connect the (possibly inverted) B
            Cin  => add_sub,       -- Connect 'add_sub' to Carry_in (for the +1 in
 2's complement)
            S    => S,             -- Connect the adder's sum output to the main
output
            Cout => Cout           -- Connect the adder's carry-out to the main
output
        );

end architecture structural;
```

**Listing 4.3.** VHDL of the 4-bit adder/subtractor, reusing the ripple-carry adder.

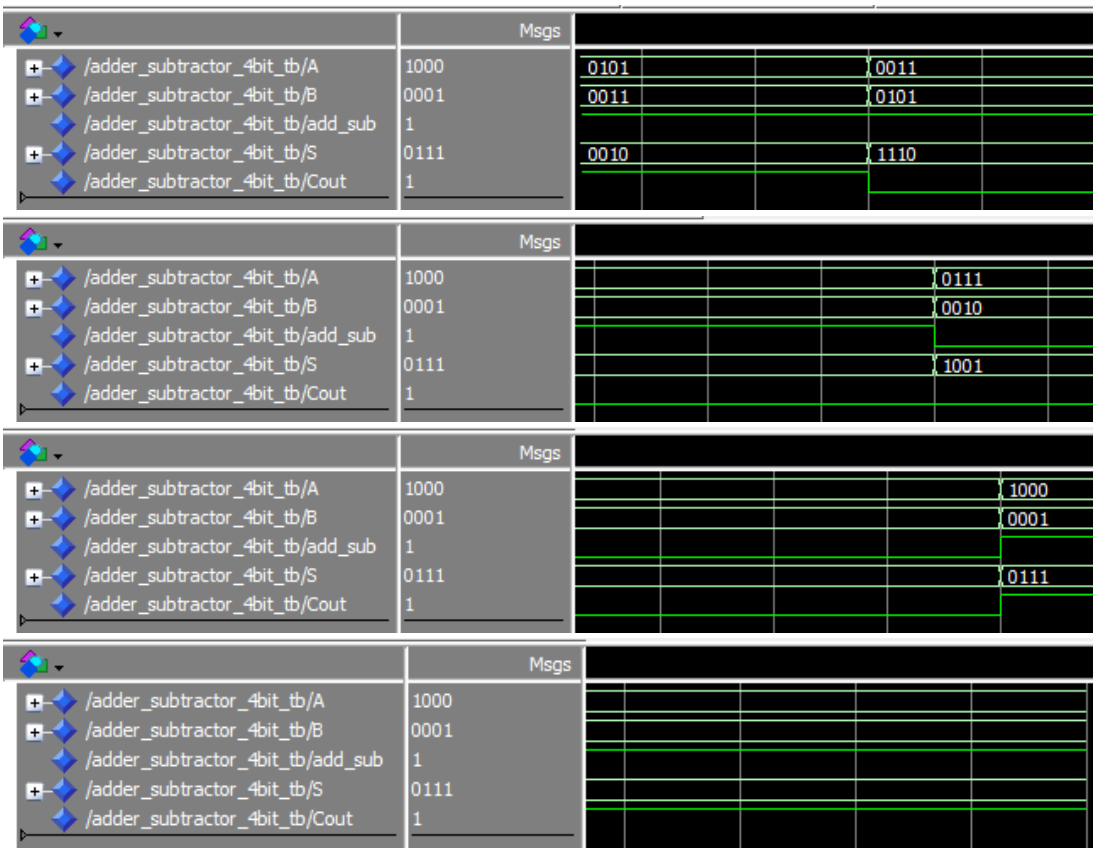



**Figure 4.6.** ModelSim simulation of the adder/subtractor: $5 + 3 = 8$, $5 - 3 = 2$, $3 - 5 = -2$ (`1110`), $7 + 2$ (overflow, `1001`) and $-8 - 1$ (overflow, `0111`). Author's screenshot.

## 4.5 The Carry Delay and the Carry-Lookahead Adder

### 4.5.1 Delay of the ripple-carry adder

The *critical path*, the longest delay between an input change and a stable output, of a ripple-carry adder passes through the carry chain. The worst case, for example `1111` + `0001`, requires the carry generated in the first stage to travel through every later stage. If each full adder adds two gate delays on the carry path, an $n$-bit adder has a delay of about $2n$ gate delays, so a 16-bit adder needs about 32 gate delays (Figure 4.7). The delay grows linearly with $n$, which is unacceptable in a fast processor.

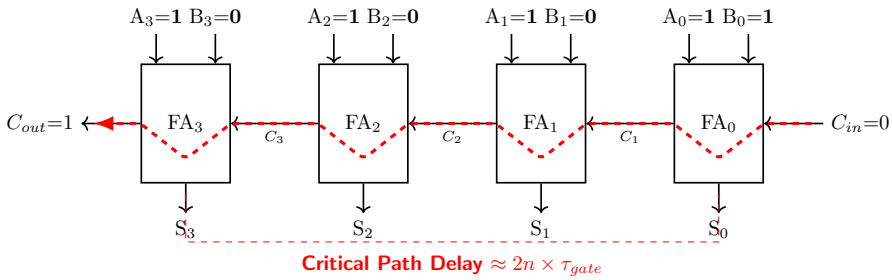



**Figure 4.7.** The carry must ripple through every stage in the worst case.

### 4.5.2 Generate and propagate

A *carry-lookahead adder* (CLA) computes all the carries in parallel directly from the inputs, which removes the long dependency chain. The key is to re-express the full adder in terms of two signals for each bit position $i$:

- the *generate* signal $G_i = A_i B_i$: stage $i$ produces a carry by itself, regardless of its carry-in;
- the *propagate* signal $P_i = A_i \oplus B_i$: stage $i$ passes its carry-in on to its carry-out.

Then

$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i. \tag{4.3}$$

Substituting repeatedly removes the dependence on the previous carry:

$$\begin{aligned} C_1 &= G_0 + P_0 C_0, \\ C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0, \\ C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0, \\ C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0. \end{aligned}$$

Each carry is a two-level AND–OR expression of the $P_i$, $G_i$ and $C_0$ only, and all four are computed at the same time (Figure 4.8).

Listing 4.4 is the lookahead logic block in VHDL and Listing 4.5 is the top-level design, which instantiates the P/G units, the lookahead block and the sum gates.

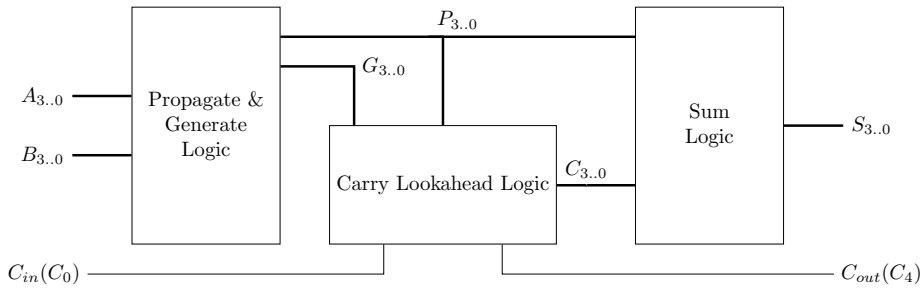



**Figure 4.8.** Block diagram of a 4-bit carry-lookahead adder: a P/G generator for each bit, the lookahead logic, and sum XOR gates.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY cla_logic IS
    PORT (
        P_in, G_in : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); -- P and G vectors (P0-P3, G0-G3)
        C0         : IN  STD_LOGIC;                    -- Global Carry-in
        C_out      : OUT STD_LOGIC_VECTOR(4 DOWNTO 1)  -- Calculated Carries (C1-C4)
    );
END ENTITY cla_logic;

ARCHITECTURE dataflow OF cla_logic IS
BEGIN
    -- C1 = G0 + P0 * C0
    C_out(1) <= G_in(0) OR (P_in(0) AND C0);

    -- C2 = G1 + P1*G0 + P1*P0*C0
    C_out(2) <= G_in(1) OR (P_in(1) AND G_in(0)) OR (P_in(1) AND P_in(0) AND C0);

    -- C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
    C_out(3) <= G_in(2) OR (P_in(2) AND G_in(1)) OR (P_in(2) AND P_in(1) AND G_in(0)) OR
                (P_in(2) AND P_in(1) AND P_in(0) AND C0);

    -- C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0 (Final Carry-out of the 4-bit block)
    C_out(4) <= G_in(3) OR (P_in(3) AND G_in(2)) OR (P_in(3) AND P_in(2) AND G_in(1)) OR
                (P_in(3) AND P_in(2) AND P_in(1) AND G_in(0)) OR
                (P_in(3) AND P_in(2) AND P_in(1) AND P_in(0) AND C0);

END ARCHITECTURE dataflow;
```

**Listing 4.4.** Carry-lookahead logic block of the 4-bit CLA.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY cla_4bit_adder IS
    PORT (
        A, B    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); -- 4-bit inputs
        Cin     : IN  STD_LOGIC;                    -- Global Carry-in (C0)
        S       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0); -- 4-bit Sum output
        Cout    : OUT STD_LOGIC                     -- Final Carry-out (C4)
    );
END ENTITY cla_4bit_adder;

ARCHITECTURE structural OF cla_4bit_adder IS

    -- Declare all necessary components
    COMPONENT pg_unit
        PORT ( A, B : IN STD_LOGIC; P_out, G_out : OUT STD_LOGIC );
    END COMPONENT;

    COMPONENT cla_logic
        PORT ( P_in, G_in : IN STD_LOGIC_VECTOR(3 DOWNTO 0); C0 : IN STD_LOGIC;
    C_out : OUT STD_LOGIC_VECTOR(4 DOWNTO 1) );
    END COMPONENT;

    COMPONENT sum_logic
        PORT ( P_in, C_in : IN STD_LOGIC; S_out : OUT STD_LOGIC );
    END COMPONENT;

    -- Internal Signals for P and G vectors
    SIGNAL P_vec : STD_LOGIC_VECTOR(3 DOWNTO 0);
    SIGNAL G_vec : STD_LOGIC_VECTOR(3 DOWNTO 0);

    -- Internal Signals for the Carries (C1, C2, C3, C4)
    SIGNAL C_vec : STD_LOGIC_VECTOR(4 DOWNTO 1);

    -- Internal Signals for correct Carry-in for each bit i=0 to 3 (C0, C1, C2, C3
    )
    SIGNAL C_in_sum : STD_LOGIC_VECTOR(3 DOWNTO 0);

BEGIN

    -- Map C0 (Cin) and C1-C3 (from C_vec) to the correct carry-in signals for the
     Sum Logic
    -- C_in_sum(0) must be C0, and C_in_sum(1 to 3) must be C_vec(1 to 3)
    C_in_sum(0) <= Cin;
    C_in_sum(3 DOWNTO 1) <= C_vec(3 DOWNTO 1);

    -- 1. Generate P and G Signals (4 instances of PG_Unit)
    PG_GEN: FOR i IN 0 TO 3 GENERATE
        PG_I: pg_unit
```

```
            PORT MAP (
                A => A(i),
                B => B(i),
                P_out => P_vec(i),
                G_out => G_vec(i)
            );
    END GENERATE;

    -- 2. CLA Logic Block (1 instance)
    CLA_BLOCK: cla_logic
        PORT MAP (
            P_in => P_vec,
            G_in => G_vec,
            C0   => Cin,
            C_out => C_vec
        );

    -- Map the final calculated carry C4 to the entity's Cout
    Cout <= C_vec(4);

    -- 3. Sum Logic (4 instances of Sum_Logic)
    SUM_GEN: FOR i IN 0 TO 3 GENERATE
        SUM_I: sum_logic
            PORT MAP (
                P_in => P_vec(i),
                -- Use the pre-calculated C_in_sum(i)
                C_in => C_in_sum(i),
                S_out => S(i)
            );
    END GENERATE;

END ARCHITECTURE structural;
```

**Listing 4.5.** Top level of the 4-bit carry-lookahead adder.

### 4.5.3 Hierarchical lookahead

The fan-in (number of inputs) of the gates in a lookahead circuit grows with the word length, so a single-level CLA is impractical for 16 or 32 bits. The remedy is to build a *hierarchy*: a 16-bit adder is made of four 4-bit CLA blocks, and a second-level lookahead unit computes the carries between the blocks. Each 4-bit block supplies a block-propagate and a block-generate signal:

$$P_0^* = P_3P_2P_1P_0, \qquad G_0^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0,$$

and similarly $P_1^*, G_1^*$ for bits 4–7, $P_2^*, G_2^*$ for bits 8–11 and $P_3^*, G_3^*$ for bits 12–15. The block carries follow the same pattern as the bit carries:

$$\begin{aligned} c_4 &= G_0^* + P_0^* c_0, \\ c_8 &= G_1^* + P_1^* G_0^* + P_1^* P_0^* c_0, \\ c_{12} &= G_2^* + P_2^* G_1^* + P_2^* P_1^* G_0^* + P_2^* P_1^* P_0^* c_0. \end{aligned}$$

The structure is shown in Figure 4.9.

**Example 4.4: Delay comparison for 16 bits**

Take one gate delay as $t$.
**Ripple-carry:** the carry passes through 16 full adders at $2t$ each: $16 \times 2t = 32t$.
**Hierarchical CLA:** (1) all $P_i$ and $G_i$: $1t$; (2) block $P^*$ and $G^*$: $2t$; (3) second-level lookahead gives $c_4, c_8, c_{12}$: $2t$; (4) the carries inside each block: $2t$; (5) the sum bits $S_i = P_i \oplus C_i$: $1t$. The total is $1 + 2 + 2 + 2 + 1 = 8t$, four times faster than the ripple-carry adder, at the price of more gates.

## 4.6 Multiplication

Binary multiplication is a process of adding *partial products*. Because each multiplier bit is 0 or 1, each partial product is either zero or a copy of the multiplicand, shifted to the position of the multiplier bit.

**Example 4.5: Multiplying 1101 × 1011**

Multiply $A =$ `1101` (13) by $B =$ `1011` (11).

```
      1101
  ×   1011
  ------
      1101
     11010
    000000
   1101000
  --------
  10001111
```

The partial products are $A \times B_0$, $A \times B_1$ (shifted one place), and so on; adding them, `1101` + `11010` + `000000` + `1101000` = 13 + 26 + 0 + 104 = 143 = `10001111`. The product of two 4-bit numbers needs up to 8 bits.

An *array multiplier* implements this directly. An AND gate forms each bit of the partial products, $A_i B_j$, and an array of adders adds them with the correct shifts. For a $4 \times 4$ multiplier (Figure 4.11) we need 16 AND gates and 12 adders (3 half adders and 9 full adders in the design of Listing 4.6). Listing 4.6 also shows the VHDL `generate` statement, which replicates hardware: here it creates the 16 AND gates.

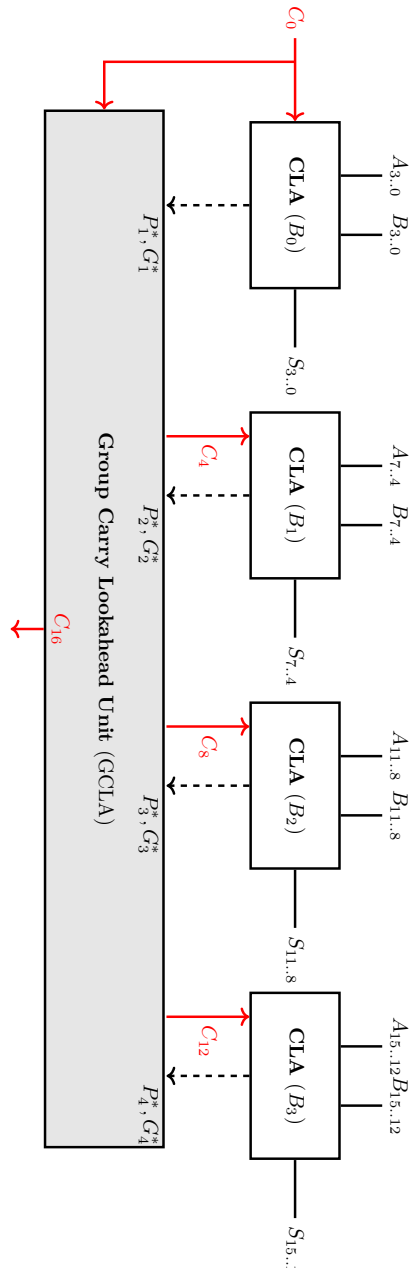



**Figure 4.9.** A 16-bit adder with four 4-bit carry-lookahead blocks and a second-level lookahead unit.

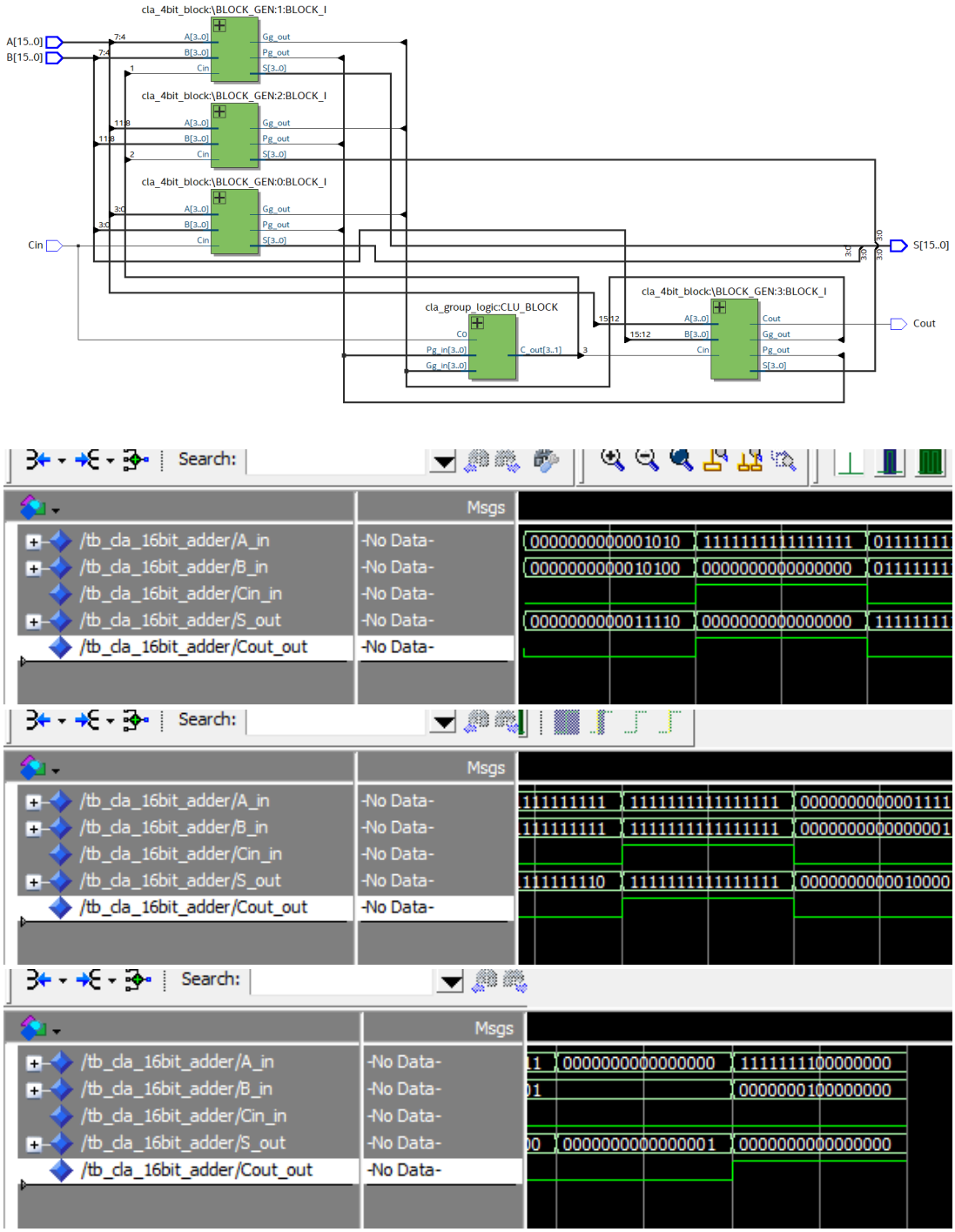



**Figure 4.10.** 16-bit carry-lookahead adder: RTL viewer (top) and ModelSim simulation (bottom). Author's screenshots. The VHDL files `cla_4bit_block`, `cla_group_logic` and `cla_16bit_adder` accompany the book.

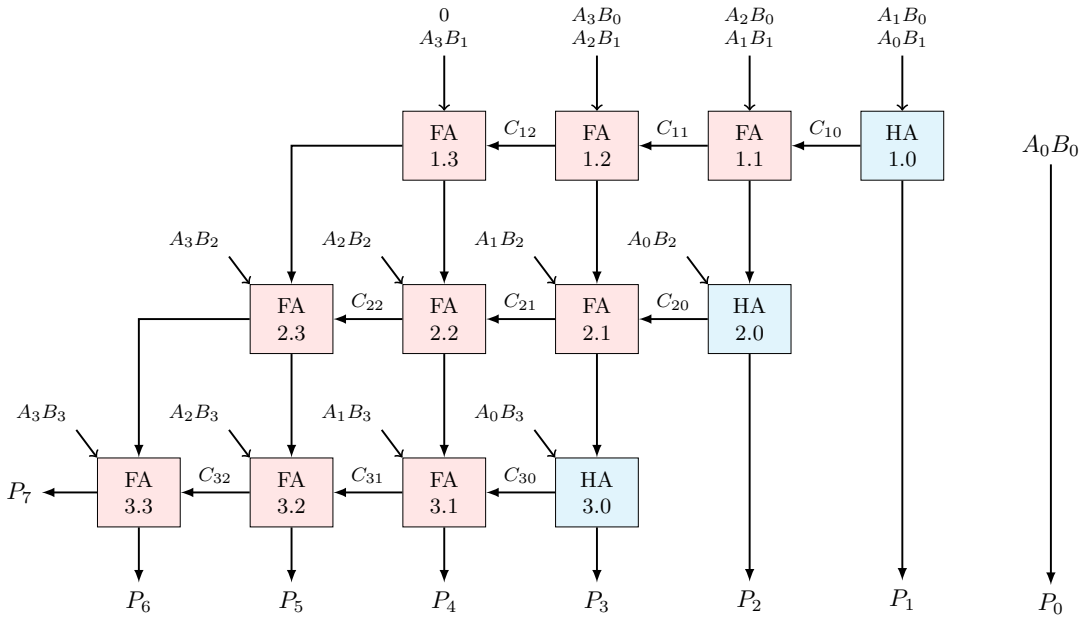



**Figure 4.11.** A 4 × 4 array multiplier: partial products are formed by AND gates and summed by rows of adders.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-----------------------------------------------------------------------
-- ENTITY: 4x4 Multiplier Interface (A * B = P)
-----------------------------------------------------------------------
ENTITY multiplier_4x4 IS
    PORT (
        A, B : IN  STD_LOGIC_VECTOR(3 DOWNTO 0); -- Two 4-bit multiplicands
        P    : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)  -- 8-bit product (P7 to P0)
    );
END ENTITY multiplier_4x4;

-----------------------------------------------------------------------
-- ARCHITECTURE: Structural Implementation (Array Multiplier)
-----------------------------------------------------------------------
ARCHITECTURE structural OF multiplier_4x4 IS

    COMPONENT half_adder
        PORT ( X, Y : IN STD_LOGIC; S, C : OUT STD_LOGIC );
    END COMPONENT;

    COMPONENT full_adder
        PORT ( A, B, Cin : IN STD_LOGIC; S, Cout : OUT STD_LOGIC );
    END COMPONENT;

```

```
    -- Internal Signals
    TYPE pp_matrix IS ARRAY (3 DOWNTO 0, 3 DOWNTO 0) OF STD_LOGIC;
    SIGNAL PP : pp_matrix;

    -- Row 1 Signals (Sum of A*B0 and A*B1)
    SIGNAL S1 : STD_LOGIC_VECTOR(3 DOWNTO 0); -- S1(0) is P(1), others pass to Row
     2
    SIGNAL C1 : STD_LOGIC_VECTOR(3 DOWNTO 0);

    -- Row 2 Signals (Sum of Row 1 and A*B2)
    SIGNAL S2 : STD_LOGIC_VECTOR(3 DOWNTO 0); -- S2(0) is P(2), others pass to Row
     3
    SIGNAL C2 : STD_LOGIC_VECTOR(3 DOWNTO 0);

    -- Row 3 Signals (Sum of Row 2 and A*B3)
    SIGNAL S3 : STD_LOGIC_VECTOR(3 DOWNTO 0); -- S3(0) is P(3), S3(1) is P(4)...
    SIGNAL C3 : STD_LOGIC_VECTOR(3 DOWNTO 0); -- Carry out

BEGIN
    -- --- Stage 1: Partial Product Generation ---
    PP_GEN: FOR i IN 0 TO 3 GENERATE
        PP_ROW: FOR j IN 0 TO 3 GENERATE
            PP(i, j) <= A(i) AND B(j);
        END GENERATE PP_ROW;
    END GENERATE PP_GEN;

    -- --- Stage 2: Adder Array ---

    -- P(0)
    P(0) <= PP(0, 0);

    -- Row 1: Add (A*B0 PPs shifted) and (A*B1 PPs)
    -- Operands:
    -- B0: PP(1,0), PP(2,0), PP(3,0), '0'
    -- B1: PP(0,1), PP(1,1), PP(2,1), PP(3,1)

    -- Adder 1.0 (Bit 1)
    HA10: half_adder PORT MAP (X => PP(1, 0), Y => PP(0, 1), S => S1(0), C => C1
    (0));
```

**Listing 4.6.** VHDL of the $4 \times 4$ array multiplier (first part: declarations, partial products and the first adder row). The remaining rows follow the same pattern.

## 4.7 Fractions and Floating Point

### 4.7.1 Fixed-point numbers

A *fixed-point* number represents a fraction with a fixed number of bits for the integer and fractional parts; the binary point is implicit. The notation $Qn.m$ means $n$ integer bits and $m$ fractional bits (unsigned unless stated otherwise); the resolution is $2^{-m}$ and

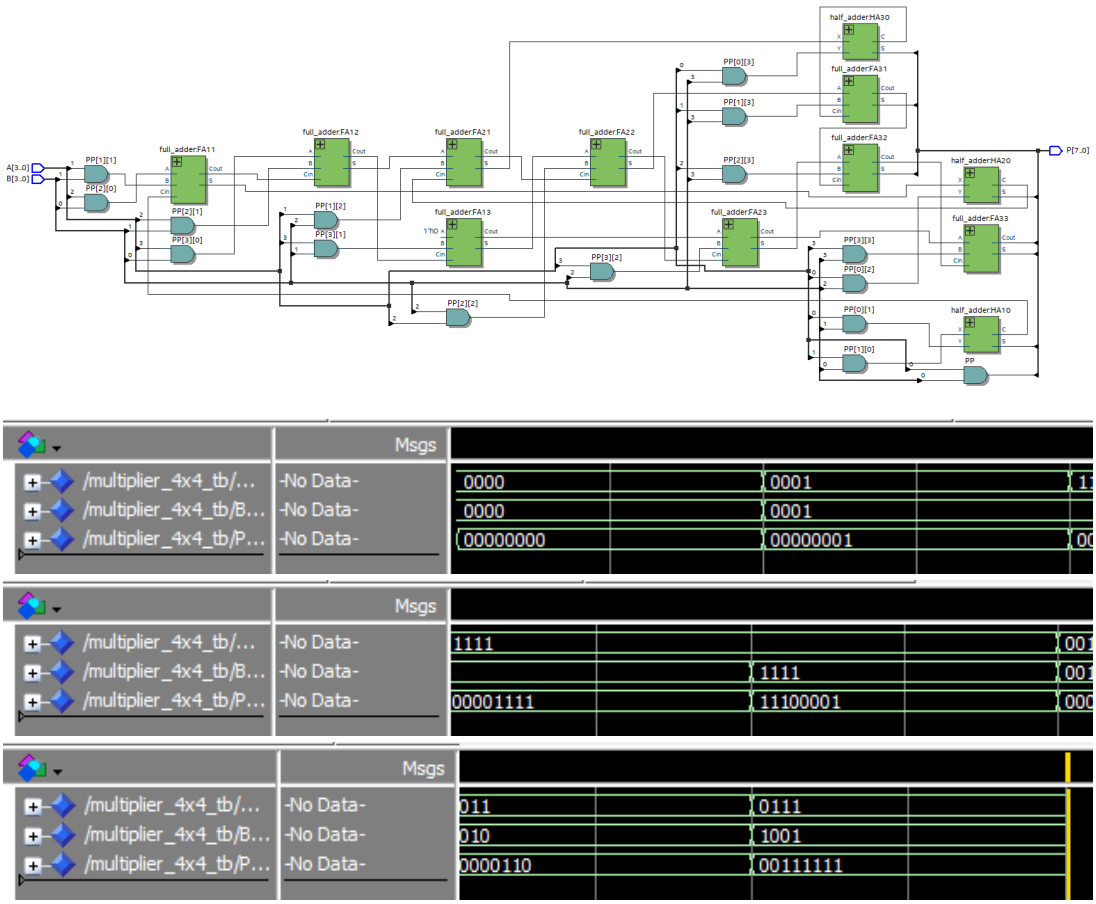



**Figure 4.12.** 4 × 4 multiplier: RTL viewer (top) and ModelSim simulation (bottom). Author's screenshots.

the range 0 to $2^{n} - 2^{-m}$. Fixed-point arithmetic uses ordinary integer hardware, so it is fast, small and low-power, but the range and precision are limited and the designer must choose the format carefully to avoid overflow and loss of precision.

**Example 4.6: Fixed-point conversion**

Represent $6.75_{10}$ in $Q4.4$ format (8 bits).
**Solution.** The integer part is $6 = \texttt{0110}$. The fraction: $0.75 \times 2 = 1.5 \to 1$; $0.5 \times 2 = 1.0 \to 1$; then $0, 0$. So $0.75 = .\texttt{1100}$ and the number is $\texttt{0110.1100} = \texttt{01101100}$. Check: $4 + 2 + 0.5 + 0.25 = 6.75$.

### 4.7.2 Floating-point numbers (IEEE 754)

Floating point represents a number as a sign, a significand (mantissa) and a power of two, much like scientific notation, and gives a huge dynamic range with a fixed word length. The *IEEE 754* standard [13] defines the formats; the 32-bit *single precision* format (Figure 4.13) has

- a **sign** bit $S$ (0 positive, 1 negative);
- an 8-bit **exponent** $E$ in *excess-127* form (true exponent $= E - 127$);
- a 23-bit **fraction** $M$. A normalised number has the form $1.M$; the leading 1 is not stored (the *hidden bit*), which gives 24 bits of precision.

$$V = (-1)^{S} \times (1.M)_{2} \times 2^{E-127}. \tag{4.4}$$

The exponent values $E = 0$ and $E = 255$ are reserved for zero and subnormal numbers, and for infinity and *not-a-number* (NaN). Normal single-precision numbers range from about $1.2 \times 10^{-38}$ to $3.4 \times 10^{38}$ with about seven significant decimal digits. A useful discussion of the pitfalls of floating-point arithmetic is given by Goldberg [14].

**Example 4.7: Encoding 3.5 and −6.25**

**3.5.** $3.5 = \texttt{11.1} = 1.11 \times 2^{1}$. $S = 0$; $E = 1{+}127 = 128 = \texttt{10000000}$; $M = \texttt{11}$ followed by 21 zeros. The word is $\texttt{0 10000000 11000000000000000000000}$, or $40600000_{16}$.
**−6.25.** $6.25 = \texttt{110.01} = \texttt{1.1001} \times 2^{2}$. $S = 1$; $E = 2 + 127 = 129 = \texttt{10000001}$; $M = \texttt{1001}$ then zeros. The word is $\texttt{1 10000001 10010000000000000000000} =$ $\mathrm{C0C80000}_{16}$.

## 4.8 Codes for Decimal Digits and Characters

**Binary-coded decimal (BCD).** *BCD* encodes each decimal digit in four bits: $0 = \texttt{0000}$ up to $9 = \texttt{1001}$; the codes $\texttt{1010}$ to $\texttt{1111}$ are unused (and are natural don't-cares in a K-map). For example $78_{10} = \texttt{0111 1000}_{\mathrm{BCD}}$, whereas the binary value of 78 is $\texttt{0100 1110}$. BCD wastes some codes and needs special adders (Chapter 5), but it makes conversion to a display trivial, which is why it is used in calculators and digital clocks

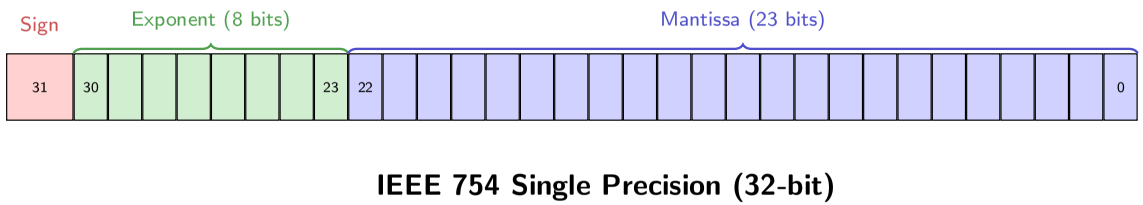



**Figure 4.13.** The IEEE 754 single-precision format.

(Figure 4.14).

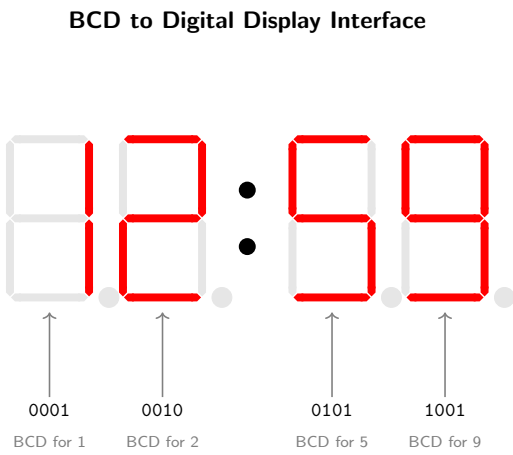



**Figure 4.14.** BCD digits drive the displays of a digital clock.

**ASCII.** The *American Standard Code for Information Interchange* [1] assigns 7-bit codes to 128 characters: codes 0–31 (and 127) are non-printing control characters, and codes 32–126 are the space, punctuation, the digits and the upper- and lower-case letters. Most modern systems use 8-bit extensions, and ASCII is the first 128 code points of Unicode and UTF-8. Some codes are listed in Table 4.4. Note that lower-case letters differ from the corresponding upper-case letters by exactly one bit (bit 5: a = A + 32), and that the digit characters have the decimal value in their low nibble ('7' = $37_{16}$).

**Table 4.4.** Example ASCII codes.

| Char | Dec | Hex | Binary | Char | Dec | Hex | Binary |
|---|---|---|---|---|---|---|---|
| A | 65 | 41 | 0100 0001 | 0 | 48 | 30 | 0011 0000 |
| B | 66 | 42 | 0100 0010 | 1 | 49 | 31 | 0011 0001 |
| a | 97 | 61 | 0110 0001 | space | 32 | 20 | 0010 0000 |
| b | 98 | 62 | 0110 0010 | ! | 33 | 21 | 0010 0001 |

**Chapter Summary**

- Binary, octal and hexadecimal are related by grouping bits in threes and fours. Convert from decimal by repeated division (integer part) and multiplication (fraction).
- **Two's complement** is the standard signed representation: negate by inverting and adding 1; range $-2^{n-1} \ldots 2^{n-1} - 1$; one zero; the same adder works for signed and unsigned numbers. Overflow $= c_{n-1} \oplus c_n$.
- A **full adder** computes $S = A \oplus B \oplus C_{in}$ and $C_{out} = AB + C_{in}(A \oplus B)$. $n$ of them

form a **ripple-carry adder**, whose delay grows linearly with $n$.

- The **adder/subtractor** uses XOR gates to complement $B$ and the control signal as the first carry-in.
- The **carry-lookahead adder** uses $G_i = A_i B_i$ and $P_i = A_i \oplus B_i$ to compute all carries in parallel; a hierarchical design reduces a 16-bit delay from about $32t$ to about $8t$.
- An **array multiplier** forms partial products with AND gates and adds them with an array of adders.
- **Fixed-point** $(Qn.m)$ is cheap but has limited range; **IEEE 754** single precision is $(-1)^S\, 1.M \times 2^{E-127}$. **BCD** and **ASCII** encode digits and characters.

## Exercises

**4.1** **Number conversion.** (a) Convert the decimal number $-75$ to an 8-bit two's-complement binary number and to hexadecimal. (b) What is the decimal value of the 8-bit two's-complement number `1011 0101`?

**4.2** **Two's-complement arithmetic.** Let $A =$ `0110 1001` and $B =$ `1101 0110` be 8-bit two's-complement numbers. (a) Calculate $A + B$. Is there an overflow? (b) Calculate $A - B$ (as $A + (-B)$). Is there an overflow?

**4.3** **Adder delay.** (a) What is the approximate worst-case delay of a 16-bit ripple-carry adder if each full adder contributes two gate delays on the carry path? (b) Define the propagate $P_i$ and generate $G_i$ signals of a carry-lookahead adder and explain why they are useful.

**4.4** **8-bit adder in VHDL.** Using the entity `ripple_adder_4bit` of Listing 4.2 as a component, write the structural VHDL of an 8-bit ripple-carry adder with inputs `A, B : std_logic_vector(7 downto 0)` and `Cin`, and outputs `S` and `Cout`. How many internal signals do you need to connect the two blocks?

**4.5** Convert (a) $1011\,0110_2$ to decimal and hexadecimal; (b) $A7_{16}$ to binary and decimal; (c) $725_8$ to binary and decimal; (d) $0.8125_{10}$ to binary.

**4.6** Write $+19$, $-19$ and $-32$ as 6-bit two's-complement numbers. What is the range of 6-bit two's-complement numbers?

**4.7** Perform the following in 5-bit two's complement, and state whether overflow occurs: (a) $9 + 7$; (b) $(-9) + (-8)$; (c) $12 - 5$.

**4.8** Show that for two's-complement addition, $S = A + B$, overflow also occurs if and only if the two operands have the same sign and the sign of the result is different, i.e. $V = A_{n-1} B_{n-1} S'_{n-1} + A'_{n-1} B'_{n-1} S_{n-1}$.

**4.9** Show from the truth table that $C_{out} = AB + AC_{in} + BC_{in}$ for the full adder, and derive the same expression from Equation (4.2). Draw the corresponding two-level circuit.

**4.10** For $A =$ `1011`, $B =$ `0110` and $C_0 = 1$, compute $G_i$, $P_i$, $C_1, \ldots, C_4$ and $S_3 S_2 S_1 S_0$

using the carry-lookahead equations. Verify the result by ordinary addition.

**4.11** Multiply `1011` × `0110` by forming the partial products. Verify the result in decimal.

**4.12** (a) Represent 10.3125 in $Q4.4$. (b) What decimal value does the $Q4.4$ word $5C_{16}$ represent? (c) What is the resolution of a $Q8.8$ number, and what is its range?

**4.13** (a) Convert 0.15625 to IEEE 754 single precision, in hexadecimal. (b) What number does the word $41480000_{16}$ represent?

**4.14** (a) Write $2026_{10}$ in BCD. How does it differ from the binary representation? (b) How many bits are needed to store a 6-digit decimal number in BCD and in binary?

**4.15** (a) Write the ASCII codes (in hexadecimal) of the string `Hi!`. (b) What simple operation converts the ASCII code of a decimal digit character to the number it represents? (c) What single-bit operation converts an upper-case letter to lower case?

**4.16** Add an *overflow* output to the adder/subtractor of Figure 4.5. Which internal signals of the ripple-carry adder do you need, and which gate? What change does this require in the entity `ripple_adder_4bit` of Listing 4.2?

## Further Reading

Number systems and arithmetic circuits are treated in Chapter 5 of Brown and Vranesic [3] and Chapters 1 and 4 of Mano and Ciletti [7]. For the numerical behaviour of floating-point arithmetic read the paper by Goldberg [14] and the standard [13]. Hwang [15] describes a VHDL carry-lookahead adder in detail.

# 5

# Combinational Circuit Building Blocks

**Learning Objectives**

After studying this chapter you will be able to:

- analyse a given combinational circuit to find its Boolean functions, truth table and purpose, and follow a systematic procedure to design a new one;
- design a code converter, such as BCD to excess-3, and a decimal (BCD) adder;
- explain the operation of decoders, demultiplexers, encoders and priority encoders, and build larger ones from smaller ones;
- use a multiplexer to implement any Boolean function, and state Shannon's expansion theorem;
- describe the structure of a simple arithmetic logic unit (ALU);
- explain three-state gates and why buses need them; and
- design a magnitude comparator, and describe all of these blocks in VHDL.

## 5.1 Combinational and Sequential Circuits

Digital circuits are of two kinds. In a *combinational circuit* the outputs depend *only on the present values of the inputs*: the circuit has no memory, and its behaviour is completely described by a truth table with $2^n$ rows for $n$ inputs. An adder is the typical example. In a *sequential circuit* the outputs depend on the inputs *and on the state* held in memory elements (flip-flops), and therefore on past inputs; a counter is the typical example. Sequential circuits are the subject of Chapters 6–8.

The combinational circuits of this chapter are so frequently needed that they are provided as standard functions: as medium-scale-integration (MSI) chips in the 74xx se-

ries, as library cells, and as ready-made components in any HDL design tool. We study them in order to use them and to understand what a synthesis tool produces. Figure 5.1 shows a combinational circuit as a block with $n$ inputs and $m$ outputs.

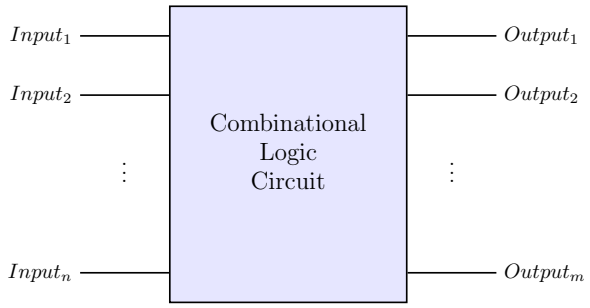



**Figure 5.1.** A combinational circuit has $n$ input variables and $m$ output variables.

## 5.2 Analysis of a Combinational Circuit

*Analysis* determines what function a given circuit implements.

1. **Check that the circuit is combinational:** it must have no feedback paths and no memory elements.
2. **Derive Boolean functions.** Label every gate output with a symbol; write the function at each gate output, starting at the inputs and working towards the outputs; then substitute to obtain the output functions in terms of the primary inputs only.
3. **Make a truth table** by evaluating the output functions for all $2^n$ input combinations.
4. **Explain the function** by observing the pattern in the truth table.

**Example 5.1: Analysis of a circuit**

Analyse the circuit of Figure 5.2, which has inputs $A, B, C$ and outputs $F_1, F_2$.
**Solution.** Labelling the internal signals $T_1, T_2, T_3$ as in the figure:

$$F_2 = AB + AC + BC, \qquad T_1 = A + B + C, \qquad T_2 = ABC,$$
$$T_3 = F_2' T_1, \qquad F_1 = T_3 + T_2.$$

Substituting, $F_1 = (AB + AC + BC)'(A + B + C) + ABC$. The first term is 1 exactly when at least one input is 1 but fewer than two are (that is, exactly one input is 1), and $ABC$ covers the case where all three are 1. Hence

$$F_1 = A'B'C + A'BC' + AB'C' + ABC = A \oplus B \oplus C.$$

The truth table (Table 5.1) agrees: $F_1$ is the *sum* bit and $F_2$ the *carry* bit of $A+B+C$.

The circuit is a **full adder**.

**Table 5.1.** Truth table obtained in Example 5.1.

| $A$ | $B$ | $C$ | $F_2$ (carry) | $F_1$ (sum) |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

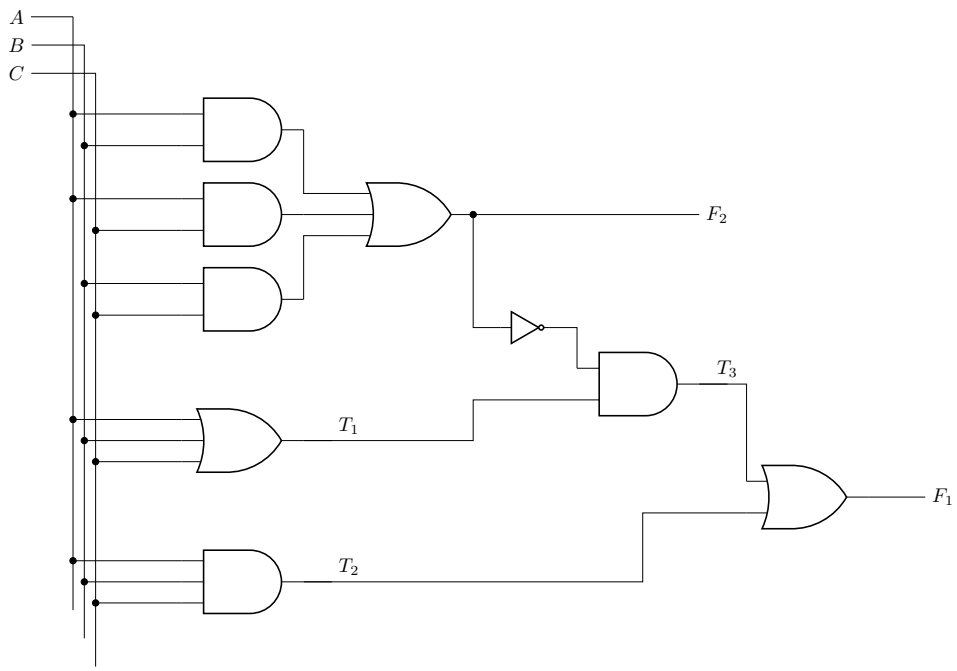



**Figure 5.2.** The circuit analysed in Example 5.1.

## 5.3 Design Procedure

*Design* is the inverse process: from a specification to a circuit.

1. State the problem and understand the required behaviour.
2. Determine the number of inputs and outputs; the inputs fix the truth-table size ($2^n$ rows).

3. Assign symbols to the inputs ($A, B, C, \ldots$ or $x, y, z, \ldots$) and outputs ($F_1, F_2, \ldots$).
4. Derive the truth table.
5. Derive and simplify the Boolean function of each output (K-maps or algebra).
6. Draw the logic diagram (or write the HDL) and verify it by simulation.

### 5.3.1 Example: BCD to excess-3 converter

The *excess-3 code* of a decimal digit is the 4-bit binary code of the digit plus 3; it is a self-complementing code used in some older decimal arithmetic circuits. We design a converter from BCD to excess-3.

**Example 5.2: BCD to excess-3 code converter**

**Steps 1–3.** The input is a BCD digit $ABCD$ ($A$ is the MSB) and the output is the excess-3 code $wxyz$ (Figure 5.3a). **Step 4.** Excess-3 is the BCD value plus 3 (Table 5.2); inputs 10–15 are not valid BCD and are don't-cares.
**Step 5.** The K-maps with don't-cares (Figure 5.4) give

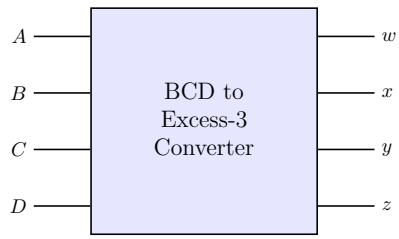
$$w = A + BC + BD = A + B(C + D), \quad x = B'C + B'D + BC'D' = B \oplus (C + D),$$
$$y = CD + C'D' = (C \oplus D)', \qquad z = D'.$$

**Step 6.** The circuit is in Figure 5.3b and its VHDL description in Listing 5.1. For $x$ we used the identity $B'(C + D) + B(C + D)' = B \oplus (C + D)$.

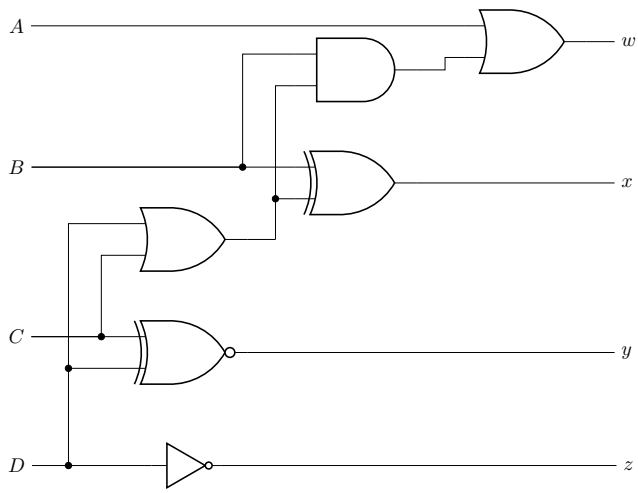
**Table 5.2.** BCD and excess-3 codes.

| Decimal | BCD ($ABCD$) | Excess-3 ($wxyz$) |
|---|---|---|
| 0 | 0000 | 0011 |
| 1 | 0001 | 0100 |
| 2 | 0010 | 0101 |
| 3 | 0011 | 0110 |
| 4 | 0100 | 0111 |
| 5 | 0101 | 1000 |
| 6 | 0110 | 1001 |
| 7 | 0111 | 1010 |
| 8 | 1000 | 1011 |
| 9 | 1001 | 1100 |
| 10–15 | 1010–1111 | xxxx |

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity bcd_to_excess3 is
5     port ( A, B, C, D : in  std_logic;
6            w, x, y, z : out std_logic );
7 end bcd_to_excess3;
```

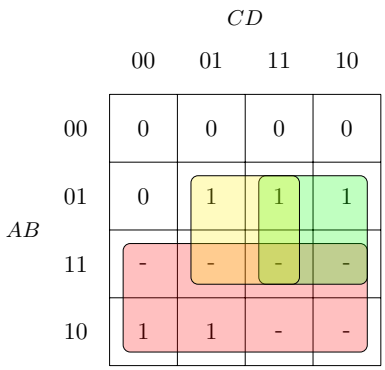



**(a)** block diagram

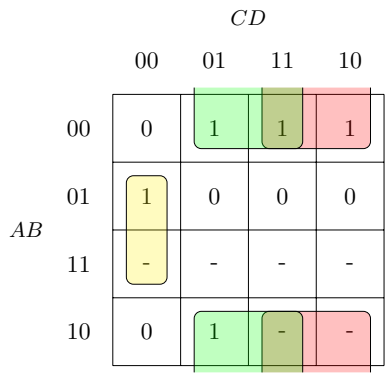



**(b)** logic diagram

**Figure 5.3.** BCD to excess-3 converter of Example 5.2.

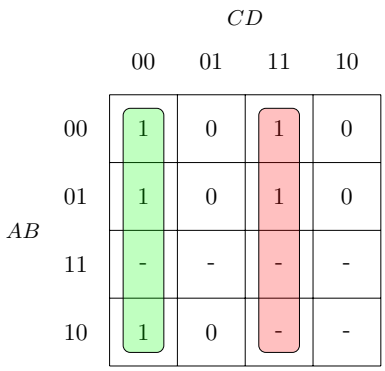



(a) Map for $w = A + BC + BD$

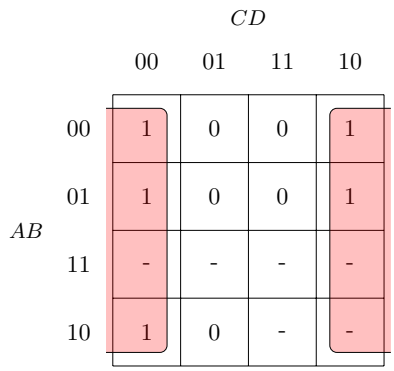



(b) Map for $x = B'C + B'D + BC'D'$

**(a)** K-maps for $w$ and $x$




(c) Map for $y = CD + C'D'$




(d) Map for $z = D'$

**(b)** K-maps for $y$ and $z$

**Figure 5.4.** K-maps for the outputs of the BCD to excess-3 converter; X marks don't-care inputs.

```vhdl

architecture dataflow of bcd_to_excess3 is
begin
    z <= not D;
    y <= C xnor D;
    x <= B xor (C or D);
    w <= A or (B and (C or D));
end dataflow;
```

**Listing 5.1.** VHDL of the BCD to excess-3 converter.

## 5.4 Decimal (BCD) Adder

A *BCD adder* adds two BCD digits (plus a carry-in) and produces a BCD sum digit and a carry-out. A direct design would have 8 inputs (plus carry-in), a truth table with 256 rows and no usable K-map. A better idea is to reuse the 4-bit binary adder:

1. add the two digits with a binary adder; the sum $Z$ (with carry $K$) lies between 0 and 19 ($9+9+1$);
2. if the binary sum is 9 or less it is already the correct BCD digit;
3. if it exceeds 9, add 6 (`0110`): the extra 6 skips the six unused codes 10–15 and produces the correct decimal digit and a carry to the next decade.

A correction is therefore needed if $K = 1$ (sums 16–19) or if $Z \in \{10, \ldots, 15\}$ ($K = 0$). In the latter range $Z_3 = 1$ always, and either $Z_2 = 1$ (12–15) or $Z_1 = 1$ (10, 11, 14, 15). The correction (and the carry-out) is

$$C = K + Z_3 Z_2 + Z_3 Z_1. \tag{5.1}$$

Figure 5.5 shows the arrangement: the first adder forms $Z$, the logic of Equation (5.1) detects sums above 9, and the second adder adds 0110 when $C = 1$ and 0000 otherwise.

**Example 5.3: Adding BCD digits**

$7+5$: the binary sum is `0111`$+$`0101` $=$ `1100` $= 12 > 9$, so add `0110`: `1100`$+$`0110` $=$ `1 0010`. The carry is 1 and the digit is `0010`: the BCD number `0001 0010` $= 12$. $9+8+1$: `1001` $+$ `1000` $+ 1 =$ `1 0010`, so $K = 1$; adding 6 gives `1 1000`, the digit `1000` $= 8$ and carry 1, i.e. 18.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bcd_adder is
    port ( A, B : in  std_logic_vector(3 downto 0); -- Input BCD digits
           Cin  : in  std_logic;                    -- Carry Input
           S    : out std_logic_vector(3 downto 0); -- Sum BCD digit
```

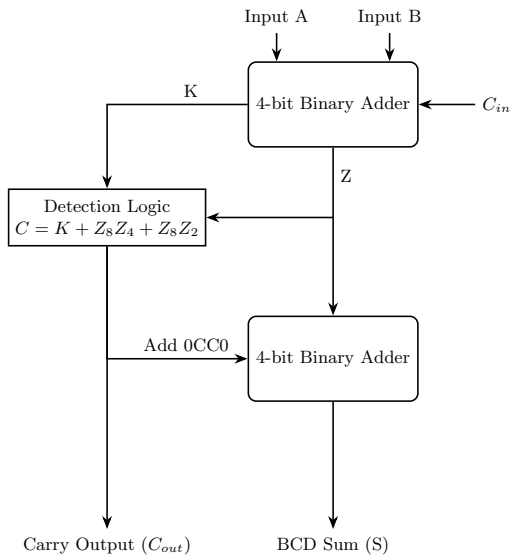



**Figure 5.5.** A one-digit BCD adder: a binary adder, the correction-detection logic and a second adder that adds 6.

```
           Cout : out std_logic );                  -- Carry Output
end bcd_adder;

architecture behavioral of bcd_adder is
    signal sum_temp : unsigned(4 downto 0); -- 5 bits to hold sum up to 19
    signal sum_corr : unsigned(4 downto 0);
begin
    -- 1. Binary Addition
    sum_temp <= resize(unsigned(A), 5) + resize(unsigned(B), 5) + ("0000" & Cin);

    -- 2. Correction Logic (Check if > 9)
    process(sum_temp)
    begin
        if (sum_temp > 9) then
            sum_corr <= sum_temp + 6; -- Add 6 to correct
            Cout <= '1';
        else
            sum_corr <= sum_temp;
            Cout <= '0';
        end if;
    end process;

    -- 3. Output Sum
    S <= std_logic_vector(sum_corr(3 downto 0));
end behavioral;
```

**Listing 5.2.** Behavioural VHDL of a one-digit BCD adder. The package `numeric_std` provides the type `unsigned` and arithmetic on it.

## 5.5 Decoders

A *decoder* converts binary information from $n$ input lines to at most $2^n$ unique output lines, so that for each input combination exactly one output is active. A decoder with $n$ inputs and $m \leq 2^n$ outputs is called an $n$-to-$m$-line decoder. Table 5.3 gives the truth table of a 3-to-8 decoder, which is simply a generator of all eight *minterms* of the three inputs: output $D_i$ is the minterm $m_i$. Figure 5.6 shows the block and its AND-gate implementation, and Listing 5.3 the VHDL.

**Table 5.3.** Truth table of a 3-to-8-line decoder (active-high outputs).

| $x$ | $y$ | $z$ | $D_0$ | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | **1** | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | **1** | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | **1** | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | **1** | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | **1** | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | **1** | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | **1** |

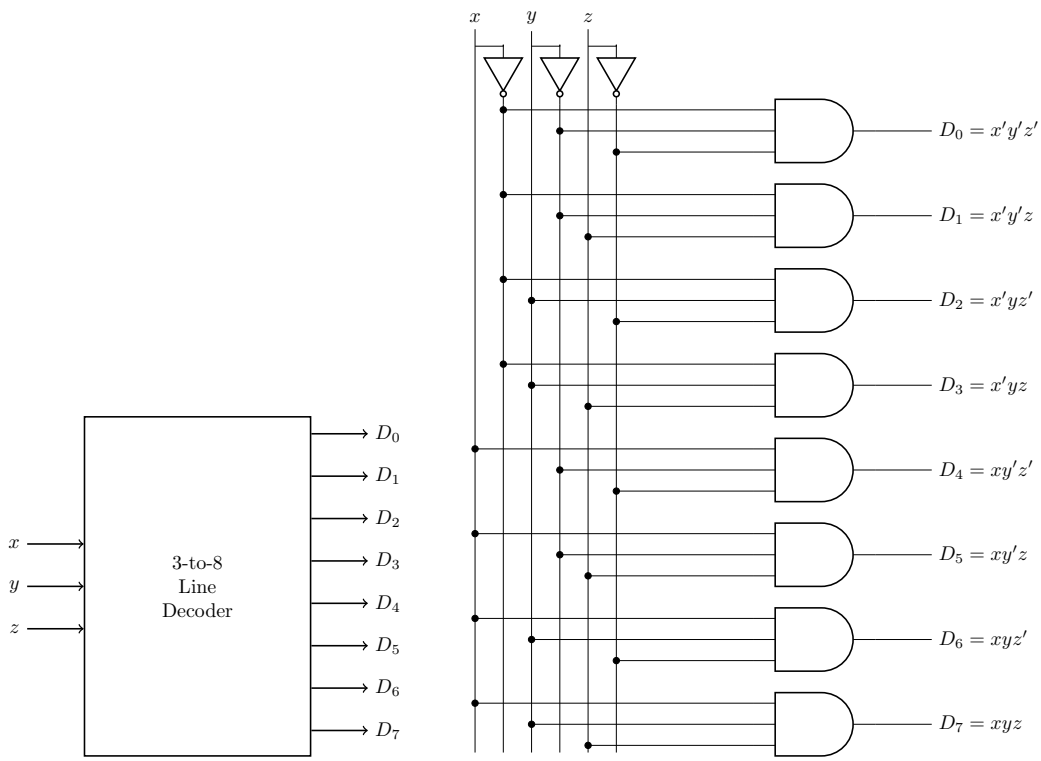
```
library ieee;
use ieee.std_logic_1164.all;

entity decoder_3to8 is
    port ( I : in  std_logic_vector(2 downto 0);
           D : out std_logic_vector(7 downto 0) );
end decoder_3to8;

architecture dataflow of decoder_3to8 is
begin
    D(0) <= not I(2) and not I(1) and not I(0);
    D(1) <= not I(2) and not I(1) and     I(0);
    D(2) <= not I(2) and     I(1) and not I(0);
    D(3) <= not I(2) and     I(1) and     I(0);
    D(4) <=     I(2) and not I(1) and not I(0);
    D(5) <=     I(2) and not I(1) and     I(0);
    D(6) <=     I(2) and     I(1) and not I(0);
    D(7) <=     I(2) and     I(1) and     I(0);
end dataflow;
```

**Listing 5.3.** A 3-to-8 decoder in VHDL.

**Decoder with enable.** Decoders usually have an *enable* input $E$. In the common NAND-gate implementation of Table 5.4 the outputs are active-low and the decoder works when $E = 0$; when $E = 1$ all outputs are inactive (1). Figure 5.7a shows the circuit. Enable inputs allow decoders to be *cascaded*: two 3-to-8 decoders form a 4-to-16




**(a)** block diagram **(b)** logic diagram

**Figure 5.6.** A 3-to-8-line decoder.

decoder if the most significant input drives the enable of one decoder directly and of the other through an inverter (Figure 5.7b).

**Table 5.4.** 3-to-8 decoder with active-low enable $E$ and active-low outputs.

| $E$ | $x$ | $y$ | $z$ | $D_0$ | $D_1$ | $D_2$ | $D_3$ | $D_4$ | $D_5$ | $D_6$ | $D_7$ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | X | X | X | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | **0** | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 1 | 1 | **0** | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 | **0** | 1 | 1 | 1 | 1 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | **0** | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | **0** | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | **0** | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | **0** | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | **0** |

**Decoders implement functions**

Because the outputs of a decoder are the minterms of its inputs, any combinational function of $n$ variables can be implemented by an $n$-to-$2^n$ decoder and an OR gate that collects the minterms for which $f = 1$. For example the full adder is $S = \sum m(1,2,4,7)$ and $C_{out} = \sum m(3,5,6,7)$: one 3-to-8 decoder and two 4-input OR gates.

### 5.5.1 Demultiplexers

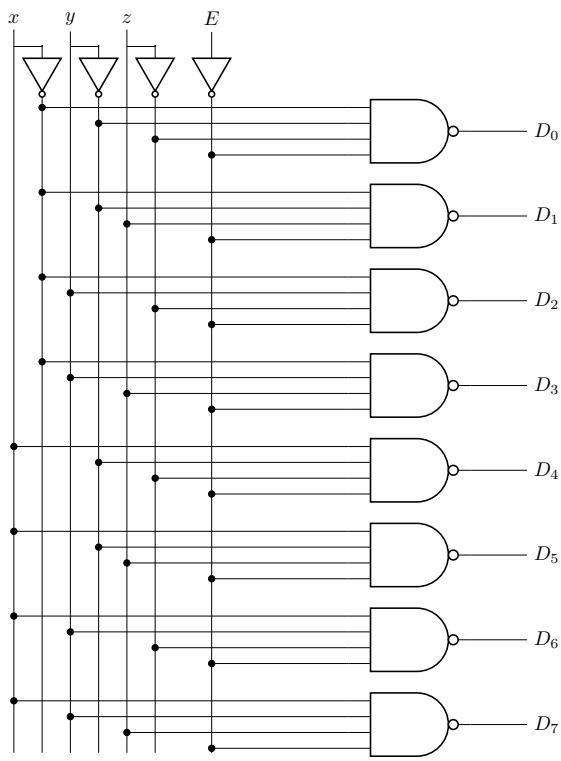
A *demultiplexer* (DEMUX) does the opposite of a multiplexer: it takes data from a single input line and routes it to one of $2^n$ output lines selected by $n$ select lines. A decoder with an enable input is a demultiplexer: the *enable* becomes the *data input* and the decoder inputs become the select inputs. Table 5.5 and Figure 5.8 show a 1-to-4 demultiplexer.

**Table 5.5.** 1-to-4 demultiplexer.

| Data | $S_1$ | $S_0$ | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
|---|---|---|---|---|---|---|
| $D$ | 0 | 0 | $D$ | 0 | 0 | 0 |
| $D$ | 0 | 1 | 0 | $D$ | 0 | 0 |
| $D$ | 1 | 0 | 0 | 0 | $D$ | 0 |
| $D$ | 1 | 1 | 0 | 0 | 0 | $D$ |

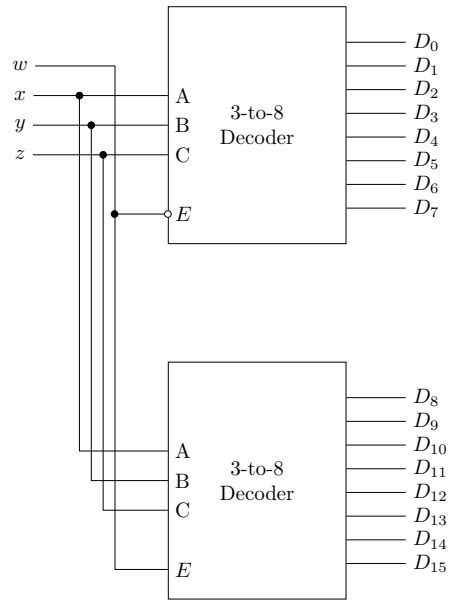
## 5.6 Encoders

An *encoder* performs the inverse of a decoder: it has $2^n$ (or fewer) input lines and $n$ output lines, and produces the binary code of the input line that is active. It is assumed that only one input is active at a time. For the octal-to-binary encoder with inputs




**(a)** 3-to-8 decoder with enable

**Figure 5.7.** Decoder with enable input and cascading.




**(b)** 4-to-16 decoder from two 3-to-8 decoders

**Figure 5.7.** Decoder with enable input and cascading.

$D_0, \ldots, D_7$ and outputs $xyz$ (Figure 5.9),

$$x = D_4 + D_5 + D_6 + D_7, \qquad y = D_2 + D_3 + D_6 + D_7, \qquad z = D_1 + D_3 + D_5 + D_7.$$

Each output is the OR of the inputs whose code has a 1 in that bit position. The simple encoder has two defects: the output is undefined if more than one input is active, and the all-zero output code is produced both by $D_0$ and by no input at all.

### 5.6.1 Priority encoders

A *priority encoder* gives each input a priority and outputs the code of the active input with the *highest priority*. It also has a *valid* output $V$ that shows whether any input is active, which resolves the second defect. Table 5.6 is the truth table of a 4-input priority encoder in which $D_3$ has the highest priority; X means don't-care. From it,

$$x = D_3 + D_2, \qquad y = D_3 + D_2' D_1, \qquad V = D_3 + D_2 + D_1 + D_0,$$

and Figure 5.10 is the circuit. Priority encoders are used, for example, to arbitrate interrupt requests.

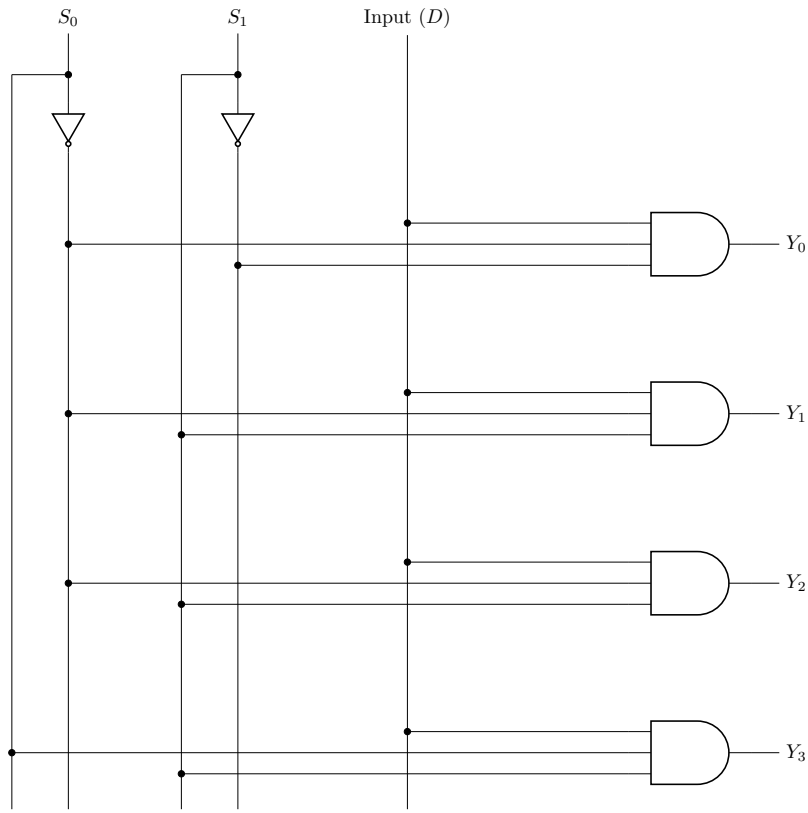



**Figure 5.8.** A 1-to-4 demultiplexer.

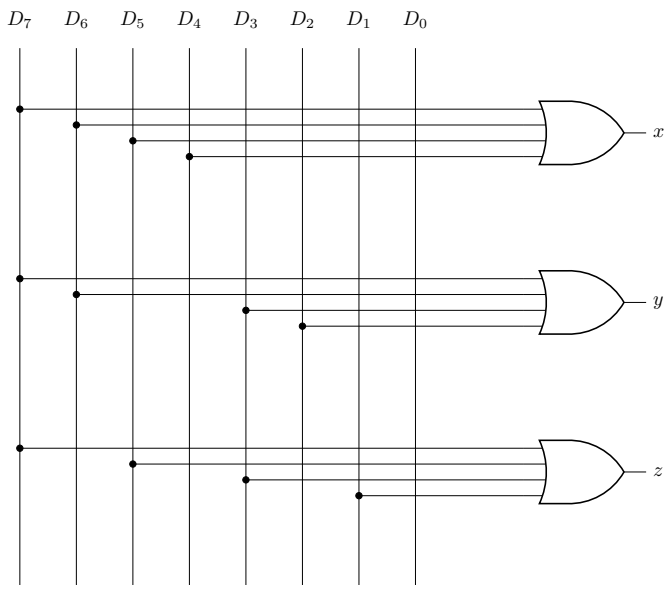



**Figure 5.9.** An octal-to-binary (8-to-3) encoder.

**Table 5.6.** 4-to-2 priority encoder ($D_3$ has the highest priority).

| $D_3$ | $D_2$ | $D_1$ | $D_0$ | $x$ | $y$ | $V$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | X | X | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | X | 0 | 1 | 1 |
| 0 | 1 | X | X | 1 | 0 | 1 |
| 1 | X | X | X | 1 | 1 | 1 |

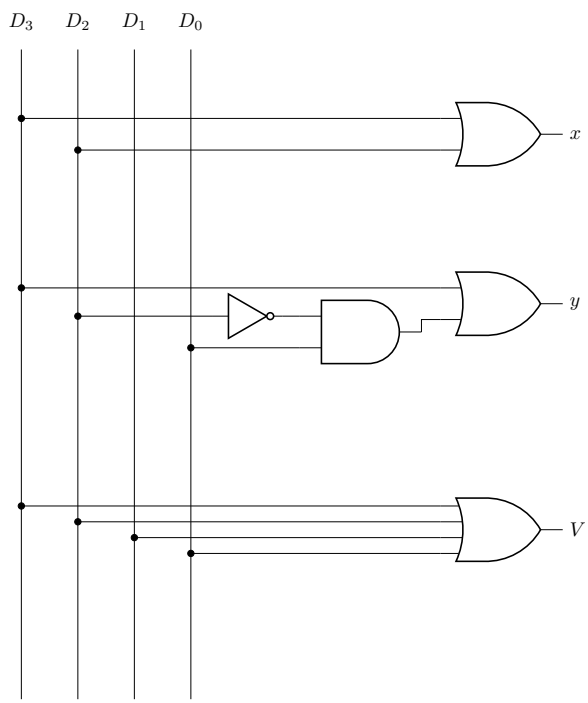



**Figure 5.10.** A 4-to-2 priority encoder.

## 5.7 Multiplexers

A *multiplexer* (MUX, or data selector) selects one of $2^n$ data inputs and connects it to a single output; $n$ *select* lines determine which. We met the 2-to-1 multiplexer in Example 2.4. For the 4-to-1 multiplexer (Table 5.7, Figure 5.11),

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3.$$

Listing 5.4 describes it behaviourally, as a selection (`when ... else`); Listing 5.5 gives the same function as a Boolean expression. Both produce the same hardware after synthesis, but the behavioural style is shorter and much easier to read and to scale to wider multiplexers.

**Table 5.7.** 4-to-1 multiplexer.

| $S_1$ | $S_0$ | $Y$ |
|---|---|---|
| 0 | 0 | $I_0$ |
| 0 | 1 | $I_1$ |
| 1 | 0 | $I_2$ |
| 1 | 1 | $I_3$ |

```
library ieee;
use ieee.std_logic_1164.all;

entity mux_4to1 is
    port ( I : in  std_logic_vector(3 downto 0);
           S : in  std_logic_vector(1 downto 0);
           Y : out std_logic );
end mux_4to1;

architecture behavioral of mux_4to1 is
begin
    Y <= I(0) when S = "00" else
         I(1) when S = "01" else
         I(2) when S = "10" else
         I(3);
end behavioral;
```

**Listing 5.4.** 4-to-1 multiplexer, behavioural architecture.

```
library ieee;
use ieee.std_logic_1164.all;

entity mux_4to1 is
    port ( I : in  std_logic_vector(3 downto 0);
           S : in  std_logic_vector(1 downto 0);
           Y : out std_logic );
end mux_4to1;
```

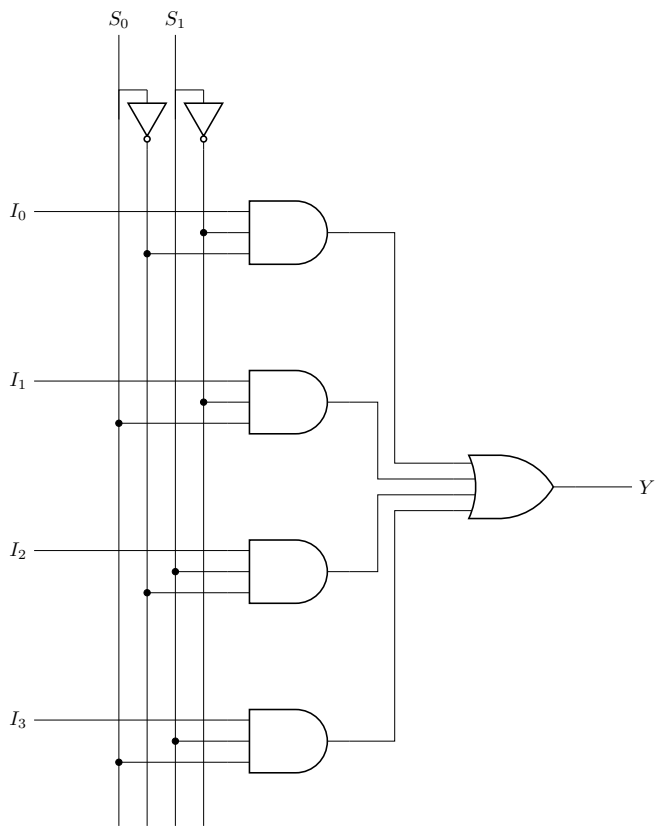



**Figure 5.11.** Logic diagram of a 4-to-1 multiplexer.

```
9
10 architecture structural of mux_4to1 is
11 begin
12     Y <= (I(0) and not S(1) and not S(0)) or
13          (I(1) and not S(1) and     S(0)) or
14          (I(2) and     S(1) and not S(0)) or
15          (I(3) and     S(1) and     S(0));
16 end structural;
```

**Listing 5.5.** 4-to-1 multiplexer written as a Boolean expression ("structural" in the lecture notes).

### 5.7.1 Implementing Boolean functions with multiplexers

A multiplexer is a *universal logic module*: any Boolean function of $n$ variables can be implemented by a multiplexer with $n-1$ select lines.

1. Connect $n-1$ of the variables to the select inputs.
2. Using an *implementation table*, express $F$ for each select combination as a function of the remaining variable $D$. It is always one of $D$, $D'$, 0 or 1.
3. Connect each data input $I_k$ to the corresponding $D$, $D'$, 0 or 1.

**Example 5.4: Function with an 8-to-1 multiplexer**

Implement $F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$ with an 8-to-1 multiplexer.

**Solution.** Connect $A, B, C$ to the select inputs $S_2, S_1, S_0$. For each $ABC$ pair of rows, see where $F = 1$ (Table 5.8). The data inputs are

$$I_0 = D,\ I_1 = D,\ I_2 = D',\ I_3 = 0,\ I_4 = 0,\ I_5 = D,\ I_6 = 1,\ I_7 = 1.$$

The circuit is Figure 5.12.

**Table 5.8.** Implementation table for Example 5.4.

| $A$ | $B$ | $C$ | $F$ for $D=0$ | $F$ for $D=1$ | Data input |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | $I_0 = D$ |
| 0 | 0 | 1 | 0 | 1 | $I_1 = D$ |
| 0 | 1 | 0 | 1 | 0 | $I_2 = D'$ |
| 0 | 1 | 1 | 0 | 0 | $I_3 = 0$ |
| 1 | 0 | 0 | 0 | 0 | $I_4 = 0$ |
| 1 | 0 | 1 | 0 | 1 | $I_5 = D$ |
| 1 | 1 | 0 | 1 | 1 | $I_6 = 1$ |
| 1 | 1 | 1 | 1 | 1 | $I_7 = 1$ |

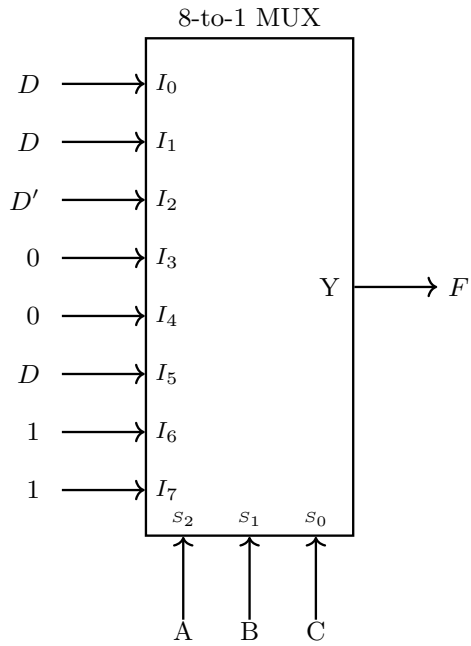



**Figure 5.12.** Example 5.4: a four-variable function with an 8-to-1 multiplexer.

### 5.7.2 Shannon's expansion theorem

*Shannon's expansion theorem* states that any Boolean function can be split about any variable $x_i$:

$$F(x_1, \dots, x_n) = x_i' F(x_1, \dots, 0, \dots, x_n) + x_i F(x_1, \dots, 1, \dots, x_n). \tag{5.2}$$

The two terms are the *cofactors* of $F$ with respect to $x_i$. Equation (5.2) is exactly the equation of a 2-to-1 multiplexer with select input $x_i$ and data inputs the two cofactors. Repeating the expansion for further variables gives a tree of 2-to-1 multiplexers: any $n$-variable function can be built from at most $2^{n-1} - 1$ of them, with the last variable (or a constant) at the leaves.

**Example 5.5: Multiplexer tree**

Implement $F(A, B, C) = AB + C$ with 2-to-1 multiplexers.

**Solution.** Expanding on $A$: $F_{A'} = C$ and $F_A = B + C$. Expanding $F_A$ on $B$: $F_{AB'} = C$ and $F_{AB} = 1$. Hence one multiplexer selected by $B$ with inputs $(C, 1)$ produces $F_A$, and a second one selected by $A$ with inputs $(C, F_A)$ produces $F$ (Figure 5.13); two multiplexers suffice.

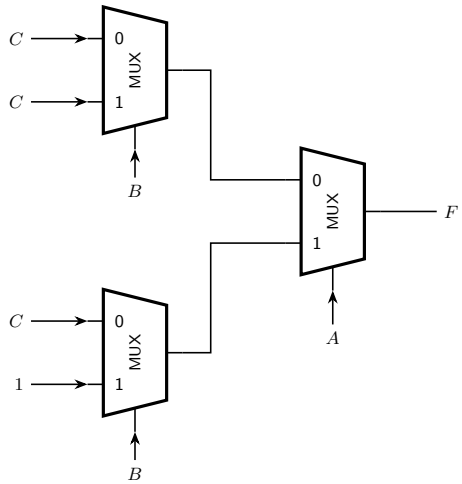



**Figure 5.13.** Multiplexer tree for a function of three variables.

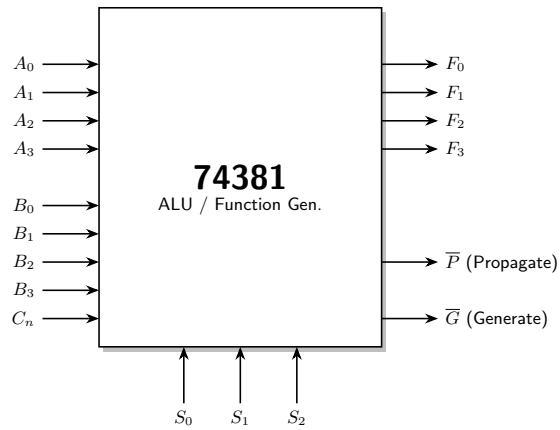
## 5.8 The Arithmetic Logic Unit

An *arithmetic logic unit* (ALU) is a combinational circuit that performs a set of arithmetic and logic operations on two binary operands, the operation being chosen by select inputs. It is the computational core of every processor. A classic example is the 4-bit 74381 ALU (Figure 5.14a), which performs the eight operations of Table 5.9 on operands $A$ and $B$ under the control of three inputs $S_2S_1S_0$. It also produces *group propagate* and *group generate* outputs, which serve as the inputs $P^*$ and $G^*$ of a carry-lookahead generator (Section 4.5), so that wider ALUs can be built without a long carry delay.

**Table 5.9.** Function table of the 74381 ALU.

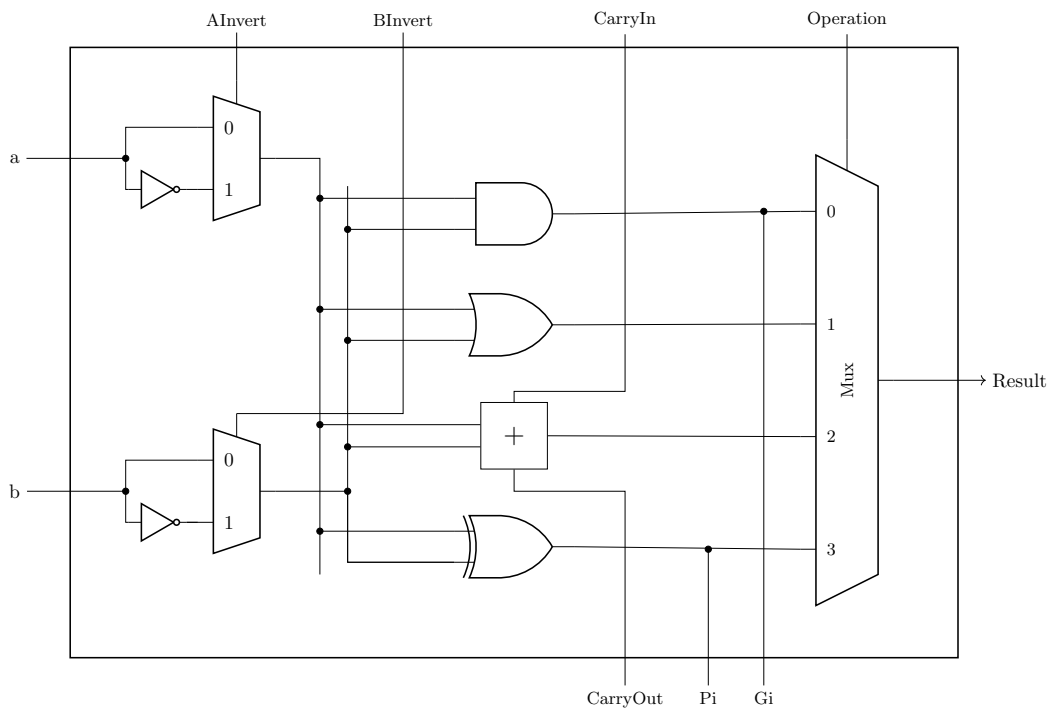
| $S_2$ | $S_1$ | $S_0$ | Function | Operation |
|---|---|---|---|---|
| 0 | 0 | 0 | Clear | $F = 0000$ |
| 0 | 0 | 1 | Subtract | $F = B - A$ |
| 0 | 1 | 0 | Subtract | $F = A - B$ |
| 0 | 1 | 1 | Add | $F = A + B$ |
| 1 | 0 | 0 | XOR | $F = A \oplus B$ |
| 1 | 0 | 1 | OR | $F = A + B$ (logical) |
| 1 | 1 | 0 | AND | $F = A \cdot B$ |
| 1 | 1 | 1 | Preset | $F = 1111$ |

Subtraction uses two's complement ($A+\overline{B}+1$, Section 4.4). Each bit slice (Figure 5.14b) contains a logic unit (AND, OR, XOR), an arithmetic unit (a full adder in propagate/generate form, $P_i = A_i \oplus B_i$, $G_i = A_iB_i$) and a multiplexer, controlled by the select inputs, that chooses the result of the slice.




**(a)** block symbol

**Figure 5.14.** The 74381 ALU: symbol and one bit slice.




**(b)** simplified 1-bit slice

**Figure 5.14.** The 74381 ALU: symbol and one bit slice.

## 5.9 Three-State Gates and Buses

A *three-state gate* (tri-state buffer) has three output states: logic 0, logic 1 and *high impedance* (Hi-Z), in which the output is effectively disconnected, like an open circuit. A control input $C$ selects the state: if $C = 1$ the buffer is enabled and Output = Input; if $C = 0$ the output is Hi-Z (Figure 5.15).

Three-state gates make it possible to connect several devices to a common wire, a *bus*, so long as *only one device drives the bus at a time*. Two ordinary gates cannot drive the same wire: if one outputs 1 and the other 0 there is a low-resistance path between the supply and ground through the two output stages, a condition called *bus contention* that causes a large current and an undefined voltage. Listing 5.6 shows the buffer in VHDL; the value `'Z'` of `std_logic` stands for high impedance. A three-state output can only be described behaviourally, because high impedance is a property of the output driver and cannot be formed from AND/OR/NOT gates.

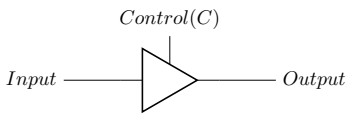



**Figure 5.15.** A three-state buffer with enable input $C$.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity tri_state_buffer is
    port ( Input   : in  std_logic;
           Control : in  std_logic;
           Output  : out std_logic );
end tri_state_buffer;

architecture behavioral of tri_state_buffer is
begin
    Output <= Input when (Control = '1') else 'Z';
end behavioral;
```

**Listing 5.6.** A three-state buffer.

## 5.10 Magnitude Comparator

A *magnitude comparator* compares two binary numbers $A$ and $B$ and produces three outputs: $A > B$, $A = B$ and $A < B$. The comparison proceeds from the most significant bit. Let $x_i = A_i \odot B_i = (A_i \oplus B_i)'$ be 1 when the bits at position $i$ are equal. Then, for

4-bit numbers,

$$
\begin{aligned}
(A = B) &= x_3x_2x_1x_0, \\
(A > B) &= A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0', \\
(A < B) &= A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0.
\end{aligned}
$$

$A > B$ if at some position $A_i = 1$, $B_i = 0$ and all more significant bits are equal. The circuit is Figure 5.16; Listings 5.7 and 5.8 describe it behaviourally, and structurally with the equations above.

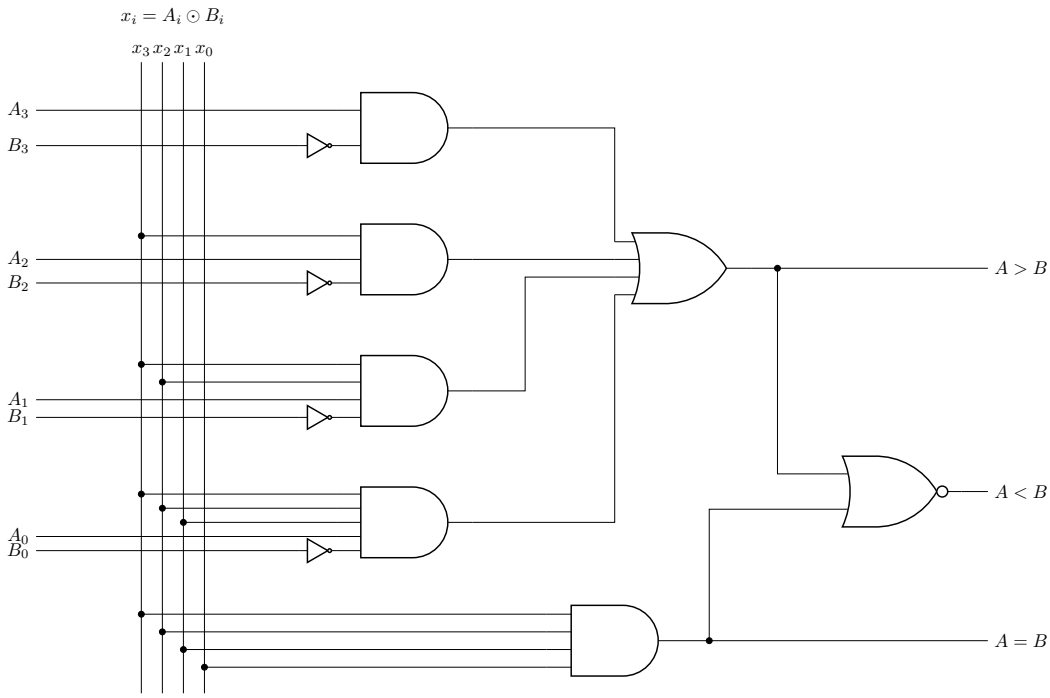



**Figure 5.16.** A 4-bit magnitude comparator.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity magnitude_comparator is
    port ( A : in  std_logic_vector(3 downto 0);
           B : in  std_logic_vector(3 downto 0);
           A_Greater_B : out std_logic;
           A_Equal_B : out std_logic;
           A_Less_B : out std_logic );
end magnitude_comparator;

architecture behavioral of magnitude_comparator is
begin
    A_Greater_B <= '1' when A > B else '0';
```

```
    A_Equal_B <= '1' when A = B else '0';
    A_Less_B <= '1' when A < B else '0';
end behavioral;
```

**Listing 5.7.** Behavioural description of the comparator using the relational operators.

```
library ieee;
use ieee.std_logic_1164.all;

entity magnitude_comparator is
    port ( A : in  std_logic_vector(3 downto 0);
           B : in  std_logic_vector(3 downto 0);
           A_Greater_B : out std_logic;
           A_Equal_B : out std_logic;
           A_Less_B : out std_logic );
end magnitude_comparator;

architecture structural of magnitude_comparator is
    signal x : std_logic_vector(3 downto 0);
    signal term0, term1, term2, term3 : std_logic;
    signal A_Greater_B_int, A_Equal_B_int : std_logic;
begin
    -- Equivalence (x_i = A_i XNOR B_i)
    x <= not (A xor B);

    -- Terms for A > B (AND Gates)
    term3 <= A(3) and not B(3);
    term2 <= x(3) and A(2) and not B(2);
    term1 <= x(3) and x(2) and A(1) and not B(1);
    term0 <= x(3) and x(2) and x(1) and A(0) and not B(0);

    -- Output Logic
    A_Greater_B_int <= term3 or term2 or term1 or term0; -- OR Gate
    A_Equal_B_int   <= x(3) and x(2) and x(1) and x(0);  -- AND Gate

    -- Final Outputs
    A_Greater_B <= A_Greater_B_int;
    A_Equal_B   <= A_Equal_B_int;
    A_Less_B    <= not (A_Greater_B_int or A_Equal_B_int); -- NOR Gate logic
end structural;
```

**Listing 5.8.** Gate-level description of the comparator.

**Chapter Summary**

- A **combinational circuit** has no memory. **Analysis**: label gate outputs, derive functions, build the truth table, explain. **Design**: specification, truth table, simplification, logic diagram, verification.
- **Code converters** such as BCD to excess-3 are designed with K-maps and don't-

cares. The **BCD adder** adds 6 to a binary sum greater than 9, with $C = K + Z_3Z_2 + Z_3Z_1$.

- A **decoder** generates the minterms of its inputs; with an enable input it is also a **demultiplexer** and can be cascaded. Any function can be built from a decoder and an OR gate.
- An **encoder** produces the code of the active input; a **priority encoder** handles several active inputs and has a valid output.
- A **multiplexer** selects one of $2^n$ inputs; it implements any function of $n + 1$ variables, and Shannon's expansion gives a tree of 2-to-1 multiplexers.
- An **ALU** selects among arithmetic and logic operations; its group $P$ and $G$ outputs permit fast wide ALUs.
- **Three-state gates** (Hi-Z) allow many drivers on one bus, provided only one is enabled at a time.
- A **magnitude comparator** compares from the MSB downward using $x_i = A_i \odot B_i$.

## Exercises

**5.1** **Multiplexer expansion.** Construct a 4-to-1 multiplexer using only 2-to-1 multiplexers. How many are needed? *Hint:* use the LSB select line $S_0$ in the first stage to choose within the pairs $(I_0, I_1)$ and $(I_2, I_3)$, and the MSB $S_1$ in the second stage.

**5.2** **Function with a 4-to-1 multiplexer.** Implement $F(A, B, C) = \sum m(1, 3, 5, 6, 7)$ with a single 4-to-1 multiplexer. Connect $A$ and $B$ to the select lines and find $I_0, \ldots, I_3$ in terms of $C$.

**5.3** **Tri-state bus.** Explain why the outputs of two ordinary AND gates cannot be connected together to the same wire. What happens if one gate outputs 1 and the other 0? How does a three-state buffer solve the problem?

**5.4** **Comparator analysis.** For $A = \texttt{1011}$ and $B = \texttt{1001}$, find the equivalence signals $x_3, x_2, x_1, x_0$ and the outputs $A > B$, $A = B$, $A < B$.

**5.5** Analyse the circuit defined by $T_1 = AB'$, $T_2 = A'B$, $F = T_1 + T_2 + C$. Write $F$ as a function of $A, B, C$, make the truth table and describe the function in words.

**5.6** Implement the full adder ($S$ and $C_{out}$) with a 3-to-8 decoder (active-high outputs) and OR gates.

**5.7** Design a 4-to-2 priority encoder in which $D_0$ has the *highest* priority and $D_3$ the lowest. Give the equations for $x$, $y$ and $V$.

**5.8** Implement $F(A, B, C, D) = \sum m(0, 2, 6, 7, 8, 10, 13, 15)$ with an 8-to-1 multiplexer, using $A, B, C$ as the select inputs. Give the data inputs in terms of $D$.

**5.9** Implement $F = xy + x'z$ with a single 2-to-1 multiplexer. Identify the select and data inputs.

**5.10** Show how to build a 5-to-32 decoder from 3-to-8 decoders with enable inputs

and one 2-to-4 decoder. How many of each are needed?

**5.11** How many 2-to-1 multiplexers are needed to build an 8-to-1 multiplexer?

**5.12** Add the BCD digits (a) `0111` + `0101` and (b) `1001` + `1001` + 1 using the BCD adder method, giving the binary sum, the correction and the result.

**5.13** Design a 2-bit magnitude comparator with outputs $A > B$, $A = B$, $A < B$ and draw the logic diagram. *Hint:* $A > B = A_1 B_1' + (A_1 \odot B_1) A_0 B_0'$.

**5.14** Design a combinational circuit that converts a 4-bit *binary* number in the range 0–9 into the corresponding Gray code of that number, ($g_3 = b_3$, $g_2 = b_3 \oplus b_2$, $g_1 = b_2 \oplus b_1$, $g_0 = b_1 \oplus b_0$). Write its VHDL in one architecture with four concurrent assignments.

**5.15** A three-state buffer drives the bus with the value of an 8-bit register when the signal `rd` is 1. Write the VHDL, using a `std_logic_vector` of width 8 and the value `(others => 'Z')`.

## Further Reading

All the blocks of this chapter are treated in Chapter 4 of Mano and Ciletti [7] and in Chapters 6 and 7 of Brown and Vranesic [3], including the VHDL descriptions; Wakerly [8] also discusses MSI parts in detail. Ashenden [9] is a good reference for the VHDL features used in the listings.

# 6

# Latches, Flip-Flops and Synchronous Sequential Circuits

**Learning Objectives**

After studying this chapter you will be able to:

- explain why a digital system needs memory and describe the general structure of a sequential circuit;
- distinguish synchronous from asynchronous circuits, and level-triggered latches from edge-triggered flip-flops;
- describe the operation of the SR, $\overline{SR}$, gated SR and D latches, and of the D, JK and T flip-flops, using characteristic tables and equations;
- write VHDL for latches and flip-flops, with synchronous and asynchronous reset;
- analyse a clocked sequential circuit by deriving its state table and state diagram;
- distinguish the Mealy and Moore models; and
- design a synchronous sequential circuit with D, JK and T flip-flops using excitation tables.

## 6.1 Why Sequential Circuits?

The combinational circuits of Chapters 2–5 have one limitation: their output depends *only* on the present input, and they have no memory of the past. Many real systems need such a memory:

- a **vending machine** must know how much money *has been* inserted;
- a **combination lock** must know whether the *previous* digits were correct;
- a **counter** must know its *present* value to compute the next.

The information that must be remembered is called the *state* of the system, and circuits that store it are built from *memory elements*: latches and flip-flops.

## 6.2 Structure of a Sequential Circuit

A *sequential circuit* (Figure 6.1) consists of a combinational circuit and memory elements connected in a loop. The memory elements hold the *present state*; the combinational logic computes the *next state* and the *outputs* from the inputs and the present state; and the next state is stored in the memory elements at the next clock pulse. The feedback loop through the memory is what distinguishes a sequential from a combinational circuit.

Sequential circuits are of two kinds.

- In a *synchronous* sequential circuit state changes happen only at discrete instants, determined by a periodic signal called the *clock*. The memory elements are *flip-flops*. Synchronous design is the dominant style because it is easier to design and verify and avoids many timing problems.
- In an *asynchronous* sequential circuit the state may change whenever an input changes. The memory elements are often simple *latches*. Asynchronous circuits are faster but difficult to design reliably, and are outside the scope of this book.

## 6.3 Latches

A *latch* is the simplest memory element: a circuit with two stable states, a *bistable* circuit, that can store one bit as long as power is applied.

### 6.3.1 The SR latch

The *SR latch* (Figure 6.2a) consists of two cross-coupled NOR gates; the inputs are *Set* ($S$) and *Reset* ($R$). Its behaviour (Table 6.1) is:

- $S = R = 0$: no change; the latch remembers its state;
- $S = 1, R = 0$: *set*: $Q = 1$;
- $S = 0, R = 1$: *reset*: $Q = 0$;
- $S = R = 1$: *forbidden*: both outputs go to 0, violating $Q' = \overline{Q}$, and when both inputs return to 0 simultaneously the final state is unpredictable. This input combination must be avoided.

The memory comes from the feedback: when $S = R = 0$ each gate acts as an inverter,

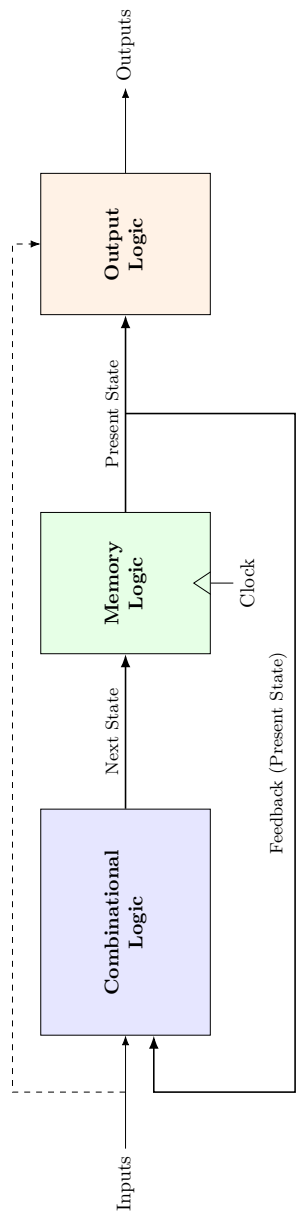



**Figure 6.1.** Block diagram of a sequential circuit: combinational logic, memory elements and the state feedback.

and the two inverters in a loop hold whatever state they have.

**Table 6.1.** SR latch with NOR gates.

| $S$ | $R$ | $Q(t+1)$ | $Q'(t+1)$ | State |
|---|---|---|---|---|
| 0 | 0 | $Q$ | $Q'$ | no change |
| 0 | 1 | 0 | 1 | reset |
| 1 | 0 | 1 | 0 | set |
| 1 | 1 | 0 | 0 | invalid |

### 6.3.2 The $\overline{SR}$ latch

With NAND gates the same circuit has *active-low* inputs, $\overline{S}$ and $\overline{R}$ (Figure 6.2b, Table 6.2): $(\overline{S}, \overline{R}) = (1,1)$ holds the state, $(0,1)$ sets, $(1,0)$ resets, and $(0,0)$ is invalid (both outputs 1).

**Table 6.2.** $\overline{SR}$ latch with NAND gates.

| $\overline{S}$ | $\overline{R}$ | $Q(t+1)$ | $Q'(t+1)$ | State |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | invalid |
| 0 | 1 | 1 | 0 | set |
| 1 | 0 | 0 | 1 | reset |
| 1 | 1 | $Q$ | $Q'$ | no change |




**(a)** NOR SR latch

**(b)** NAND $\overline{SR}$ latch

**Figure 6.2.** Basic latches built from cross-coupled gates.

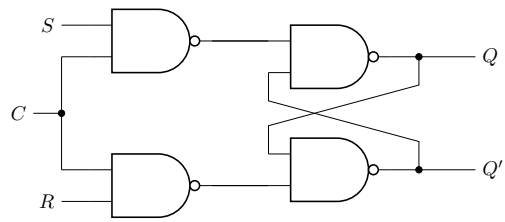
### 6.3.3 Gated SR latch and D latch

Adding two gates and a control input $C$ makes the latch respond to $S$ and $R$ only while $C = 1$ (Figure 6.3a): when $C = 0$ the latch holds its state. This is the *gated SR latch*. To remove the forbidden input combination, an inverter ensures that $S$ and $R$ are never 1 together: connect $S = D$ and $R = D'$. The result is the *D latch* (Figure 6.3b, Table 6.3): while $C = 1$, $Q$ follows $D$ (the latch is *transparent*); when $C = 0$ the latch holds the last value of $D$. Its equation is $Q(t+1) = D$ when $C = 1$.

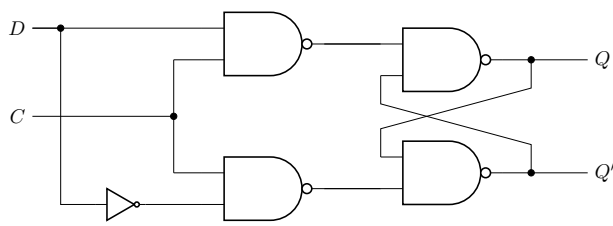
```
1 library ieee;
2 use ieee.std_logic_1164.all;
```

**Table 6.3.** D latch.

| $C$ | $D$ | $Q(t+1)$ |
|---|---|---|
| 0 | X | no change |
| 1 | 0 | 0 (reset) |
| 1 | 1 | 1 (set) |




**(a)** gated SR latch




**(b)** D latch

**Figure 6.3.** Gated latches.

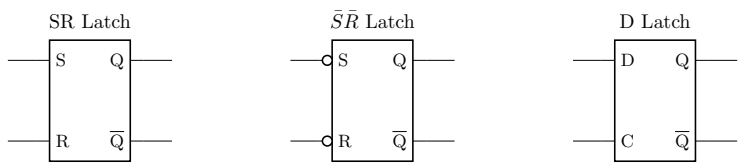



**Figure 6.4.** Graphical symbols of the SR, $\overline{SR}$ and D latches.

```
3
4  entity d_latch is
5      port ( D, C : in  std_logic;
6             Q    : out std_logic );
7  end d_latch;
8
9  architecture behavioral of d_latch is
10 begin
11     process(D, C)
12     begin
13         if C = '1' then
14             Q <= D;
15         end if;
16     end process;
17 end behavioral;
```

**Listing 6.1.** A D latch. The process is sensitive to both `D` and `C`; since no value is assigned when `C` is 0, the synthesis tool infers a latch.

Listing 6.1 shows how a latch is described: the process has no `else` branch, so `Q` keeps its value when `C = '0'`, which means memory. Unintentional latches caused by incomplete `if` statements are a common mistake in combinational VHDL descriptions.

## 6.4 Flip-Flops

A latch is *level-triggered*: its output may change at any time while the control input is active. In a circuit in which the latch output feeds back into its own input through combinational logic this causes uncontrolled multiple transitions during one clock pulse. A *flip-flop* is *edge-triggered*: it samples its inputs and changes state only at one *edge* of the clock (Figure 6.5), so that all state changes in a synchronous system happen at the same moment. In a *positive-edge* flip-flop the transition is on the rising edge $(0 \to 1)$, in a *negative-edge* flip-flop on the falling edge $(1 \to 0)$.

### 6.4.1 The D flip-flop

The D flip-flop is the most common flip-flop. At the active clock edge it captures the value of $D$ and stores it in $Q$. One way to build it is the *master–slave* configuration of Figure 6.6: two D latches in series with opposite clock polarity. While CLK is 1 the master latch is transparent and follows $D$ while the slave is closed and holds the old value. When CLK falls to 0 the master closes, capturing $D$, and the slave opens and passes the captured value to $Q$. $Q$ therefore changes only when the clock falls: a negative-edge D flip-flop. The characteristic equation is $Q(t+1) = D$.

### 6.4.2 The JK flip-flop

The *JK flip-flop* refines the SR flip-flop by giving meaning to the forbidden input. $J$ sets, $K$ resets, and when $J = K = 1$ the flip-flop *toggles*: $Q$ becomes $Q'$. Its characteristic

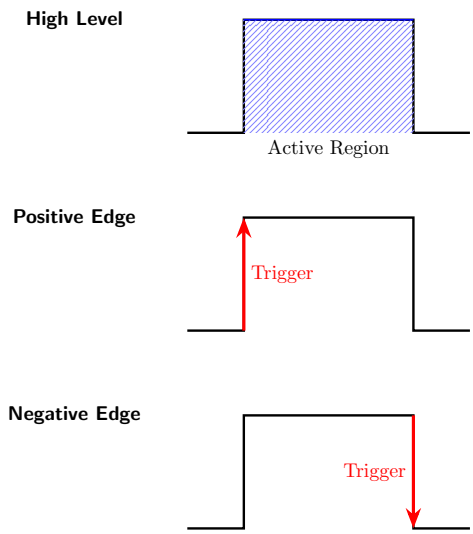
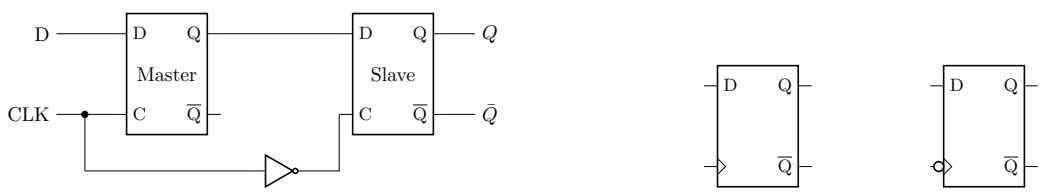



**Figure 6.5.** Level triggering, positive-edge triggering and negative-edge triggering.




**(a)** master–slave construction **(b)** symbols

**Figure 6.6.** Edge-triggered D flip-flop.

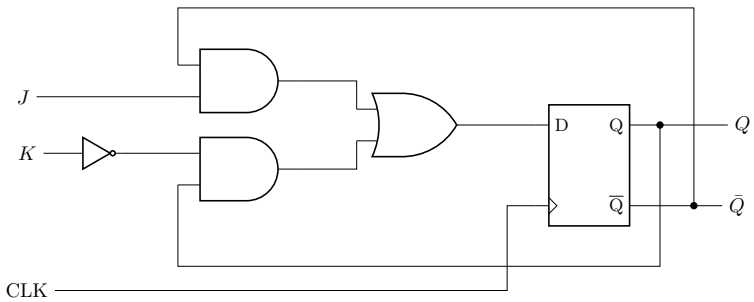
equation is

$$Q(t+1) = JQ' + K'Q.$$

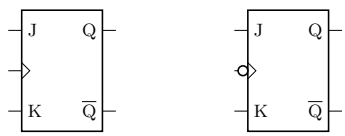
A JK flip-flop can be built from a D flip-flop with the input equation $D = JQ' + K'Q$ (Figure 6.7).

### 6.4.3 The T flip-flop

The *T flip-flop* has a single input $T$: if $T = 0$ it holds its state and if $T = 1$ it toggles, $Q(t+1) = T \oplus Q$. It is useful for counters. It is obtained from a JK flip-flop by tying $J = K = T$, or from a D flip-flop with an XOR gate, $D = T \oplus Q$ (Figure 6.8).




**(a)** JK from a D flip-flop




**(b)** JK symbol

**Figure 6.7.** JK flip-flop.

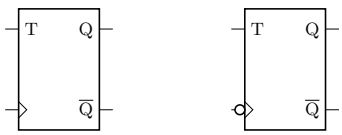
### 6.4.4 Characteristic tables

The behaviour of each flip-flop is summarised by its *characteristic table*, which gives the next state $Q(t+1)$ as a function of the inputs and the present state $Q(t)$, and by the equivalent *characteristic equation* (Table 6.4). The SR equation $Q(t+1) = S + R'Q$ holds under the restriction $SR = 0$.

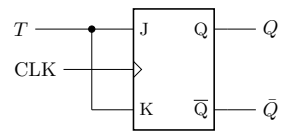
### 6.4.5 VHDL descriptions

In VHDL a flip-flop is described by a process that is sensitive to the clock and does something only at the active edge, detected with the function `rising_edge(CLK)` (or `falling_edge`). The assignments made in such a process are *registered*: the synthesis tool creates flip-flops. Listings 6.2 to 6.4 show the D, JK and T flip-flops.

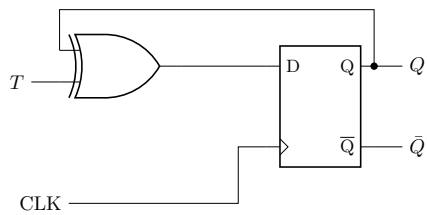
```
1 library ieee;
2 use ieee.std_logic_1164.all;
```




**(a)** symbol




**(b)** from JK




**(c)** from D and XOR

**Figure 6.8.** T flip-flop.

**Table 6.4.** Characteristic tables and equations of the flip-flops.

**SR**

$Q(t{+}1) = S + R'Q$

| $S$ | $R$ | $Q(t{+}1)$ |
|---|---|---|
| 0 | 0 | $Q$ |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | ? |

**D**

$Q(t{+}1) = D$

| $D$ | $Q(t{+}1)$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

**JK**

$Q(t{+}1) = JQ' + K'Q$

| $J$ | $K$ | $Q(t{+}1)$ |
|---|---|---|
| 0 | 0 | $Q$ |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | $Q'$ |

**T**

$Q(t{+}1) = T \oplus Q$

| $T$ | $Q(t{+}1)$ |
|---|---|
| 0 | $Q$ |
| 1 | $Q'$ |

```
3
4  entity d_ff is
5      port ( D, CLK : in  std_logic;
6             Q      : out std_logic );
7  end d_ff;
8
9  architecture behavioral of d_ff is
10 begin
11     process(CLK)
12     begin
13         if rising_edge(CLK) then
14             Q <= D;
15         end if;
16     end process;
17 end behavioral;
```

**Listing 6.2.** A positive-edge-triggered D flip-flop.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity jk_ff is
5      port ( J, K, CLK : in  std_logic;
6             Q         : out std_logic );
7  end jk_ff;
8
9  architecture behavioral of jk_ff is
10     signal q_int : std_logic := '0';
11 begin
12     process(CLK)
13     begin
14         if rising_edge(CLK) then
15             if (J='0' and K='0') then
16                 null; -- No change
17             elsif (J='0' and K='1') then
18                 q_int <= '0'; -- Reset
19             elsif (J='1' and K='0') then
20                 q_int <= '1'; -- Set
21             elsif (J='1' and K='1') then
22                 q_int <= not q_int; -- Toggle
23             end if;
24         end if;
25     end process;
26     Q <= q_int;
27 end behavioral;
```

**Listing 6.3.** A JK flip-flop. The internal signal `q_int` lets the value of `Q` be read back for the toggle operation.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
```

```
3
4  entity t_ff is
5      port ( T, CLK : in  std_logic;
6             Q      : out std_logic );
7  end t_ff;
8
9  architecture behavioral of t_ff is
10     signal q_int : std_logic := '0';
11 begin
12     process(CLK)
13     begin
14         if rising_edge(CLK) then
15             if T='1' then
16                  q_int <= not q_int;
17             end if;
18         end if;
19     end process;
20     Q <= q_int;
21 end behavioral;
```

**Listing 6.4.** A T flip-flop.

### 6.4.6 Asynchronous and synchronous reset

Flip-flops often have *preset* (set $Q = 1$) and *clear* or *reset* (set $Q = 0$) inputs, shown with a bubble if active-low (Figure 6.9). A reset can act in two ways.

- An *asynchronous* reset overrides the clock and the data input: the output changes *immediately* when the reset becomes active. In VHDL the reset is tested *before* the clock-edge condition (Listing 6.5).
- A *synchronous* reset takes effect only at the active clock edge. In VHDL the reset is tested *inside* the clock-edge condition (Listing 6.6).

Asynchronous reset is convenient for initialising a system at power-up; synchronous reset keeps all state changes aligned with the clock. Releasing an asynchronous reset close to a clock edge can cause timing violations, so in practice the reset signal is itself synchronised to the clock.

```
process(clk, reset)
begin
  if reset = '1' then -- Async Reset
    Q <= '0';
  elsif rising_edge(clk) then
    Q <= D;
  end if;
end process;
```

**Listing 6.5.** Process with asynchronous reset.

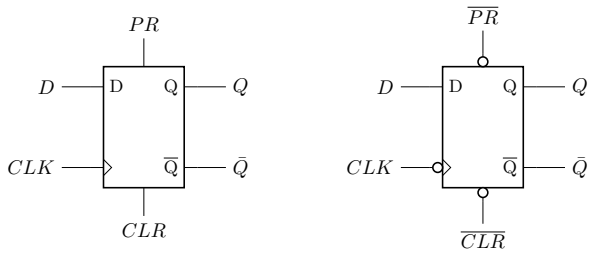



**Figure 6.9.** D flip-flops with asynchronous preset and clear inputs.

```
process(clk)
begin
  if rising_edge(clk) then
    if reset = '1' then -- Sync Reset
        Q <= '0';
    else
        Q <= D;
    end if;
  end if;
end process;
```

**Listing 6.6.** Process with synchronous reset.

## 6.5 Analysis of Clocked Sequential Circuits

*Analysis* derives the behaviour of a given sequential circuit, expressed as a *state table* or *state diagram*.

1. From the logic diagram, write the *flip-flop input equations* (excitation equations) and the *output equations*.
2. Substitute the input equations in the characteristic equation of each flip-flop to obtain the *next-state equations*: $Q(t+1) = D$ for a D flip-flop, $Q(t+1) = JQ' + K'Q$ for JK, $Q(t+1) = T \oplus Q$ for T.
3. Construct the *state table*, which lists the next state and the output for every combination of present state and input.
4. Draw the *state diagram*: a circle for each state and an arrow for each transition, labelled `input/output`.

**Example 6.1: Analysis with D flip-flops**

The circuit of Figure 6.10a has two D flip-flops $A$ and $B$, an input $x$ and an output $y$.

**Equations.** $D_A = Ax + Bx$, $D_B = A'x$, $y = A + Bx'$. Since $A(t+1) = D_A$ and $B(t+1) = D_B$, the next-state equations are $A(t+1) = Ax + Bx$ and $B(t+1) = A'x$.
**State table.** Evaluating these equations for the eight combinations of $AB$ and $x$ gives Table 6.5; the corresponding state diagram is Figure 6.10b.

**Table 6.5.** State table of Example 6.1.

| Present state $A$ | $B$ | Input $x$ | Next state $A$ | $B$ | Output $y$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 |

**Example 6.2: Analysis with JK flip-flops**

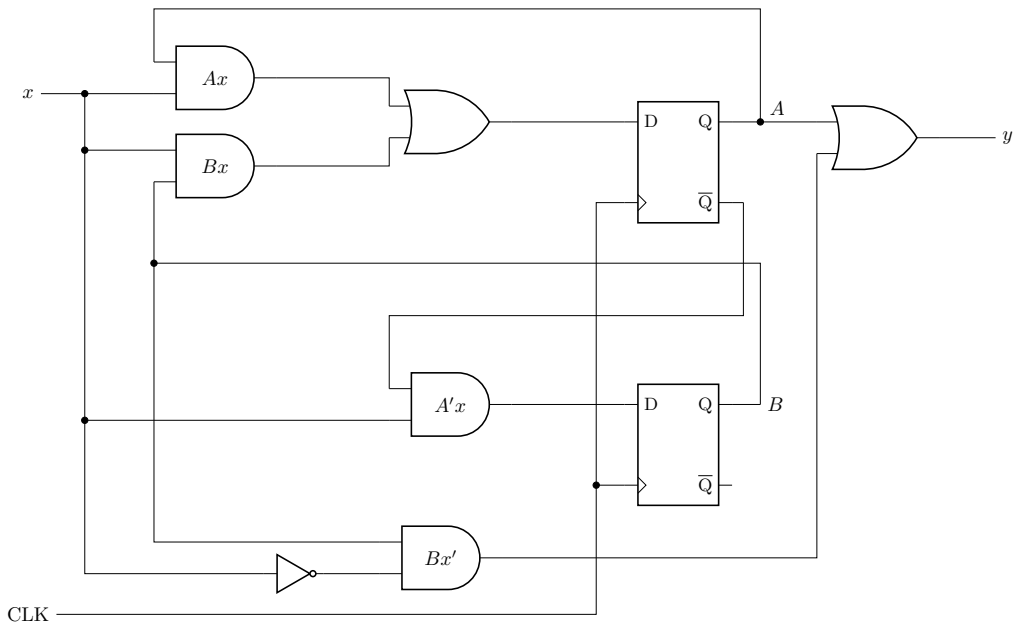
In Figure 6.11a the inputs are $J_A = B$, $K_A = Bx'$, $J_B = x'$, $K_B = A'x + Ax' = A \oplus x$. With $Q(t+1) = JQ' + K'Q$,

$$A(t+1) = BA' + (Bx')'A, \qquad B(t+1) = x'B' + (A \oplus x)'B.$$

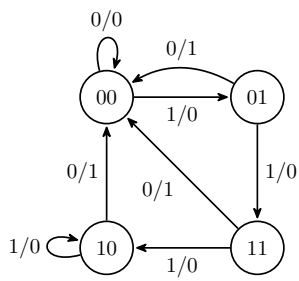
Table 6.6 lists the flip-flop inputs for each row and the resulting next state, which gives the state diagram of Figure 6.11b. (Row $AB = 01$, $x = 0$: $J_A = 1, K_A = 1$ toggles $A$ from 0 to 1, and $J_B = 1, K_B = 0$ sets $B$, so the next state is 11.)

**Table 6.6.** State table of Example 6.2.

| Present state $A$ | $B$ | Input $x$ | Flip-flop inputs $J_A$ | $K_A$ | $J_B$ | $K_B$ | Next state $A$ | $B$ |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 |

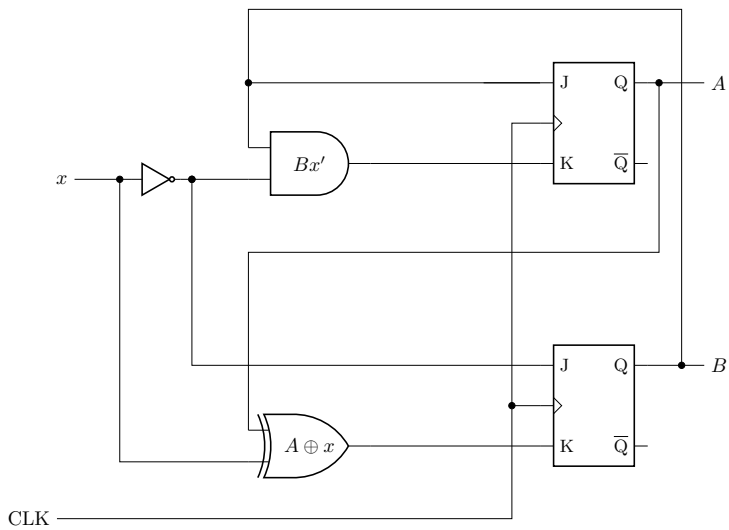



**(a)** circuit

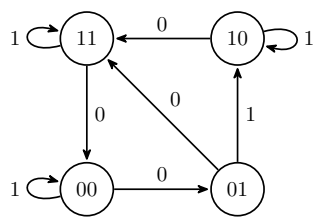



**(b)** state diagram

**Figure 6.10.** The circuit analysed in Example 6.1.




**(a)** circuit




**(b)** state diagram

**Figure 6.11.** The circuit analysed in Example 6.2.

**Example 6.3: Analysis with T flip-flops**

In Figure 6.12a: $T_A = Bx$, $T_B = x$ and $y = AB$. With $Q(t+1) = T \oplus Q$, $A(t+1) = Bx \oplus A$ and $B(t+1) = x \oplus B$. The state table is Table 6.7. Because $y = AB$ depends on the state only, the machine is a *Moore* machine and its diagram (Figure 6.12b) writes the output inside each state, `state/output`, and labels the arrows with the input.

**Table 6.7.** State table of Example 6.3.

| Present state | | Input | Next state | | Output |
|---|---|---|---|---|---|
| $A$ | $B$ | $x$ | $A$ | $B$ | $y$ |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 |

## 6.6 Mealy and Moore Models

Synchronous sequential circuits are classified by how their outputs are generated.

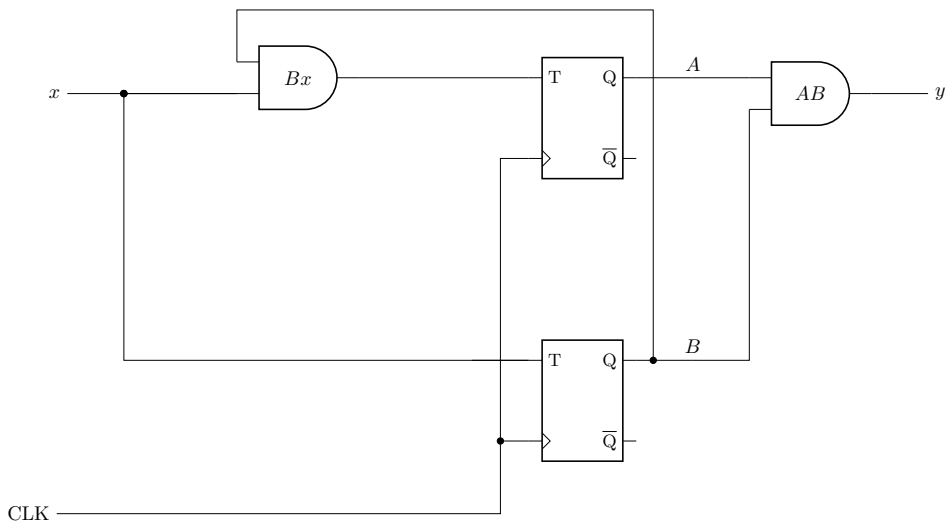
- In the *Mealy model* [16] the outputs depend on the *present state and the present inputs*. In a state diagram the output is written on each transition, `input/output`. Because the output depends directly on the inputs, it can change between clock edges whenever the input changes, and may show momentary false values (glitches).
- In the *Moore model* [17] the outputs depend *only on the present state*. In a state diagram the output is written inside each state circle. Moore outputs change only when the state changes, which is just after a clock edge, so they are synchronous and glitch-free.

The two models are compared in detail, with timing diagrams, in Section 8.6.

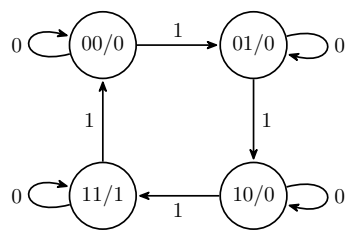
## 6.7 Design of Sequential Circuits

*Design* is the reverse of analysis: from a specification to a logic diagram.

1. Derive a **state diagram** from the word description.
2. Perform **state reduction**: remove states that are equivalent.
3. Perform **state assignment**: assign a unique binary code to each state. A machine




**(a)** circuit




**(b)** state diagram

**Figure 6.12.** The circuit analysed in Example 6.3.

with $N$ states needs at least $\lceil \log_2 N \rceil$ flip-flops.

4. Write the binary-coded **state table**.
5. Choose the **type of flip-flop**.
6. Derive the simplified **flip-flop input equations** and **output equations** with K-maps (unused states give don't-cares).
7. Draw the **logic diagram** (or write the VHDL) and verify it.

### 6.7.1 Example: a sequence detector

**Example 6.4: 101 sequence detector**

Design a circuit with input $x$ and output $y$ that produces $y = 1$ whenever the input sequence 101 has just been received. Overlapping sequences count (10101 contains two).

**State diagram.** Three states are needed to remember how much of the pattern has been seen: $S_0$, the initial state (no useful input yet, or the last input was 0 without a preceding 1); $S_1$, the last input was 1; $S_2$, the last two inputs were 10. In $S_2$ a 1 completes the sequence: the output is 1 and, since the last input was 1, the next state is $S_1$. The output depends on the input, so this is a Mealy machine (Figure 6.13).

**State assignment.** $S_0 = 00$, $S_1 = 01$, $S_2 = 10$; two flip-flops $A$ and $B$. The state 11 is unused.

**State table** (Table 6.8).

**Table 6.8.** State table of the 101 detector, $S_0 = 00$, $S_1 = 01$, $S_2 = 10$.

| Present state | | Input | Next state | | Output |
|---|---|---|---|---|---|
| $A$ | $B$ | $x$ | $A$ | $B$ | $y$ |
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 1 |

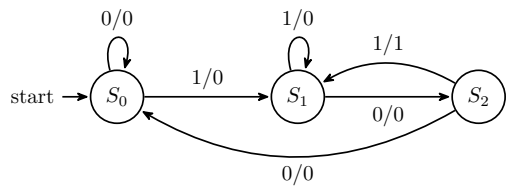



**Figure 6.13.** State diagram of the 101 sequence detector (Mealy).

**Implementation with D flip-flops.** For D flip-flops the input equation is the next-state value. From K-maps of Table 6.8 (with the unused state 11 as don't-care):

$$D_A = Bx', \qquad D_B = x, \qquad y = Ax.$$

The circuit is shown in Figure 6.14.

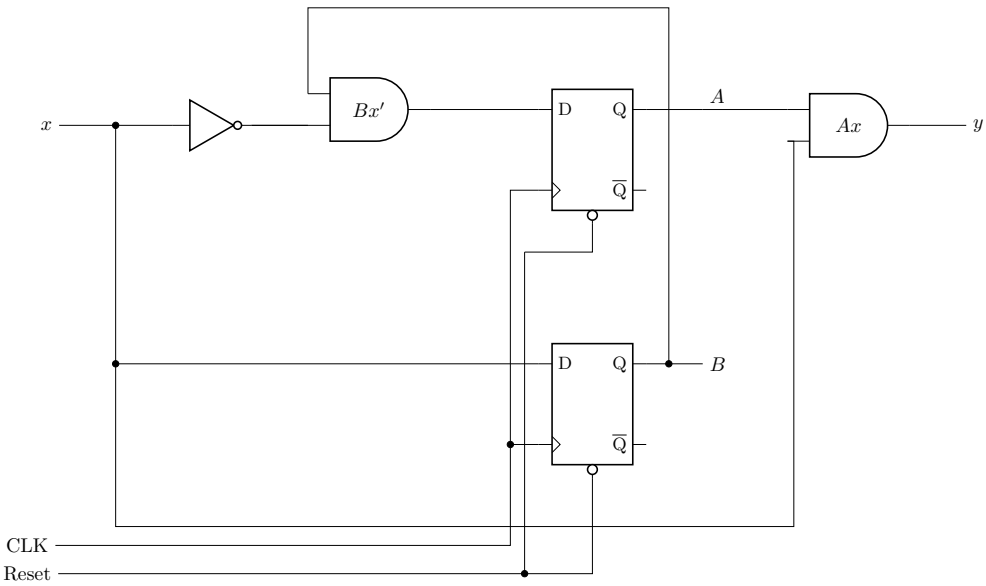



**Figure 6.14.** Logic diagram of the 101 detector with D flip-flops.

**Excitation tables.** For JK and T flip-flops we need the *excitation table*, the inverse of the characteristic table: it states which flip-flop inputs produce a required transition $Q(t) \to Q(t+1)$ (Table 6.9). The X entries are don't-cares and are what make JK designs sometimes simpler than D designs.

**Table 6.9.** Excitation tables of the JK and T flip-flops.

| JK | | | |
|---|---|---|---|
| $Q(t)$ | $Q(t{+}1)$ | $J$ | $K$ |
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | X |
| 1 | 0 | X | 1 |
| 1 | 1 | X | 0 |

| T | | |
|---|---|---|
| $Q(t)$ | $Q(t{+}1)$ | $T$ |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Implementation with JK flip-flops.** We extend the state table with the required $J$ and $K$ inputs of each flip-flop (Table 6.10). For flip-flop $A$: transition $0 \to 0$ needs $J_A = 0$; $0 \to 1$ (row $AB = 01$, $x = 0$) needs $J_A = 1$; $1 \to 0$ (both rows with $AB = 10$)

needs $K_A = 1$. For flip-flop $B$: $0 \to 1$ occurs when $x = 1$ in states 00 and 10; $1 \to 0$ in state 01 when $x = 0$; $1 \to 1$ in state 01 when $x = 1$. The K-maps (the unused state 11 and all entries X are don't-cares) give

$$J_A = Bx', \qquad K_A = 1, \qquad J_B = x, \qquad K_B = x', \qquad y = Ax.$$

Note that $K_A$ is the constant 1: in state 10 the flip-flop $A$ must always return to 0, whatever the input, and in all other states the value of $K_A$ is a don't-care. The flip-flop input logic of the JK design is thus one AND gate and one inverter (shared by $J_A$ and $K_B$), the same as for the D design; the advantage of JK flip-flops appears in larger machines, where the don't-cares often remove literals.

**Table 6.10.** Excitation table for the 101 detector with JK flip-flops.

| PS | | Input | NS | | $A$ inputs | | $B$ inputs | |
|---|---|---|---|---|---|---|---|---|
| $A$ | $B$ | $x$ | $A$ | $B$ | $J_A$ | $K_A$ | $J_B$ | $K_B$ |
| 0 | 0 | 0 | 0 | 0 | 0 | X | 0 | X |
| 0 | 0 | 1 | 0 | 1 | 0 | X | 1 | X |
| 0 | 1 | 0 | 1 | 0 | 1 | X | X | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | X | X | 0 |
| 1 | 0 | 0 | 0 | 0 | X | 1 | 0 | X |
| 1 | 0 | 1 | 0 | 1 | X | 1 | 1 | X |

**VHDL descriptions.** In VHDL a machine is best described with an *enumerated type* for the states, e.g. `type state_type is (S0, S1, S2);`; the synthesis tool then chooses the state encoding (binary, one-hot, ...). Listing 6.7 describes the detector with two processes: a clocked process holding the state register and a combinational process computing the next state and the output. Listing 6.8 is the gate-level (structural) description that follows the D-flip-flop design.

```
library ieee;
use ieee.std_logic_1164.all;

entity sequence_detector is
  port (clk, reset : in  std_logic;
        x          : in  std_logic;
        y          : out std_logic);
end sequence_detector;

architecture fsm of sequence_detector is
  type state_type is (S0, S1, S2);
  signal current_state, next_state : state_type;
begin
  -- state register (sequential part)
  process (clk, reset)
  begin
```

```
    if reset = '1' then
      current_state <= S0;
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

  -- next-state and output logic (combinational part)
  process (current_state, x)
  begin
    y <= '0';                                   -- default output
    case current_state is
      when S0 => if x = '1' then next_state <= S1; else next_state <= S0; end if;
      when S1 => if x = '0' then next_state <= S2; else next_state <= S1; end if;
      when S2 => if x = '1' then y <= '1'; next_state <= S1;
                 else next_state <= S0; end if;
    end case;
  end process;
end fsm;
```

**Listing 6.7.** The 101 detector as a state machine with an enumerated state type.

```
library ieee;
use ieee.std_logic_1164.all;

-- D flip-flop with asynchronous reset (used as a component)
entity d_ff_r is
  port (D, clk, rst : in  std_logic;
        Q           : out std_logic);
end d_ff_r;

architecture behavioral of d_ff_r is
begin
  process (clk, rst)
  begin
    if rst = '1' then
      Q <= '0';
    elsif rising_edge(clk) then
      Q <= D;
    end if;
  end process;
end behavioral;

library ieee;
use ieee.std_logic_1164.all;

entity sequence_detector_s is
  port (clk, reset : in  std_logic;
        x          : in  std_logic;
        y          : out std_logic);
end sequence_detector_s;
```

```
30
31 architecture structural of sequence_detector_s is
32   component d_ff_r
33     port (D, clk, rst : in  std_logic;
34           Q           : out std_logic);
35   end component;
36   signal A, B     : std_logic;                  -- state variables
37   signal D_A, D_B : std_logic;                  -- next-state logic
38 begin
39   FF_A : d_ff_r port map (D => D_A, clk => clk, rst => reset, Q => A);
40   FF_B : d_ff_r port map (D => D_B, clk => clk, rst => reset, Q => B);
41
42   D_A <= B and (not x);                         -- D_A = B x'
43   D_B <= x;                                     -- D_B = x
44   y   <= A and x;                               -- y   = A x   (Mealy output)
45 end structural;
```

**Listing 6.8.** The 101 detector as two D flip-flops and gates (structural description).

### 6.7.2 Example: a binary counter with T flip-flops

**Example 6.5: 3-bit binary counter**

Design a counter that cycles through $000, 001, \ldots, 111$ and back to 000, using T flip-flops. There are no inputs except the clock.

**Solution.** A T flip-flop must toggle ($T = 1$) exactly when its bit changes between the present and next states. In the table, $A_0$ changes every clock; $A_1$ changes when $A_0 = 1$; $A_2$ changes when $A_1 = A_0 = 1$ (Table 6.11). Hence

$$T_{A_0} = 1, \qquad T_{A_1} = A_0, \qquad T_{A_2} = A_1 A_0.$$

The circuit (Figure 6.15) uses three flip-flops and one AND gate.

**Table 6.11.** State table and T inputs of the 3-bit counter.

| Present state | | | Next state | | | Flip-flop inputs | | |
|---|---|---|---|---|---|---|---|---|
| $A_2$ | $A_1$ | $A_0$ | $A_2$ | $A_1$ | $A_0$ | $T_{A_2}$ | $T_{A_1}$ | $T_{A_0}$ |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |

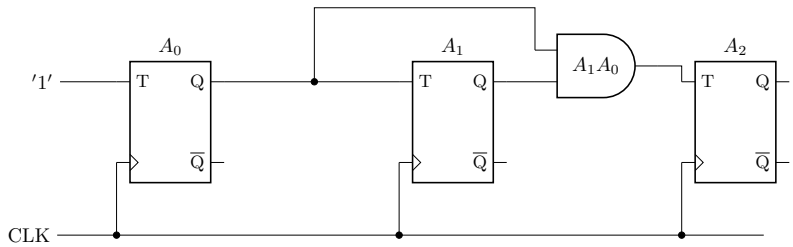



**Figure 6.15.** A 3-bit binary counter with T flip-flops.

**Chapter Summary**

- A **sequential circuit** contains memory; its output depends on the inputs and the **state**. **Synchronous** circuits change state at clock edges, using flip-flops.
- **Latches** (SR, $\overline{SR}$, gated SR, D) are level-triggered. **Flip-flops** (D, JK, T) are edge-triggered. Characteristic equations: $Q^+ = D$; $Q^+ = JQ' + K'Q$; $Q^+ = T \oplus Q$; $Q^+ = S + R'Q$ $(SR = 0)$.
- **Asynchronous** reset overrides the clock; **synchronous** reset acts at the clock edge. In VHDL, `rising_edge(clk)` describes a flip-flop.
- **Analysis**: logic diagram → flip-flop input equations → next-state equations → state table → state diagram.
- **Mealy**: output depends on state and input. **Moore**: output depends on state only.
- **Design**: word problem → state diagram → (reduction) → state assignment → state table → excitation → flip-flop input equations → logic diagram. Use the **excitation tables** for JK and T flip-flops.

## Exercises

**6.1** **Next-state equations.** A sequential circuit has two D flip-flops $A, B$ and an input $x$, with $D_A = Ax + Bx$ and $D_B = A'x$. Write the next-state equations $A(t+1)$ and $B(t+1)$.

**6.2** **State diagram.** A circuit has one T flip-flop and an input $x$, with $T = A \oplus x$. Draw the state diagram. Describe what the circuit does.

**6.3** **JK flip-flop behaviour.** A JK flip-flop has $J = K = 1$ permanently. (a) Describe how $Q$ changes on each clock pulse. (b) Draw its state diagram. (c) If the clock frequency is 1 MHz, what is the frequency of $Q$?

**6.4** **Mealy or Moore?** "A sequence detector produces $y = 1$ whenever the internal counter holds the value 101. The output remains 1 as long as the counter is in this state, regardless of the input during that clock cycle." Classify the machine and justify your answer.

**6.5** An NOR SR latch initially has $Q = 0$. The inputs $(S, R)$ are applied in turn: $(0,0)$,

$(1,0)$, $(0,0)$, $(0,1)$, $(0,0)$. Give $Q$ after each step.

**6.6** Explain why a D latch cannot be used directly in place of a D flip-flop in a counter in which $Q'$ is fed back to $D$. What happens while the latch is transparent?

**6.7** Show how to build (a) a T flip-flop and (b) a D flip-flop from a JK flip-flop.

**6.8** Derive the characteristic equation $Q(t+1) = S + R'Q$ of the SR flip-flop from its characteristic table, with a K-map in which the combination $S = R = 1$ is a don't-care.

**6.9** **Analysis.** A circuit has two D flip-flops with $D_A = x \oplus B$, $D_B = A$ and output $y = AB'$. Draw the state table and state diagram. Is it a Mealy or a Moore machine?

**6.10** **Design.** Design a Mealy sequence detector with input $x$ and output $y$ that detects the sequence 110 (overlapping allowed) using two D flip-flops with the state assignment $S_0 = 00$, $S_1 = 01$, $S_2 = 10$. Give the state diagram, the state table, the equations, and the VHDL of the machine.

**6.11** Design a 3-bit binary counter with JK flip-flops.

**6.12** How many flip-flops are required to implement a state machine with (a) 5 states; (b) 9 states; (c) a one-hot encoding of 9 states?

**6.13** Write VHDL for a D flip-flop with synchronous reset and an enable input `en`: the register loads `D` only when `en = '1'`.

**6.14** Write the VHDL of an 8-bit register with asynchronous active-low reset.

## Further Reading

The treatment of latches, flip-flops and the analysis and design of clocked sequential circuits follows Chapter 5 of Mano and Ciletti [7]. Brown and Vranesic [3] (Chapters 7 and 8) and Wakerly [8] discuss timing parameters (setup, hold, propagation delay) in detail, which we have not covered. Mealy [16] and Moore [17] introduced the two models. For VHDL coding style for registers and state machines, see Ashenden [9].

# 7

# Registers and Counters

**Learning Objectives**

After studying this chapter you will be able to:

- design registers with parallel load and shift registers, and describe serial transfer and serial addition;
- describe a universal shift register and write it in VHDL;
- explain the operation, delay and frequency limit of asynchronous (ripple) counters;
- design synchronous binary counters, up/down counters and counters with parallel load;
- build modulo-$N$ counters, including the BCD counter, and make a counter self-correcting;
- use ring counters and counter–decoder combinations to generate timing signals.

## 7.1 Registers

A *register* is a group of flip-flops, each of which stores one bit. An $n$-bit register consists of $n$ flip-flops sharing one clock and stores one $n$-bit word. The simplest register (Figure 7.1) is a row of D flip-flops: at each clock edge the four inputs are copied to the outputs. Registers may also contain gates that control *how* and *when* new information is accepted.

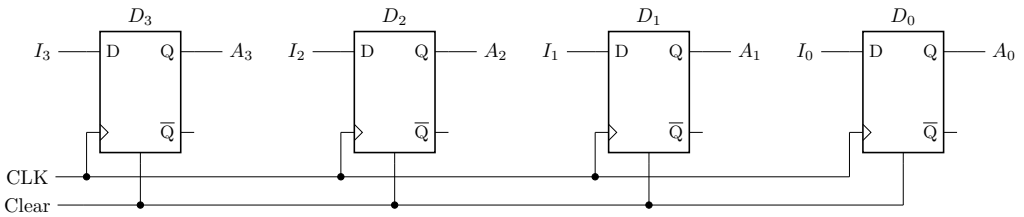



**Figure 7.1.** A 4-bit register built from D flip-flops.

### 7.1.1 Register with parallel load

Transferring new information into a register is called *loading*; if all bits are transferred at the same moment it is a *parallel load*. A register that loads on every clock pulse cannot hold its data. A *load* control input solves this (Figure 7.2): a 2-to-1 multiplexer at each flip-flop input selects either the external data $I_i$ (when $Load = 1$) or the flip-flop's own output (when $Load = 0$), so that the register keeps its value through the feedback path. Equivalently, the clock may be *gated*, but this is poor practice in synchronous design because it can introduce clock skew and glitches; a multiplexer (or a flip-flop with a clock-enable input) is preferred.

## 7.2 Shift Registers

A *shift register* can move its stored bits to the left or right. The simplest one is a chain of D flip-flops in which the output of each is connected to the input of the next and all flip-flops share a clock (Figure 7.3). At each clock pulse every bit moves one position along the chain, and a new bit enters at the *serial input*. Because all flip-flops change simultaneously, each captures the *old* output of its neighbour, which is why the data moves exactly one position and not through the whole chain.

### 7.2.1 Serial transfer

In a *serial transfer*, information is moved from a register $A$ to a register $B$ one bit at a time: the serial output of $A$ is connected to the serial input of $B$, and both registers are clocked together (Figure 7.4a). After $n$ clock pulses the $n$-bit word has been transferred. If the serial output of $A$ is also fed back to its own serial input, $A$ is *rotated* and keeps its data. Table 7.1 traces the transfer of $A = \texttt{1011}$, and Figure 7.4b is the timing diagram; a serial transfer needs far fewer wires than a parallel transfer, but takes $n$ clock periods instead of one.

### 7.2.2 Serial adder

A *serial adder* adds two binary numbers one bit at a time, using one full adder instead of $n$ (Figure 7.5). Two shift registers hold the operands $A$ (augend) and $B$ (addend); at each clock pulse the least significant bits of both registers and a stored carry are

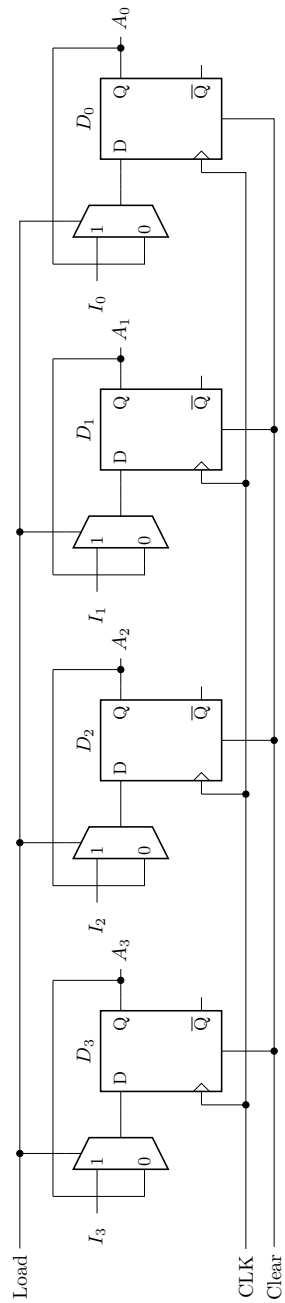



**Figure 7.2.** A 4-bit register with parallel load. When $Load = 0$ the flip-flops recirculate their own outputs.

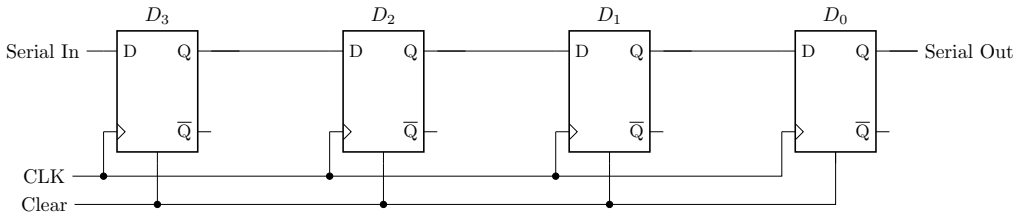
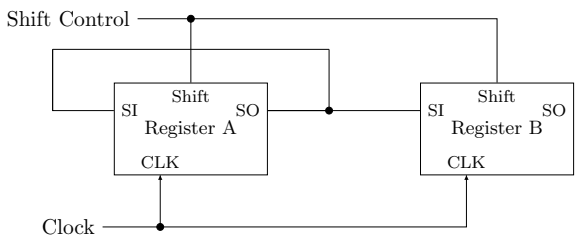
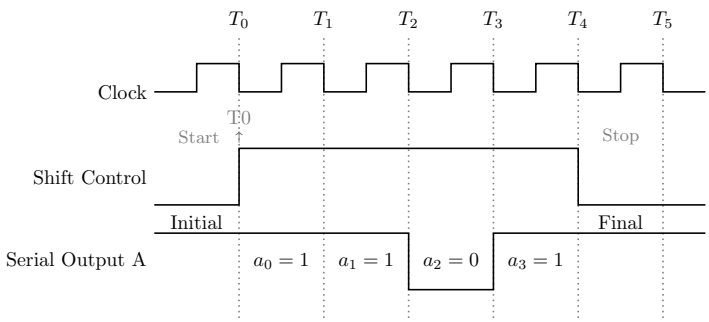



**Figure 7.3.** A 4-bit shift register: serial input at the left, serial output at the right.




**(a)** circuit




**(b)** timing diagram

**Figure 7.4.** Serial transfer from register $A$ to register $B$.

**Table 7.1.** Serial transfer of $A = 1011$ into $B$ ($A$ is rotated).

| Timing pulse | Serial output | Register $A$ | Register $B$ |
|---|---|---|---|
| Initial | — | 1011 | 0000 |
| $T_1$ | 1 | 1101 | 1000 |
| $T_2$ | 1 | 1110 | 1100 |
| $T_3$ | 0 | 0111 | 0110 |
| $T_4$ | 1 | 1011 | 1011 |

added by the full adder; the sum bit is shifted into the most significant end of $A$ (so that the sum replaces the augend); $B$ is rotated to preserve its value; and the carry-out is stored in a D flip-flop for the next bit position. The flip-flop is cleared to 0 initially. Table 7.2 traces $A = \texttt{1001}$ (9) plus $B = \texttt{0011}$ (3): after four pulses $A = \texttt{1100} = 12$. The adder is slow ($n$ clock periods) but needs very little hardware.

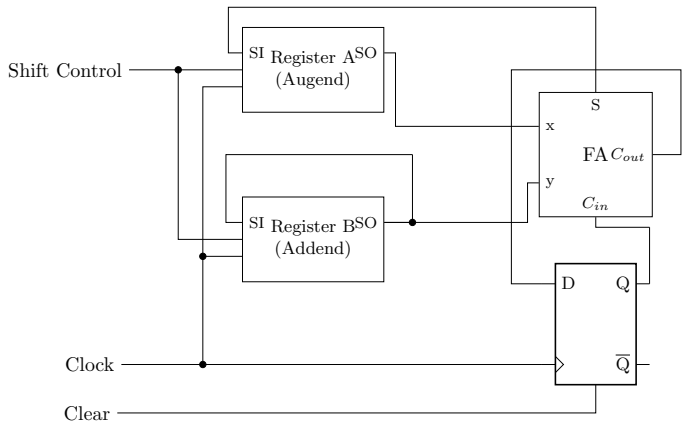



**Figure 7.5.** A serial adder: one full adder and a carry flip-flop.

**Table 7.2.** Trace of the serial addition 1001 + 0011.

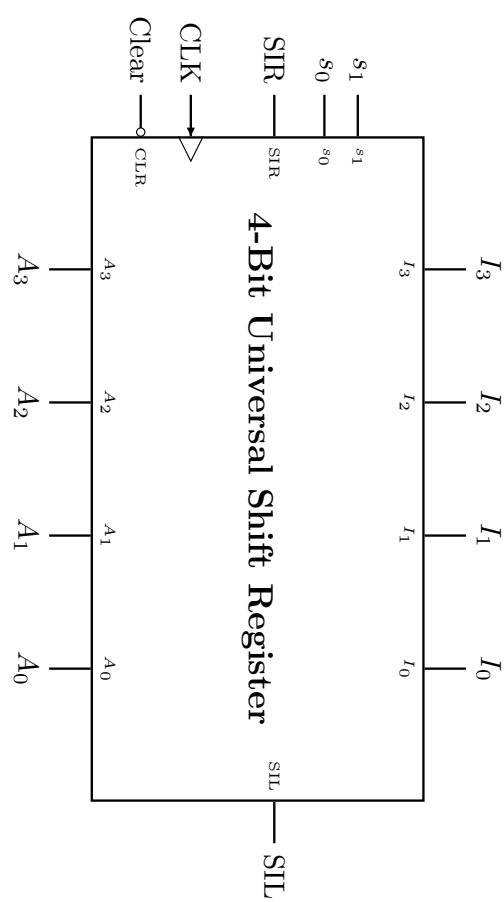
| Pulse | Reg. $A$ | Reg. $B$ | $x$ | $y$ | $C_{in}$ | $S$ | $C_{out}$ |
|---|---|---|---|---|---|---|---|
| Initial | 1001 | 0011 | — | — | 0 | — | — |
| $T_1$ | 0100 | 1001 | 1 | 1 | 0 | 0 | 1 |
| $T_2$ | 0010 | 1100 | 0 | 1 | 1 | 0 | 1 |
| $T_3$ | 1001 | 0110 | 0 | 0 | 1 | 1 | 0 |
| $T_4$ | 1100 | 0011 | 1 | 0 | 0 | 1 | 0 |

$x$ and $y$ are the bits of $A$ and $B$ added in each step (before the shift); the registers show their contents after the pulse.

### 7.2.3 Universal shift register

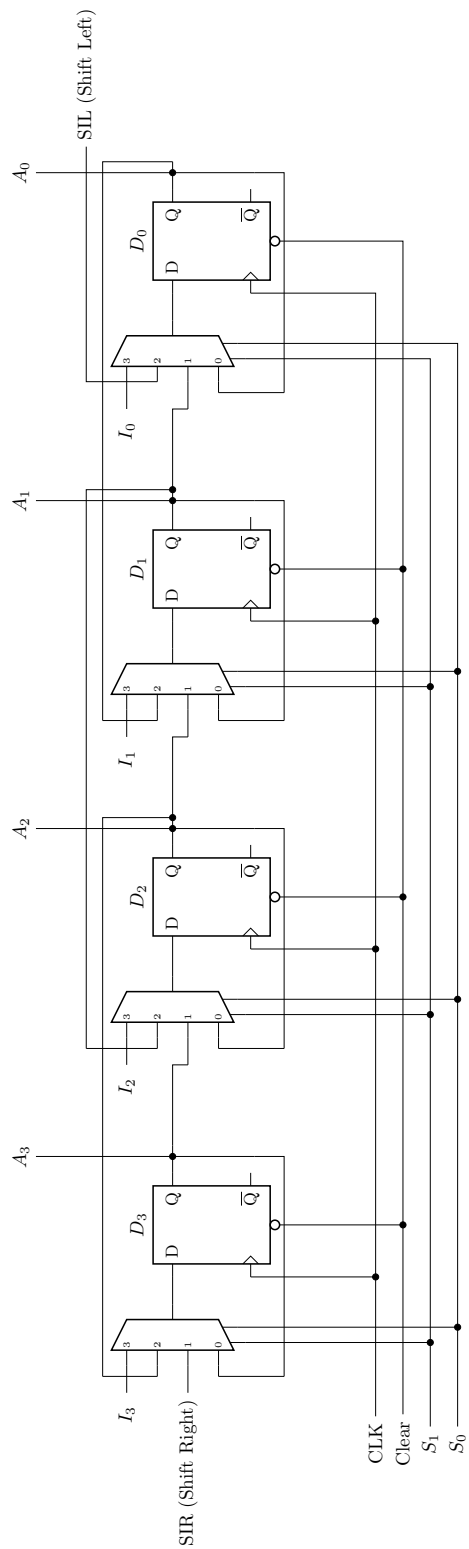
A *universal shift register* combines several operations in one register: clear, shift right, shift left, parallel load and no change. Two mode inputs $s_1 s_0$ select the operation (Table 7.3). In the implementation (Figure 7.6) a 4-to-1 multiplexer at the input of each flip-flop selects among its own output (hold), its left neighbour (shift right), its right neighbour (shift left) and the parallel input. Listing 7.1 is a behavioural VHDL description: the concatenation operator `&` builds the shifted word, e.g. `SR_IN & reg_val(3 downto 1)` places the serial input in the MSB and drops the LSB.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
```




**(a)** block diagram

**Figure 7.6.** A 4-bit universal shift register.




**(b)** implementation with multiplexers

**Figure 7.6.** A 4-bit universal shift register.

**Table 7.3.** Function table of a 4-bit universal shift register ($A_i^+$ is the next value of $A_i$).

| $s_1$ | $s_0$ | Operation | $A_3^+$ | $A_2^+$ | $A_1^+$ | $A_0^+$ |
|---|---|---|---|---|---|---|
| 0 | 0 | no change | $A_3$ | $A_2$ | $A_1$ | $A_0$ |
| 0 | 1 | shift right | SIR | $A_3$ | $A_2$ | $A_1$ |
| 1 | 0 | shift left | $A_2$ | $A_1$ | $A_0$ | SIL |
| 1 | 1 | parallel load | $I_3$ | $I_2$ | $I_1$ | $I_0$ |

```
4
5  entity universal_shift_register is
6      port ( CLK, CLR      : in  std_logic;
7             S             : in  std_logic_vector(1 downto 0); -- Mode select
8             P_IN          : in  std_logic_vector(3 downto 0); -- Parallel input
9             SL_IN, SR_IN : in  std_logic;                     -- Serial inputs
10            Q_OUT         : out std_logic_vector(3 downto 0) );
11 end universal_shift_register;
12
13 architecture behavioral of universal_shift_register is
14     signal reg_val : std_logic_vector(3 downto 0);
15 begin
16     process(CLK, CLR)
17     begin
18         if CLR = '1' then
19             reg_val <= (others => '0');
20         elsif rising_edge(CLK) then
21             case S is
22                 when "00" => reg_val <= reg_val; -- No change
23                 when "01" => reg_val <= SR_IN & reg_val(3 downto 1); -- Shift
       Right
24                 when "10" => reg_val <= reg_val(2 downto 0) & SL_IN; -- Shift Left
25                 when "11" => reg_val <= P_IN; -- Parallel Load
26                 when others => reg_val <= reg_val;
27             end case;
28         end if;
29     end process;
30     Q_OUT <= reg_val;
31 end behavioral;
```

**Listing 7.1.** Behavioural VHDL of a 4-bit universal shift register with asynchronous clear.

**Shifts are multiplications**

Shifting an unsigned binary number one place to the left multiplies it by 2 (provided no 1 is shifted out), and shifting right with a 0 entering the MSB divides it by 2 (discarding the remainder). Shifters are therefore used to multiply and divide by powers of two, and in the shift-and-add algorithms for multiplication.

## 7.3 Counters

A *counter* is a register that goes through a predetermined sequence of states when it is driven by input pulses, most often the clock. The sequence can be a binary count or any other. Counters are classified by how the clock reaches the flip-flops (Figure 7.7):

- in a *ripple* (asynchronous) counter, the output of one flip-flop clocks the next, so there is no common clock;
- in a *synchronous* counter, a common clock drives all flip-flops simultaneously.

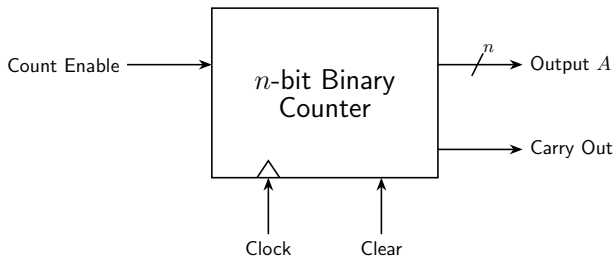



**Figure 7.7.** General $n$-bit counter.

### 7.3.1 Ripple counters

In a *ripple counter* the clock is applied only to the first flip-flop, and the output of each flip-flop provides the clock of the next (Figure 7.8). Each stage is a T flip-flop with $T = 1$ (a JK flip-flop with $J = K = 1$), so each stage divides the frequency of its clock by 2, and the outputs form a binary count. The counter is very simple, but it is slower than a synchronous counter because the changes propagate, or "ripple", along the chain.

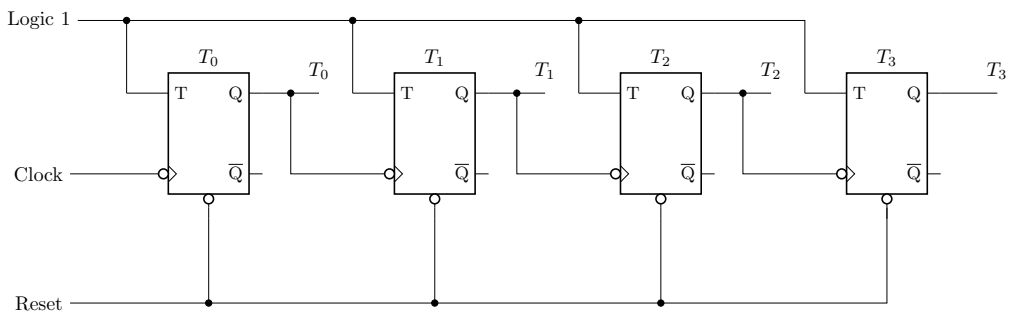



**Figure 7.8.** A 4-bit binary ripple counter built from T flip-flops with $T = 1$.

**Characteristics.** The flip-flops change one after another, not together (Figure 7.9).

- **Propagation delay.** Each flip-flop has a delay $t_{pd}$ between a clock edge and the

change of its output; in a ripple counter the delays add, so an $n$-bit counter needs up to $n\,t_{pd}$ to settle after a clock edge.

- **Glitches.** While the counter ripples to its final state, the outputs pass through *invalid intermediate states* (for example 0111 → 0110 → 0100 → 0000 when counting from 7 to 8). Combinational logic or decoders connected to the outputs may see these states and produce momentary false outputs.
- **Frequency limit.** The next clock edge must not arrive before the previous ripple has finished: $f_{clk} < 1/(n\,t_{pd})$.

For example, a 4-bit ripple counter with $t_{pd} = 10$ ns per stage settles in at most 40 ns, so the clock must be slower than 25 MHz.

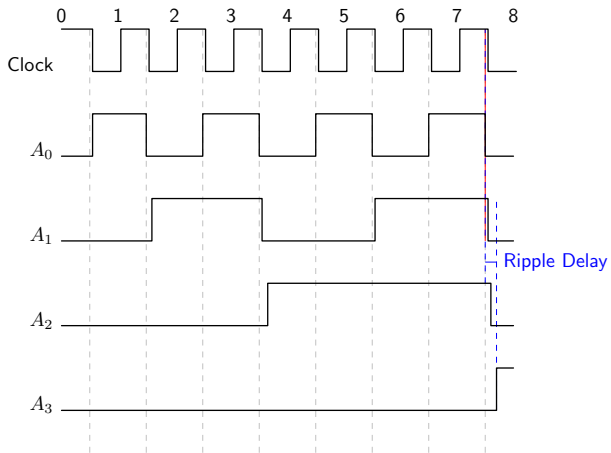
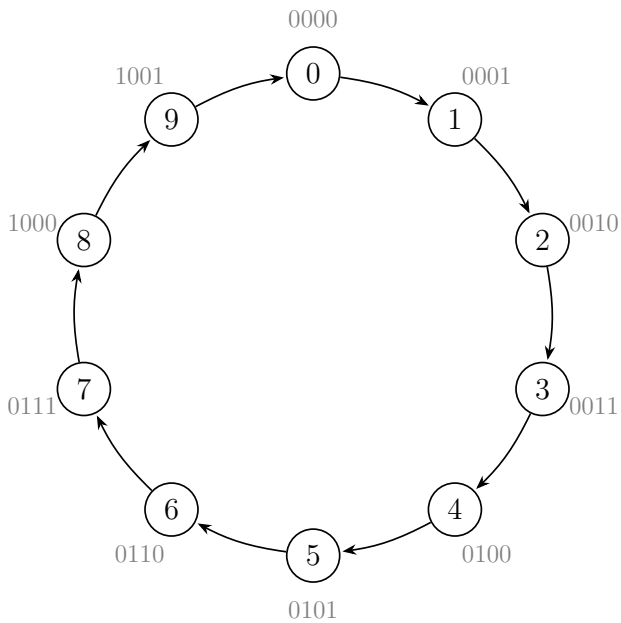



**Figure 7.9.** Timing diagram of a ripple counter: each flip-flop changes a delay after the previous one.

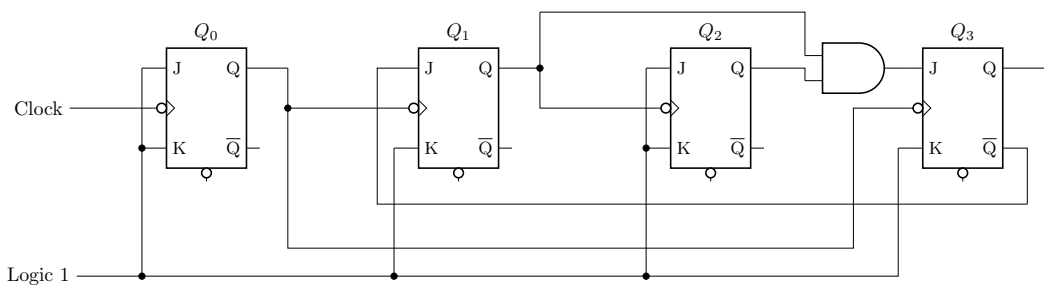
### 7.3.2 BCD ripple counter

A *BCD counter* counts from 0 to 9 (`0000` to `1001`) and returns to 0 (Figure 7.10a). BCD counters are essential in instruments and clocks that display decimal digits. A BCD ripple counter is obtained from the binary ripple counter by modifying two stages so that the state 1001 is followed by 0000 and states 10–15 are skipped (Figure 7.10b).

- The $J$ input of the $Q_1$ flip-flop is connected to $\overline{Q_3}$. When the count reaches 1000 ($Q_3 = 1$), $J = 0$ and $Q_1$ cannot toggle to 1 on the next pulse, so the sequence 1001 → 1010 never occurs.
- The $J$ input of the $Q_3$ flip-flop is connected to $Q_2Q_1$, so $Q_3$ sets only on the transition 0111 → 1000; $Q_3$ is reset on the transition 1001 → 0000 by its own $K = 1$.




**(a)** state diagram




**(b)** BCD ripple counter

**Figure 7.10.** The BCD counter.

### 7.3.3 Synchronous binary counters

In a *synchronous counter* all flip-flops are clocked together, and combinational logic at the $J$, $K$ or $T$ inputs decides which of them toggle. There is no accumulation of delays, so synchronous counters are used for high frequencies. They are designed with the standard procedure of Section 6.7. From the 3-bit design of Example 6.5 we know that in a binary up-counter a bit toggles when *all lower-order bits are 1*:

$$T_{A_i} = A_{i-1}A_{i-2}\cdots A_0, \quad T_{A_0} = 1.$$

Figure 7.11 shows the 4-bit counter: the toggle condition of each stage is formed by a chain of AND gates. The delay of a synchronous counter is therefore the flip-flop delay plus the delay of one AND chain, independent of the number of bits if wider AND gates or a lookahead structure are used.

### 7.3.4 Up/down counter

An *up/down counter* counts up when $Up = 1$ and down when $Down = 1$ (Figure 7.12a). In the down direction a bit toggles when all lower-order bits are 0, so

$$T_{A_i} = Up \cdot A_{i-1}\cdots A_0 \;+\; Down \cdot \overline{A_{i-1}}\cdots\overline{A_0}.$$

Figure 7.12b shows the circuit.

### 7.3.5 Counter with parallel load

Counters in digital systems often need to be preset to a given value as well as counted. Table 7.4 gives the function table of a counter with *clear*, *load* and *count* controls (Figure 7.13), with a priority order: clear first, then load, then count. Listing 7.2 is the VHDL; the type `unsigned` of the package `numeric_std` allows the expression `count_val + 1`.

**Table 7.4.** Function table of a 4-bit counter with parallel load (the clear is asynchronous and active-high in the VHDL of Listing 7.2).

| Clear | CLK | Load | Count | Function |
|---|---|---|---|---|
| 1 | X | X | X | clear to 0 |
| 0 | ↑ | 1 | X | load parallel inputs |
| 0 | ↑ | 0 | 1 | count up |
| 0 | ↑ | 0 | 0 | no change |

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity counter_parallel_load is
6     port ( CLK, CLR, LOAD, COUNT : in  std_logic;
```

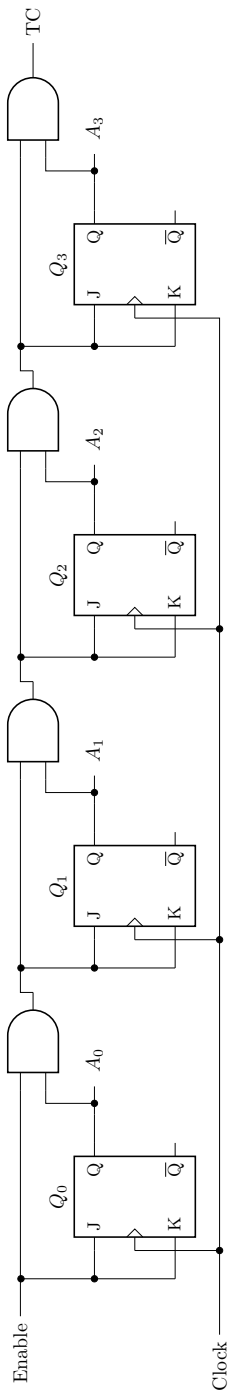
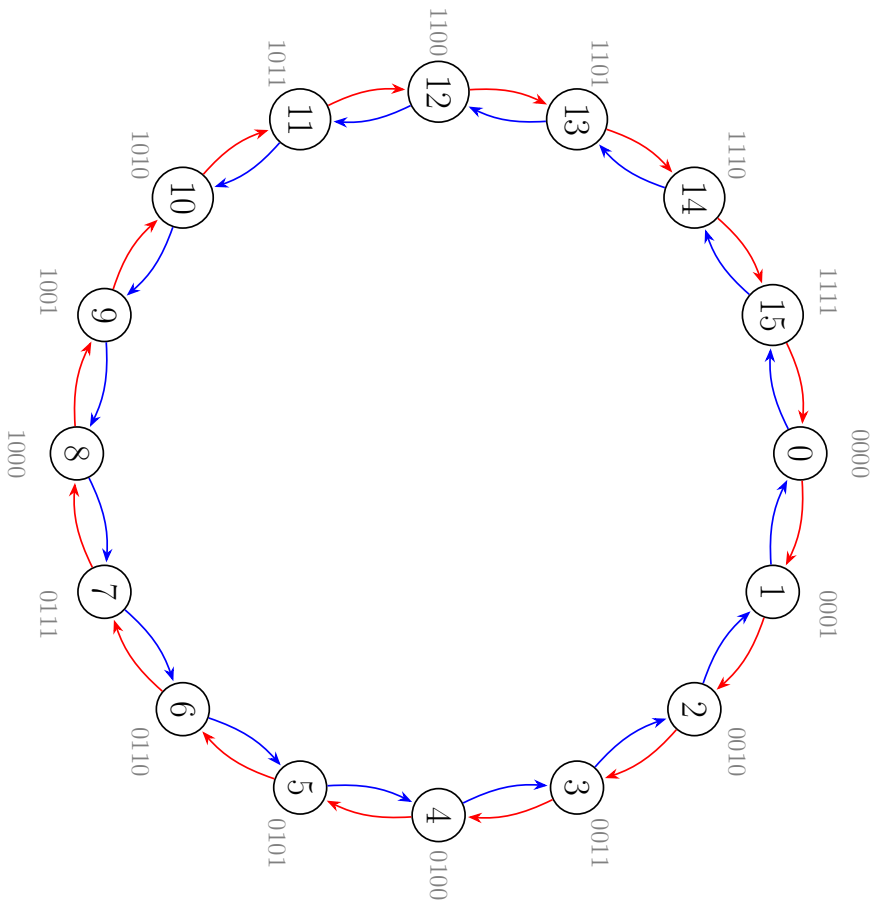
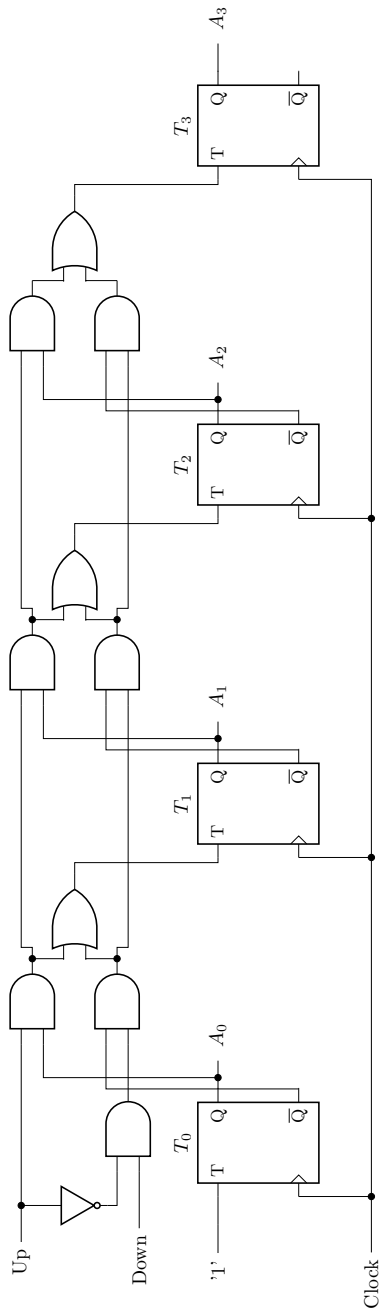



**Figure 7.11.** A 4-bit synchronous binary counter with T flip-flops.




(a) state diagram

**Figure 7.12.** Up/down counter.




**(b)** 4-bit synchronous up/down counter

**Figure 7.12.** Up/down counter.

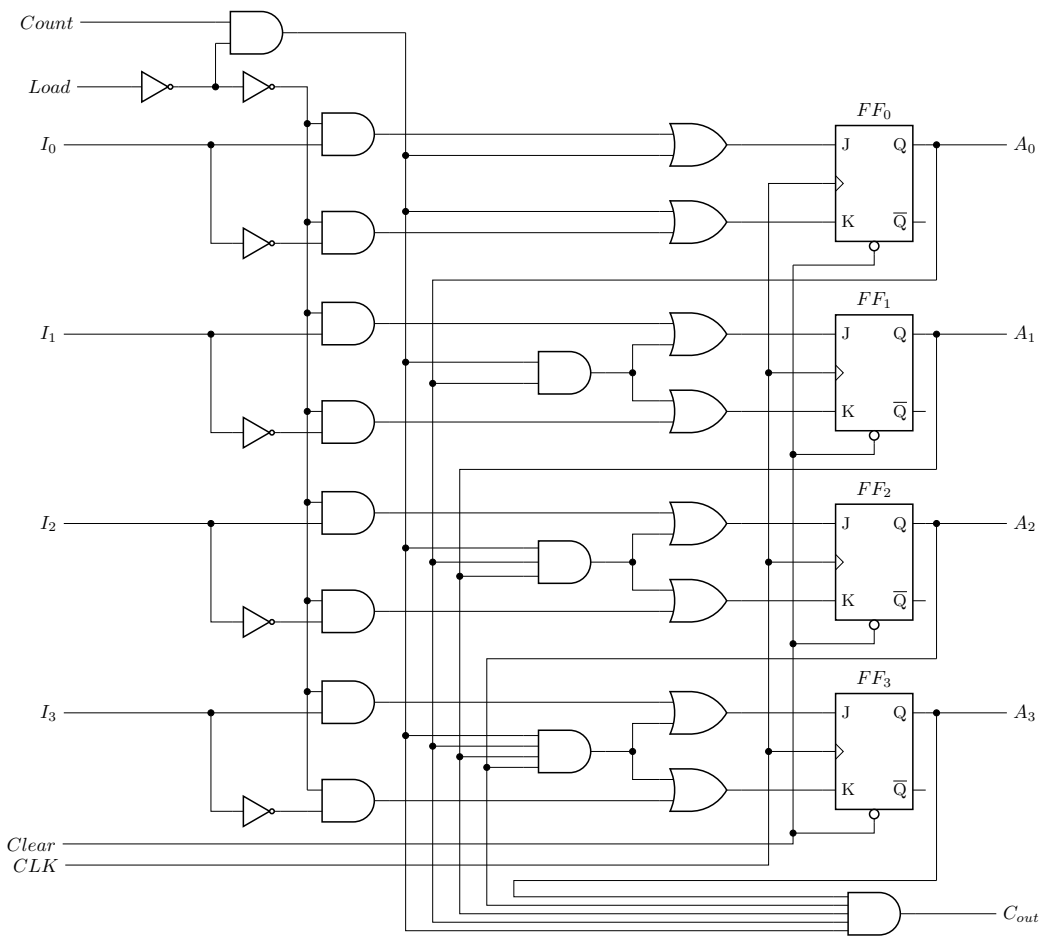



**Figure 7.13.** A 4-bit counter with parallel load.

```
7           P_IN                : in  std_logic_vector(3 downto 0);
8           Q_OUT               : out std_logic_vector(3 downto 0) );
9  end counter_parallel_load;
10
11 architecture behavioral of counter_parallel_load is
12     signal count_val : unsigned(3 downto 0);
13 begin
14     process(CLK, CLR)
15     begin
16         if CLR = '1' then
17             count_val <= (others => '0');
18         elsif rising_edge(CLK) then
19             if LOAD = '1' then
20                 count_val <= unsigned(P_IN);
21             elsif COUNT = '1' then
22                 count_val <= count_val + 1;
23             end if;
24         end if;
25     end process;
26     Q_OUT <= std_logic_vector(count_val);
27 end behavioral;
```

**Listing 7.2.** VHDL of a 4-bit counter with asynchronous clear, parallel load and count enable.

### 7.3.6 Modulo-$N$ counters

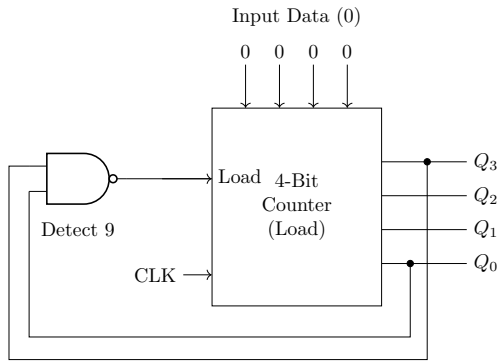
A *modulo-N counter* goes through a repeating sequence of $N$ states and so divides the clock frequency by $N$. The BCD counter is a modulo-10 counter. A modulo-$N$ counter can be built from a binary counter with parallel load or clear in two ways (Figure 7.14):

- **synchronous load**: detect the last count (for BCD, $9 = \texttt{1001}$) and, on the *next clock edge*, load `0000`; the count 9 appears for a full clock period;
- **asynchronous clear**: detect the first *invalid* count ($10 = \texttt{1010}$) and clear the counter *immediately*; the state 10 appears for a brief moment (a glitch) before the counter resets, which may disturb other circuits.

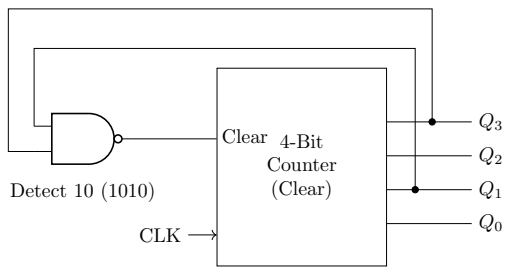
The synchronous method is cleaner. For BCD the detection of 9 needs only $Q_3 Q_0$ since 9 is the only count between 0 and 9 with both bits 1.

### 7.3.7 Self-correcting counters

A counter with *unused* states can enter one of them by noise or at power-up. If there is no path out of the unused states, the counter is stuck. A *self-correcting* design makes sure that every unused state eventually leads to a valid state. In the modulo-6 counter of Figure 7.16 the unused states 110 and 111 both lead to 000 (Table 7.5). Note that don't-cares used during minimisation must be checked afterwards: the next states of the unused states are whatever the simplified equations produce.




**(a)** synchronous load (detect 9)




**(b)** asynchronous clear (detect 10)

**Figure 7.14.** Two ways of making a BCD counter from a binary counter,.

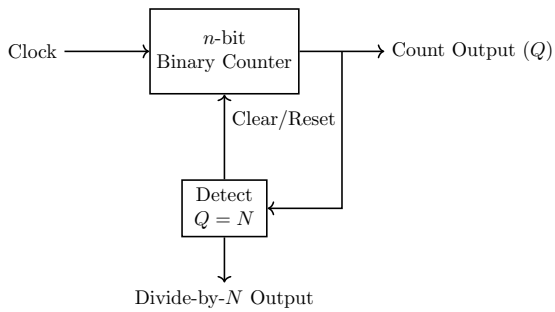



**Figure 7.15.** Block diagram of a modulo-$N$ counter.

**Table 7.5.** State table of the self-correcting modulo-6 counter.

| $Q_2$ | $Q_1$ | $Q_0$ | $Q_2^+$ | $Q_1^+$ | $Q_0^+$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 |

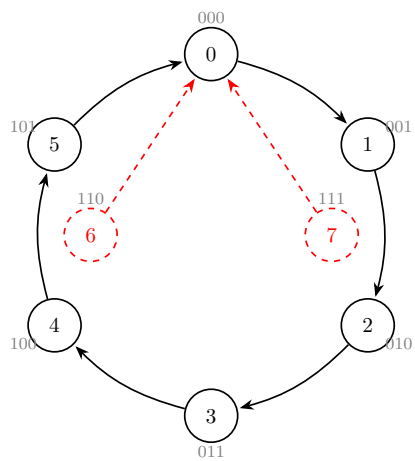


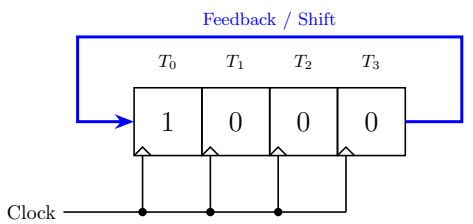

**Figure 7.16.** State diagram of the self-correcting modulo-6 counter.

### 7.3.8 Ring counters

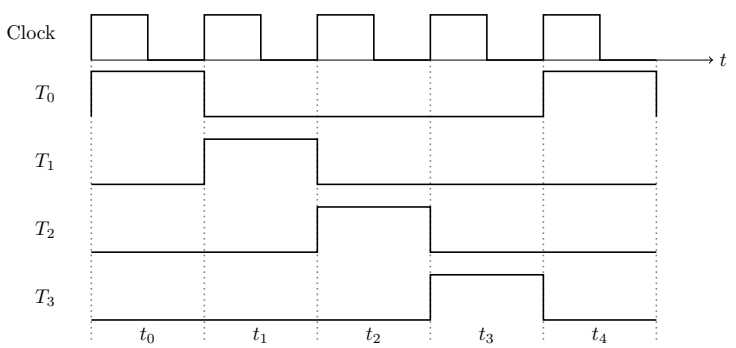
A *ring counter* is a circular shift register in which a single 1 circulates (Figure 7.17a): it is initialised to 1000 and the 1 moves one place at every pulse, producing the sequence of Table 7.6. An $n$-bit ring counter has $n$ states, and the output of each flip-flop is high for one clock period out of $n$, so the outputs are $n$ non-overlapping timing signals (Figure 7.17b) with no decoder needed. Each output has a frequency $f_{clk}/n$. A *Johnson* (twisted-ring) counter feeds back the *complement* of the last output instead, which doubles the number of states to $2n$ at the cost of a decoder.

**Table 7.6.** Sequence of a 4-bit ring counter.

| Clock | $T_0$ | $T_1$ | $T_2$ | $T_3$ |
|---|---|---|---|---|
| $t_0$ | 1 | 0 | 0 | 0 |
| $t_1$ | 0 | 1 | 0 | 0 |
| $t_2$ | 0 | 0 | 1 | 0 |
| $t_3$ | 0 | 0 | 0 | 1 |
| $t_4$ | 1 | 0 | 0 | 0 |




**(a)** 4-bit ring counter




**(b)** timing signals

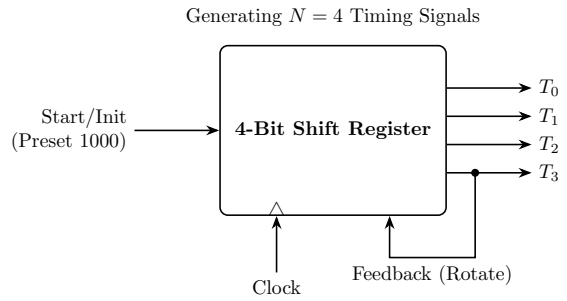
**Figure 7.17.** Ring counter and the timing signals it generates.

### 7.3.9 Application: generating timing signals

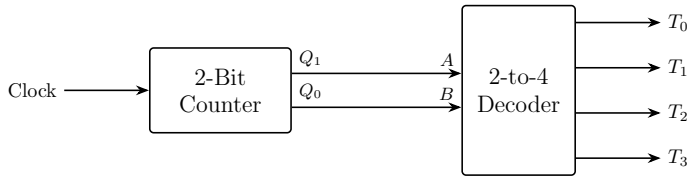
Controllers need timing signals that start the successive steps of an operation. Two methods are common (Figure 7.18):

1. a **ring counter**: $N$ flip-flops give $N$ signals directly, with no decoding, glitch-free and fast, but it needs many flip-flops;
2. a **counter and decoder**: an $n$-bit binary counter followed by an $n$-to-$2^n$ decoder gives $N = 2^n$ signals with only $n$ flip-flops, at the cost of the decoder gates (and, with a ripple counter, glitches).

For 16 signals the ring counter needs 16 flip-flops, whereas the counter and decoder need 4 flip-flops and a 4-to-16 decoder. Timing signals of this kind drive the control unit of the processors in Chapters 9 and 10.




**(a)** ring counter




**(b)** counter and decoder

**Figure 7.18.** Two timing-signal generators.

**Chapter Summary**

- A **register** is a group of flip-flops; **parallel load** uses a multiplexer at each input to choose between new data and hold.
- A **shift register** moves its bits one place per clock; serial transfer and the **serial adder** (one full adder and a carry flip-flop) take $n$ clock periods. The **universal shift register** supports hold, shift right, shift left and load.
- **Ripple counters** are simple but their delays accumulate ($f_{clk} < 1/(n\,t_{pd})$) and they glitch. **Synchronous counters** toggle bit $i$ when all lower bits are 1 ($T_i =$

$A_{i-1} \cdots A_0$).

- **Up/down counters** combine both toggle conditions. **Counters with parallel load** can be preset and truncated: a modulo-$N$ counter uses synchronous load (detect $N-1$) or asynchronous clear (detect $N$).
- **Self-correcting** counters guarantee that unused states lead to a valid state.
- A **ring counter** generates $N$ timing signals with $N$ flip-flops; a counter with a decoder uses only $\log_2 N$ flip-flops.

## Exercises

**7.1** **Register design.** Design a 4-bit register with inputs `CLK`, `CLR`, `LD`, `INC` and `I[3:0]`. `CLR = 1` resets it to `0000` at the clock edge; `LD = 1` loads `I` (with priority over `INC`); `INC = 1` increments it; otherwise it holds its value. Write the behavioural VHDL.

**7.2** **Shift register analysis.** A 4-bit universal shift register (Table 7.3) initially holds `1010`. Find its content after (a) shift right with SIR = 1; then (b) shift left with SIL = 0; then (c) no change; then (d) parallel load of $I =$ `1100`.

**7.3** **Counter design.** Design a synchronous modulo-6 counter (sequence 0, 1, 2, 3, 4, 5, 0, ...) with JK flip-flops. Draw the state diagram and the excitation table, and find the $J$ and $K$ equations. Does the counter correct itself from the unused states 6 and 7?

**7.4** **Timing diagram.** Draw the timing diagram of a 4-bit ring counter initially in the state `1000` ($Q_3 = 1$) for five clock pulses. What is the relation between the frequency of a single output $Q_i$ and the clock frequency? How does a Johnson counter differ?

**7.5** Trace the serial transfer of $A =$ `1100` into a register $B$ initially `0000` (as in Table 7.1), with $A$ rotating. What do $A$ and $B$ contain after four pulses?

**7.6** Trace the serial addition of $A =$ `0110` and $B =$ `0101` in the format of Table 7.2, and give the sum.

**7.7** A 4-bit ripple counter uses flip-flops with $t_{pd} = 10$ ns. What is the highest clock frequency, and what is the worst-case settling time when the counter changes from 0111 to 1000?

**7.8** Show how to use a 4-bit counter with synchronous parallel load to make a BCD counter. Give the load condition and the data inputs.

**7.9** Derive the $T$ input equations of a 3-bit synchronous *up/down* counter with control inputs *Up* and *Down*.

**7.10** You need eight timing signals. Compare (a) a ring counter and (b) a counter and decoder in terms of the number of flip-flops and other gates.

**7.11** A 4-bit universal shift register contains the unsigned number 0011 (3). What number does it hold after one shift left with SIL = 0? What is the effect on a

number if the shift-left is applied $k$ times? What does a shift right with SIR $= 0$ do to `0110`?

**7.12** Write VHDL for an 8-bit shift register with serial input `sin`, serial output `sout`, a clock and a shift-enable input `en`.

**7.13** Write the VHDL of a modulo-10 counter that counts 0–9 (4-bit output) using `unsigned` arithmetic and a synchronous reset to 0 after 9.

**7.14** Explain, with a timing sketch, how an asynchronous-clear BCD counter can produce a short glitch on the output $Q_1$ when it passes through the state 10.

## Further Reading

Registers and counters are covered in Chapter 6 of Mano and Ciletti [7] and in Chapter 7 of Brown and Vranesic [3], which also treat Johnson counters and the VHDL of counters in more detail; see also Wakerly [8] for the design of synchronous counters and a thorough treatment of the timing of ripple counters.

# 8

# Finite State Machines

**Learning Objectives**

After studying this chapter you will be able to:

- describe a finite state machine (FSM) by its next-state logic, state memory and output logic, and distinguish the Moore from the Mealy model;
- analyse an FSM from its circuit and synthesise one from a word description, using state diagrams, state tables and K-maps;
- apply the design procedure to counters, controllers and one-shot circuits;
- write FSMs in VHDL with the two-process method and the `numeric_std` type conversions;
- compare Moore and Mealy implementations of the same function, in the number of states, the output timing and the risk of glitches; and
- state the clock-period constraint of a synchronous FSM and choose a state encoding.

## 8.1 Finite State Machines

A *finite state machine* (FSM) is a computational model with a finite number of *states*; it moves between states according to its inputs and its present state, and produces outputs. Every synchronous sequential circuit of Chapter 6 is an FSM, and an FSM is the standard way to describe a controller. It has three parts (Figure 8.1):

1. **next-state logic**, a combinational circuit that computes the next state from the present state and the inputs;
2. **state memory**, the flip-flops (the *state register*) that hold the present state and are

updated at each clock edge; and
3. **output logic**, a combinational circuit that generates the outputs.

## 8.2 Moore and Mealy Machines

The two FSM models differ in their output logic (Figure 8.2).

- **Moore machine**: output $= f(\text{present state})$. Outputs change synchronously with the state, are stable for the whole clock period and are glitch-free. The output logic is simple, but the machine often needs more states.
- **Mealy machine**: output $= f(\text{present state}, \text{inputs})$. Outputs can change as soon as the inputs change, which gives a faster response and often *fewer states*, but the output can also show glitches and is not synchronised with the clock.

## 8.3 Analysis of an FSM

*Analysis* determines the behaviour of a given circuit. The steps are those of Section 6.5: (1) determine the flip-flop input equations and output equations; (2) construct the state table; (3) draw the state diagram, with states as nodes and transitions as arrows labelled `input/output`. Examples with D, JK and T flip-flops were given in Chapter 6; here is one with two inputs.

**Example 8.1: Analysis of a one-flip-flop machine**

The circuit of Figure 8.3a has one D flip-flop $A$, two inputs $x$, $y$ and an output $z$.
**Equations.** $D_A = A \oplus x \oplus y$ and $z = Ax$. Since $A^+ = D_A$, the state table is Table 8.1. For example, in state $A = 1$ with $x = 1, y = 0$, $A^+ = 1 \oplus 1 \oplus 0 = 0$ and $z = 1$.
**Diagram.** The machine has two states. The output depends on the input $x$, so it is a Mealy machine (Figure 8.3b).

**Table 8.1.** State table of Example 8.1.

| PS $A$ | $x$ | $y$ | NS $A^+ = A \oplus x \oplus y$ | $z = Ax$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

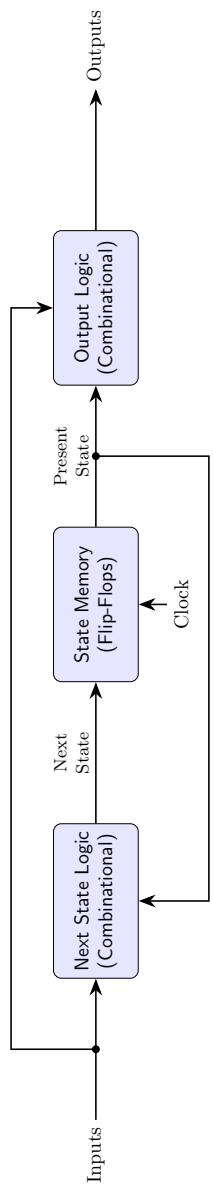



**Figure 8.1.** General structure of an FSM: next-state logic, state register and output logic.

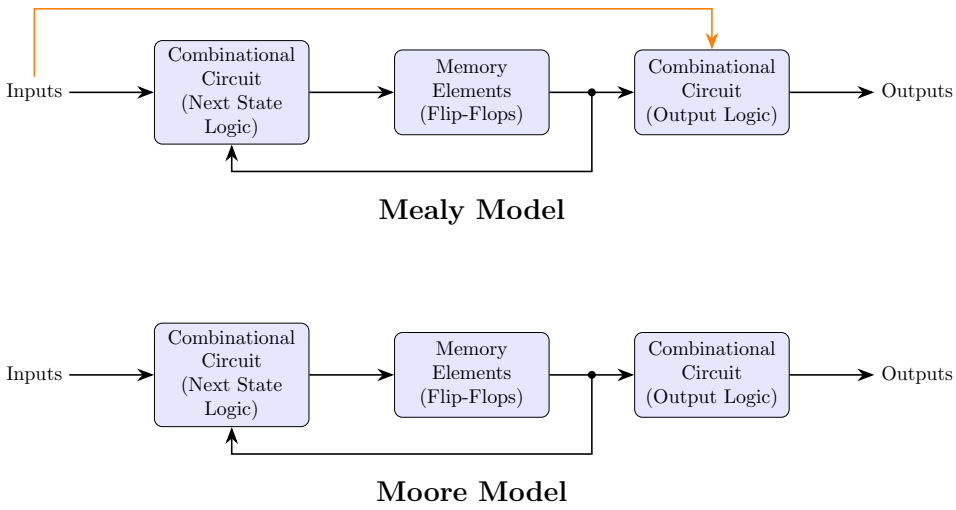
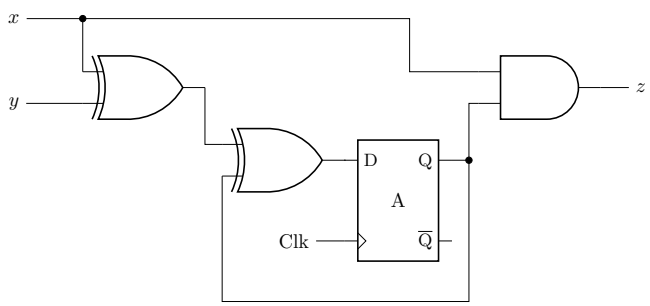
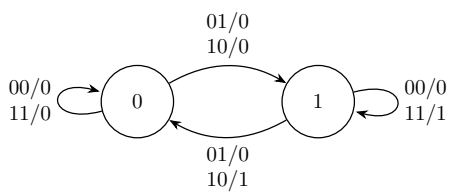



**Figure 8.2.** Block diagrams of the Moore and Mealy models; the only difference is whether the inputs feed the output logic.




**(a)** circuit




**(b)** state diagram

**Figure 8.3.** The circuit analysed in Example 8.1.

## 8.4 Synthesis of an FSM

*Synthesis* creates a circuit from a description of behaviour.

1. **Understand the specification**: inputs, outputs, initial state.
2. Create the **state diagram**.
3. Create the **state table**.
4. (Optional) **state minimisation**: merge equivalent states.
5. **State assignment**: assign binary codes to states.
6. Choose the **flip-flop type** (D, JK, T).
7. Derive the **logic equations** for the flip-flop inputs and outputs with K-maps.
8. Draw the **logic diagram** and, in a modern flow, write the VHDL.

**State encoding.** A machine with $N$ states needs at least $\lceil \log_2 N \rceil$ flip-flops. *Binary* encoding uses this minimum, but the next-state logic can be complex. *Gray* encoding gives adjacent states that differ in one bit, which suits counters and reduces glitches. *One-hot* encoding uses $N$ flip-flops, exactly one of which is 1 in each state; it needs more flip-flops but simpler and faster logic, and is popular in FPGAs, which are rich in flip-flops. In VHDL an enumerated state type leaves the choice of encoding to the synthesis tool.

## 8.5 Design Examples

### 8.5.1 A 2-bit up/down counter

**Example 8.2: 2-bit up/down counter**

Design a synchronous 2-bit counter that counts up when $x = 1$ and down when $x = 0$, using JK flip-flops.

**State table.** The states are 00, 01, 10, 11. For $x = 1$: $00 \to 01 \to 10 \to 11 \to 00$; for $x = 0$: $00 \to 11 \to 10 \to 01 \to 00$. The excitation table (Table 8.2) is obtained from the transitions of each flip-flop with the JK excitation table of Table 6.9.

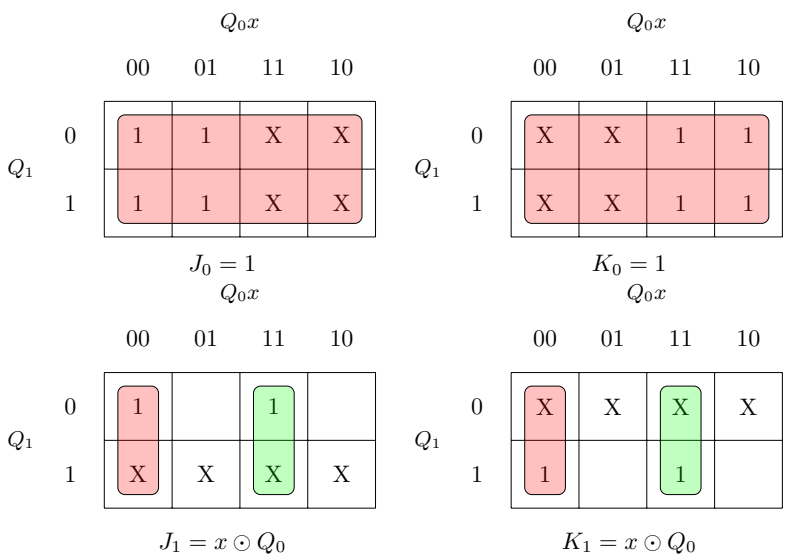
**Equations.** All entries for $Q_0$ are 1 or X, so $J_0 = K_0 = 1$ ($Q_0$ toggles on every clock). For $Q_1$ the K-maps (Figure 8.4a) give

$$J_1 = K_1 = x \odot Q_0 = \overline{x \oplus Q_0}.$$

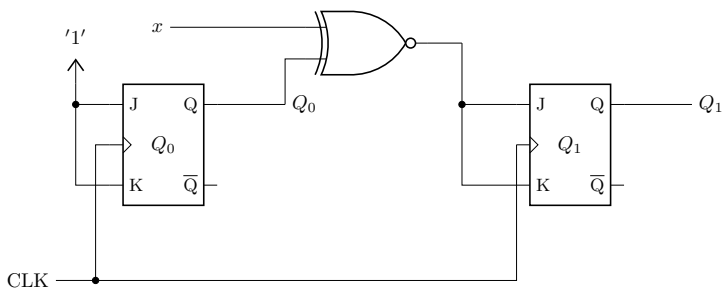
Check: counting up ($x = 1$), $Q_1$ must toggle when $Q_0 = 1$; counting down ($x = 0$) it toggles when $Q_0 = 0$; in both cases $x \odot Q_0 = 1$. The circuit is Figure 8.4b.

**Table 8.2.** Excitation table of the 2-bit up/down counter.

| PS $Q_1$ | PS $Q_0$ | Input $x$ | NS $Q_1^+$ | NS $Q_0^+$ | $J_1$ | $K_1$ | $J_0$ | $K_0$ |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | X | 1 | X |
| 0 | 0 | 1 | 0 | 1 | 0 | X | 1 | X |
| 0 | 1 | 0 | 0 | 0 | 0 | X | X | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | X | X | 1 |
| 1 | 0 | 0 | 0 | 1 | X | 1 | 1 | X |
| 1 | 0 | 1 | 1 | 1 | X | 0 | 1 | X |
| 1 | 1 | 0 | 1 | 0 | X | 0 | X | 1 |
| 1 | 1 | 1 | 0 | 0 | X | 1 | X | 1 |




**(a)** K-maps




**(b)** logic diagram

**Figure 8.4.** Synthesis of the 2-bit up/down counter.

### 8.5.2 A car security system (Moore)

**Example 8.3: Car security controller**

Design a controller with inputs $M$ (master switch), $D$ (door sensor), $V$ (vibration sensor) and output $A$ (alarm siren). The system is initially disarmed. If $M = 1$ it is armed; if it is armed and a door or the vibration sensor trips, the siren sounds. Once the siren sounds it stays on until $M = 0$.

**State diagram.** Two states: *No_Siren* ($Q = 0, A = 0$) and *Siren* ($Q = 1, A = 1$) (Figure 8.5a). The output depends only on the state: a Moore machine.

**State table** (Table 8.3). The K-maps (Figure 8.5b) give

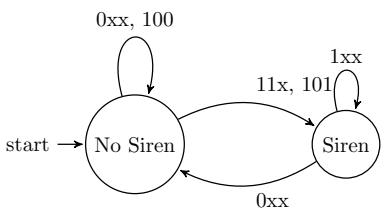
$$Q^+ = M\,(Q + D + V), \qquad A = Q.$$

The circuit is Figure 8.5c and Listing 8.1 the VHDL.

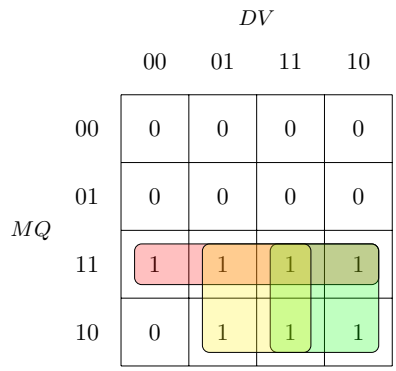
**Table 8.3.** State table of the car security controller.

| State $Q$ | $M$ | $D$ | $V$ | $Q^+$ | $A$ |
|---|---|---|---|---|---|
| 0 (No_Siren) | 0 | X | X | 0 | 0 |
| 0 (No_Siren) | 1 | 0 | 0 | 0 | 0 |
| 0 (No_Siren) | 1 | 1 | X | 1 | 0 |
| 0 (No_Siren) | 1 | X | 1 | 1 | 0 |
| 1 (Siren) | 0 | X | X | 0 | 1 |
| 1 (Siren) | 1 | X | X | 1 | 1 |

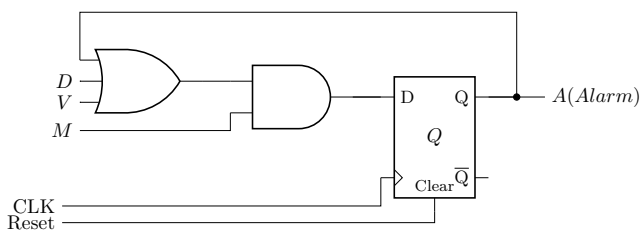
```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity Car_Security is
5      Port ( clk, reset : in std_logic;
6             M, D, V : in std_logic;
7             A : out std_logic);
8  end Car_Security;
9
10 architecture Behavioral of Car_Security is
11     type state_type is (No_Siren, Siren);
12     signal current_state, next_state : state_type;
13 begin
14     -- State Register
15     process(clk, reset)
16     begin
17         if reset = '1' then
18             current_state <= No_Siren;
19         elsif rising_edge(clk) then
20             current_state <= next_state;
21         end if;
22     end process;
```




**(a)** state diagram




**(b)** K-map




**(c)** circuit

**Figure 8.5.** Car security controller.

```
23
24      -- Next State and Output Logic
25      process(current_state, M, D, V)
26      begin
27          -- Default assignments
28          next_state <= current_state;
29          A <= '0';
30
31          case current_state is
32              when No_Siren =>
33                  if M='1' and (D='1' or V='1') then
34                      next_state <= Siren;
35                  end if;
36              when Siren =>
37                  A <= '1';
38                  if M='0' then
39                      next_state <= No_Siren;
40                  end if;
41          end case;
42      end process;
43  end Behavioral;
```

**Listing 8.1.** VHDL of the car security controller: an enumerated state type, a state-register process and a combinational process (the two-process method).

The VHDL uses an *enumerated type* to give names to the states, so the designer does not have to choose binary codes by hand. It follows the *two-process method*: a *sequential* process updates `current_state` on the clock edge (with an asynchronous reset), and a *combinational* process computes `next_state` and the outputs from `current_state` and the inputs. Default assignments at the beginning of the combinational process (`next_state <= current_state;` and `A <= '0';`) guarantee that every signal is assigned in every path, which prevents unintended latches.

### 8.5.3 A modulo-6 counter with enable

**Example 8.4: Modulo-6 up-counter**

Design a synchronous counter that counts $0, 1, \ldots, 5$ and repeats, with a count-enable input $C$ and an output $Y = 1$ when the state is 5, using D flip-flops.
**State table.** States 000 to 101 (Table 8.4); the unused states 110 and 111 are don't-cares. K-maps (Figure 8.6b) give

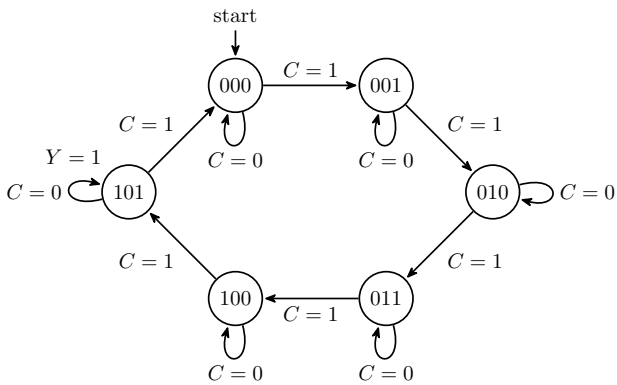
$$D_2 = Q_2Q_0' + Q_2C' + Q_1Q_0C, \qquad D_1 = Q_1C' + Q_1Q_0' + Q_2'Q_1'Q_0C,$$
$$D_0 = Q_0 \oplus C, \qquad Y = Q_2Q_1'Q_0.$$

The circuit is Figure 8.6c. Listing 8.2 is a behavioural VHDL description of the same machine, which is much shorter than the equations: *describing what the ma-*

*chine does* lets the tool derive the logic.

**Table 8.4.** State table of the modulo-6 counter with enable $C$.

| PS | NS, $C = 0$ | NS, $C = 1$ | $Y$ |
|---|---|---|---|
| 0 0 0 | 0 0 0 | 0 0 1 | 0 |
| 0 0 1 | 0 0 1 | 0 1 0 | 0 |
| 0 1 0 | 0 1 0 | 0 1 1 | 0 |
| 0 1 1 | 0 1 1 | 1 0 0 | 0 |
| 1 0 0 | 1 0 0 | 1 0 1 | 0 |
| 1 0 1 | 1 0 1 | 0 0 0 | 1 |
| 1xx (110, 111) | xxx | xxx | x |



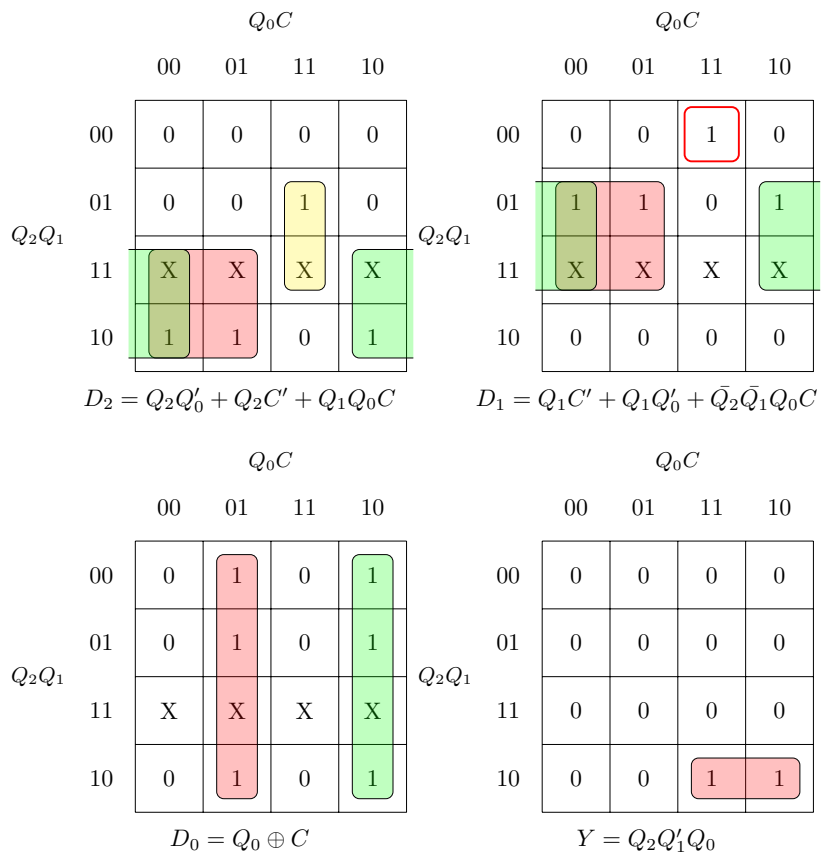

**(a)** state diagram

**Figure 8.6.** Modulo-6 counter with enable.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

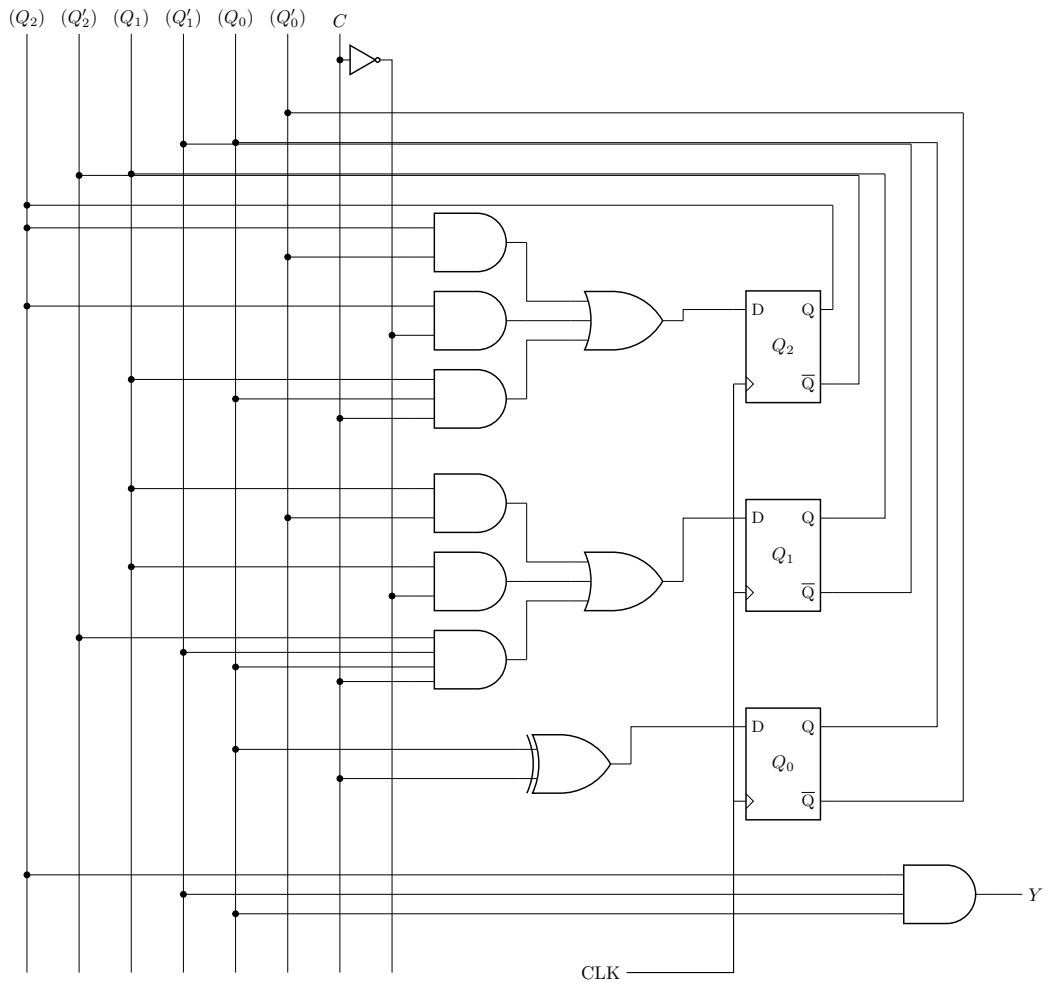
entity Modulo6_Behav is
    Port ( clk, reset, C : in std_logic;
           Y : out std_logic);
end Modulo6_Behav;

architecture Behav of Modulo6_Behav is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, reset)
    begin
        if reset='1' then
            count <= "000";
        elsif rising_edge(clk) then
```




**(b)** K-maps

**Figure 8.6.** Modulo-6 counter with enable.




**(c)** logic diagram

**Figure 8.6.** Modulo-6 counter with enable.

```
            if C = '1' then
                if count = "101" then
                    count <= "000";
                else
                    count <= std_logic_vector(unsigned(count) + 1);
                end if;
            end if;
        end if;
    end process;

    -- Output Logic
    Y <= '1' when count = "101" else '0';
end Behav;
```

**Listing 8.2.** Behavioural VHDL of the modulo-6 counter with enable.

**Type conversions in VHDL.** Listing 8.2 adds one to a vector with the expression

```
std_logic_vector(unsigned(count) + 1)
```

VHDL is a strongly typed language: arithmetic is defined for the types `unsigned` and `signed` of the package `numeric_std`, not for `std_logic_vector`, so one converts explicitly (Figure 8.7). Type-cast functions: `unsigned(slv)` and `signed(slv)` reinterpret a `std_logic_vector`; `std_logic_vector(u)` converts back. Integers are converted with `to_unsigned(int, size)` and `to_signed(int, size)`, and numbers with `to_integer(u)` or `to_integer(s)`. The conversions of vectors are only reinterpretations of the same bits and cost no hardware.

### 8.5.4 A one-shot circuit

**Example 8.5: One-shot**

Design a circuit that produces an output pulse $S = 1$ for *exactly one clock cycle* when a push-button input $B$ is pressed, however long the button is held. (Assume the button is already debounced.)

**Mealy design.** Three states: $S_0$ waits for $B = 1$; $S_1$ is the pulse state, always followed by $S_2$ on the next clock; $S_2$ waits for $B = 0$ (release) and then returns to $S_0$. Encoding $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, with the unused state 11 returning to 00 (Table 8.5, Figure 8.8a), and taking the output as $S = 1$ in $S_0$ when $B = 1$ (the Mealy output), the K-maps (Figure 8.8b) give

$$D_1 = Q_1'Q_0 + Q_1Q_0'B, \qquad D_0 = Q_1'Q_0'B, \qquad S = Q_1'Q_0'B.$$

The circuit is Figure 8.8c, the response to a long press is Figure 8.8d, and Listing 8.3 the VHDL.

```
library ieee;
```

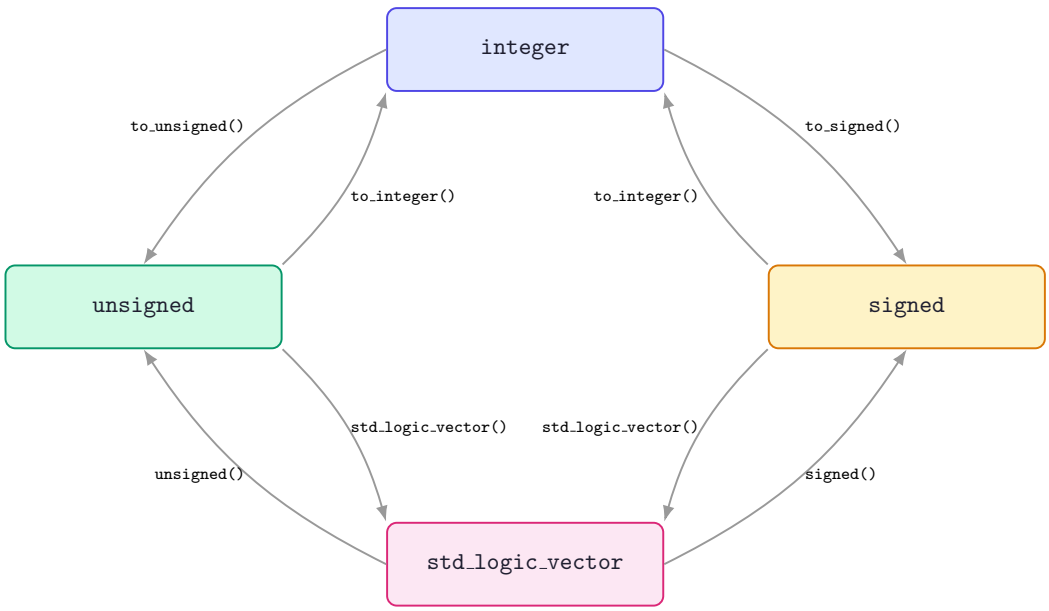
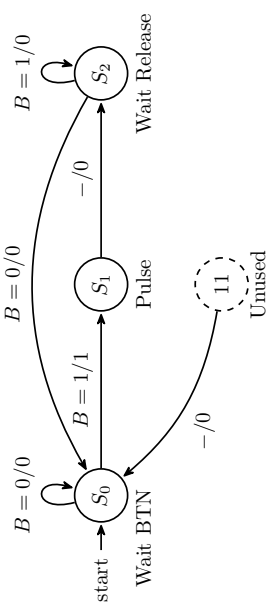



**Figure 8.7.** Conversions among `std_logic_vector`, `unsigned`, `signed` and `integer` in `numeric_std`.

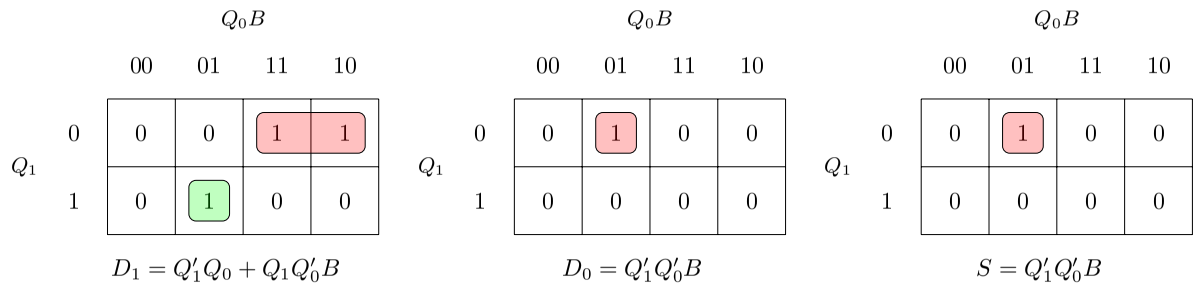
**Table 8.5.** State table of the Mealy one-shot.

| PS $(Q_1Q_0)$ | NS $(B = 0)$ | NS $(B = 1)$ | $S$ (with $B = 1$) |
|---|---|---|---|
| $S_0$ (00) | 00 | 01 | 1 |
| $S_1$ (01) | 10 | 10 | 0 |
| $S_2$ (10) | 00 | 10 | 0 |
| unused (11) | 00 | 00 | 0 |



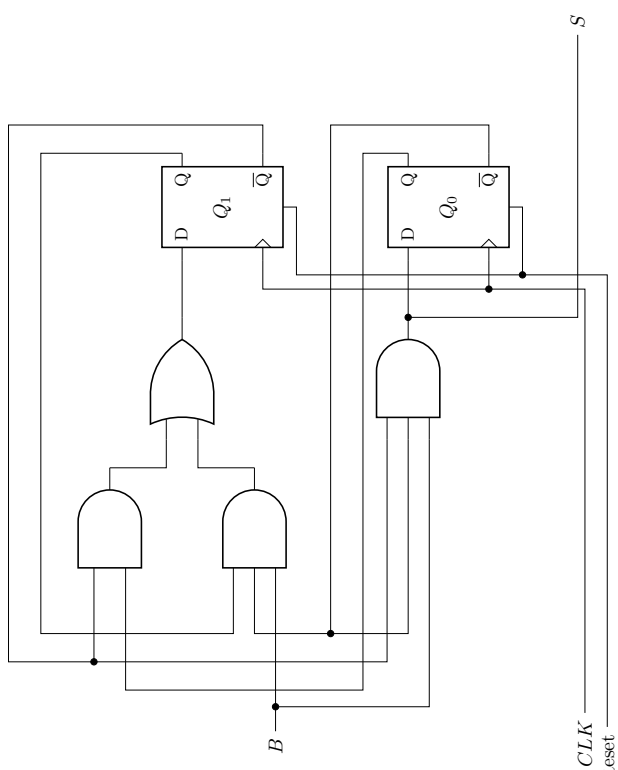

**(a)** state diagram

**Figure 8.8.** The Mealy one-shot circuit.



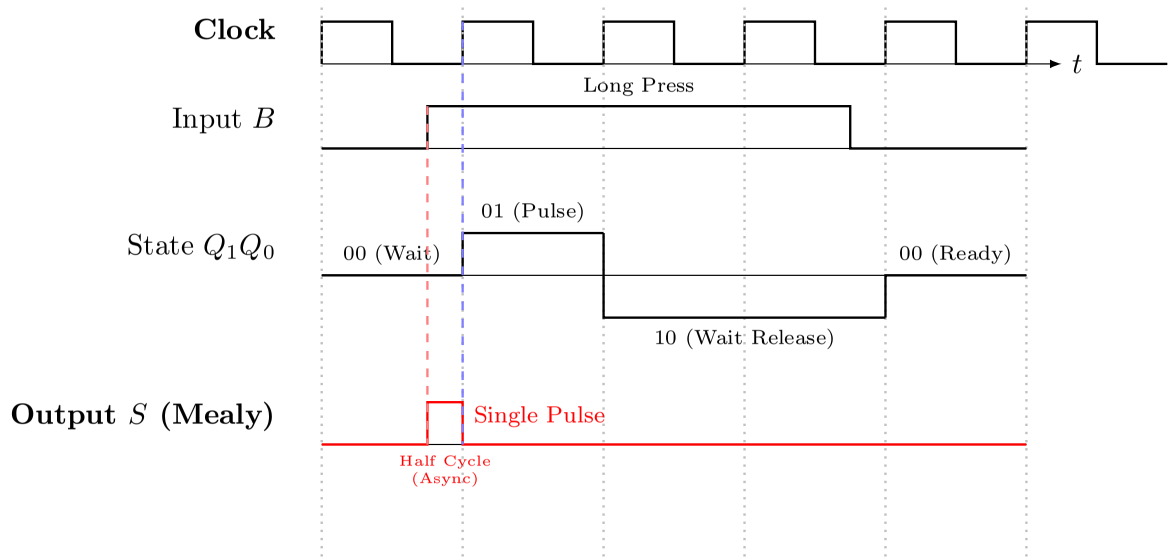

**(b)** K-maps

**Figure 8.8.** The Mealy one-shot circuit.




**(c)** logic diagram

**Figure 8.8.** The Mealy one-shot circuit.




**(d)** response to a long press

**Figure 8.8.** The Mealy one-shot circuit.

```vhdl
use ieee.std_logic_1164.all;

entity One_Shot is
    Port ( clk, reset : in std_logic;
           B : in std_logic;
           S : out std_logic);
end One_Shot;

architecture Behavioral of One_Shot is
    signal Q : std_logic_vector(1 downto 0);
begin
    process(clk, reset)
    begin
        if reset='1' then
            Q <= "00";
        elsif rising_edge(clk) then
            case Q is
                when "00" => -- S0
                    if B='1' then Q <= "01"; end if;
                when "01" => -- S1
                    Q <= "10";
                when "10" => -- S2
                    if B='0' then Q <= "00"; end if;
                when others =>
                    Q <= "00";
            end case;
        end if;
    end process;

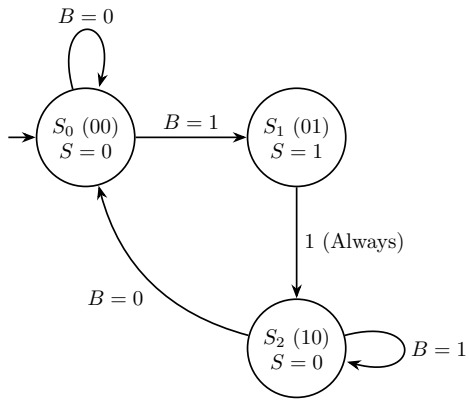
    -- Output Logic (Mealy: S=1 if State=00 and B=1)
    S <= '1' when (Q="00" and B='1') else '0';
end Behavioral;
```

**Listing 8.3.** VHDL of the Mealy one-shot.

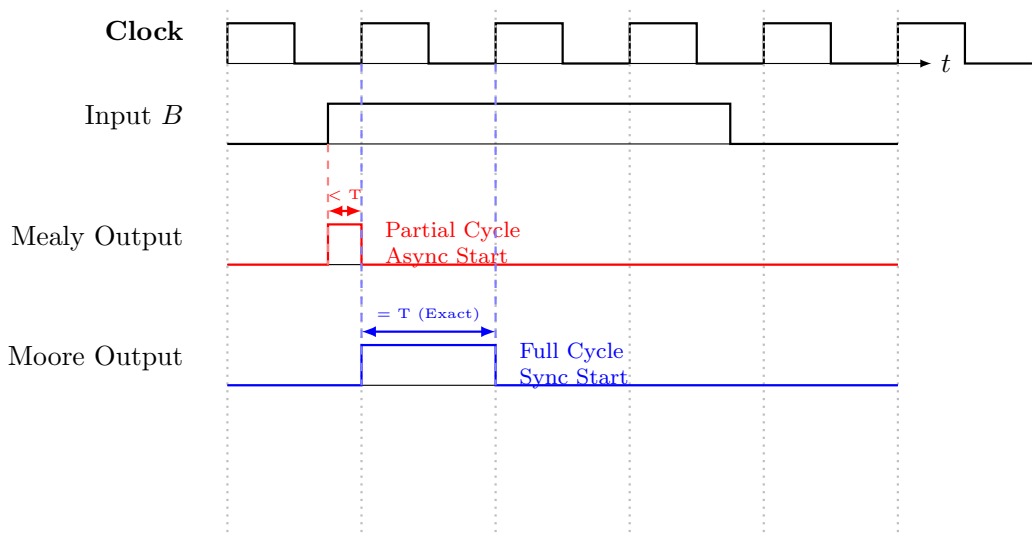
**A problem with the Mealy design.** Because $S = Q_1' Q_0' B$ depends directly on the input $B$, which is not synchronised with the clock, the pulse starts when the button is pressed and ends at the next clock edge; its width is therefore *less than one clock period* and varies with the moment of pressing (and any noise on $B$ passes straight to $S$). A **Moore** design solves the problem: the machine has states $S_0$ (idle, output 0, wait for $B = 1$), $S_1$ (pulse, output 1, unconditionally to $S_2$) and $S_2$ (wait for $B = 0$), and the output $S = 1$ only in the state $S_1$. Since states change only at clock edges, the pulse starts and ends at clock edges and lasts *exactly one period* (Figures 8.9a and 8.9b, Listing 8.4). The price is a delay of one clock period between the press and the pulse.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity One_Shot_Moore is
```




**(a)** Moore state diagram




**(b)** Mealy (asynchronous) and Moore (synchronous) pulse

**Figure 8.9.** The Moore one-shot and the comparison of the two outputs.

```
    Port ( clk, reset : in std_logic;
           B : in std_logic;
           S : out std_logic);
end One_Shot_Moore;

architecture Behavioral of One_Shot_Moore is
    type state_type is (S0, S1, S2);
    signal current_state, next_state : state_type;
begin
    -- State Register
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= S0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    -- Next State and Output Logic
    process(current_state, B)
    begin
        -- Default assignments
        next_state <= current_state;
        S <= '0';

        case current_state is
            when S0 => -- Idle
                if B='1' then
                    next_state <= S1;
                end if;
            when S1 => -- Pulse
                S <= '1'; -- Moore Output
                next_state <= S2;
            when S2 => -- Wait
                if B='0' then
                    next_state <= S0;
                end if;
        end case;
    end process;
end Behavioral;
```

**Listing 8.4.** VHDL of the Moore one-shot (exact one-cycle pulse).

**VHDL coding of Moore and Mealy machines.** The two styles differ in a recognisable way, summarised in Table 8.6. In a Moore machine the outputs are assigned depending on `current_state` only; in a Mealy machine the output assignments are inside the `if` statements that test the inputs, so the process must be sensitive to the inputs as well.

**Table 8.6.** VHDL coding differences between Moore and Mealy machines.

| | **Moore** | **Mealy** |
|---|---|---|
| Sensitivity list | `process(current_state)` (and the inputs for the next-state logic) | `process(current_state, inputs)` |
| Output assignment | outside any test of inputs within the `case` branch: `when S0 => A <= '1'; next_state <= S1;` | inside `if/case` tests of inputs: `when S0 => if X='1' then A<='1'; else A<='0'; end if;` |
| Timing | output stable between clock edges (comes from the state register) | output may change whenever an input changes |

### 8.5.5 A control unit

**Example 8.6: A simple processor control unit**

An FSM is also the form of the *control unit* of a processor (Chapter 9): it steps through the actions of a computation in response to status signals. The machine in Figure 8.10 has states $s_0, \ldots, s_3$ and outputs $x$ and $y$; the transitions out of $s_0$ depend on the input *Start* (*Start* and *Start*′), and those out of $s_2$ on the status signal $n = 9$ ($n = 9$ and $(n = 9)'$). The remaining transitions are unconditional. Such a machine is synthesised exactly as the others in this chapter; note that its inputs are *comparisons* produced by other circuits, a feature that we exploit in Chapters 9 and 10.

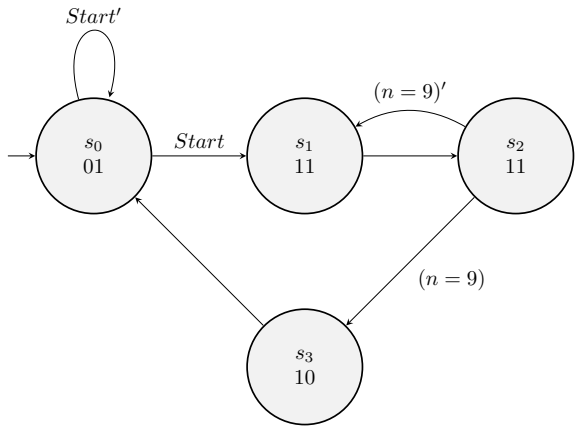



**Figure 8.10.** State diagram of the control unit of Example 8.6.

### 8.5.6 An elevator controller: Moore and Mealy

**Example 8.7: Two-floor elevator controller (Moore)**

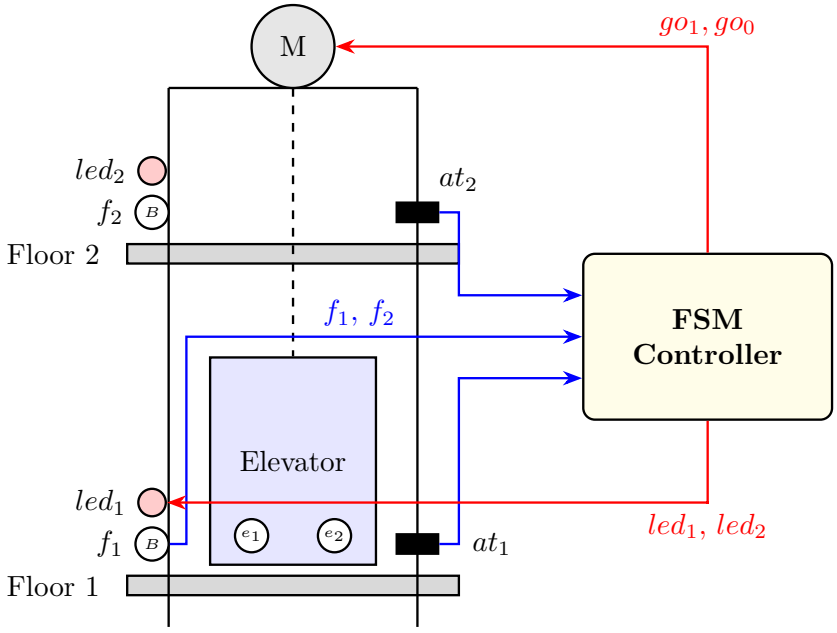
Design the FSM for an elevator serving two floors (Figure 8.11a). Inputs: $f_1$,$f_2$ are the call buttons for floors 1 and 2 (hall and cabin buttons combined, assumed to stay pressed until served) and $at_1$,$at_2$ are position sensors, asserted when the cabin is at floor 1 or 2. Outputs: $go_1$ (motor on) and $go_0$ (direction, 0 = down, 1 = up), and the floor indicators $led_1$ and $led_2$.

**State diagram** (Figure 8.11b). Four states: **00** floor 1 (idle; $led_1$ on; on $f_2$ go to 11); **11** moving up ($go_1 = go_0 = 1$; on $at_2$ go to 10); **10** floor 2 (idle; $led_2$ on; on $f_1$ go to 01); **01** moving down ($go_1 = 1, go_0 = 0$; on $at_1$ go to 00). The motor outputs depend on the state only, so this is a Moore machine.

**Equations.** From the K-maps of Figure 8.11c,

$$D_1 = Q_1'Q_0'f_2 + Q_1Q_0 + Q_1Q_0'f_1',$$
$$D_0 = Q_1'Q_0'f_2 + Q_1Q_0\,at_2' + Q_1Q_0'f_1 + Q_1'Q_0\,at_1'.$$

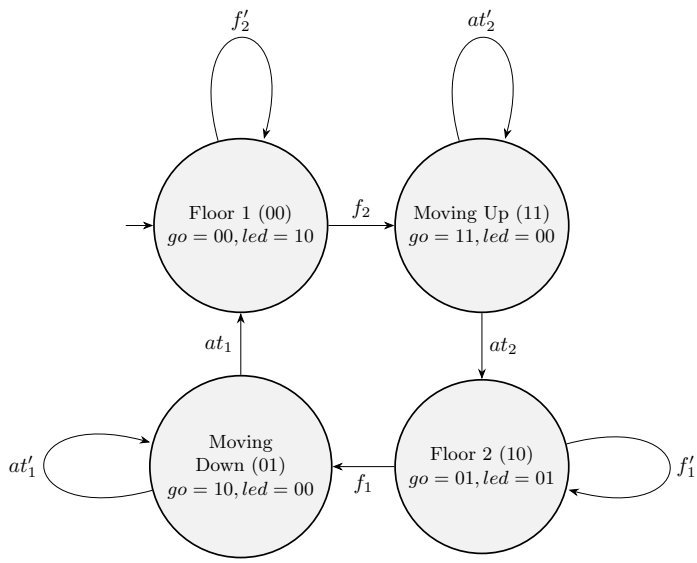
Each term describes one transition: for example $Q_1Q_0\,at_2'$ keeps $D_0 = 1$ while the cabin moves up and has not yet arrived. Listing 8.5 is the VHDL.



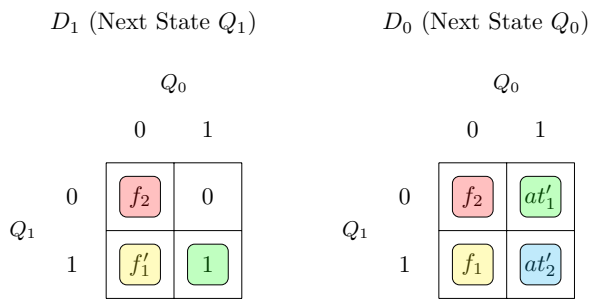

**(a)** system

**Figure 8.11.** Two-floor elevator controller, Moore version.

```
1 library IEEE;
```




**(b)** Moore state diagram




**(c)** K-maps for $D_1, D_0$

**Figure 8.11.** Two-floor elevator controller, Moore version.

```
use IEEE.STD_LOGIC_1164.ALL;

entity Elevator_Moore is
    Port ( clk, reset : in std_logic;
           f1, f2, at1, at2 : in std_logic;
           go1, go0, led1, led2 : out std_logic);
end Elevator_Moore;

architecture Behavioral of Elevator_Moore is
    type state_type is (Floor1, MovingUp, Floor2, MovingDown);
    signal current_state, next_state : state_type;
begin
    -- State Register
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= Floor1;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    -- Next State and Output Logic
    process(current_state, f1, f2, at1, at2)
    begin
        -- Defaults
        next_state <= current_state;
        go1 <= '0'; go0 <= '0'; -- Motor Off
        led1 <= '0'; led2 <= '0';

        case current_state is
            when Floor1 =>
                led1 <= '1';
                if f2 = '1' then
                    next_state <= MovingUp;
                end if;

            when MovingUp =>
                go1 <= '1'; go0 <= '1'; -- Move Up
                if at2 = '1' then
                    next_state <= Floor2;
                end if;

            when Floor2 =>
                led2 <= '1';
                if f1 = '1' then
                    next_state <= MovingDown;
                end if;

            when MovingDown =>
                go1 <= '1'; go0 <= '0'; -- Move Down
```

```
                if at1 = '1' then
                    next_state <= Floor1;
                end if;
        end case;
    end process;
end Behavioral;
```

**Listing 8.5.** VHDL of the Moore elevator controller (four states).

**Example 8.8: Two-floor elevator controller (Mealy)**

The same controller can be built with only *two* states, *Floor1* and *Floor2*, if the motor outputs are allowed to depend on the buttons. The state changes only when the cabin *arrives* at a floor ($at_x$); the motor is turned on by the call button of the *other* floor (Figure 8.12a):

$$D = \overline{Q}\,(f_2 \cdot at_2) + Q\,\overline{(f_1 \cdot at_1)},$$
$$led_1 = \overline{Q}, \quad led_2 = Q, \quad go_1 = \overline{Q} f_2 + Q f_1, \quad go_0 = \overline{Q}.$$

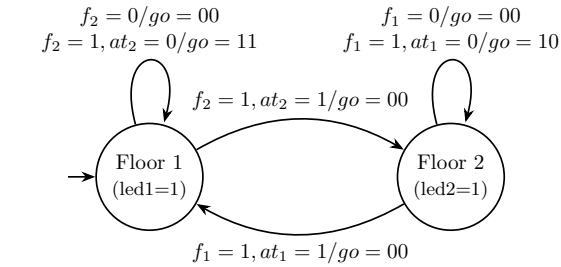
With $Q = 0$ (floor 1) and $f_2$ pressed the motor runs upwards until $at_2$ arrives; the K-maps are Figure 8.12b and the VHDL Listing 8.6. The state memory shrinks from two flip-flops to one, but the output logic becomes more complex, and the outputs now respond directly to the buttons. In this version the indicators show the last floor served, not the cabin position while it moves.

```
library ieee;
use ieee.std_logic_1164.all;

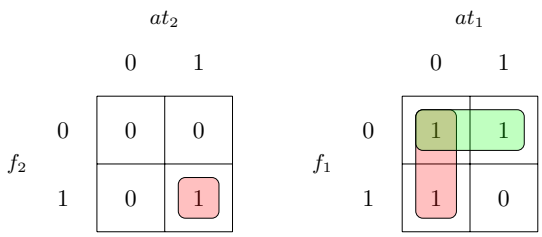
entity Elevator_Mealy is
    Port ( clk, reset : in std_logic;
           f1, f2, at1, at2 : in std_logic;
           go1, go0, led1, led2 : out std_logic);
end Elevator_Mealy;

architecture Behavioral of Elevator_Mealy is
    type state_type is (Floor1, Floor2);
    signal current_state, next_state : state_type;
begin
    -- State Register
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= Floor1;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

```




**(a)** Mealy state diagram




**(b)** K-maps

**Figure 8.12.** Two-floor elevator controller, Mealy version.

```
24      -- Next State and Output Logic
25      process(current_state, f1, f2, at1, at2)
26      begin
27          -- Defaults
28          next_state <= current_state;
29          go1 <= '0'; go0 <= '0'; -- Motor Off
30          led1 <= '0'; led2 <= '0';
31
32          case current_state is
33              when Floor1 =>
34                  led1 <= '1';
35                  if f2 = '1' then
36                      go1 <= '1'; go0 <= '1'; -- Move Up
37                      if at2 = '1' then
38                          next_state <= Floor2;
39                      end if;
40                  end if;
41
42              when Floor2 =>
43                  led2 <= '1';
44                  if f1 = '1' then
45                      go1 <= '1'; go0 <= '0'; -- Move Down
46                      if at1 = '1' then
47                          next_state <= Floor1;
48                      end if;
49                  end if;
50          end case;
```

```
51      end process;
52  end Behavioral;
```

**Listing 8.6.** VHDL of the Mealy elevator controller (two states).

## 8.6 Moore versus Mealy Timing

The essential difference is the timing of the outputs (Figure 8.13a). A Moore output changes only just after a clock edge, because it is a function of the state register, so it is synchronous and stable for the whole period. A Mealy output can change at any time during the period when an input changes: it *reacts one clock period earlier* than the equivalent Moore output, but it can also show glitches if the inputs change at different times, and in a chain of Mealy machines such combinational paths from input to output add up. The elevator timing diagram, Figure 8.13b, shows a complete trip with the two versions.

**Choosing between Moore and Mealy**

> Use a **Moore** machine when the outputs must be glitch-free and aligned with the clock, or drive other synchronous logic (the usual choice). Use a **Mealy** machine when you need the fastest reaction or the smallest number of states, and take care that the inputs are synchronised. Any Mealy machine has an equivalent Moore machine, usually with more states.

## 8.7 Timing of a Synchronous FSM

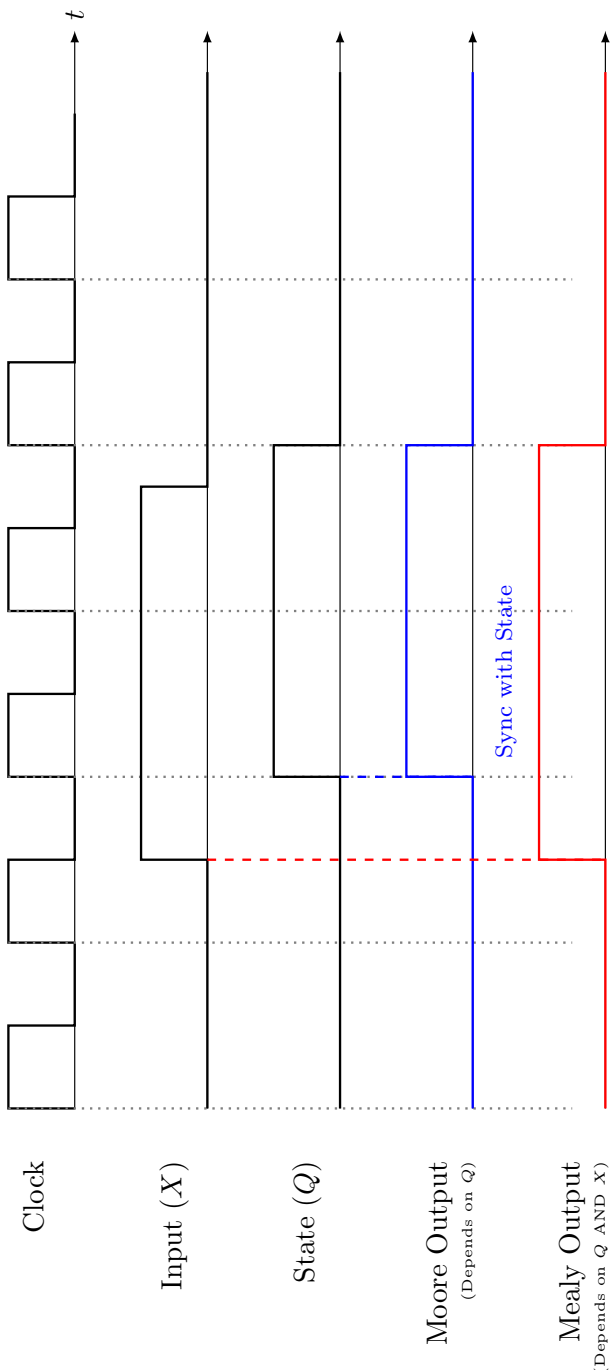
All state changes occur at the active clock edge, but the next state must be *ready* before the next edge. The signal path is: clock edge → flip-flop output ($t_{pcq}$, the propagation delay from clock to $Q$) → next-state logic ($t_{comb}$) → flip-flop input, which must be stable for the *setup time* $t_{setup}$ before the next edge. Hence

$$T_{clk} \geq t_{pcq} + t_{comb} + t_{setup}, \qquad f_{max} = \frac{1}{t_{pcq} + t_{comb} + t_{setup}}. \tag{8.1}$$

If $t_{pcq} = 0.5$ ns, $t_{comb} = 3$ ns and $t_{setup} = 0.2$ ns, then $T_{clk} \geq 3.7$ ns and $f_{max} \approx 270$ MHz. In addition the inputs must remain stable for the *hold time* $t_{hold}$ after the edge. Equation (8.1) explains why a design with more complex next-state logic (a longer $t_{comb}$) must run slower, and why one-hot encoding, with its simpler logic, is often faster.

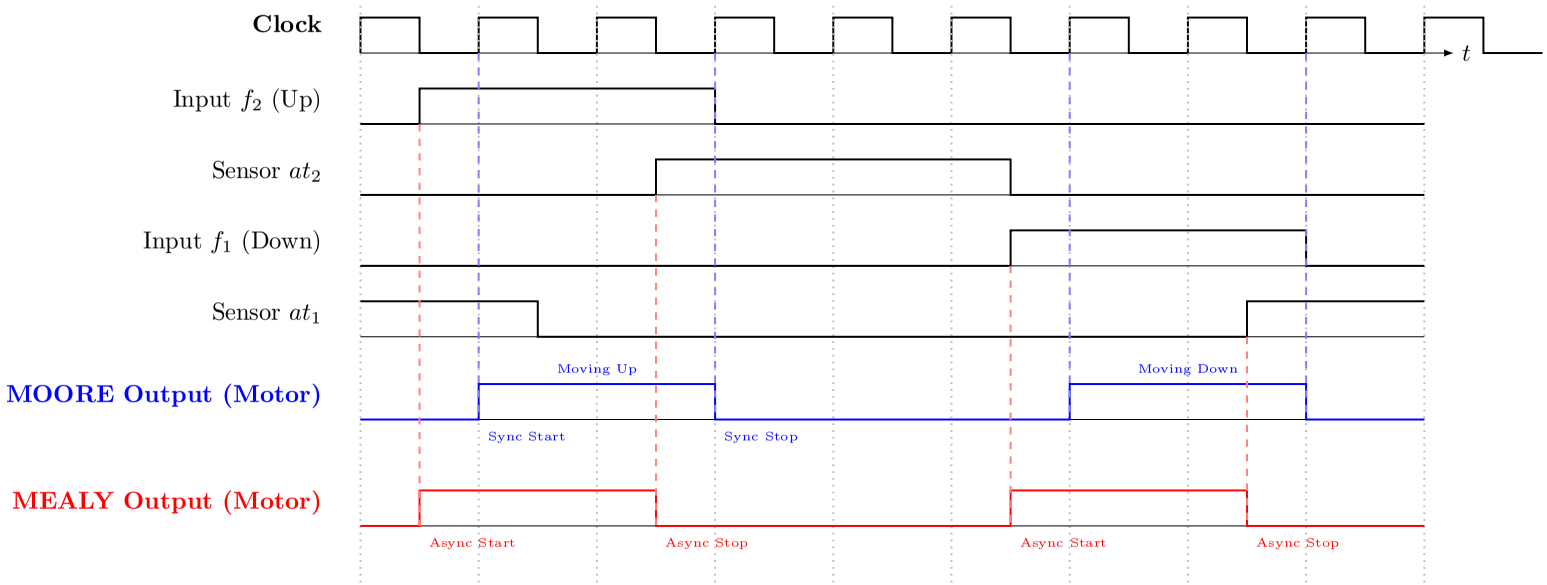
## 8.8 A Template for VHDL State Machines

Most FSMs in this book follow one template.




**(a)** generic timing comparison

**Figure 8.13.** Moore and Mealy output timing.




**(b)** elevator, up and down

**Figure 8.13.** Moore and Mealy output timing.

```vhdl
architecture fsm of my_machine is
  type state_type is (S0, S1, S2);                  -- list the states
  signal current_state, next_state : state_type;
begin
  -- Process 1: state register (sequential)
  process (clk, reset)
  begin
    if reset = '1' then
      current_state <= S0;                          -- initial state
    elsif rising_edge(clk) then
      current_state <= next_state;
    end if;
  end process;

  -- Process 2: next-state and output logic (combinational)
  process (current_state, x)                        -- all signals that are read
  begin
    next_state <= current_state;                    -- defaults: stay, outputs
    inactive
    y <= '0';
    case current_state is
      when S0 => if x = '1' then next_state <= S1; end if;
      when S1 => y <= '1'; next_state <= S2;        -- Moore output
      when S2 => if x = '0' then next_state <= S0; end if;
    end case;
  end process;
end fsm;
```

**Listing 8.7.** Template of the two-process FSM description (the state register is process 1 and the combinational logic process 2).

A common error is to omit a signal from the sensitivity list: for example, writing `process(clk)` for the register above without `reset` makes the asynchronous reset ineffective in simulation. In process 2 the sensitivity list must contain *every signal that is read*.

**Chapter Summary**

- An **FSM** has next-state logic, a state register and output logic. **Moore**: output $= f(\text{state})$. **Mealy**: output $= f(\text{state}, \text{inputs})$.
- **Analysis**: equations → state table → state diagram. **Synthesis**: specification → state diagram → state table → (minimisation) → state assignment → flip-flop type → K-maps → logic diagram or VHDL.
- **State encodings**: binary (fewest flip-flops), Gray, one-hot ($N$ flip-flops, simpler logic).
- In VHDL use an **enumerated state type** and the **two-process method**; assign defaults to avoid latches; use `numeric_std` and explicit conversions for arith-

metic.

- The **one-shot** shows the difference between the models: the Mealy output is asynchronous (less than one clock), the Moore output is exactly one cycle but delayed.
- The clock period must satisfy $T_{clk} \geq t_{pcq} + t_{comb} + t_{setup}$.

## Exercises

**8.1** **Moore or Mealy?** Machine (A) has next-state logic with inputs {inputs, present state} and output logic with inputs {present state}. Machine (B) has output logic with inputs {inputs, present state}. Which is Moore and which is Mealy?

**8.2** **State analysis.** A state machine has one JK flip-flop with $J = x$ and $K = 1$. If the present state is $Q = 0$ and $x = 1$, what is the next state? Give the complete state table.

**8.3** **110 detector, overlapping.** Draw the state diagram of a Mealy sequence detector for the sequence `110` in which overlapping sequences are valid. For the input `0 1 1 0 1 1 0` the output must be `0 0 0 1 0 0 1`.

**8.4** **110 detector, non-overlapping.** Draw the state diagram of a *non-overlapping* detector for `110`: after a detection the machine returns to the initial state. Explain why the two diagrams are the same for the sequence `110` but would differ for `101`.

**8.5** **State encoding.** An FSM has 12 states. How many flip-flops are needed with (a) binary encoding; (b) one-hot encoding?

**8.6** **VHDL syntax.** What is missing in the sensitivity list of this synchronous-FSM state-register process?

```
process (clk)
begin
  if reset = '1' then current_state <= S0;
  elsif rising_edge(clk) then current_state <= next_state;
  end if;
end process;
```

**8.7** **State-table construction.** Counter sequence $00 \to 01 \to 10 \to 11 \to 00$ with an input that is 0 for hold and 1 for count. Give the missing next state for present state 01 and input 1.

**8.8** **Logic equation.** In the K-map of Figure 8.14 for $D_0$ (rows $Q$, columns $X$), derive the simplified Boolean equation.

**8.9** **Timing.** Which of the following correctly bounds the clock period of an FSM? (1) $T_{clk} \geq t_{setup} + t_{hold}$; (2) $T_{clk} \geq t_{pcq} + t_{comb} + t_{setup}$; (3) $T_{clk} \geq t_{pcq} + t_{comb} - t_{hold}$. Compute $f_{max}$ if $t_{pcq} = 0.4$ ns, $t_{comb} = 2.1$ ns and $t_{setup} = 0.3$ ns.

**8.10** **Gray-code counter.** Design a 2-bit Gray-code counter with the sequence $00 \to$

01 → 11 → 10 → 00: draw the state diagram, find the D flip-flop equations and the circuit.

**8.11** Convert the Mealy 101 detector of Example 6.4 to a *Moore* machine. How many states are needed? Draw the state diagram.

**8.12** Design a Moore FSM for a simple traffic light with states Green, Yellow and Red, which changes state on every clock pulse when an input `tick` is 1. Give the state diagram, a binary state assignment, the equations and VHDL.

**8.13** Write the VHDL of a Moore machine whose output $z$ is 1 while the input $x$ has been 1 for at least three consecutive clock cycles.

**8.14** Explain, with a sketch, why the output of a Mealy machine can show a glitch when two inputs change at slightly different times, and why a Moore output cannot.

**8.15** For the Mealy one-shot, draw the waveforms of $B$, the clock and $S$ if $B$ rises between two clock edges and falls after three clock periods. Compare with the Moore version.

| $Q$ \ $X$ | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 1 |

**Figure 8.14.** K-map for Exercise 8.8.

## Further Reading

Mealy's and Moore's original papers [16, 17] introduced the two models. The synthesis of FSMs and their VHDL descriptions are treated in Chapter 8 of Brown and Vranesic [3] and Chapter 5 of Hwang [15]; the coding templates for state machines follow Ashenden [9]. Gajski [18] extends FSMs with a datapath, the topic of the next chapter.

# 9

# Dedicated Microprocessors

**Learning Objectives**

After studying this chapter you will be able to:

- describe the von Neumann model of a computer and the layers of abstraction from transistors to processors;
- explain what a dedicated microprocessor is and how it divides into a *datapath* and a *control unit*;
- describe computations at the register-transfer level (RTL);
- design a datapath for a given algorithm and write its *control words*;
- design the control unit as an FSM driven by status signals, including a wait state when needed for timing;
- implement a dedicated processor in VHDL in the FSM+D style and in the FSMD style.

## 9.1 The Von Neumann Model

Almost every computer follows the model described by John von Neumann in 1945 [19]. It has five components (Figure 9.1): **input** supplies data, **output** presents results, **memory** stores programs and data, and the **microprocessor** (the "brain") processes the data. The microprocessor has a **datapath**, which carries out the data processing (arithmetic and logic), and a **control unit**, which determines the sequence of operations.

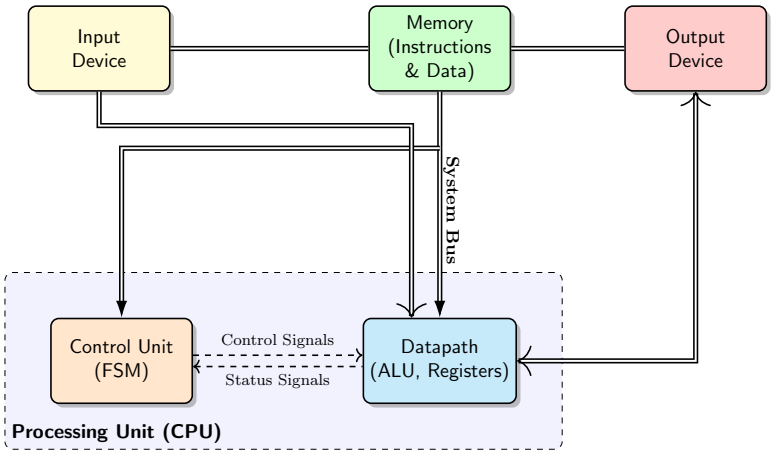



**Figure 9.1.** The von Neumann model of a computer.

## 9.2 From Transistors to Microprocessors

Digital systems are so complex that they can only be designed by managing complexity through *layers of abstraction* (Figure 9.2). Transistors combine into logic gates; gates form combinational and sequential circuits; circuits form functional components such as adders and multiplexers (Chapters 4 and 5) and registers and counters (Chapter 7); and components are integrated into a datapath and a control unit, which together form the microprocessor. Each layer hides the details of the layer below it, so a designer can work at the level of components without thinking of transistors. This chapter works at the top two layers.

## 9.3 Dedicated Microprocessors

A *dedicated microprocessor* (special-purpose processor) is designed to perform one specific task. The "program" is not stored in memory but is *hardwired* in the structure of the processor, which makes it optimised for the task in speed, power and size. Its function cannot be changed after manufacture (if implemented as an ASIC; an FPGA implementation can be reconfigured). Examples are the image-processing chip in a digital camera, the controller in a microwave oven and a GCD engine inside a cryptographic circuit. In contrast, a *general-purpose* processor executes arbitrary programs stored in memory and is the subject of Chapter 10.

## 9.4 Datapath and Control Unit

A microprocessor is partitioned into two cooperating parts (Figure 9.3).

- The *datapath* performs the data processing operations. It contains functional units (ALUs, adders, comparators), registers, counters and multiplexers; this is where

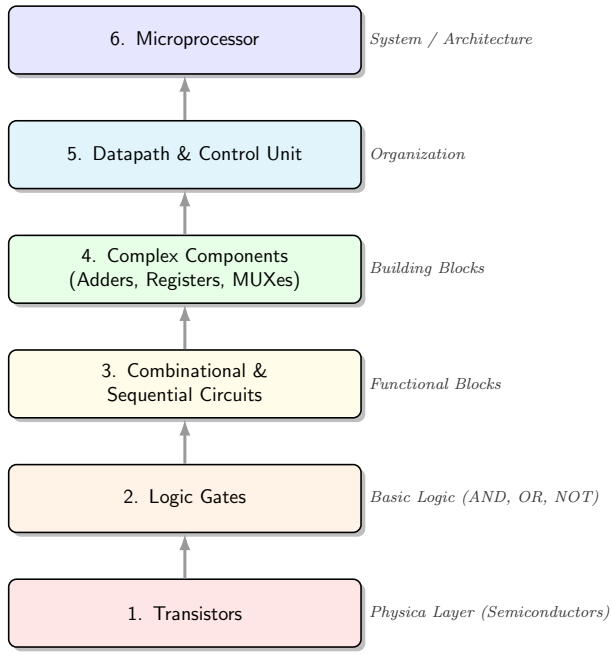



**Figure 9.2.** Layers of abstraction: from transistors to microprocessors.

the "number crunching" happens.

- The *control unit* determines the *sequence* of datapath operations. In each clock cycle it generates *control signals*, which tell the datapath what to do (load this register, select that input), and it receives *status signals* from the datapath (the result is zero, $A > B$) which it uses to make decisions. The control unit is an FSM, as studied in Chapter 8.

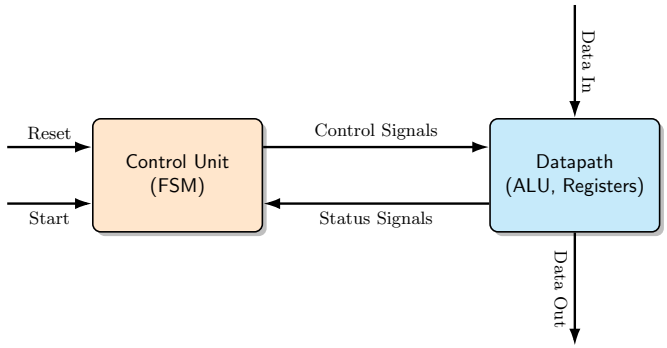



**Figure 9.3.** A dedicated microprocessor: the control unit sends control signals to the datapath and receives status signals from it.

## 9.5 Register Transfer Level

The *register-transfer level* (RTL) is the level of abstraction at which a synchronous digital system is described by (1) the flow of data between registers and (2) the logical or arithmetic operations performed on the data on the way (Figure 9.4). At RTL we consider information moving "in bulk" from register to register at each clock tick, ignoring the gates and flip-flops that implement it. This is the level of abstraction of VHDL and Verilog, and of the notation $R_1 \leftarrow R_2 + R_3$ used in this chapter, meaning "at the next active clock edge, load $R_1$ with the sum of the contents of $R_2$ and $R_3$".

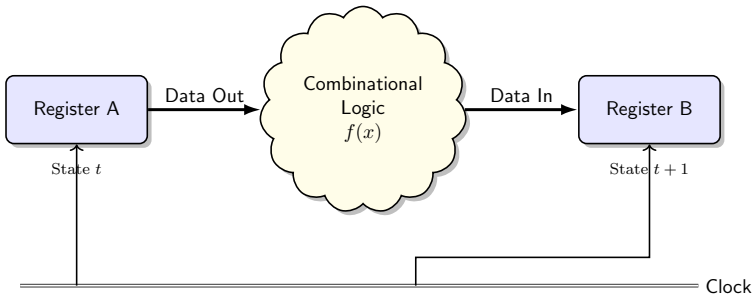



**Figure 9.4.** The RTL model: registers, combinational logic between them, and a common clock.

## 9.6 Datapath Design

To design a datapath for a single problem, assemble the registers and functional units needed to perform the required operations, then connect them so that every transfer in the algorithm has a path.

**Example 9.1: Datapath for $A = B + C$**

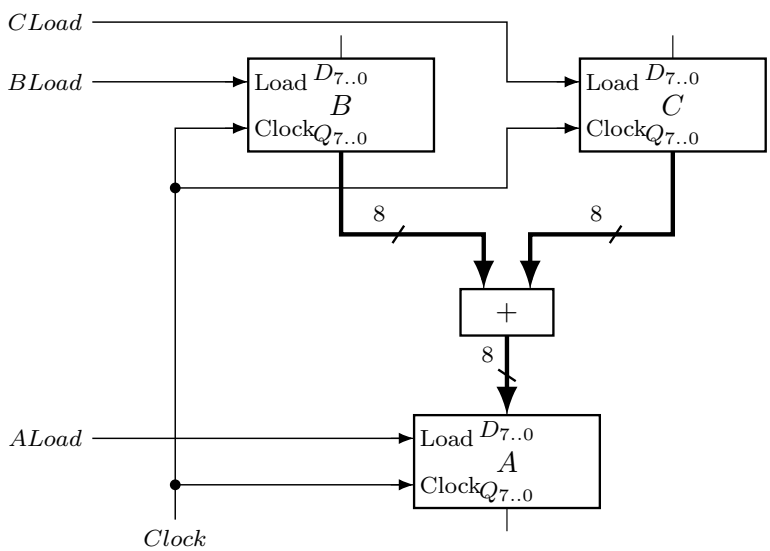
Registers $B$ and $C$ hold the operands; an adder forms $B + C$; register $A$ stores the result when the control signal *ALoad* is asserted (Figure 9.5a).

**Example 9.2: Datapath for $A = A + 3$**

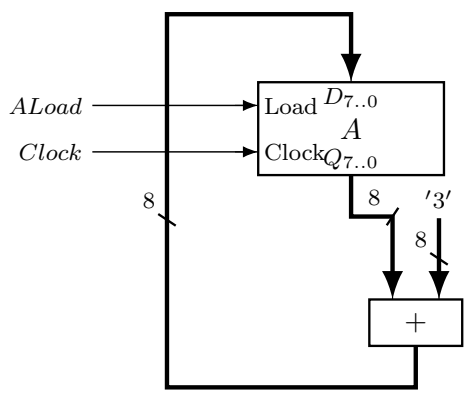
The output of register $A$ is fed back to one input of an adder; the other input is the constant 3; the sum is loaded back into $A$ (Figure 9.5b). The control unit asserts *ALoad* once for each addition.

### 9.6.1 Sharing a functional unit with multiplexers

Suppose the processor must perform *both* $A = A+3$ and $A = B+C$. Two adders would work, but a single adder preceded by multiplexers is cheaper. A multiplexer in front of each adder input selects the operand: the first chooses between $B$ (0) and $A$ (1), the




(a) $A = B + C$




(b) $A = A + 3$

**Figure 9.5.** Two dedicated datapaths.

second between $C$ (0) and the constant 3 (1) (Figure 9.6). Because the two operations need either $(B, C)$ or $(A, 3)$, one control signal *Mux* can drive both multiplexers. The control unit selects the operation by the *control word*, i.e. the values of all control signals needed in a clock cycle (Table 9.1).

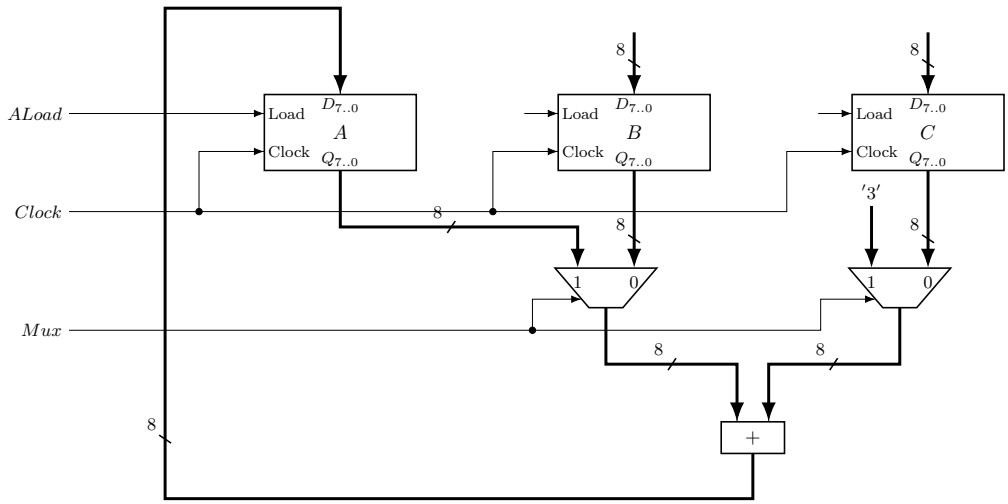



**Figure 9.6.** A shared-adder datapath that performs $A = A + 3$ and $A = B + C$. Both multiplexers are driven by the same control signal *Mux* (input 1 selects $A$ and the constant 3, input 0 selects $B$ and $C$); the bus width is marked with a slash, and the clock is common to all registers.

The two multiplexers can also be placed *in front of the destination register* instead: the adder outputs $B + C$ and $A + 3$ feed a single 2-to-1 multiplexer *Amux* that selects what is loaded into $A$ (Figure 9.7). This version needs two adders but only one multiplexer, and the same *ALoad* and *Amux* signals again define the two operations.

**Table 9.1.** Control words of the shared-adder datapath.

| Operation | *ALoad* | *Mux* |
|---|---|---|
| $A = A + 3$ | 1 | 1 |
| $A = B + C$ | 1 | 0 |

### 9.6.2 Status signals and conditional operations

To implement a conditional such as **IF** $(A = 0)$ the datapath must produce a *status signal* for the control unit. An equality comparator can compare a register with a constant (Figure 9.8a). For $A = 0$ in an 8-bit register there is a cheaper solution: a single 8-input NOR gate, whose output is 1 only if every bit of $A$ is 0 (Figure 9.8b). Similarly, a magnitude comparator (Section 5.10) produces $A > B$.

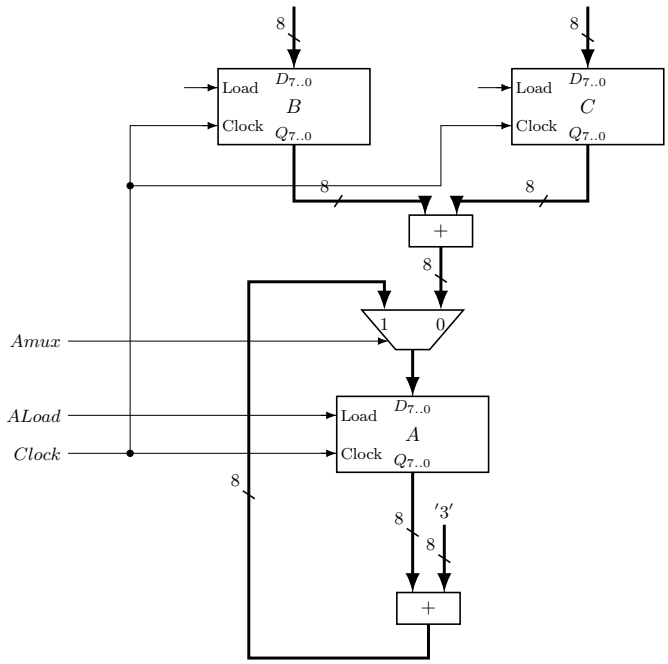



**Figure 9.7.** Alternative datapath for $A = B + C$ and $A = A + 3$: one multiplexer *Amux* in front of register $A$.

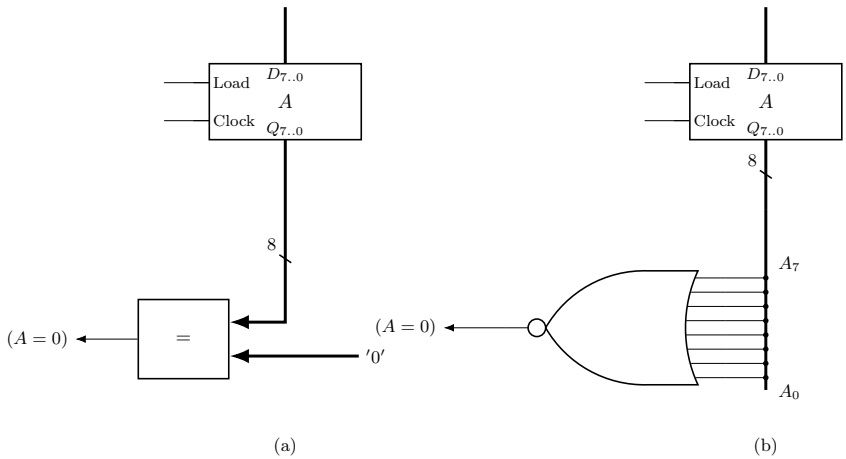



**Figure 9.8.** Generating the status signal $(A = 0)$: (a) comparator with the constant 0; (b) 8-input NOR gate.

### 9.6.3 An if-then-else datapath

Consider the algorithm

```
INPUT A;  IF (A = 5) THEN B = 8 ELSE B = 13;   OUTPUT B
```

The datapath of Figure 9.9 has register $A$ (the input), register $B$ (the result), a multiplexer selecting the constant 8 or 13, an AND gate that generates the status signal $(A = 5)$ by decoding the bit pattern 0101 (the inputs for $A_3$ and $A_1$ are inverted), and an output buffer controlled by *Out*. The control signals are *ALoad*, *BLoad*, *Muxsel* (1 selects 8, 0 selects 13) and *Out*. The algorithm is converted into a sequence of control words (Table 9.2); the control unit decides between control words 2 and 3 using the status signal $(A = 5)$.

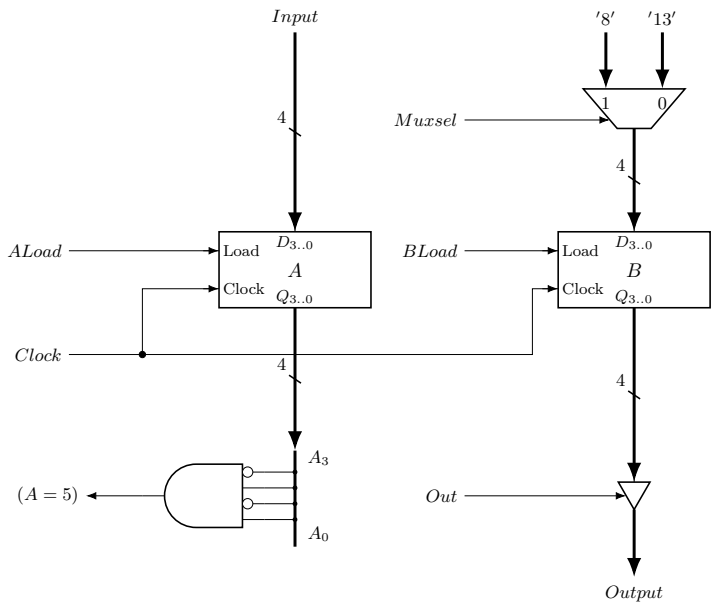



**Figure 9.9.** Datapath for the if-then-else algorithm.

**Table 9.2.** Control words for the if-then-else algorithm.

| CW | Instruction | *ALoad* | *Muxsel* | *BLoad* | *Out* |
|---|---|---|---|---|---|
| 1 | INPUT A | 1 | X | 0 | 0 |
| 2 | $B = 8$ | 0 | 1 | 1 | 0 |
| 3 | $B = 13$ | 0 | 0 | 1 | 0 |
| 4 | OUTPUT B | 0 | X | 0 | 1 |

### 9.6.4 A datapath with a loop: summing the numbers 1 to 10

```
Sum = 0
i = 1
DO {
  Sum = Sum + i
  i = i + 1
} WHILE (i != 11)
OUTPUT Sum
```

The datapath needs two registers, *Sum* and *i*, an adder, and a gate that signals when $i \neq 11$. We build it (Figure 9.10) with *one* adder, whose first input is always *i* and whose second input is chosen by *addMux*: *Sum* (0) or the constant 1 (1). The adder output *R* goes to both registers through input multiplexers: *sumMux* chooses between *R* (0) and the constant 0 (1), and *iMux* between *R* (0) and the constant 1 (1). With these selections each instruction of the algorithm is one control word (Table 9.3): $Sum = 0$ is $sumLoad = 1, sumMux = 1$; $i = 1$ is $iLoad = 1, iMux = 1$; $Sum = Sum + i$ is $sumLoad = 1$, $sumMux = 0$, $addMux = 0$; $i = i + 1$ is $iLoad = 1$, $iMux = 0$, $addMux = 1$; and the output is $OE = 1$. The loop is made by the control unit (Section 9.7).

**Table 9.3.** Control words for the summation algorithm.

| CW | Instruction | *sumLoad* | *sumMux* | *iLoad* | *iMux* | *addMux* | *OE* |
|---|---|---|---|---|---|---|---|
| 1 | $Sum = 0$ | 1 | 1 | 0 | X | X | 0 |
| 2 | $i = 1$ | 0 | X | 1 | 1 | X | 0 |
| 3 | $Sum = Sum + i$ | 1 | 0 | 0 | X | 0 | 0 |
| 4 | $i = i + 1$ | 0 | X | 1 | 0 | 1 | 0 |
| 5 | OUTPUT *Sum* | 0 | X | 0 | X | X | 1 |

## 9.7 Control Unit Design

The control unit is an FSM (Figure 9.3) that steps through the control words of the algorithm and branches on the status signals. In a Moore machine, each state outputs one control word. We design three control units.

### 9.7.1 Example 1: counting from 1 to 10

**Example 9.3: Counter control unit**

The algorithm is `i = 0; WHILE (i != 10) { i = i + 1 }`. The datapath is a counter *i* with a clear input and a count input and a comparator producing the status signal ($i = 10$) (Figure 9.11b). The control signals are *Clear* and *Count*.
**State diagram** (Figure 9.11a). $S_0$: clear the counter. $S_1$: increment; remain in $S_1$ while $i \neq 10$ and leave when $i = 10$. $S_2$: the loop is finished. A fourth state $S_3$ (halt) keeps the machine idle after the computation. With the state assignment

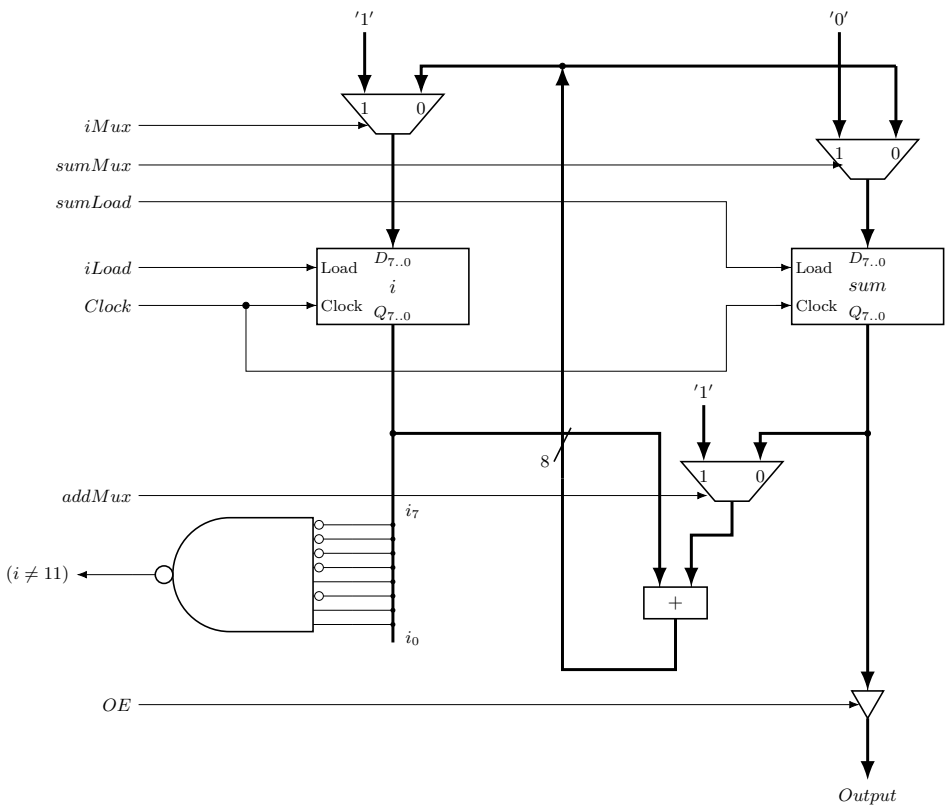



**Figure 9.10.** A datapath for summing the numbers from 1 to 10 with one adder and three multiplexers. The status signal $(i \neq 11)$ is a NAND gate whose inputs for the zero bits of $11 = 00001011_2$ are inverted; the output *Sum* is enabled by *OE*.

$S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$ and $x$ denoting the status signal $(i = 10)$, the next-state table (Table 9.4) and the Moore output table give

$$D_1 = Q_1 + Q_0 x, \qquad D_0 = Q_1 + Q_0' + x',$$
$$Clear = Q_1' Q_0' = \overline{Q_1 + Q_0}, \qquad Count = Q_1' Q_0.$$

The control logic is Figure 9.11c; connected to the datapath it forms the complete dedicated microprocessor of Figure 9.11d.

**Table 9.4.** Tables of the counter control unit $(x = (i = 10))$.

| PS $Q_1Q_0$ | Input $x$ | NS $D_1D_0$ |
|---|---|---|
| 00 $(S_0)$ | X | 01 $(S_1)$ |
| 01 $(S_1)$ | 0 | 01 $(S_1)$ |
| 01 $(S_1)$ | 1 | 10 $(S_2)$ |
| 10 $(S_2)$ | X | 11 $(S_3)$ |
| 11 $(S_3)$ | X | 11 $(S_3)$ |

| State | Clear | Count |
|---|---|---|
| 00 $(S_0)$ | 1 | 0 |
| 01 $(S_1)$ | 0 | 1 |
| 10 $(S_2)$ | 0 | 0 |
| 11 $(S_3)$ | 0 | 0 |

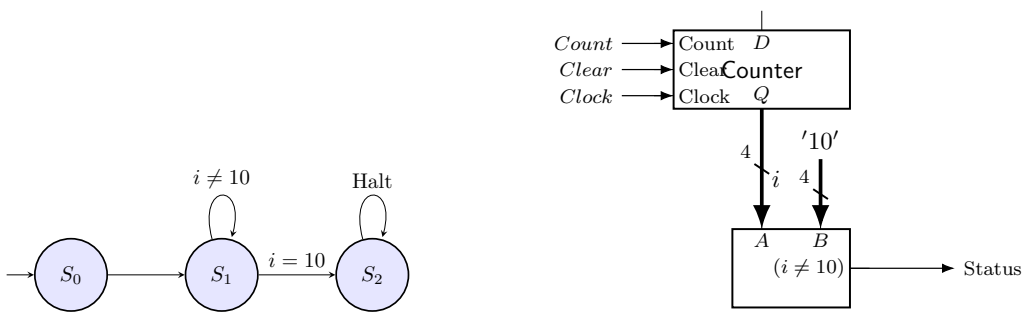
### 9.7.2 Example 2: the if-then-else control unit

**Example 9.4: if-then-else control unit**

The control signals and control words are those of Table 9.2; the status signal is $(A = 5)$. A first design has the states $S_{input}$ (load $A$), $S_{equal}$ $(B = 8)$, $S_{notequal}$ $(B = 13)$ and $S_{output}$, with $S_{input}$ branching directly on the status signal (Figure 9.12a).
**The timing problem.** In $S_{input}$ register $A$ is loaded *at the end of the clock cycle*. The control unit takes its branching decision at the same clock edge, so it sees the status signal computed from the *old* contents of $A$ and may go to the wrong state. The simulation of Figure 9.13 shows this error. The remedy is an extra **wait state** $S_{extra}$ after $S_{input}$, in which nothing happens for one clock period: register $A$ takes its new value and the comparator settles, and the next state is decided in $S_{extra}$ using the correct status (Figures 9.12b and 9.13).
**Synthesis.** With the state assignment $S_{input} = 000$, $S_{extra} = 001$, $S_{notequal} = 010$, $S_{equal} = 011$, $S_{output} = 100$ the next-state table is Table 9.5. Writing $I$ for the status signal $(A = 5)$:

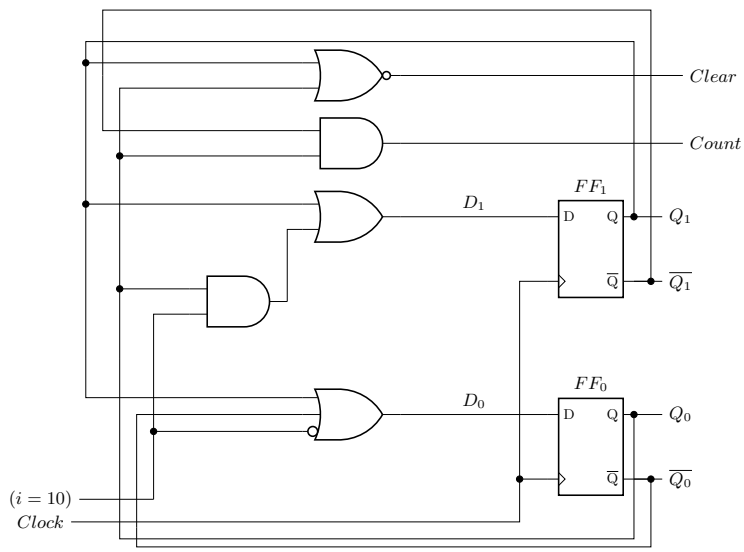
$$D_2 = Q_2 + Q_1, \quad D_1 = Q_1' Q_0, \quad D_0 = Q_2' Q_1' (Q_0' + I),$$
$$ALoad = Q_2' Q_1' Q_0', \quad BLoad = Q_2' Q_1, \quad Muxsel = Q_0, \quad Out = Q_2.$$

Muxsel is 1 in $S_{equal}$ (select 8) and 0 in $S_{notequal}$ (select 13); in $S_{extra}$ it is a don't-care. The VHDL is Listing 9.1, in which the state register is built of D flip-flops



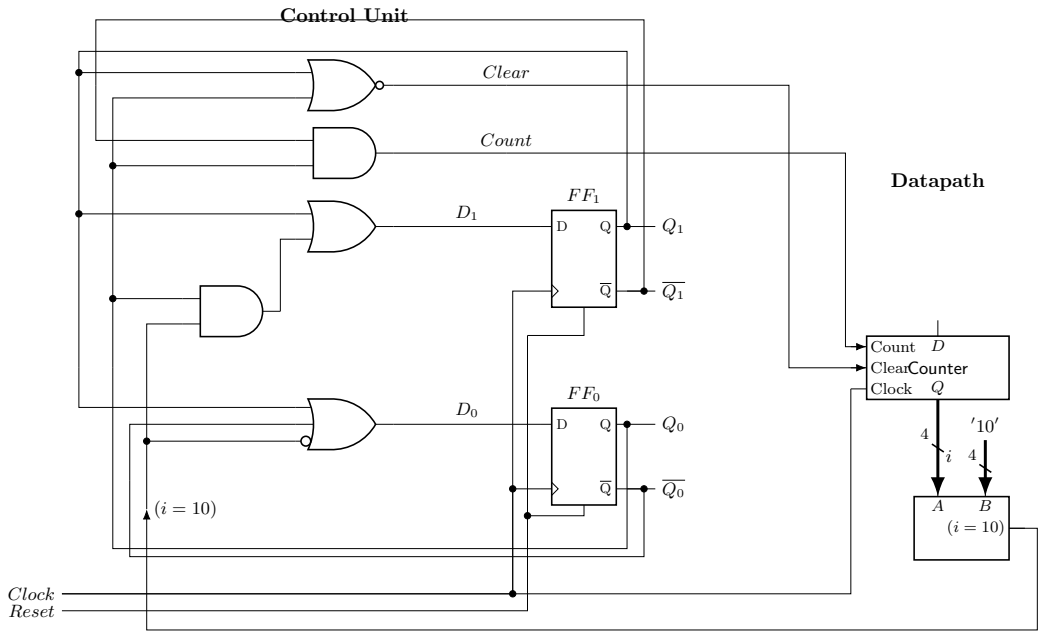

**(a)** control FSM

**(b)** datapath




**(c)** control logic

**Figure 9.11.** The counter processor of Example 9.3.




(d) complete processor

**Figure 9.11.** The counter processor of Example 9.3.

and the logic is written as the equations above, and Figure 9.14 shows the control unit connected to the datapath.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IfThenElseControl is
    Port ( Clk      : in  STD_LOGIC;
           Reset    : in  STD_LOGIC;
           A_eq_5   : in  STD_LOGIC;
           ALoad    : out STD_LOGIC;
           BLoad    : out STD_LOGIC;
           Muxsel   : out STD_LOGIC;
           Out_sig  : out STD_LOGIC);
end IfThenElseControl;

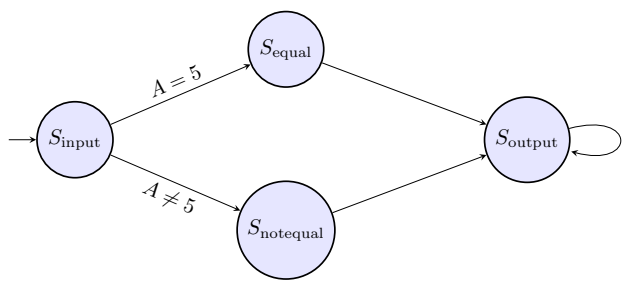
architecture Structural of IfThenElseControl is

    component d_ff_r
        Port ( D     : in  STD_LOGIC;
               clk   : in  STD_LOGIC;
               rst   : in  STD_LOGIC;
               Q     : out STD_LOGIC);
    end component;
```

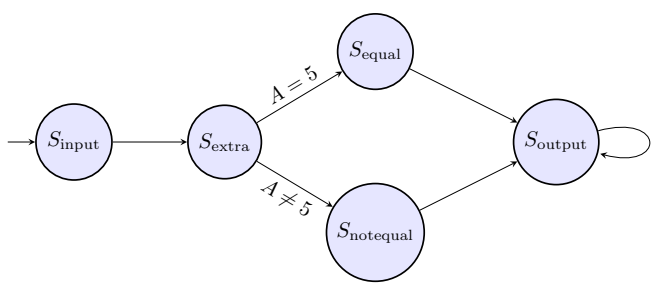
**Table 9.5.** Next-state and output tables of the if-then-else control unit with wait state ($I = (A = 5)$).

| Present state | $I$ | Next state |
|---|---|---|
| 000 ($S_{input}$) | X | 001 |
| 001 ($S_{extra}$) | 0 | 010 |
| 001 ($S_{extra}$) | 1 | 011 |
| 010 ($S_{notequal}$) | X | 100 |
| 011 ($S_{equal}$) | X | 100 |
| 100 ($S_{output}$) | X | 100 |

| State | Name | *ALoad* | *Muxsel* | *BLoad* | *Out* |
|---|---|---|---|---|---|
| 000 | $S_{input}$ | 1 | X | 0 | 0 |
| 001 | $S_{extra}$ | 0 | X | 0 | 0 |
| 010 | $S_{notequal}$ | 0 | 0 | 1 | 0 |
| 011 | $S_{equal}$ | 0 | 1 | 1 | 0 |
| 100 | $S_{output}$ | 0 | X | 0 | 1 |

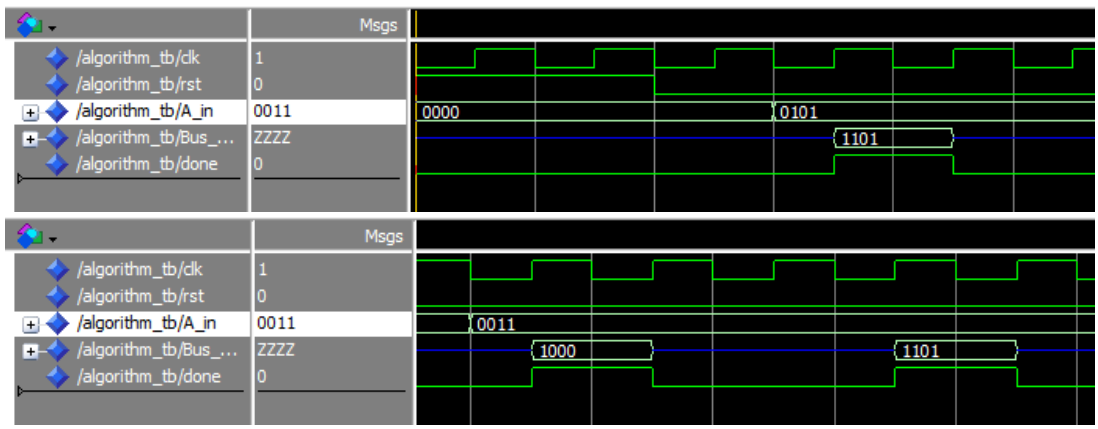



**(a)** without wait state

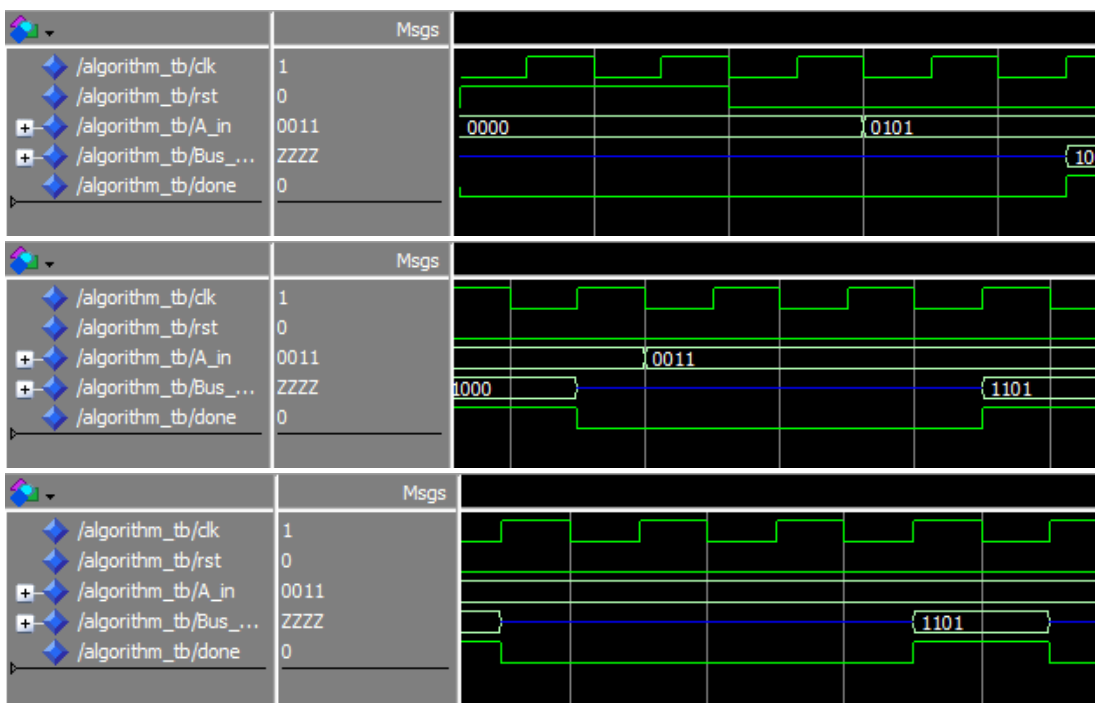



**(b)** with wait state

**Figure 9.12.** Control-unit state diagrams for the if-then-else algorithm.




(a) without wait state: the wrong branch is taken




(b) with wait state: the correct branch is taken

**Figure 9.13.** Simulation waveforms of the if-then-else processor (author's screenshots).

```
22
23      signal D, Q : STD_LOGIC_VECTOR(2 downto 0);
24
25  begin
26
27      -- Instantiate D Flip-Flops for State Register
28      DFF2: d_ff_r port map (D(2), Clk, Reset, Q(2));
29      DFF1: d_ff_r port map (D(1), Clk, Reset, Q(1));
30      DFF0: d_ff_r port map (D(0), Clk, Reset, Q(0));
31
32      -- Next State Logic
33      D(2) <= Q(2) or Q(1);
34      D(1) <= (not Q(1)) and Q(0);
35      D(0) <= (not Q(2)) and (not Q(1)) and ((not Q(0)) or A_eq_5);
36
37      -- Output Logic
38      ALoad   <= (not Q(2)) and (not Q(1)) and (not Q(0));
39      BLoad   <= (not Q(2)) and Q(1);
40      Muxsel  <= Q(0);
41      Out_sig <= Q(2);
42
43  end Structural;
```

**Listing 9.1.** Structural VHDL of the if-then-else control unit (state register of D flip-flops and the synthesised equations). The component `d_ff_r` is the flip-flop with asynchronous reset of Listing 6.8.

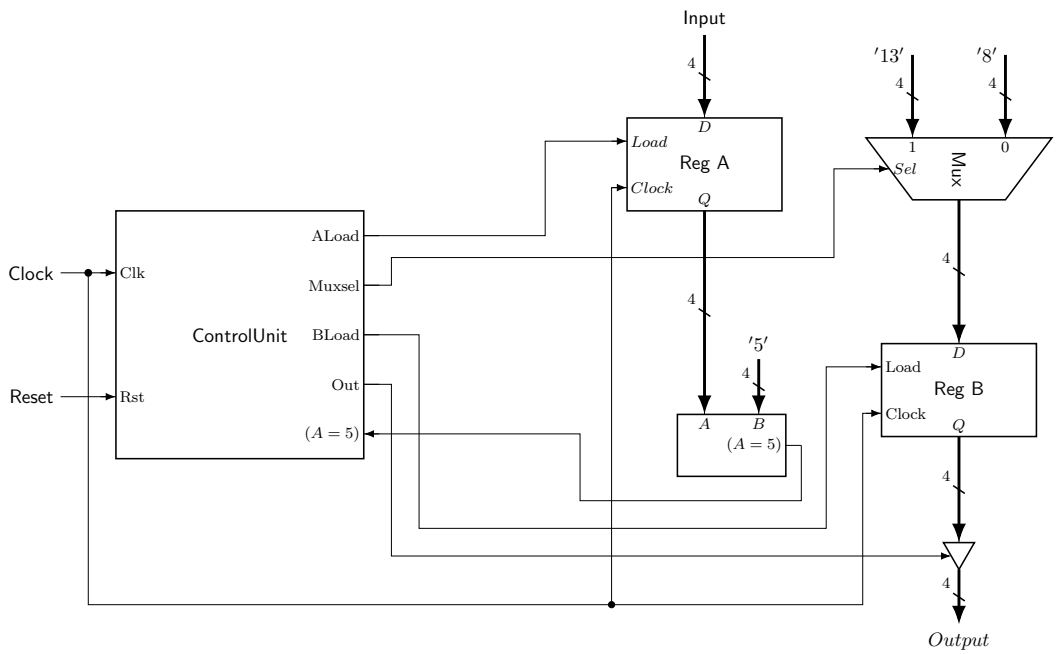



**Figure 9.14.** The if-then-else control unit connected to its datapath.

### 9.7.3 Example 3: the GCD processor

**Example 9.5: Greatest common divisor**

The *greatest common divisor* of two positive integers $X$ and $Y$ can be found by repeated subtraction (Euclid's algorithm): the larger number is replaced by the difference until the two are equal.

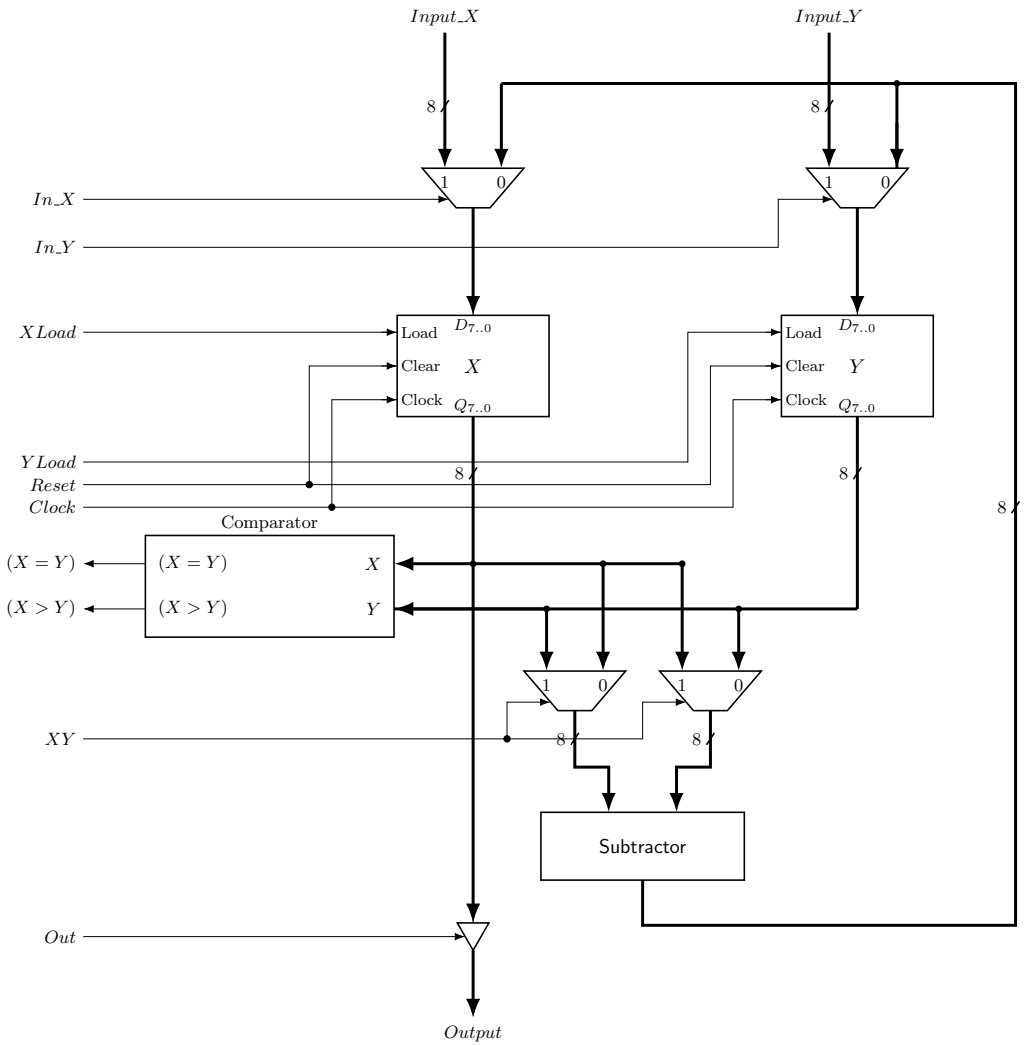
```
INPUT X
INPUT Y
WHILE (X != Y) {
  IF (X > Y) THEN X = X - Y
  ELSE Y = Y - X
}
OUTPUT X
```

For $X = 25$, $Y = 15$: $X > Y$, so $X = 10$; $X < Y$, so $Y = 5$; $X > Y$, so $X = 5$; now $X = Y = 5$, the GCD.

**Datapath** (Figure 9.15a): registers $X$ and $Y$, each with an input multiplexer that selects the external input or the result of the subtractor, a subtractor whose operands are chosen by multiplexers so that it can compute $X - Y$ or $Y - X$, and comparators that produce the status signals $XeqY$ and $XgtY$. The control signals are *In_X*, *In_Y* (select the external input, 1, or the subtractor result, 0), *XLoad*, *YLoad*, *XY* (0: $X - Y$; 1: $Y - X$) and *Out*.

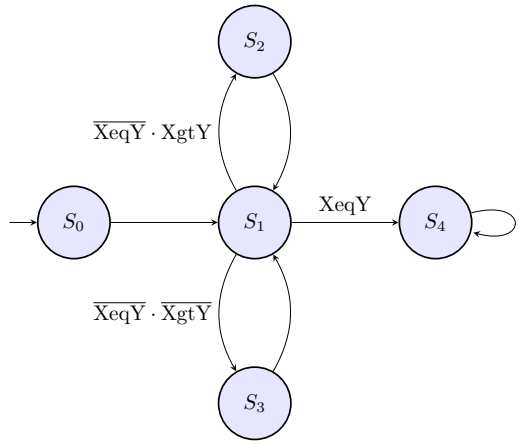
**Control unit** (Figure 9.15b). $S_0$ (init): load $X$ and $Y$ from the inputs. $S_1$ (check): compare; go to $S_4$ if $X = Y$, to $S_2$ if $X > Y$, otherwise to $S_3$. $S_2$: $X \leftarrow X - Y$, then back to $S_1$. $S_3$: $Y \leftarrow Y - X$, then back to $S_1$. $S_4$ (done): output the result. Listing 9.2 is the VHDL of the control unit, with the control signals as Moore outputs. The datapath components and the top-level connection are in Chapter C, and Figure 9.16 shows the simulation.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity gcd_control is
5      Port ( Clk      : in  STD_LOGIC;
6             Reset    : in  STD_LOGIC;
7
8             -- Status Signals
9             XeqY     : in  STD_LOGIC;
10            XgtY     : in  STD_LOGIC;
11
12            -- Control Signals
13            In_X     : out STD_LOGIC;
14            In_Y     : out STD_LOGIC;
15            XLoad    : out STD_LOGIC;
16            YLoad    : out STD_LOGIC;
17            XY       : out STD_LOGIC;
18            Out_En   : out STD_LOGIC);
```




(a) datapath

**Figure 9.15.** The GCD processor.




**(b)** control FSM

**Figure 9.15.** The GCD processor.

```
19 end gcd_control;
20
21 architecture Behavioral of gcd_control is
22     type StateType is (S0, S1, S2, S3, S4);
23     signal CurrentState, NextState : StateType;
24 begin
25
26     -- State Register
27     process(Clk)
28     begin
29         if rising_edge(Clk) then
30             if Reset = '1' then
31                 CurrentState <= S0;
32             else
33                 CurrentState <= NextState;
34             end if;
35         end if;
36     end process;
37
38     -- Next State Logic
39     process(CurrentState, XeqY, XgtY)
40     begin
41         case CurrentState is
42             when S0 => -- Init: Load X and Y
43                 NextState <= S1;
44
45             when S1 => -- Check
46                 if XeqY = '1' then
47                     NextState <= S4; -- Done
48                 elsif XgtY = '1' then
49                     NextState <= S2; -- X > Y
```

```
                else
                    NextState <= S3; -- Y > X
                end if;

            when S2 => -- X = X - Y
                NextState <= S1;

            when S3 => -- Y = Y - X
                NextState <= S1;

            when S4 => -- Done
                NextState <= S4; -- Stay here until Reset

            when others =>
                NextState <= S0;
        end case;
    end process;

    -- Output Logic
    process(CurrentState)
    begin
        -- Default values
        In_X   <= '0';
        In_Y   <= '0';
        XLoad  <= '0';
        YLoad  <= '0';
        XY     <= '0';
        Out_En <= '0';

        case CurrentState is
            when S0 => -- Load Inputs
                In_X  <= '1'; -- Select Input
                In_Y  <= '1'; -- Select Input
                XLoad <= '1';
                YLoad <= '1';

            when S1 => -- Check (No operation)
                NULL;

            when S2 => -- X = X - Y
                XY    <= '0'; -- X-Y
                In_X  <= '0'; -- Select Result
                XLoad <= '1';

            when S3 => -- Y = Y - X
                XY    <= '1'; -- Y-X (Swap)
                In_Y  <= '0'; -- Select Result
                YLoad <= '1';

            when S4 => -- Output
                Out_En <= '1';
```

```
101
102             when others =>
103                 NULL;
104         end case;
105     end process;
106
107 end Behavioral;
```

**Listing 9.2.** VHDL of the GCD control unit (FSM with five states).

## 9.8 FSM+D versus FSMD

There are two ways to describe a dedicated processor in an HDL.

**FSM+D (FSM plus datapath)** The control unit (FSM) and the datapath are designed as *separate, explicitly constructed* units and connected in a top-level module, as in the examples so far. The designer has full control over the datapath structure, which is useful for highly optimised or unusual datapaths.

**FSMD (FSM with datapath)** The whole design is described as *one behavioural FSM*, with the datapath operations (such as `X <= X - Y;`) written inside the states. The synthesis tool infers the registers, adders and multiplexers and connects them to the control logic. This is faster and simpler for standard operations and is the most common modern approach [18].

Listing 9.3 is the GCD calculator as an FSMD. The state diagram is the same as Figure 9.15b, but each state now contains the register transfer it performs. Compare its length with the FSM+D version; the RTL viewer (Figure 9.17) shows that the synthesis tool has generated the comparators, subtractors and registers by itself.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity gcd_fsmd is
6      Port ( Clk      : in  STD_LOGIC;
7             Reset    : in  STD_LOGIC;
8             InputX   : in  UNSIGNED(7 downto 0);
9             InputY   : in  UNSIGNED(7 downto 0);
10            Output   : out UNSIGNED(7 downto 0));
11 end gcd_fsmd;
12
13 architecture Behavioral of gcd_fsmd is
14     type StateType is (S0, S1, S2, S3, S4);
15     signal State : StateType;
16     signal X, Y  : UNSIGNED(7 downto 0);
17 begin
18
19     process(Clk)
```




**Figure 9.16.** Simulation of the GCD processor (author's screenshot).

```
    begin
        if rising_edge(Clk) then
            if Reset = '1' then
                State <= S0;
                Output <= (others => '0');
            else
                case State is
                    when S0 =>  -- Init
                        X <= InputX; Y <= InputY;
                        State <= S1;
                    when S1 =>  -- Check
                        if (X = Y) then State <= S4;
                        elsif (X > Y) then State <= S2;
                        else State <= S3;
                        end if;
                    when S2 =>  -- X = X - Y
                        X <= X - Y;
                        State <= S1;
                    when S3 =>  -- Y = Y - X
                        Y <= Y - X;
                        State <= S1;
                    when S4 =>  -- Done
                        Output <= X;
                end case;
            end if;
        end if;
    end process;

end Behavioral;
```

**Listing 9.3.** The GCD calculator as an FSMD: datapath operations inside a single behavioural state machine.

**Chapter Summary**

- The **von Neumann model** has input, output, memory and a microprocessor made of a **datapath** and a **control unit**. A **dedicated microprocessor** hard-wires a single algorithm.
- The **datapath** (registers, adders, multiplexers, comparators) performs the data operations; the **control unit** is an FSM that sends **control signals** and receives **status signals**.
- At the **RTL**, a computation is a sequence of register transfers $R_1 \leftarrow f(R_2, R_3)$.
- Each instruction of an algorithm becomes a **control word**; multiplexers let one functional unit serve several operations.
- The control unit's FSM has one state per control word; the next-state logic uses status signals. A **wait state** is required when a decision depends on a register loaded in the preceding state.

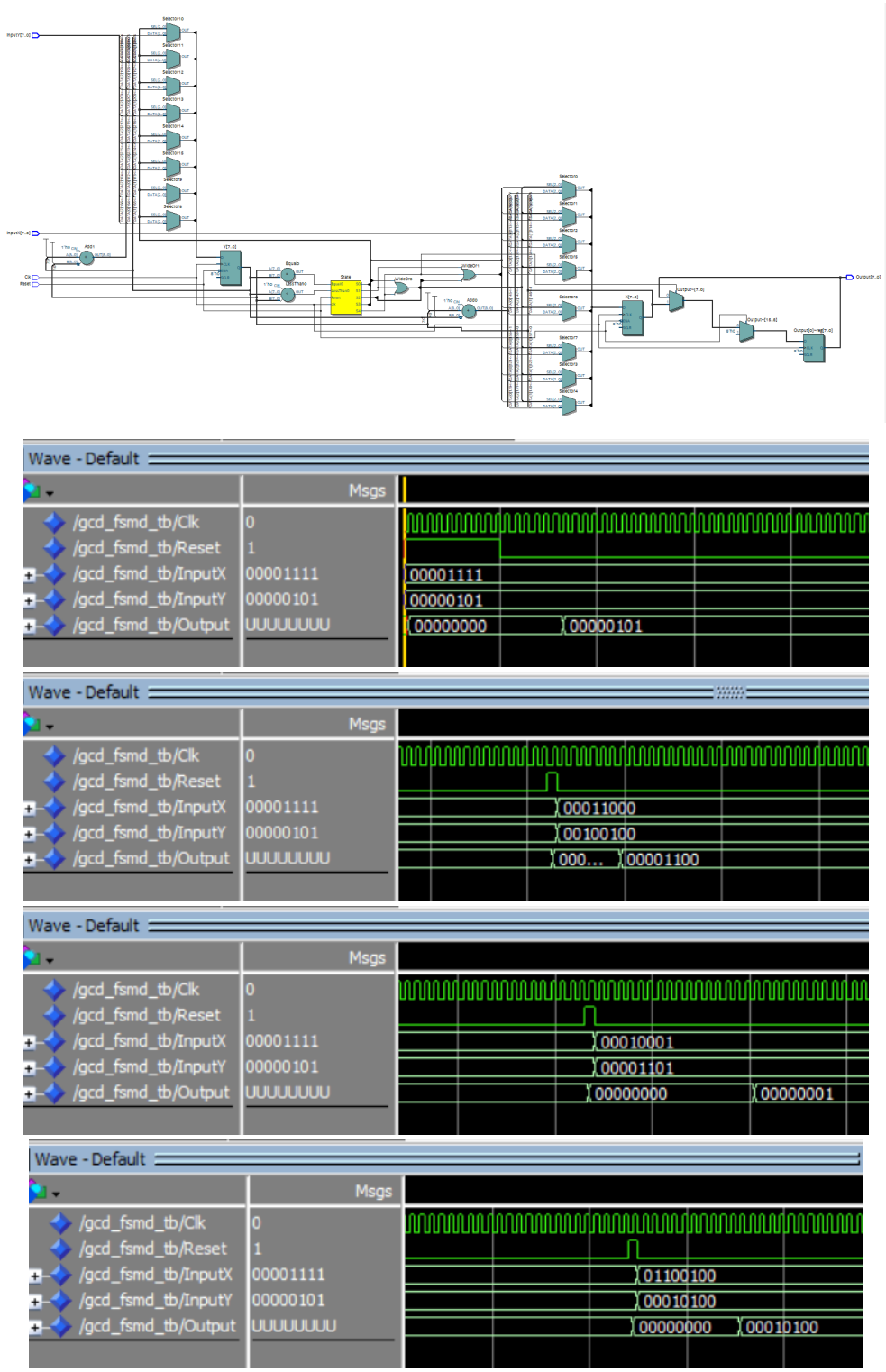



**Figure 9.17.** The FSMD GCD: logic diagram from the Quartus RTL viewer (top) and simulation (bottom). Author's screenshots.

- **FSM+D** builds the datapath and control unit explicitly; **FSMD** describes everything as one behavioural FSM and lets the tool infer the datapath.

## Exercises

**9.1** **Datapath design.** Draw a dedicated datapath that performs $R_1 \leftarrow R_1 - R_2$. List the control signals.

**9.2** **Control signals.** In the shared-adder datapath of Figure 9.6, suppose the two multiplexers had separate select signals $Mux1$ (first multiplexer: $0 = B$, $1 = A$) and $Mux2$ (second multiplexer: $0 = C$, $1 =$ constant $3$). Give the values of $Mux1$, $Mux2$ and $ALoad$ that perform $A = B + 3$.

**9.3** **Conditional logic.** What hardware must be added to a datapath to support the instruction **IF** $(A > 10)$ **THEN** ...? Which signal is delivered to the control unit?

**9.4** **Control-word sequence.** Write the control-word sequence for *swapping* the contents of registers $A$ and $B$, assuming a temporary register $Temp$. Define the control signals and give one control word per step.

**9.5** Write the control words and draw the FSM of the control unit for the summation algorithm of Table 9.3. How many states are needed? What is the status signal and in which state is it examined?

**9.6** Trace Euclid's algorithm for $(X, Y) = (12, 18)$ and $(X, Y) = (48, 18)$. Count the number of clock cycles the FSM of Figure 9.15b takes (from $S_0$ to the first cycle in $S_4$).

**9.7** Draw a datapath for $F \leftarrow (A + B) - C$ with one adder/subtractor and one result register. Write the sequence of control words.

**9.8** Derive the next-state equations $D_2, D_1, D_0$ of the if-then-else control unit from Table 9.5. Verify that the unused states 101, 110 and 111 do or do not lead to a valid state, and discuss.

**9.9** Explain in your own words why the if-then-else control unit needs the state $S_{extra}$, with a timing sketch of the register load and the comparator output.

**9.10** Rewrite the if-then-else processor as an FSMD in VHDL: a single process with the states *input*, *extra*, *equal*, *notequal*, *output*.

**9.11** Write an FSMD in VHDL that computes $Sum = 1 + 2 + \cdots + 10$ and presents the result in the last state.

**9.12** Compare FSM+D and FSMD for a GCD processor: which would you use to build a processor whose adders must be hand-placed on an FPGA to meet timing, and why?

## Further Reading

The method of designing a processor as a datapath and a control unit, with control words and status signals, follows Hwang [4, 15]. The FSMD model is described by

Gajski [18] and by Harris and Harris [20], who give a good account of the relation between architecture and microarchitecture; Brown and Vranesic [3] discuss datapath/controller design in their treatment of simple processors.

# 10

# General-Purpose Microprocessors

**Learning Objectives**

After studying this chapter you will be able to:

- explain how a general-purpose processor differs from a dedicated one, and describe the instruction cycle (fetch, decode, execute);
- define an instruction set and its binary encoding;
- describe the datapath and control unit of the simple processors EC-1 and EC-2 and trace the execution of a program;
- explain why the control unit needs wait states when memory is synchronous;
- write, assemble and trace small programs in machine code, and load them into memory with an Intel HEX file; and
- describe how the step from ROM to RAM leads to the von Neumann stored-program computer.

## 10.1 General-Purpose Microprocessors

The dedicated processors of Chapter 9 perform one task, wired into their structure. A *general-purpose microprocessor* can perform many different functions: its function is determined by the *instructions* it executes, which are stored in memory as a *program*. Changing the program changes the function without changing the hardware. The Intel Core and ARM Cortex processors are examples.

**A processor is a dedicated processor**

> A general-purpose processor can be viewed as a *dedicated* processor whose single task is to **fetch and execute instructions**. The "data" that it processes include the instructions themselves.

We therefore design it with the methods of the previous chapter. The designs of this chapter, the 8-bit processors **EC-1** and **EC-2**, follow the educational processor designs of Hwang [15, 4]; the structure, the VHDL descriptions, the programs and the simulations presented here are the author's implementations for this course.

## 10.2 Overview of CPU Design

Designing a central processing unit (CPU) has three steps.

1. **Define the instruction set**: the operations the processor performs (add, load, jump, ...) and how each instruction is encoded in binary (an *opcode* and its *operands*).
2. **Design the datapath**: a datapath that can execute *every* instruction of the set. It contains the *program counter* (PC), which holds the address of the next instruction; the *instruction register* (IR), which holds the current instruction; the ALU or other functional units; registers; and memory.
3. **Design the control unit**: an FSM that executes the *instruction cycle*: **fetch** the instruction from memory (at the address in the PC) into the IR, **decode** it by examining the opcode, and **execute** it by issuing the control signals for the operation.

## 10.3 The EC-1 Microprocessor

### 10.3.1 Overview

To keep the design small enough to do by hand, EC-1 is a simple 8-bit processor with:

- an **8-bit** data width;
- a **4-bit** address, hence $2^4 = 16$ memory words;
- a **read-only memory** (ROM) that holds the program; and
- one data register, the *accumulator* $A$.

It is an *accumulator machine*: every operation takes its operand from, and returns its result to, register $A$.

### 10.3.2 Instruction set

EC-1 has only five instructions. The opcode occupies the 3 most significant bits of the 8-bit instruction (Table 10.1). Opcodes 000, 001 and 010 are unused (no operation); the bits shown as `x` are don't-cares and `aaaa` is a 4-bit address.

**Table 10.1.** The EC-1 instruction set.

| Mnemonic | Encoding | Operation | Description |
|---|---|---|---|
| `IN A` | `011 xxxxx` | $A \leftarrow$ Input | input a byte into $A$ |
| `OUT A` | `100 xxxxx` | Output $\leftarrow A$ | output $A$ |
| `DEC A` | `101 xxxxx` | $A \leftarrow A - 1$ | decrement $A$ |
| `JNZ addr` | `110 xaaaa` | if $A \neq 0$, $PC \leftarrow aaaa$ | jump if not zero |
| `HALT` | `111 xxxxx` | halt | stop execution |

### 10.3.3 Datapath

The datapath (Figure 10.1) must support fetching instructions and executing them.

- The **program counter** PC (4 bits) addresses the ROM. Normally $PC \leftarrow PC + 1$ (the incrementer). For `JNZ` with $A \neq 0$, $PC \leftarrow IR[3:0]$; the multiplexer *JNZmux* chooses between the incremented PC and the jump address.
- The **instruction register** IR (8 bits) holds the instruction read from the ROM. Its top three bits $IR[7:5]$ are the opcode that goes to the control unit.
- The **accumulator** $A$ (8 bits) takes its input either from the external input port or from the decrementer ($A-1$); the multiplexer *INmux* chooses ($1 =$ Input, $0 =$ Dec).
- A **zero detector** (an 8-input OR gate on $A$) generates the status signal $A \neq 0$ for the control unit.
- A **three-state buffer** controlled by *OutE* drives the output port with $A$.

The control signals, produced by the control unit, are *IRload*, *PCload*, *INmux*, *Aload*, *JNZmux*, *OutE* and *Halt*.

### 10.3.4 ROM implementation

FPGA vendors provide optimised memory blocks as *IP cores* (intellectual-property modules). Using them guarantees that the memory is mapped to the FPGA's dedicated block-RAM resources rather than to general logic cells. In our implementation the program ROM is a one-port ROM generated by Intel's megafunction wizard (type `altsyncram`): an 8-bit wide, 32-word memory with a single clock and *unregistered* output, initialised at programming time from a file in Intel HEX format. The memory is *synchronous*: the address is sampled on the rising clock edge. As we shall see, this has consequences for the control unit.

### 10.3.5 Intel HEX format

*Intel HEX* is a text format that stores binary data for programming or initialising memories. Each line is a *record* beginning with a colon: `:` *LL AAAA TT DD…DD CC*, where *LL* is the number of data bytes, *AAAA* the starting address, *TT* the record type (00 = data, 01 = end of file), *DD* the data bytes and *CC* a checksum chosen so that the sum

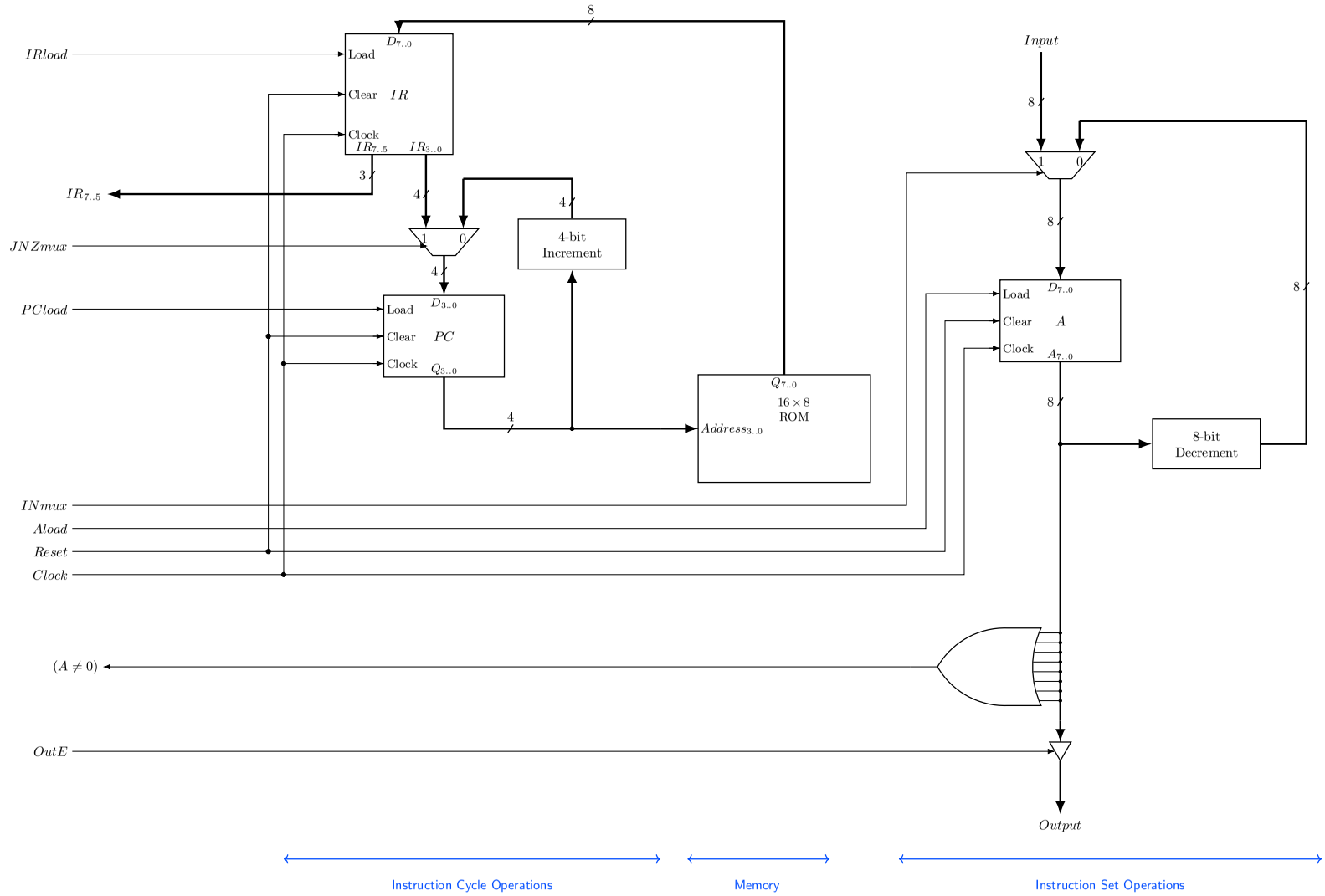



**Figure 10.1.** Datapath of the EC-1 microprocessor. Multiplexer inputs are numbered 1 (left) and 0 (right); a slash on a bus gives its width in bits. The clock and the reset are common to all registers, and the control signals (left) come from the control unit.

of all bytes of the record is $00_{16}$ modulo 256. For example, the line

```
:100000006080A0C1E000000000000000000000000CF
```

holds 16 ($10_{16}$) bytes starting at address 0000, the first five being the program of Table 10.3; its checksum is CF because $10+00+00+00+60+80+A0+C1+E0 = 331_{16}$ and $100_{16} - 31_{16} = \mathrm{CF}_{16}$. The file is completed by a record for addresses 16–31 and the end-of-file record `:00000001FF`.

### 10.3.6 Control unit

The control unit is an FSM that carries out the instruction cycle, with one **fetch** state, one **decode** state and one execution state for each instruction (Figure 10.2a).

**FETCH (000)** Read the instruction at address PC into IR ($IR \leftarrow M[PC]$) and increment the PC ($PC \leftarrow PC + 1$): $IRload = PCload = 1$, $JNZmux = 0$.

**DECODE (001)** Examine the opcode $IR[7:5]$ and go to the execution state of the instruction (no control signals). Unused opcodes return to FETCH.

**EXECUTE** **Input** (011): $A \leftarrow$ Input, $INmux = 1$, $Aload = 1$. **Output** (100): $OutE = 1$. **Dec** (101): $A \leftarrow A - 1$, $INmux = 0$, $Aload = 1$. **JNZ** (110): if $A \neq 0$ then $PC \leftarrow IR[3:0]$ ($JNZmux = 1$, $PCload = 1$). **Halt** (111): stay here.

All execution states except Halt return to FETCH. The values of the control signals in each state are in Table 10.2.

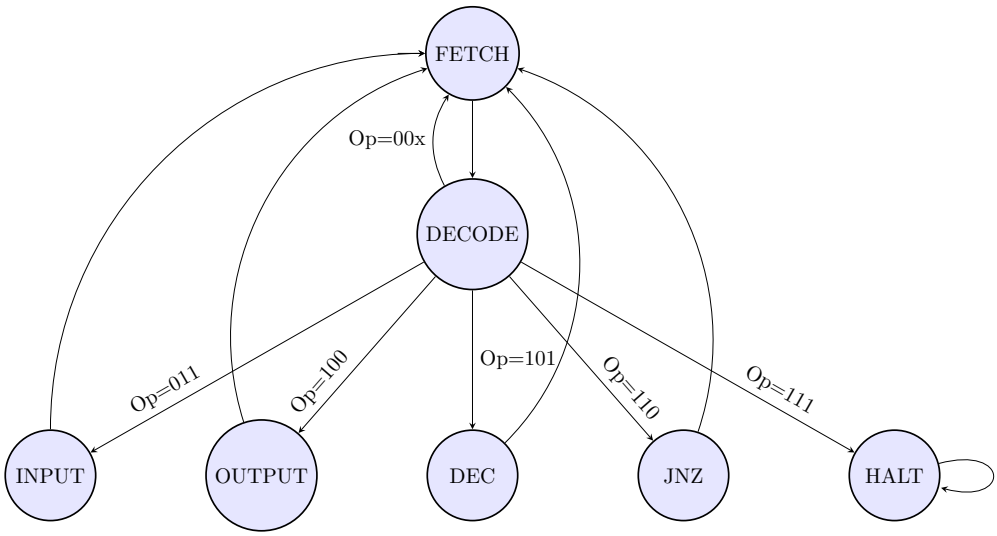
**Table 10.2.** Control signals of the EC-1 in each state ("–" = don't care).

| State | Code | IRload | PCload | INmux | Aload | JNZmux | OutE | Halt |
|---|---|---|---|---|---|---|---|---|
| Fetch | 000 | 1 | 1 | – | 0 | 0 | 0 | 0 |
| Decode | 001 | 0 | 0 | – | 0 | 0 | 0 | 0 |
| Wait | 010 | 0 | 0 | – | 0 | 0 | 0 | 0 |
| Input | 011 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| Output | 100 | 0 | 0 | – | 0 | 0 | 1 | 0 |
| Decrement | 101 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| JumpNotZero | 110 | 0 | $(A \neq 0)$ | – | 0 | 1 | 0 | 0 |
| Halt | 111 | 0 | 0 | – | 0 | 0 | 0 | 1 |

#### The wait state

The EC-1 uses a *synchronous* ROM: it samples the address at the clock edge. In normal execution the PC is stable throughout the execution state, so the next fetch reads the right instruction. But when a jump is *taken*, the PC is updated with the target address at the clock edge at the *end* of the JNZ state. If the next state were FETCH, the ROM would sample the PC at the *same edge* at which the PC changes, and may read the old address: a race condition. The solution is a **wait state**: JNZ goes to WAIT (010), a buffer cycle in which all control signals are idle, while the PC settles at the target; WAIT then goes to FETCH, in which the ROM safely samples the stable address (Figure 10.2b). This

is the same idea as the wait state of Example 9.4.




**(a)** without wait state

**Figure 10.2.** State diagram of the EC-1 control unit.

### State table and equations

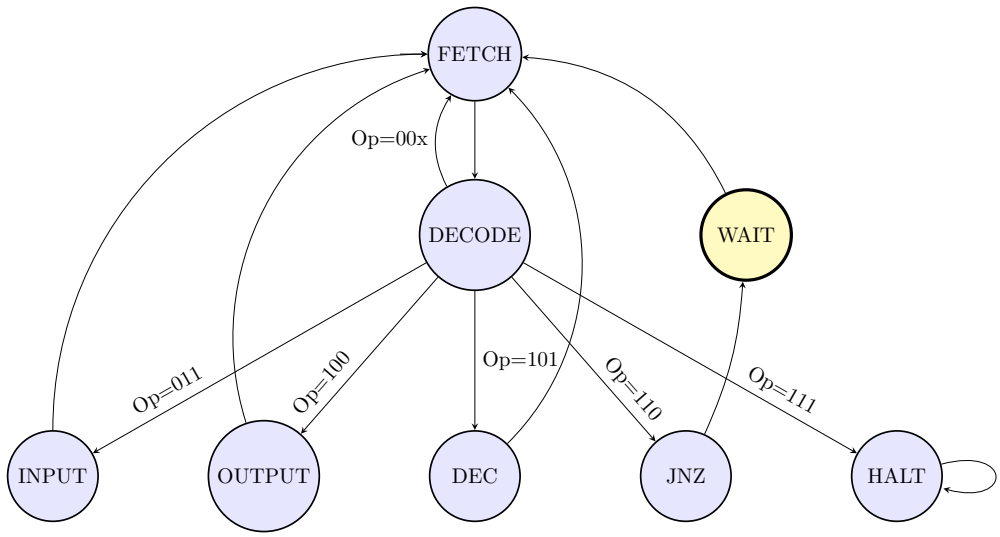
With the state codes of Table 10.2, and $O_2O_1O_0$ the opcode, the next-state equations follow from the state diagram:

$$
\begin{aligned}
D_2 &= \text{Decode} \cdot O_2 + \text{Halt},\\
D_1 &= \text{Decode} \cdot O_1 \cdot (O_2 + O_0) + \text{Halt} + \text{JNZ},\\
D_0 &= \text{Fetch} + \text{Decode} \cdot O_0 \cdot (O_2 + O_1) + \text{Halt},
\end{aligned}
$$

where the state decoders are Fetch $= Q_2'Q_1'Q_0'$, Decode $= Q_2'Q_1'Q_0$, Input $= Q_2'Q_1Q_0$, Output $= Q_2Q_1'Q_0'$, Dec $= Q_2Q_1'Q_0$, JNZ $= Q_2Q_1Q_0'$ and Halt $= Q_2Q_1Q_0$. Each term can be checked against Figure 10.2b: for instance $D_1 = 1$ in the JNZ state because the next state is WAIT (010); in the decode state $D_1 = 1$ for the opcodes 011 (Input = 011), 110 (JNZ = 110) and 111 (Halt = 111) but not 010 (unused, back to Fetch). WAIT and the other states return to 000 by $D_2 = D_1 = D_0 = 0$. The outputs are

$$
IRload = \text{Fetch}, \quad PCload = \text{Fetch} + \text{JNZ} \cdot (A \neq 0), \quad INmux = \text{Input},
$$
$$
Aload = \text{Input} + \text{Dec}, \quad JNZmux = \text{JNZ}, \quad OutE = \text{Output}, \quad Halt = \text{Halt}.
$$

Listing 10.1 is the VHDL of the control unit; it is written directly from these equations (structural style), using three D flip-flops. Listing 10.2 is the datapath. The components (flip-flops, registers, multiplexers, incrementer, decrementer, zero detector, three-state buffer) and the top level, which connects the two, are listed in Chapter C.




**(b)** with wait state

**Figure 10.2.** State diagram of the EC-1 control unit.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.EC1_Components.ALL;

entity EC1_Control_Unit is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           opcode : in STD_LOGIC_VECTOR (2 downto 0);
           A_neq_0 : in STD_LOGIC;
           IRload : out STD_LOGIC;
           PCload : out STD_LOGIC;
           INmux : out STD_LOGIC;
           Aload : out STD_LOGIC;
           JNZmux : out STD_LOGIC;
           OutE : out STD_LOGIC;
           Halt : out STD_LOGIC;
           dbg_state : out STD_LOGIC_VECTOR(2 downto 0));
end EC1_Control_Unit;

architecture Structural of EC1_Control_Unit is
    signal Q, D : std_logic_vector(2 downto 0);
    signal Q2_n, Q1_n, Q0_n : std_logic;
    signal state_Fetch, state_Decode, state_Halt : std_logic;
    signal state_Input, state_Output, state_Dec, state_JNZ, state_Wait : std_logic
    ;

begin
    -- State Registers
```

```
    FF2: d_ff port map(clk, reset, D(2), Q(2));
    FF1: d_ff port map(clk, reset, D(1), Q(1));
    FF0: d_ff port map(clk, reset, D(0), Q(0));

    Q2_n <= not Q(2); Q1_n <= not Q(1); Q0_n <= not Q(0);

    -- State Decoders
    state_Fetch <= Q2_n and Q1_n and Q0_n;   -- 000
    state_Decode <= Q2_n and Q1_n and Q(0);  -- 001
    state_Wait <= Q2_n and Q(1) and Q0_n;    -- 010 (New)
    state_Input <= Q2_n and Q(1) and Q(0);   -- 011
    state_Output <= Q(2) and Q1_n and Q0_n;  -- 100
    state_Dec <= Q(2) and Q1_n and Q(0);     -- 101
    state_JNZ <= Q(2) and Q(1) and Q0_n;     -- 110
    state_Halt <= Q(2) and Q(1) and Q(0);    -- 111

    -- Next State Logic
    -- D2 = Decode * O2 + Halt
    D(2) <= (state_Decode and opcode(2)) or state_Halt;

    -- D1 = Decode * O1 * (O2 + O0) + Halt + JNZ (Go to Wait logic: 110 -> 010)
    -- If in JNZ (110), we want next state to be Wait (010).
    -- Wait (010) next state is Fetch (000), which happens naturally as D=000.
    D(1) <= (state_Decode and opcode(1) and (opcode(2) or opcode(0))) or
    state_Halt or state_JNZ;

    -- D0 = Fetch + Decode * O0 * (O2 + O1) + Halt
    D(0) <= state_Fetch or (state_Decode and opcode(0) and (opcode(2) or opcode(1)
    )) or state_Halt;

    -- Output Logic
    IRload <= state_Fetch;
    PCload <= state_Fetch or (state_JNZ and A_neq_0);
    INmux <= state_Input;
    Aload <= state_Input or state_Dec;
    JNZmux <= state_JNZ;
    OutE <= state_Output;
    Halt <= state_Halt;

    dbg_state <= Q;

end Structural;
```

**Listing 10.1.** VHDL of the EC-1 control unit (structural, from the state equations).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.EC1_Components.ALL;

entity EC1_Datapath is
    Port ( clk, reset : in STD_LOGIC;
```

```
            input_bus : in STD_LOGIC_VECTOR(7 downto 0);
            IRload, PCload, INmux, Aload, JNZmux, OutE : in STD_LOGIC;
            opcode : out STD_LOGIC_VECTOR(2 downto 0);
            A_out_bus : out STD_LOGIC_VECTOR(7 downto 0);
            A_neq_0 : out STD_LOGIC;
            dbg_PC : out STD_LOGIC_VECTOR(3 downto 0);
            dbg_IR : out STD_LOGIC_VECTOR(7 downto 0));
end EC1_Datapath;

architecture Structural of EC1_Datapath is
    signal IR_out, A_out, A_in, Dec_out : std_logic_vector(7 downto 0);
    signal PC_out, PC_in, PC_inc : std_logic_vector(3 downto 0);
    signal ROM_data : std_logic_vector(7 downto 0);

begin
    -- ROM (Program Memory)
    ROM_Unit: rom32x8 port map(
        address(4) => '0',
        address(3 downto 0) => PC_out,
        clock => clk,
        q => ROM_data
    );

    -- Instruction Register
    IR_Reg: Reg8 port map(clk, reset, IRload, ROM_data, IR_out);
    opcode <= IR_out(7 downto 5);

    -- Program Counter Logic
    PC_Inc_Unit: Inc4 port map(PC_out, PC_inc);
    PC_Mux: Mux2_4 port map(JNZmux, PC_inc, IR_out(3 downto 0), PC_in);
    PC_Reg: Reg4 port map(clk, reset, PCload, PC_in, PC_out);

    -- Accumulator Logic
    Dec_Unit: Dec8 port map(A_out, Dec_out);
    A_Mux: Mux2_8 port map(INmux, Dec_out, input_bus, A_in);

    A_Reg: Reg8 port map(clk, reset, Aload, A_in, A_out);

    -- Zero Check
    ZC: ZeroCheck8 port map(A_out, A_neq_0);

    -- Output Buffer
    OutBuf: TriBuf8 port map(OutE, A_out, A_out_bus);

    dbg_PC <= PC_out;
    dbg_IR <= IR_out;

end Structural;
```

**Listing 10.2.** VHDL of the EC-1 datapath.

### 10.3.7 Programming example: countdown from 10 to 1

A program for EC-1 is a list of instruction bytes. Table 10.3 counts down from the input value to 1, outputting each value: `IN A` reads the starting value, `OUT A` displays $A$, `DEC A` decrements it, `JNZ 01` returns to `OUT A` while $A \neq 0$, and `HALT` stops. Instructions are encoded by Table 10.1; for example `JNZ 01` $=$ `110 0 0001` $=$ C1.

**Table 10.3.** EC-1 program: count down from the input value.

| Address | Hex | Assembly | Comment |
|---|---|---|---|
| 00 | 60 | `IN A` | input to $A$ |
| 01 | 80 | `OUT A` | output $A$ |
| 02 | A0 | `DEC A` | decrement $A$ |
| 03 | C1 | `JNZ 01` | jump to 01 if $A \neq 0$ |
| 04 | E0 | `HALT` | done |

A testbench (similar to Listing 2.6) generates a clock of period 10 ns, holds the reset for 20 ns, applies the input value 10 (`00001010`) and runs until the *halt* output is 1. Figure 10.3 shows the result: the output counts $10, 9, \ldots, 2, 1$. For every value the processor goes through FETCH, DECODE and the execution state of `OUT A`, FETCH, DECODE, `DEC A`, FETCH, DECODE, `JNZ` and, when the jump is taken, WAIT; when $A$ reaches 0 the jump is not taken, PC advances to 04 and `HALT` is executed. The RTL viewer of Quartus shows the synthesised top level (Figure 10.4).

## 10.4 The EC-2 Microprocessor

### 10.4.1 Overview

EC-2 improves on EC-1 in two ways: more instructions and a *read/write* memory that stores both code and data. This is a *stored-program computer* in the von Neumann sense (Figure 10.5). EC-2 has

- eight instructions;
- a $32 \times 8$-bit **RAM** that holds both the program and its variables (unified code/data memory);
- a **5-bit** address; and
- an adder/subtractor ALU.

### 10.4.2 Instruction set

Table 10.4 lists the instructions. The three most significant bits are the opcode and the five least significant bits the memory address `aaaaa`. Memory operands make this a *memory-reference* machine. `JZ` jumps if $A = 0$ and `JPOS` if $A \geq 0$ (a non-negative two's-complement value).

(a) start: counting 10, 9, ...

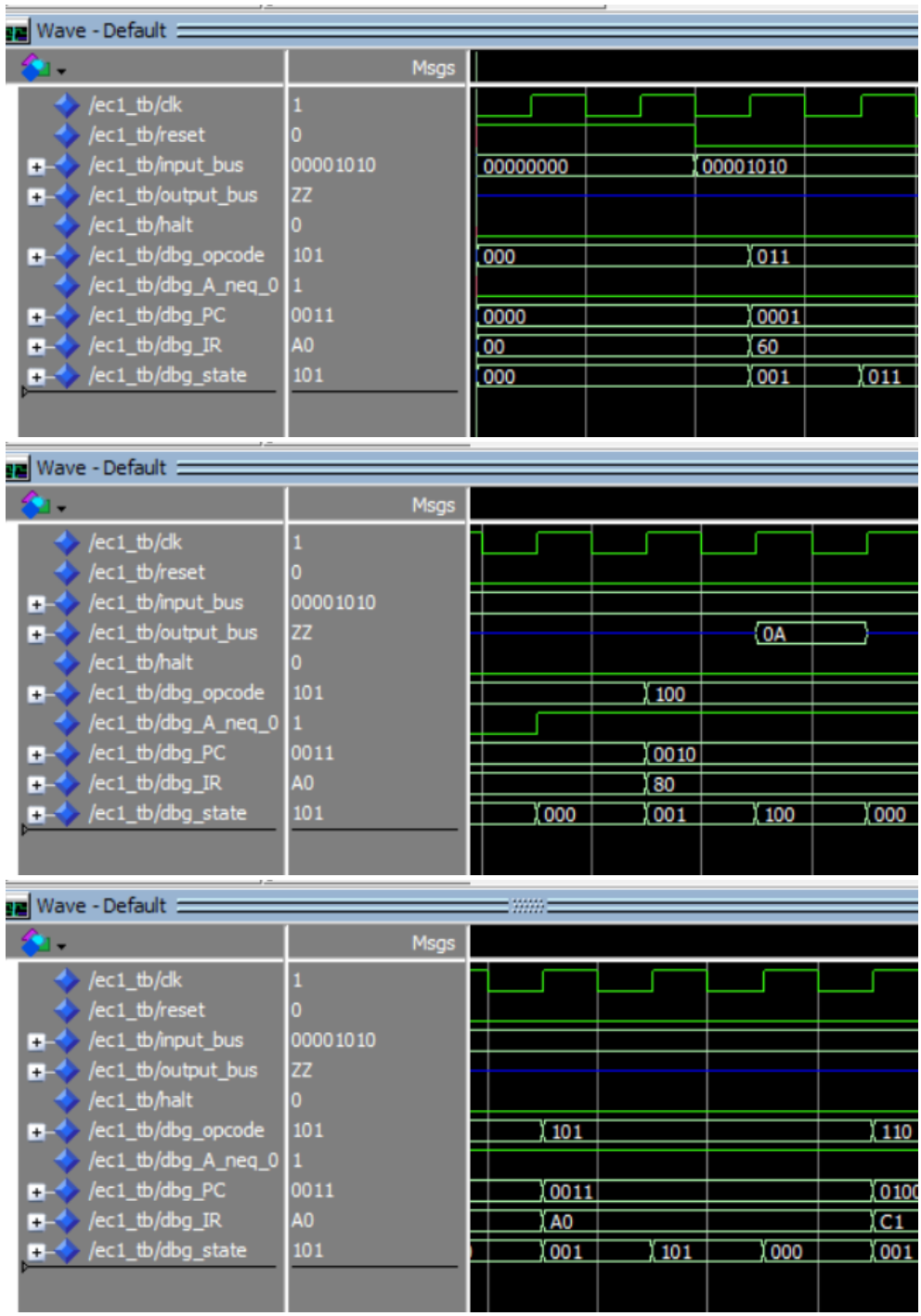
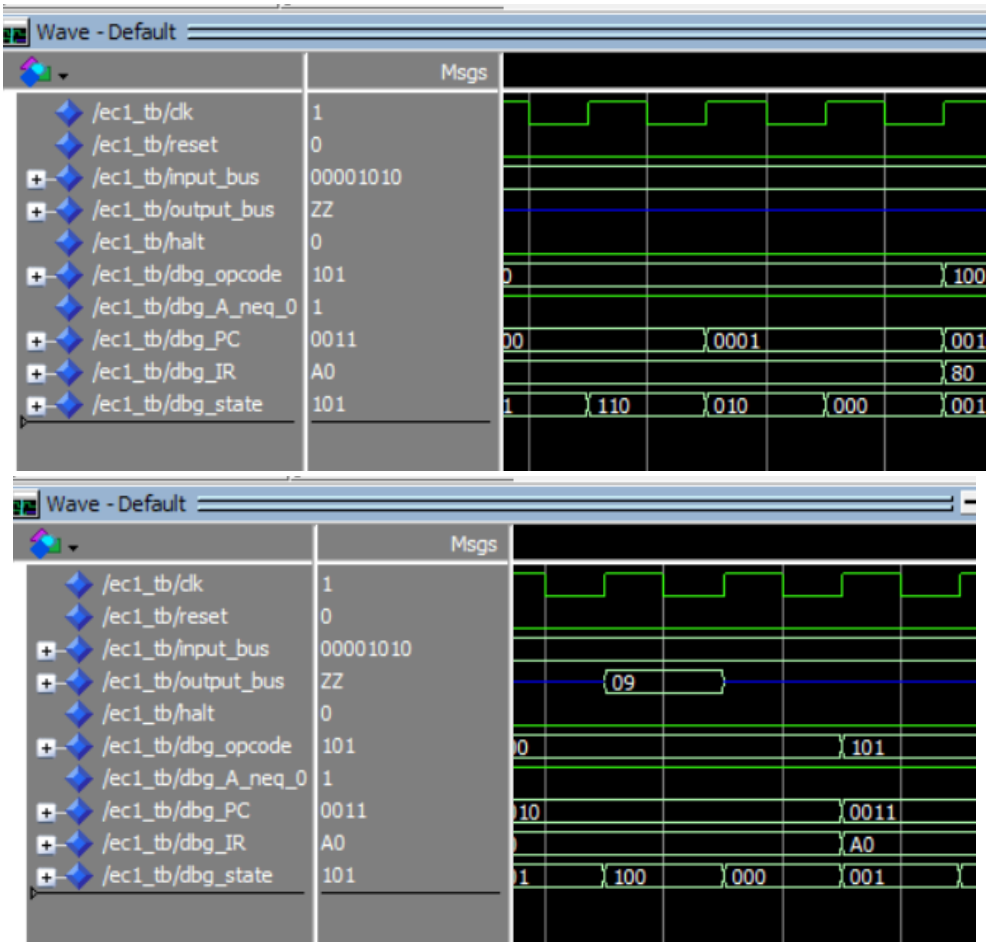



**Figure 10.3.** Simulation of the EC-1 running the countdown program (author's screenshots).




(b) end: ..., 2, 1 and halt

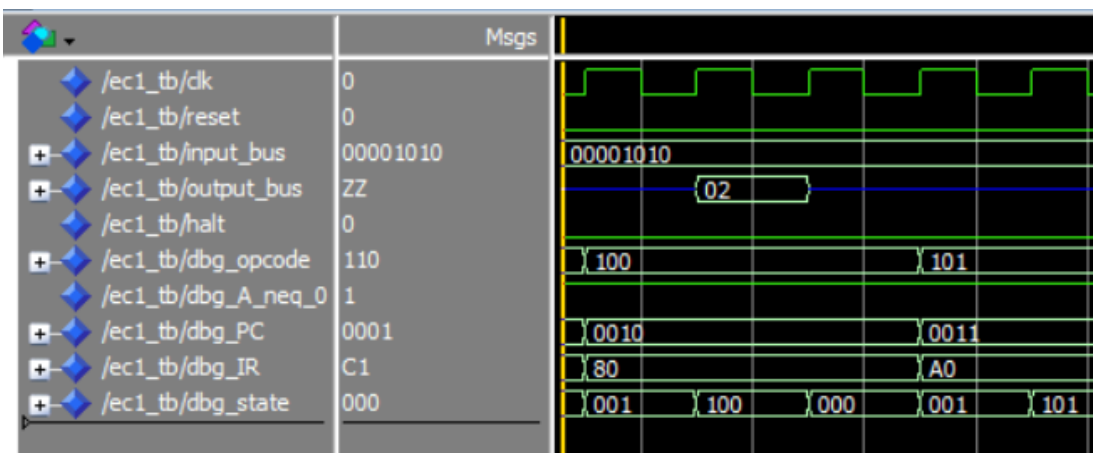



**Figure 10.3.** Simulation of the EC-1 running the countdown program (author's screenshots).

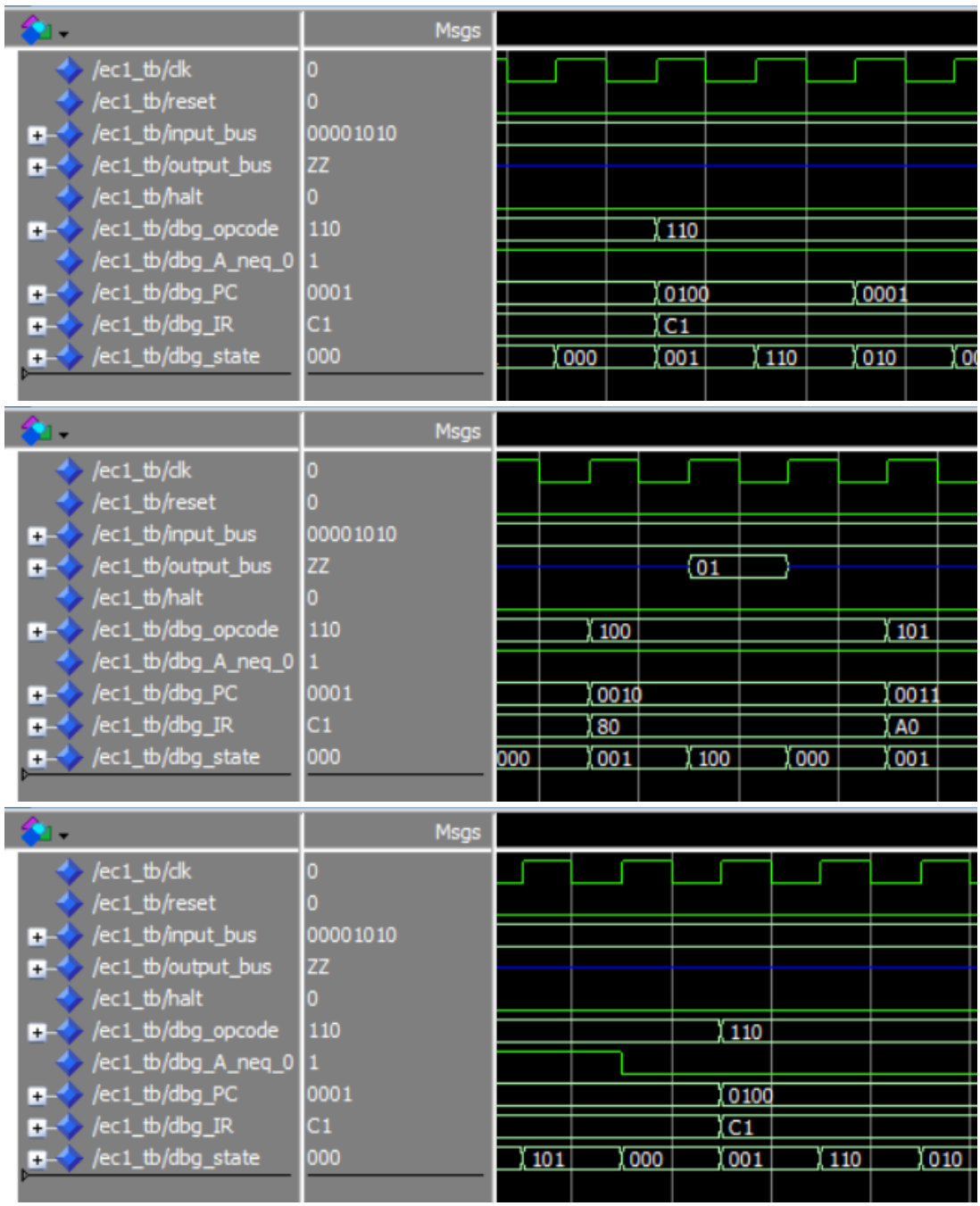



**Figure 10.3.** Simulation of the EC-1 running the countdown program (author's screenshots).

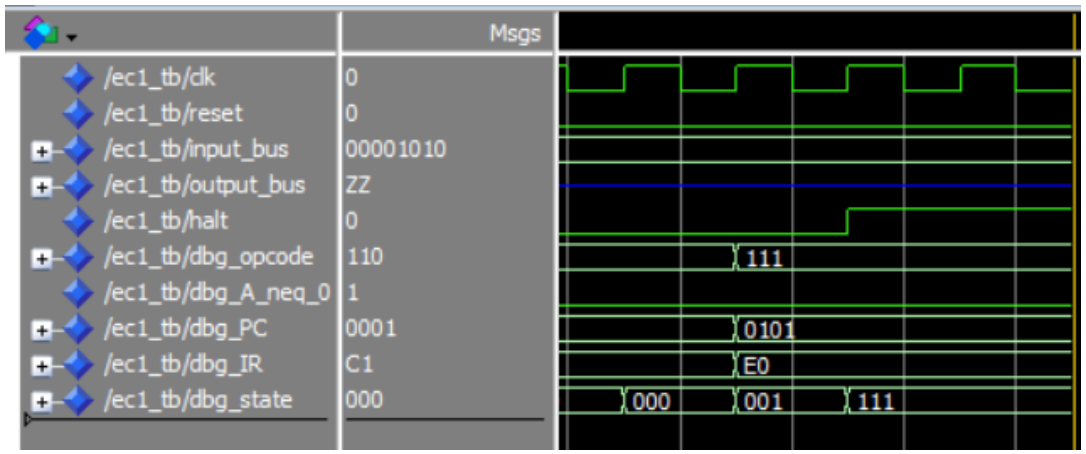



**Figure 10.3.** Simulation of the EC-1 running the countdown program (author's screenshots).

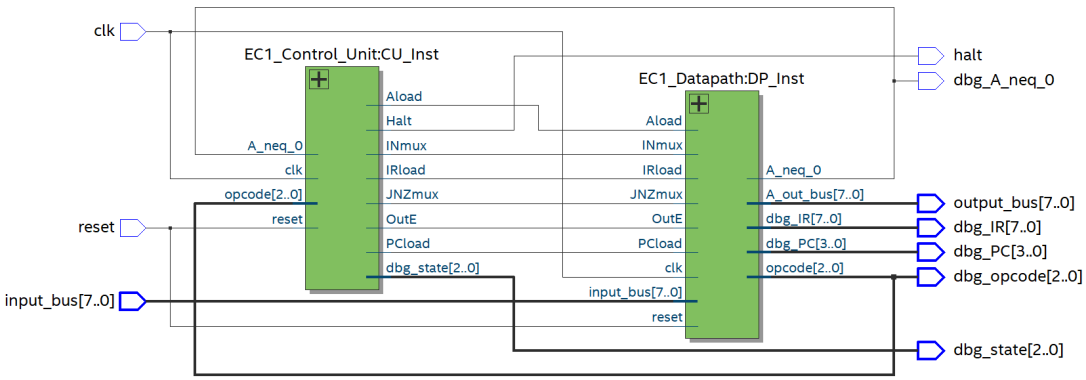



**Figure 10.4.** RTL viewer (Quartus) of the top level of the EC-1: the control unit and the datapath (author's screenshot).

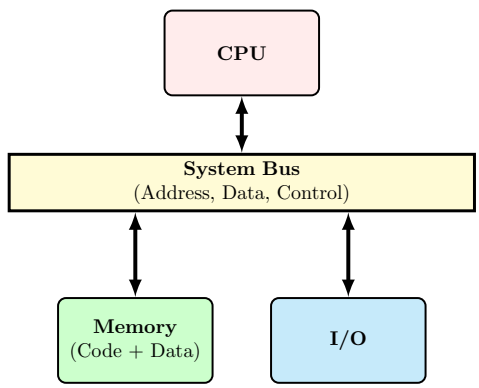



**Figure 10.5.** The von Neumann architecture: a single memory holds instructions and data, connected to the processor by a common bus.

**Table 10.4.** The EC-2 instruction set.

| Mnemonic | Encoding | Operation | Description |
|---|---|---|---|
| `LOAD A, addr` | `000 aaaaa` | $A \leftarrow M[aaaaa]$ | load $A$ from memory |
| `STORE A, addr` | `001 aaaaa` | $M[aaaaa] \leftarrow A$ | store $A$ to memory |
| `ADD A, addr` | `010 aaaaa` | $A \leftarrow A + M[aaaaa]$ | add memory to $A$ |
| `SUB A, addr` | `011 aaaaa` | $A \leftarrow A - M[aaaaa]$ | subtract memory from $A$ |
| `IN A` | `100 xxxxx` | $A \leftarrow$ Input | input to $A$ |
| `JZ addr` | `101 aaaaa` | if $A = 0$, $PC \leftarrow aaaaa$ | jump if zero |
| `JPOS addr` | `110 aaaaa` | if $A \geq 0$, $PC \leftarrow aaaaa$ | jump if non-negative |
| `HALT` | `111 xxxxx` | halt | stop execution |

### 10.4.3 Datapath

The datapath (Figure 10.6) is that of EC-1 with the following changes.

1. A **5-bit address bus** to address 32 locations; the PC and IR address fields are 5 bits wide.
2. A **RAM** instead of the ROM, so that `STORE` is possible.
3. A **memory-address multiplexer** (*MemInst*): during *fetch* the address comes from the PC (0); during the execution of LOAD, STORE, ADD and SUB it comes from $IR[4:0]$ (1). This shared address bus is the fundamental constraint of the von Neumann design.
4. An **ALU** that adds ($Sub = 0$) or subtracts ($Sub = 1$) the RAM output from $A$.
5. A **4-to-1 accumulator multiplexer** (*Asel*) that selects the input of $A$: the RAM output (LOAD, 00), the ALU output (ADD, SUB, 01) or the input port (IN, 10); code 11 selects the constant 0.
6. Flag detectors for $A = 0$ (*Zero*, an 8-input NOR gate) and $A \geq 0$ (*Pos*, the inverted sign bit $A_7$), the status signals for JZ and JPOS.

The control signals are *IRload*, *PCload*, *MemInst*, *MemWr* (RAM write enable), *Asel*, *Aload*, *Sub*, *JMPmux* (selects the jump address as the new PC) and *Halt*.

### 10.4.4 Control unit

The instruction cycle now includes memory access (Table 10.5). In **fetch** ($MemInst = 0$) the instruction is read from the address in the PC into IR and the PC is incremented. In **decode** the opcode selects the execution state. In **execute**: `LOAD` sets $MemInst = 1$, reads the RAM and loads $A$; `STORE` sets $MemInst = 1$ and $MemWr = 1$, writing $A$ into the RAM; `ADD` and `SUB` set $MemInst = 1$, read the RAM and load $A$ with the ALU result; `IN` loads $A$ from the input port; `JZ` and `JPOS` load the PC with the address field if the flag $Z$ or $P$ is true; `HALT` stops. The FSM is Figure 10.7, with ten states coded in 4 bits.

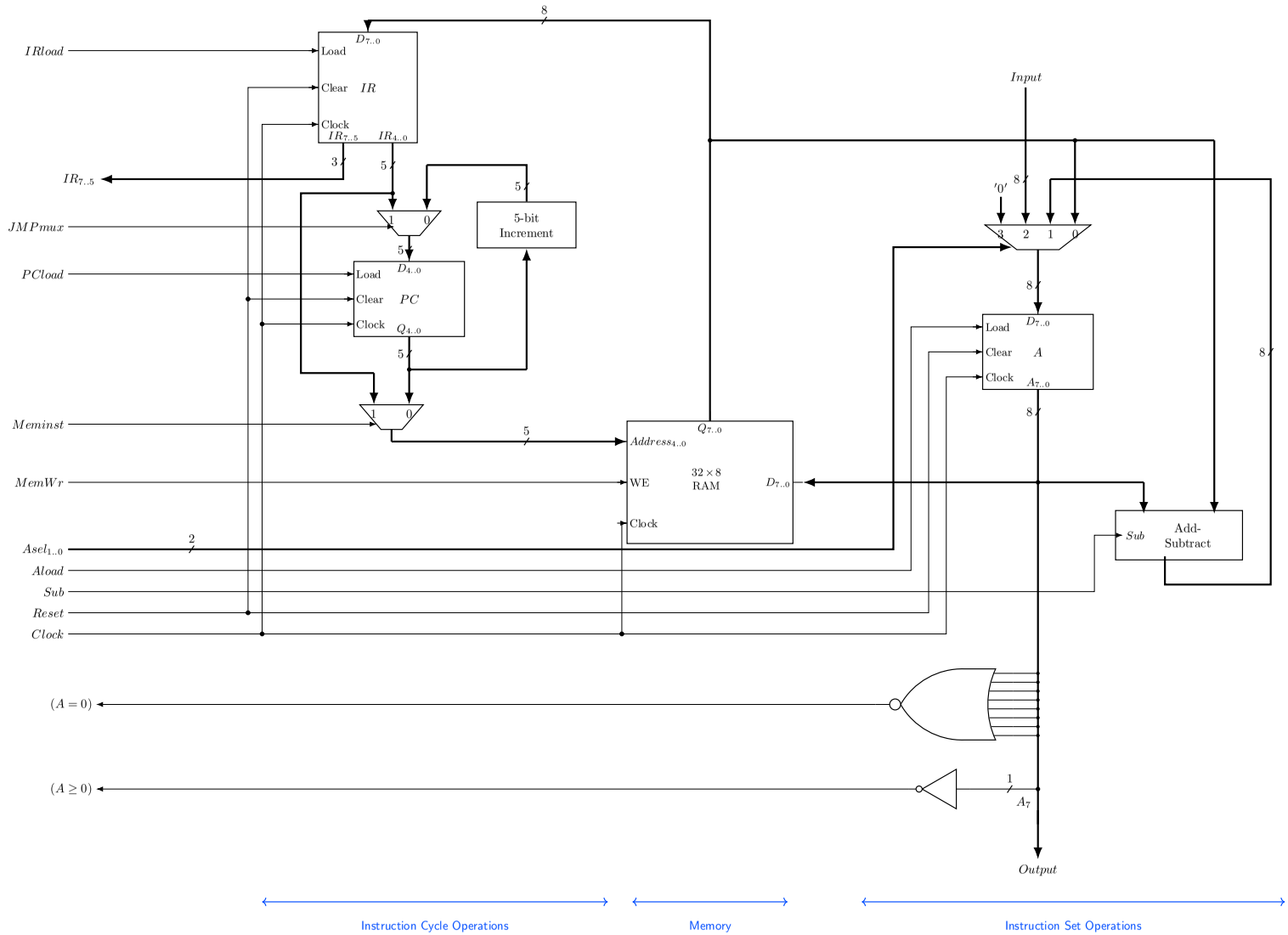



**Figure 10.6.** Datapath of the EC-2 microprocessor. The memory-address multiplexer *MemInst* selects the PC (0) in the fetch state and the address field of the IR (1) when a memory operand is accessed; the accumulator multiplexer is selected by $Asel_{1..0}$ (00 RAM, 01 ALU, 10 input port, 11 constant 0). $(A = 0)$ and $(A \geq 0)$ are the status signals for `JZ` and `JPOS`.

**Table 10.5.** Control signals of the EC-2 in each state ($Z$: $A = 0$; $P$: $A \geq 0$).

| State | Code | IRload | PCload | MemInst | MemWr | Asel | Aload | Sub | JMPmux | Halt |
|---|---|---|---|---|---|---|---|---|---|---|
| Fetch | 0000 | 1 | 1 | 0 | 0 | – | 0 | 0 | 0 | 0 |
| Decode | 0001 | 0 | 0 | 0 | 0 | – | 0 | 0 | 0 | 0 |
| Load | 0010 | 0 | 0 | 1 | 0 | 00 | 1 | 0 | 0 | 0 |
| Store | 0011 | 0 | 0 | 1 | 1 | – | 0 | 0 | 0 | 0 |
| Add | 0100 | 0 | 0 | 1 | 0 | 01 | 1 | 0 | 0 | 0 |
| Sub | 0101 | 0 | 0 | 1 | 0 | 01 | 1 | 1 | 0 | 0 |
| In | 0110 | 0 | 0 | 0 | 0 | 10 | 1 | 0 | 0 | 0 |
| Jz | 0111 | 0 | $Z$ | 0 | 0 | – | 0 | 0 | 1 | 0 |
| Jpos | 1000 | 0 | $P$ | 0 | 0 | – | 0 | 0 | 1 | 0 |
| Halt | 1001 | 0 | 0 | 0 | 0 | – | 0 | 0 | 0 | 1 |

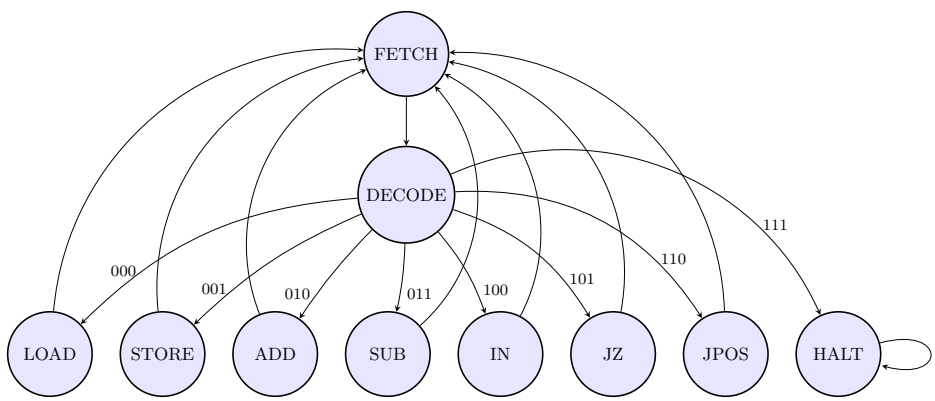



**Figure 10.7.** State diagram of the EC-2 control unit.

With the state codes of Table 10.5, and the opcode $O_2O_1O_0$ examined in the decode state, the next-state equations are

$$D_3 = \text{Decode} \cdot O_2O_1 + \text{Halt}, \qquad D_2 = \text{Decode} \cdot (O_2 \oplus O_1),$$
$$D_1 = \text{Decode} \cdot O_1', \qquad D_0 = \text{Fetch} + \text{Decode} \cdot O_0 + \text{Halt},$$

and the outputs are

$$IRload = \text{Fetch}, \quad PCload = \text{Fetch} + \text{Jz} \cdot Z + \text{Jpos} \cdot P, \quad JMPmux = \text{Jz} + \text{Jpos},$$
$$MemInst = \text{Load} + \text{Store} + \text{Add} + \text{Sub}, \quad MemWr = \text{Store},$$
$$Asel_1 = \text{In}, \quad Asel_0 = \text{Add} + \text{Sub}, \quad Aload = \text{Load} + \text{Add} + \text{Sub} + \text{In}, \quad Sub = \text{Sub}, \quad Halt = \text{Halt}.$$

As a check of $D_2$: Add (010 → 0100), Sub (011 → 0101), In (100 → 0110) and Jz (101 → 0111) have $O_2 \oplus O_1 = 1$; Load, Store and Halt/Jpos (11$x$) have $O_2 \oplus O_1 = 0$ and $D_2 = 0$. The complete VHDL of the control unit, the datapath, the components (including the RAM with its initial contents) and the top level is in Chapter C.

### 10.4.5 Memory wait states and timing

EC-2 is a *multi-cycle* processor: every instruction takes several clock cycles, and some states act as wait states for the memory (Figure 10.8). The whole **fetch** cycle is a wait for the instruction to be read. The execution states of the memory-reading instructions (`LOAD`, `ADD`, `SUB`) drive the address bus and wait for the RAM data, which is captured in $A$ at the end of the cycle (asynchronous read). A `STORE` sets up the address and data and asserts *MemWr*, and the RAM writes on the clock edge (synchronous write). Waiting for one cycle guarantees that the signals are stable through the chain address multiplexer → RAM → data out.

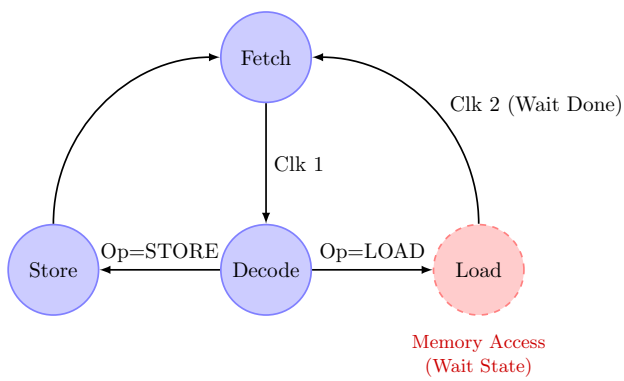



**Figure 10.8.** Multi-cycle memory access in the EC-2.

### 10.4.6 Programming example: GCD

The program of Table 10.6 implements Euclid's algorithm of Example 9.5: `while (X != Y) { if (X > Y) X -= Y; else Y -= X; }`. The variables occupy memory locations 30 ($X$) and 31 ($Y$), which are initialised to 15 and 25. Because EC-2 has only `JZ` and `JPOS`, the compare is done by subtraction: $A = X - Y$ is zero if they are equal (jump to the end) and positive if $X > Y$; a `JPOS` after a result that is known to be positive acts as an unconditional jump.

**Table 10.6.** EC-2 program that computes the GCD of the numbers in locations 30 and 31 (addresses in decimal).

| Address | Hex | Assembly | Comment |
|---|---|---|---|
| 00 | 1E | `LOAD A, 30` | load $X$ |
| 01 | 7F | `SUB A, 31` | $A = X - Y$ |
| 02 | AA | `JZ 10` | if $X = Y$, done |
| 03 | C8 | `JPOS 08` | if $X > Y$, update $X$ |
| 04 | 1F | `LOAD A, 31` | else ($X < Y$) load $Y$ |
| 05 | 7E | `SUB A, 30` | $A = Y - X$ |
| 06 | 3F | `STORE A, 31` | save new $Y$ |
| 07 | C0 | `JPOS 00` | loop |
| 08 | 3E | `STORE A, 30` | save new $X$ |
| 09 | C0 | `JPOS 00` | loop |
| 10 | E0 | `HALT` | result in $M[30]$ |

**Trace.** Starting with $X = 15$, $Y = 25$: $A = X - Y = -10$; `JZ` and `JPOS` do not jump; $A \leftarrow Y = 25$, $A \leftarrow A - X = 10$, $Y \leftarrow 10$ and the program loops. Now $A = 15 - 10 = 5 > 0$: `JPOS 08`; $X \leftarrow 5$; loop. Then $A = 5 - 10 = -5$: $A \leftarrow 10$, $A \leftarrow 5$, $Y \leftarrow 5$; loop. Now $A = 5 - 5 = 0$ and `JZ 10` jumps to `HALT`; $M[30] = 5$, the GCD of 15 and 25. Figure 10.9 shows the simulation of the processor running this program, Figure 10.10 the synthesised top level.

### 10.4.7 Program loading and self-modifying code

In Listing C.10 (Appendix) the program is hard-coded into the RAM initial contents. A real system loads programs from storage into RAM using a *bootloader*, a small program that is already in memory. EC-2 has no *indirect addressing* (no way to address memory through a pointer), so the loader cannot write to `RAM[i]` with a variable $i$. The solution exploits the von Neumann property that *code is data*: the program **modifies its own** `STORE` instruction to change its destination (Table 10.7):

1. read a byte from the input;
2. store it at the target address (the address field of the `STORE` at location 01);
3. read the `STORE` instruction itself (`LOAD A, 01`), add 1 (the constant at location 06)

(a)

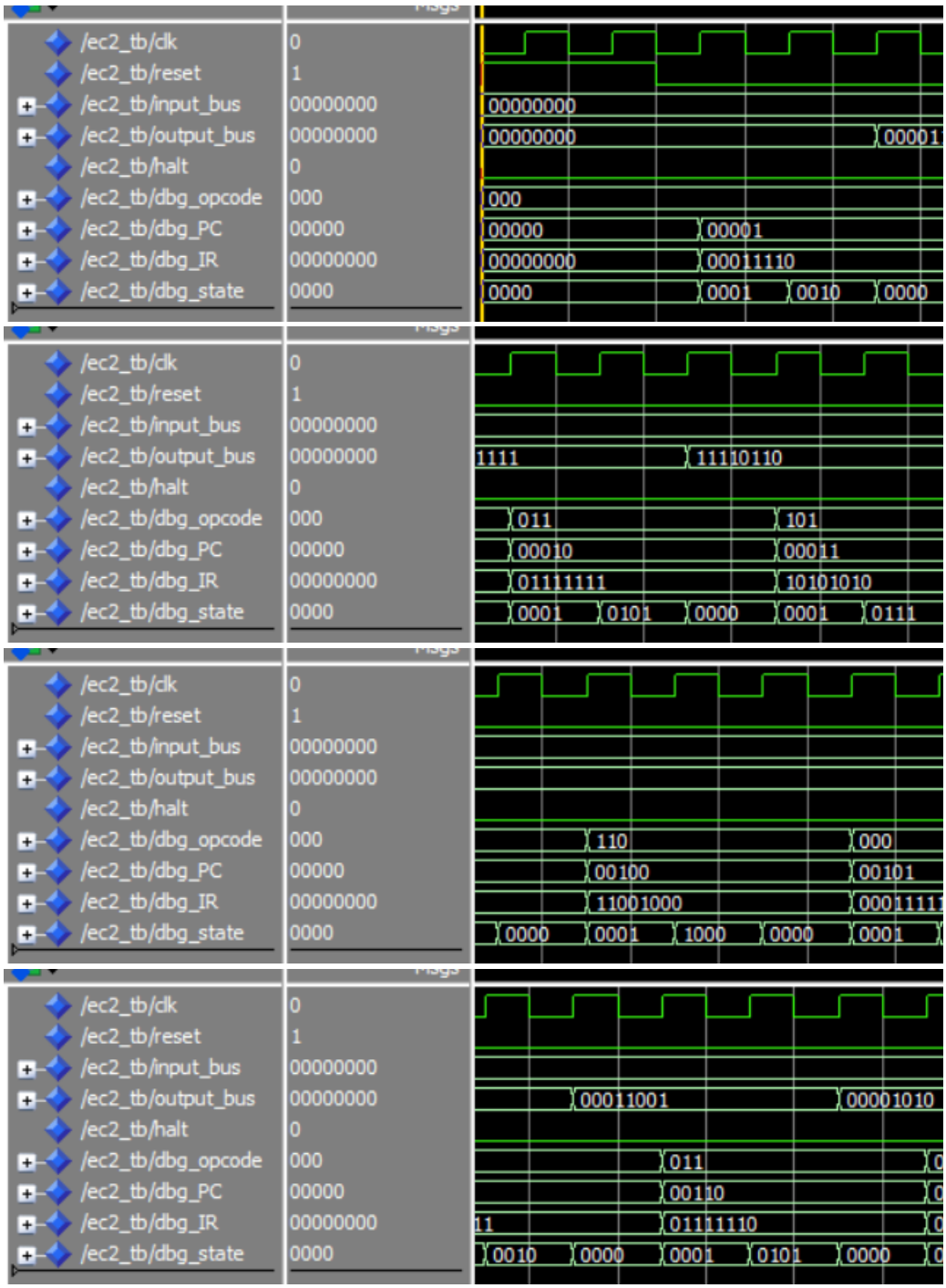
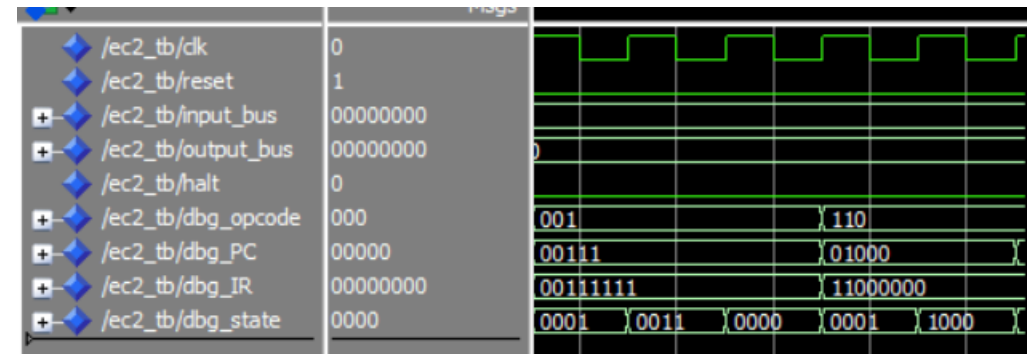



**Figure 10.9.** Simulation of the EC-2 computing the GCD of 25 and 15: (a)–(d) successive parts of the run: $Y$ becomes 10, then $X$ becomes 5, then $Y$ becomes 5, and finally $A = X - Y = 0$, `JZ` is taken and the processor halts. Author's screenshots.




(b)

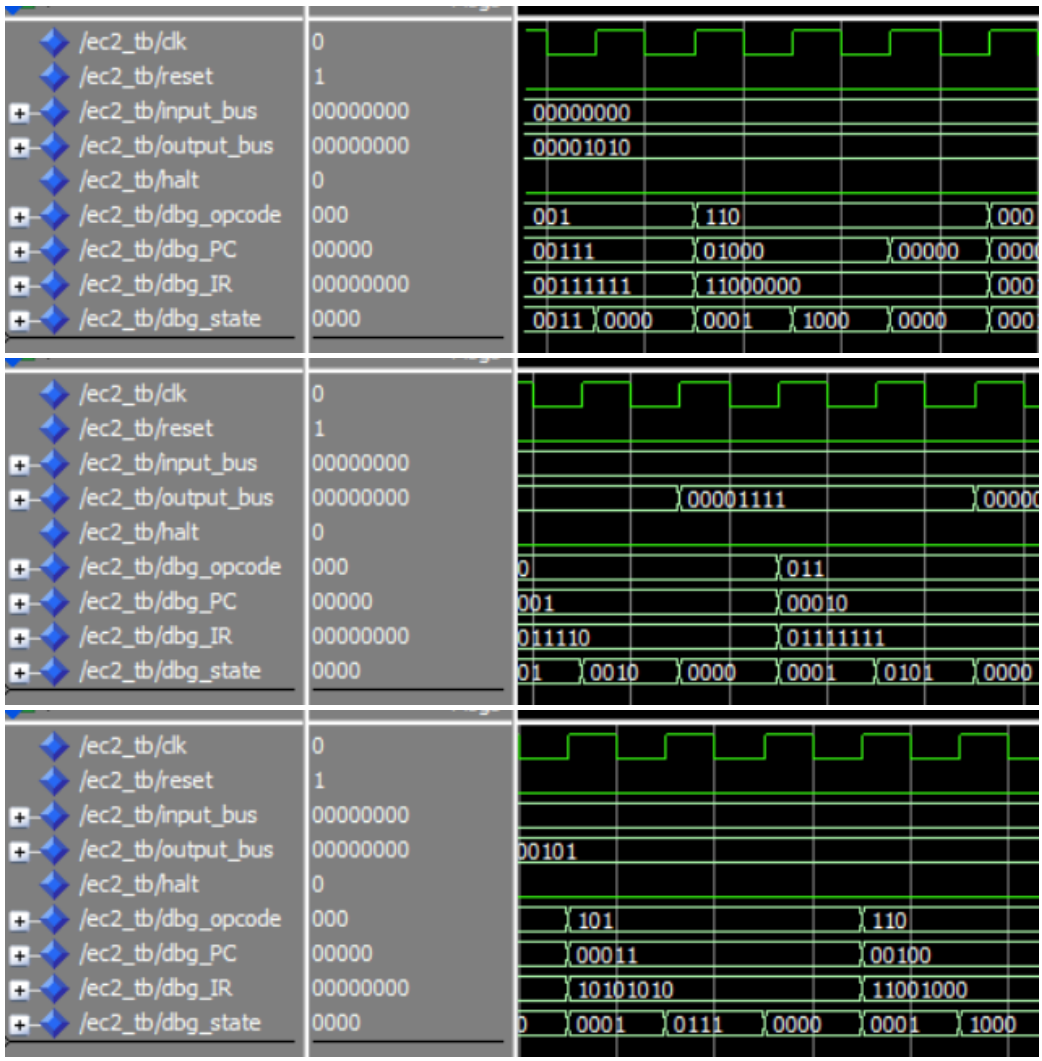
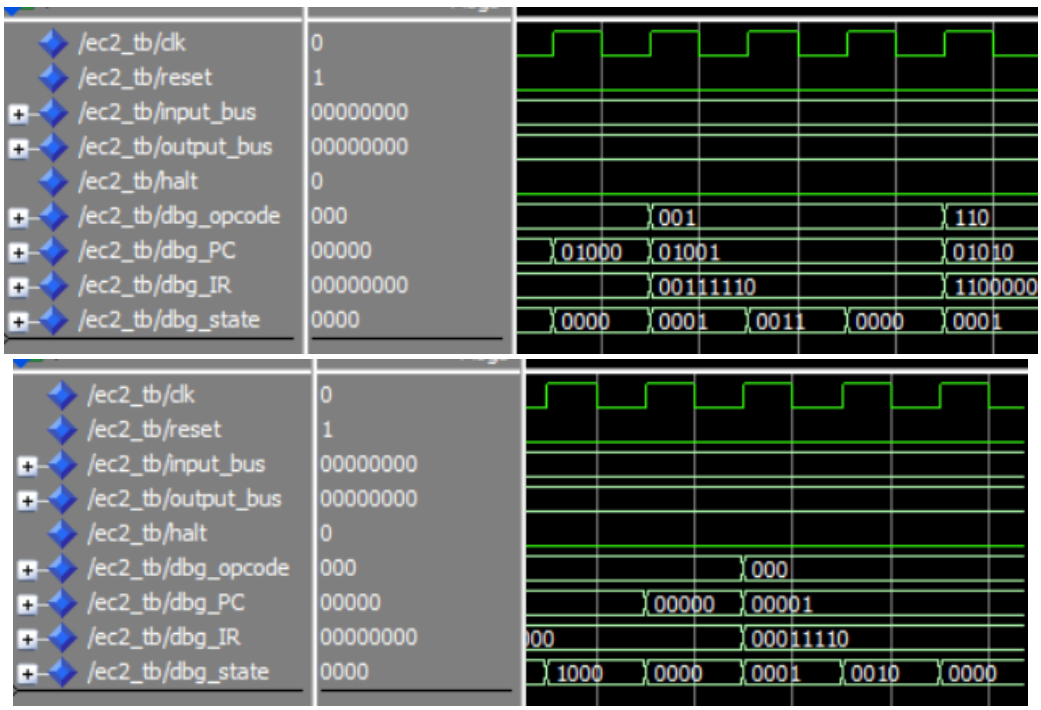



**Figure 10.9.** Simulation of the EC-2 computing the GCD of 25 and 15




(c)

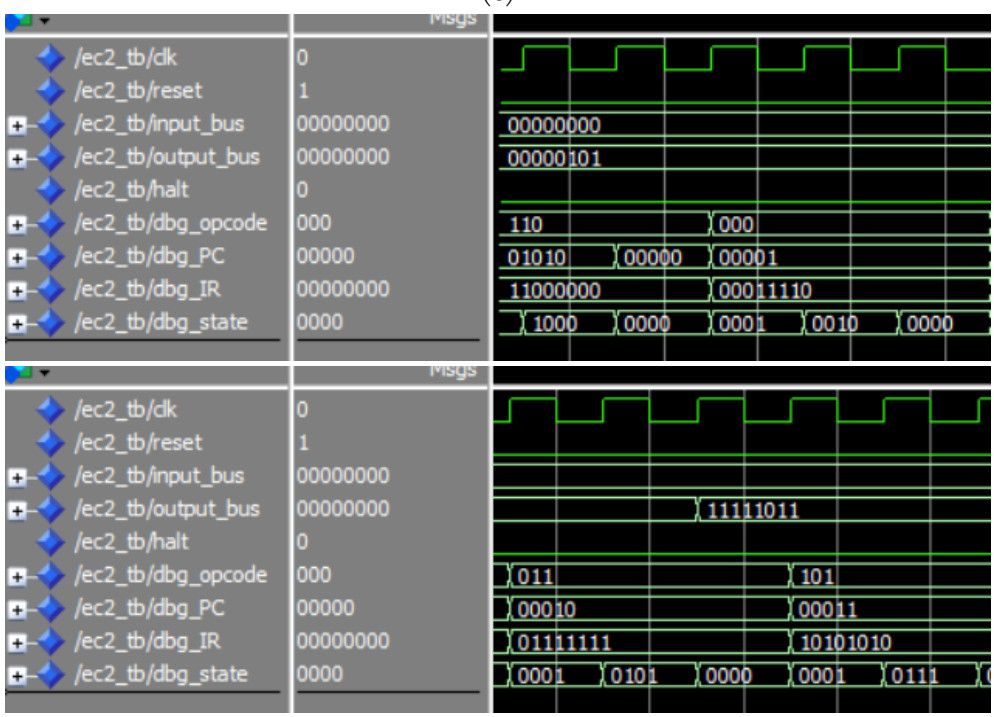



**Figure 10.9.** Simulation of the EC-2 computing the GCD of 25 and 15

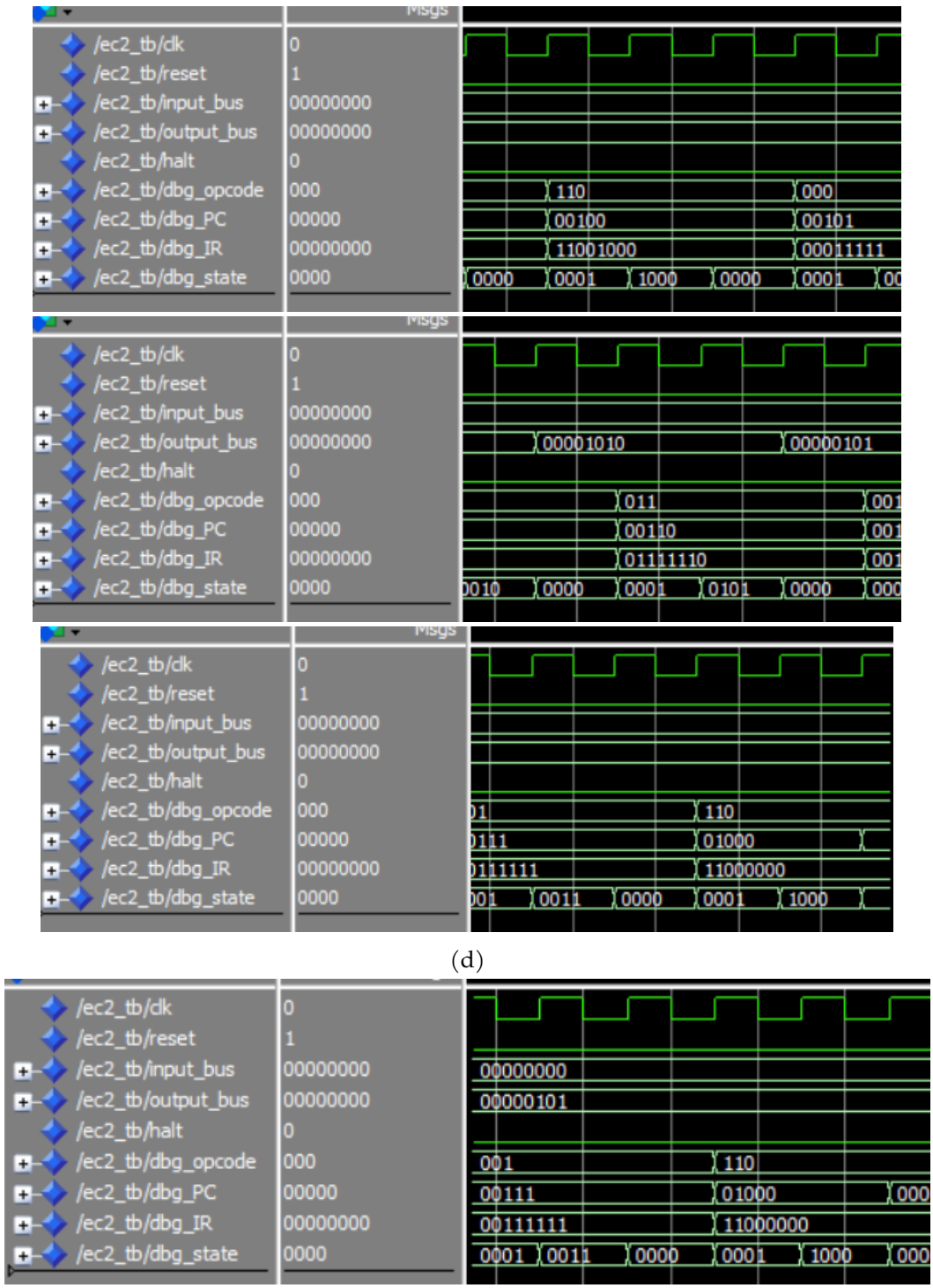



**Figure 10.9.** Simulation of the EC-2 computing the GCD of 25 and 15

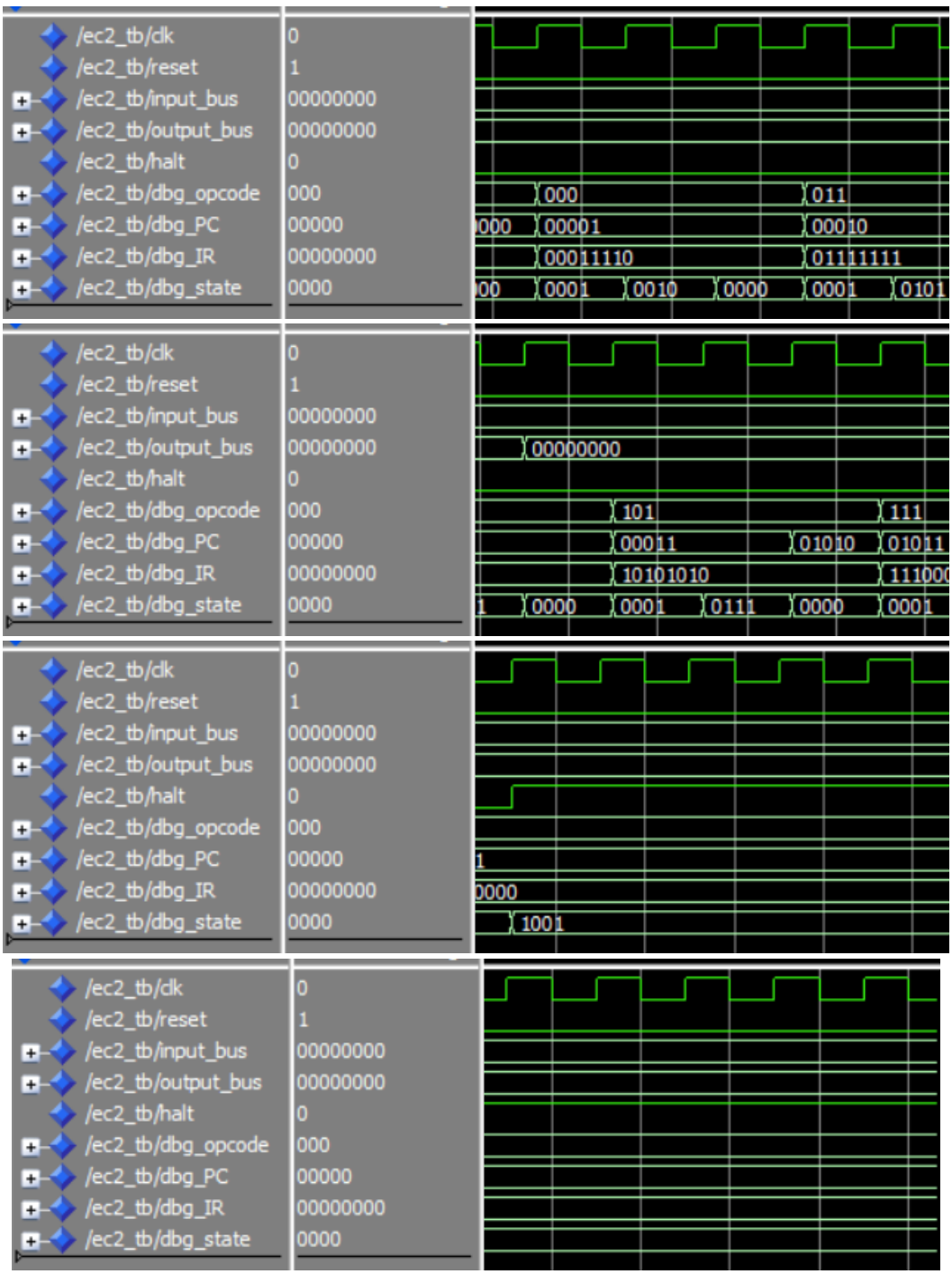



**Figure 10.9.** Simulation of the EC-2 computing the GCD of 25 and 15

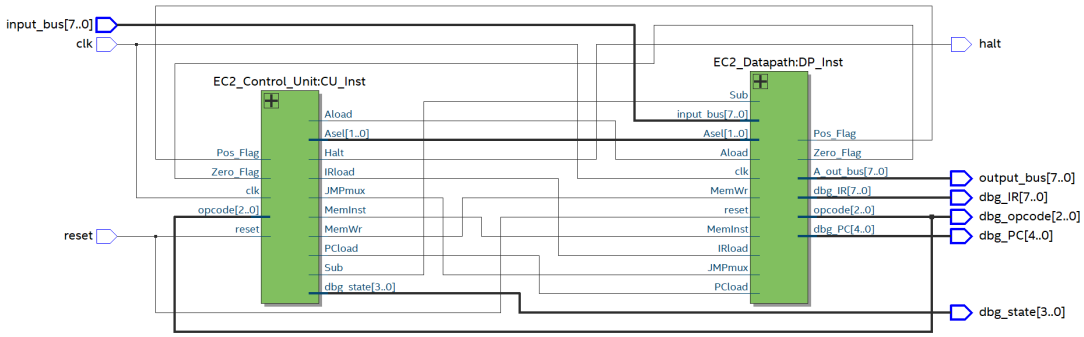



**Figure 10.10.** RTL viewer (Quartus) of the EC-2 top level (author's screenshot).

to its address field, and write it back: `STORE A,10` becomes `STORE A,11`;

4. loop to read the next byte.

**Table 10.7.** A bootloader for the EC-2 using self-modifying code; the loaded program starts at address 10.

| Address | Assembly | Comment |
|---|---|---|
| 00 | `IN A` | read a byte from storage |
| 01 | `STORE A, 10` | write to address 10 (the target) – *this instruction is modified* |
| 02 | `LOAD A, 01` | load the instruction at 01 |
| 03 | `ADD A, 06` | add 1 to the address field (constant at 06) |
| 04 | `STORE A, 01` | save the modified instruction (`STORE A,11`) |
| 05 | `JPOS 00` | loop for the next byte |
| 06 | `01` | the constant 1 |
| 10 | xx | start of the destination of the loaded program |

**Self-modifying code**

> The same property that makes a stored-program computer powerful also makes it vulnerable: a program that can change its own instructions, or data that can be executed as instructions, is a source of bugs and of security attacks. Modern processors separate executable from writable memory.

## 10.5 EC-1 versus EC-2

Table 10.8 summarises the architectural step from a small accumulator machine with a program ROM to a stored-program computer.

**Table 10.8.** Comparison of EC-1 and EC-2.

| Feature | EC-1 | EC-2 |
|---|---|---|
| Architecture | Hardwired control, accumulator machine with a program ROM | Von Neumann: unified memory for code and data |
| Memory | 16×8 ROM (program only); no data memory | 32 × 8 RAM (code and data) |
| Instruction set | 5 instructions (input, output, decrement, conditional jump, halt) | 8 instructions (load/store, add/subtract, input, conditional jumps, halt) |
| Addressing | 4-bit program address; operand is the accumulator | 5-bit direct memory addressing |
| Control unit | Fetch–decode–execute with one wait state (jump) | Multi-state memory access with wait cycles |
| Program can modify itself | no | yes (code in RAM) |

**Chapter Summary**

- A **general-purpose processor** is a dedicated processor whose task is to **fetch, decode and execute** stored instructions. Its design consists of the **instruction set**, the **datapath** and the **control unit**.
- **EC-1**: 8-bit accumulator machine, 5 instructions, 16-word ROM. The control unit has FETCH, DECODE and execution states; a **wait state** after JNZ prevents a race with the synchronous ROM.
- Programs are loaded into memory with an **Intel HEX** file; each record has a byte count, address, type, data and checksum.
- **EC-2**: 8 instructions, 32 × 8 RAM for code and data, ALU, flags, and multi-cycle memory access. The memory-address multiplexer shares one address bus between instruction fetch and data access.
- With the code in RAM the processor becomes a **von Neumann** computer: programs can be loaded by a bootloader and can modify themselves.
- The processor is an **interpreter**: the FSM translates opcodes into control signals. Flexibility costs control-unit complexity.

## Exercises

**10.1 Ramp generator (EC-1).** Write an EC-1 program that outputs a value counting down from the input $n$ to $0$ (a descending ramp, as would be sent to a digital-to-analogue converter to generate a sawtooth). Show the waveform simulation. (*Hint:* Table 10.3 outputs $n, \ldots, 1$.) Why can the EC-1 not repeat the ramp forever, and what instruction would it need?

**10.2 Odd/even detector (EC-1).** Determine whether the input number is odd or

even. The program may not test bit 0 directly. *Hint:* "ping-pong" between two decrement points: subtract 1, test, subtract 1, test; the `HALT` at which the program stops tells the parity. Write the program, encode it in hexadecimal and trace it for the inputs 5 and 6.

**10.3** **Multiplication (EC-2).** Compute the product of the numbers in $M[30]$ and $M[31]$ by repeated addition: $Sum = 0$; loop: add $A$ to $Sum$, decrement $B$; stop when $B = 0$. Write the program, using constants in memory, and trace it for $3 \times 4$. (EC-2 has no decrement instruction: use `SUB` with a memory location containing 1.)

**10.4** **Absolute difference (EC-2).** Compute $|A - B|$ for the numbers in $M[30]$, $M[31]$ and store it in $M[31]$.

**10.5** Encode in hexadecimal: (a) EC-1: `IN A`, `DEC A`, `DEC A`, `OUT A`, `HALT`; (b) EC-2: `LOAD A,5`, `ADD A,6`, `STORE A,7`, `HALT`.

**10.6** Write the Intel HEX record for the 5-byte EC-1 program `60 80 A0 C1 E0` at address 0000, compute its checksum, and give the end-of-file record.

**10.7** Trace the EC-1 program of Table 10.3 for the input 2: list for each clock cycle the state, PC, IR, $A$ and output.

**10.8** Explain what would go wrong in the EC-1 if the wait state after JNZ were removed and the ROM were a synchronous ROM, with a timing sketch of PC, the ROM address sampling and the instruction register.

**10.9** Why does the EC-2 need the *MemInst* multiplexer? What would be the consequence of using two separate memories for instructions and data (the Harvard architecture)?

**10.10** How many clock cycles does the EC-2 need to execute `LOAD A,30`, `SUB A,31` and `HALT`? (Use the state diagram: fetch, decode and execute each take one cycle.)

**10.11** Extend the EC-1 with a new instruction `INC A` (opcode 010). Which changes are needed in the datapath, in the state diagram and in the next-state and output equations?

**10.12** Add a `JMP addr` (unconditional jump) to EC-2 using the unused states. Which parts of the design change?

**10.13** Compute the contents of $M[30]$ when the EC-2 GCD program is run with $X = 48$, $Y = 18$, and the number of times that `SUB` is executed.

## Further Reading

The idea of building a processor from a datapath and an FSM, and the EC-1 and EC-2 designs on which this chapter is based, are described in Hwang [15, 4]. For modern microarchitecture, Harris and Harris [20] develop a complete MIPS processor in the same style (single-cycle, multi-cycle and pipelined). The stored-program concept was first described by von Neumann [19].

# A

# VHDL Quick Reference

This appendix summarises the VHDL used in the book [10, 9]. VHDL is *not* case sensitive; `--` starts a comment that runs to the end of the line.

## A.1 Structure of a design

```
library ieee;                           -- library clause
use ieee.std_logic_1164.all;            -- std_logic, std_logic_vector
use ieee.numeric_std.all;               -- unsigned, signed, resize, to_integer ...

entity my_circuit is                    -- the interface
  port (a, b : in  std_logic;
        sel  : in  std_logic_vector(1 downto 0);
        y    : out std_logic);
end entity my_circuit;

architecture rtl of my_circuit is       -- the implementation
  signal tmp : std_logic;               -- internal wires
begin
  tmp <= a and b;                       -- concurrent statements
  y   <= tmp;
end architecture rtl;
```

**Listing A.1.** Skeleton of a VHDL design unit.

## A.2 Data types

**Table A.1.** Common VHDL types.

| Type | Use |
|---|---|
| `std_logic` | one bit with values `'0'`, `'1'`, `'Z'` (high impedance), `'X'` (unknown), `'-'` (don't care), ... |
| `std_logic_vector(7 downto 0)` | array of bits (bus); `"1010"` is a vector literal, `'1'` a bit literal |
| `unsigned(3 downto 0)`, `signed(3 downto 0)` | vectors with arithmetic (package `numeric_std`) |
| `integer`, `boolean` | integers (for simulation and constants) and truth values |
| `type state_type is (S0, S1, S2);` | enumerated type, e.g. for FSM states |
| `time` | simulation time, e.g. `10 ns` |

## A.3 Operators

- **Logical**: `and or nand nor xor xnor not`. In an expression with mixed operators use parentheses (VHDL has no precedence between `and` and `or`).
- **Relational**: `= /= < <= > >=` (result: boolean).
- **Arithmetic** (on `unsigned`, `signed`, `integer`): `+ - * / mod rem abs`.
- **Concatenation**: `&`, e.g. `x <= a(2 downto 0) & '0';` is a shift left.
- **Assignment**: `<=` for signals, `:=` for variables and initial values.

## A.4 Concurrent statements

```
y <= (a and b) or c;                                    -- simple assignment
y <= a when sel = '0' else b;                           -- conditional assignment
with sel select                                         -- selected assignment
  y <= i0 when "00",
       i1 when "01",
       i2 when "10",
       i3 when others;
u1 : entity work.half_adder port map (x => a, y => b, s => sum, c => cy);   --
    instance
g1 : for i in 0 to 3 generate                           -- replicated hardware
       y(i) <= a(i) and b(i);
     end generate g1;
```

**Listing A.2.** Concurrent signal assignments.

## A.5 Processes

A `process` contains *sequential* statements, which execute in order when a signal in its *sensitivity list* changes. Signals assigned in a process get their new value only when the process suspends.

```vhdl
-- combinational process: list every signal that is read
process (a, b, sel)
begin
  if sel = '1' then y <= a; else y <= b; end if;       -- assign y in every branch
end process;

-- flip-flop with asynchronous reset
process (clk, reset)
begin
  if reset = '1' then q <= '0';
  elsif rising_edge(clk) then q <= d; end if;
end process;

-- case statement (all choices must be covered)
case state is
  when S0     => next_state <= S1;
  when S1 | S2 => next_state <= S0;
  when others => next_state <= S0;
end case;
```

**Listing A.3.** Processes for combinational logic and flip-flops.

**Rules of thumb.** (1) A combinational process must assign each of its outputs in every branch (give defaults at the top), otherwise a latch is inferred. (2) A clocked process uses `rising_edge(clk)` and describes flip-flops. (3) Do not assign the same signal from two processes.

## A.6 Type conversions (package `numeric_std`)

**Table A.2.** Conversion functions.

| From → to | Expression |
|---|---|
| `std_logic_vector` → `unsigned/signed` | `unsigned(v)`, `signed(v)` |
| `unsigned/signed` → `std_logic_vector` | `std_logic_vector(u)` |
| `integer` → `unsigned/signed` | `to_unsigned(n, width)`, `to_signed(n, width)` |
| `unsigned/signed` → `integer` | `to_integer(u)` |
| resize | `resize(u, new_width)` |

## A.7 Testbench constructs

```vhdl
constant T : time := 10 ns;
signal clk : std_logic := '0';
...
clk <= not clk after T/2;                              -- free-running clock (
    concurrent)

stim : process
```

```
begin
  reset <= '1'; wait for 2*T;  reset <= '0';
  a <= "0101"; b <= "0011";    wait for T;
  assert s = "1000" report "5 + 3 failed" severity error;
  report "end of simulation" severity note;
  wait;                                                -- stop this process
    forever
end process;
```

**Listing A.4.** Clock generation, stimulus and a self-checking assertion.

## A.8 Common mistakes

1. **Unintended latch**: a combinational process that does not assign an output in every branch.
2. **Missing signal in the sensitivity list** of a combinational process.
3. **Mixing `std_logic_vector` arithmetic**: use `unsigned`/`signed` and explicit conversions.
4. **Multiple drivers**: two concurrent assignments to the same `std_logic` signal produce `'X'`.
5. **Confusing `:=` and `<=`**: variables update immediately, signals at the end of the process.
6. **Comparing with the wrong width**: `count = "101"` requires a 3-bit vector.

# B

# Reference Tables

## B.1 Logic gates

**Table B.1.** Two-input gate functions.

| $x$ | $y$ | AND | OR | NAND | NOR | XOR | XNOR | $x'$ |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 0 |

**Table B.2.** Expressions and VHDL operators of the gates.

| Gate | Expression | VHDL |
|---|---|---|
| AND | $xy$ | `x and y` |
| OR | $x + y$ | `x or y` |
| NOT | $x'$ | `not x` |
| NAND | $(xy)'$ | `x nand y` |
| NOR | $(x + y)'$ | `x nor y` |
| XOR | $x \oplus y = x'y + xy'$ | `x xor y` |
| XNOR | $(x \oplus y)' = xy + x'y'$ | `x xnor y` |

## B.2 Number systems

## B.3 Seven-segment display

## B.4 ASCII printable characters

**Table B.3.** Powers of two.

| $n$ | $2^n$ | $n$ | $2^n$ | $n$ | $2^n$ |
|---|---|---|---|---|---|
| 0 | 1 | 6 | 64 | 12 | 4 096 |
| 1 | 2 | 7 | 128 | 13 | 8 192 |
| 2 | 4 | 8 | 256 | 14 | 16 384 |
| 3 | 8 | 9 | 512 | 15 | 32 768 |
| 4 | 16 | 10 | 1 024 | 16 | 65 536 |
| 5 | 32 | 11 | 2 048 | 20 | 1 048 576 |

**Table B.4.** Four-bit binary, Gray and excess-3 codes (excess-3 is used for the decimal digits 0–9 only).

| Decimal | Binary | Gray | Excess-3 |
|---|---|---|---|
| 0 | 0000 | 0000 | 0011 |
| 1 | 0001 | 0001 | 0100 |
| 2 | 0010 | 0011 | 0101 |
| 3 | 0011 | 0010 | 0110 |
| 4 | 0100 | 0110 | 0111 |
| 5 | 0101 | 0111 | 1000 |
| 6 | 0110 | 0101 | 1001 |
| 7 | 0111 | 0100 | 1010 |
| 8 | 1000 | 1100 | 1011 |
| 9 | 1001 | 1101 | 1100 |
| 10 | 1010 | 1111 | – |
| 11 | 1011 | 1110 | – |
| 12 | 1100 | 1010 | – |
| 13 | 1101 | 1011 | – |
| 14 | 1110 | 1001 | – |
| 15 | 1111 | 1000 | – |

**Table B.5.** Ranges of $n$-bit integers.

| $n$ | Unsigned | Two's complement | Hex digits |
|---|---|---|---|
| 4 | 0 to 15 | −8 to 7 | 1 |
| 8 | 0 to 255 | −128 to 127 | 2 |
| 12 | 0 to 4 095 | −2048 to 2 047 | 3 |
| 16 | 0 to 65 535 | −32 768 to 32 767 | 4 |
| 32 | 0 to 4 294 967 295 | −2 147 483 648 to 2 147 483 647 | 8 |

**Table B.6.** Segments that are *lit* for each decimal digit (1 = lit; segment letters as in Figure 2.12). For a common-anode display the control signal is the complement.

| Digit | *a* | *b* | *c* | *d* | *e* | *f* | *g* |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 3 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 4 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 6 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 7 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

**Table B.7.** The printable ASCII characters (codes 32–126) [1].

| Dec | Hex | Char | Dec | Hex | Char | Dec | Hex | Char | Dec | Hex | Char |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 32 | 20 | space | 56 | 38 | 8 | 80 | 50 | P | 104 | 68 | h |
| 33 | 21 | ! | 57 | 39 | 9 | 81 | 51 | Q | 105 | 69 | i |
| 34 | 22 | " | 58 | 3A | : | 82 | 52 | R | 106 | 6A | j |
| 35 | 23 | # | 59 | 3B | ; | 83 | 53 | S | 107 | 6B | k |
| 36 | 24 | $ | 60 | 3C | < | 84 | 54 | T | 108 | 6C | l |
| 37 | 25 | % | 61 | 3D | = | 85 | 55 | U | 109 | 6D | m |
| 38 | 26 | & | 62 | 3E | > | 86 | 56 | V | 110 | 6E | n |
| 39 | 27 | ' | 63 | 3F | ? | 87 | 57 | W | 111 | 6F | o |
| 40 | 28 | ( | 64 | 40 | @ | 88 | 58 | X | 112 | 70 | p |
| 41 | 29 | ) | 65 | 41 | A | 89 | 59 | Y | 113 | 71 | q |
| 42 | 2A | * | 66 | 42 | B | 90 | 5A | Z | 114 | 72 | r |
| 43 | 2B | + | 67 | 43 | C | 91 | 5B | [ | 115 | 73 | s |
| 44 | 2C | , | 68 | 44 | D | 92 | 5C | \ | 116 | 74 | t |
| 45 | 2D | - | 69 | 45 | E | 93 | 5D | ] | 117 | 75 | u |
| 46 | 2E | . | 70 | 46 | F | 94 | 5E | ^ | 118 | 76 | v |
| 47 | 2F | / | 71 | 47 | G | 95 | 5F | _ | 119 | 77 | w |
| 48 | 30 | 0 | 72 | 48 | H | 96 | 60 | ` | 120 | 78 | x |
| 49 | 31 | 1 | 73 | 49 | I | 97 | 61 | a | 121 | 79 | y |
| 50 | 32 | 2 | 74 | 4A | J | 98 | 62 | b | 122 | 7A | z |
| 51 | 33 | 3 | 75 | 4B | K | 99 | 63 | c | 123 | 7B | { |
| 52 | 34 | 4 | 76 | 4C | L | 100 | 64 | d | 124 | 7C | \| |
| 53 | 35 | 5 | 77 | 4D | M | 101 | 65 | e | 125 | 7D | } |
| 54 | 36 | 6 | 78 | 4E | N | 102 | 66 | f | 126 | 7E | ~ |
| 55 | 37 | 7 | 79 | 4F | O | 103 | 67 | g | | | |

# C

# Complete VHDL Listings

This appendix lists the VHDL files of the dedicated and general-purpose processors that are not printed in the chapters. The files are also supplied with the book in the directory `vhdl/`, so they can be compiled and simulated with any VHDL-2008 simulator. All listings are the author's work.

## C.1 GCD processor (Chapter 9)

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  package gcd_components is
6
7      component mux_2to1_8bit
8          Port ( SEL : in  STD_LOGIC;
9                 A   : in  STD_LOGIC_VECTOR (7 downto 0);
10                B   : in  STD_LOGIC_VECTOR (7 downto 0);
11                Y   : out STD_LOGIC_VECTOR (7 downto 0));
12     end component;
13
14     component reg_8bit
15         Port ( D     : in  STD_LOGIC_VECTOR (7 downto 0);
16                Reset : in  STD_LOGIC;
17                Clk   : in  STD_LOGIC;
18                Load  : in  STD_LOGIC;
19                Q     : out STD_LOGIC_VECTOR (7 downto 0));
20     end component;
21
22     component subtractor_8bit
23         Port ( A    : in  STD_LOGIC_VECTOR (7 downto 0);
24                B    : in  STD_LOGIC_VECTOR (7 downto 0);
25                Diff : out STD_LOGIC_VECTOR (7 downto 0));
26     end component;
27
```

```
    component comparator_8bit
        Port ( A    : in  STD_LOGIC_VECTOR (7 downto 0);
               B    : in  STD_LOGIC_VECTOR (7 downto 0);
               AeqB : out STD_LOGIC;
               AgtB : out STD_LOGIC);
    end component;

    component tristate_buffer_8bit
        Port ( Input  : in  STD_LOGIC_VECTOR (7 downto 0);
               Enable : in  STD_LOGIC;
               Output : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

end gcd_components;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- MUX 2:1 Implementation
entity mux_2to1_8bit is
    Port ( SEL : in  STD_LOGIC;
           A   : in  STD_LOGIC_VECTOR (7 downto 0);
           B   : in  STD_LOGIC_VECTOR (7 downto 0);
           Y   : out STD_LOGIC_VECTOR (7 downto 0));
end mux_2to1_8bit;

architecture Behavioral of mux_2to1_8bit is
begin
    Y <= A when SEL = '0' else B;
end Behavioral;

-- Register Implementation
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity reg_8bit is
    Port ( D     : in  STD_LOGIC_VECTOR (7 downto 0);
               Reset : in  STD_LOGIC;
               Clk   : in  STD_LOGIC;
               Load  : in  STD_LOGIC;
               Q     : out STD_LOGIC_VECTOR (7 downto 0));
end reg_8bit;

architecture Behavioral of reg_8bit is
signal storage : STD_LOGIC_VECTOR(7 downto 0);
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            if Reset = '1' then
                storage <= (others => '0');
            elsif Load = '1' then
                storage <= D;
            end if;
        end if;
    end process;
    Q <= storage;
```

```vhdl
end Behavioral;

-- Subtractor Implementation
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity subtractor_8bit is
    Port ( A    : in  STD_LOGIC_VECTOR (7 downto 0);
           B    : in  STD_LOGIC_VECTOR (7 downto 0);
           Diff : out STD_LOGIC_VECTOR (7 downto 0));
end subtractor_8bit;

architecture Behavioral of subtractor_8bit is
begin
    Diff <= std_logic_vector(unsigned(A) - unsigned(B));
end Behavioral;

-- Comparator Implementation
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity comparator_8bit is
    Port ( A    : in  STD_LOGIC_VECTOR (7 downto 0);
           B    : in  STD_LOGIC_VECTOR (7 downto 0);
           AeqB : out STD_LOGIC;
           AgtB : out STD_LOGIC);
end comparator_8bit;

architecture Behavioral of comparator_8bit is
begin
    AeqB <= '1' when unsigned(A) = unsigned(B) else '0';
    AgtB <= '1' when unsigned(A) > unsigned(B) else '0';
end Behavioral;

-- Tristate Buffer Implementation
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tristate_buffer_8bit is
    Port ( Input  : in  STD_LOGIC_VECTOR (7 downto 0);
           Enable : in  STD_LOGIC;
           Output : out STD_LOGIC_VECTOR (7 downto 0));
end tristate_buffer_8bit;

architecture Behavioral of tristate_buffer_8bit is
begin
    Output <= Input when Enable = '1' else (others => 'Z');
end Behavioral;
```

**Listing C.1.** GCD processor: library of components (registers, multiplexers, subtractor, comparator, ...).

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.gcd_components.all; -- Use the package we defined
```

```
5
6  entity gcd_datapath is
7      Port ( Clk      : in  STD_LOGIC;
8             Reset    : in  STD_LOGIC;
9             InputX   : in  STD_LOGIC_VECTOR (7 downto 0);
10            InputY   : in  STD_LOGIC_VECTOR (7 downto 0);
11
12            -- Control Signals
13            In_X     : in  STD_LOGIC; -- 1 = Load External X, 0 = Load Result
14            In_Y     : in  STD_LOGIC; -- 1 = Load External Y, 0 = Load Result
15            XLoad    : in  STD_LOGIC;
16            YLoad    : in  STD_LOGIC;
17            XY       : in  STD_LOGIC; -- 0 = X-Y, 1 = Y-X
18            Out_En   : in  STD_LOGIC;
19
20            -- Status Signals
21            XeqY     : out STD_LOGIC;
22            XgtY     : out STD_LOGIC;
23
24            -- Data Output
25            OutputData : out STD_LOGIC_VECTOR (7 downto 0));
26 end gcd_datapath;
27
28 architecture Structural of gcd_datapath is
29
30     signal X_Reg_Q, Y_Reg_Q : STD_LOGIC_VECTOR(7 downto 0);
31     signal X_Mux_Out, Y_Mux_Out : STD_LOGIC_VECTOR(7 downto 0);
32     signal Sub_A, Sub_B, Sub_Diff : STD_LOGIC_VECTOR(7 downto 0);
33
34 begin
35
36     -- Register X
37     -- Input MUX for X: Selects between Subtractor Result (0) and InputX (1)
38     Mux_In_X: mux_2to1_8bit port map (
39         SEL => In_X,
40         A   => Sub_Diff,
41         B   => InputX,
42         Y   => X_Mux_Out
43     );
44
45     Reg_X: reg_8bit port map (
46         D     => X_Mux_Out,
47         Reset => Reset,
48         Clk   => Clk,
49         Load  => XLoad,
50         Q     => X_Reg_Q
51     );
52
53     -- Register Y
54     -- Input MUX for Y: Selects between Subtractor Result (0) and InputY (1)
55     Mux_In_Y: mux_2to1_8bit port map (
56         SEL => In_Y,
57         A   => Sub_Diff,
58         B   => InputY,
59         Y   => Y_Mux_Out
60     );
61
62     Reg_Y: reg_8bit port map (
63         D     => Y_Mux_Out,
```

```
        Reset => Reset,
        Clk   => Clk,
        Load  => YLoad,
        Q     => Y_Reg_Q
    );

    -- Comparator
    Comp: comparator_8bit port map (
        A    => X_Reg_Q,
        B    => Y_Reg_Q,
        AeqB => XeqY,
        AgtB => XgtY
    );

    -- Subtractor Operand Selection (Swap Logic)
    -- If XY=0 (X-Y): SubA=X, SubB=Y
    -- If XY=1 (Y-X): SubA=Y, SubB=X

    Mux_Sub_A: mux_2to1_8bit port map (
        SEL => XY,
        A   => X_Reg_Q,
        B   => Y_Reg_Q,
        Y   => Sub_A
    );

    Mux_Sub_B: mux_2to1_8bit port map (
        SEL => XY,
        A   => Y_Reg_Q,
        B   => X_Reg_Q,
        Y   => Sub_B
    );

    -- Subtractor
    Sub: subtractor_8bit port map (
        A    => Sub_A,
        B    => Sub_B,
        Diff => Sub_Diff
    );

    -- Output Tristate Buffer (Outputs X)
    Out_Buf: tristate_buffer_8bit port map (
        Input  => X_Reg_Q,
        Enable => Out_En,
        Output => OutputData
    );

end Structural;
```

**Listing C.2.** GCD processor: datapath.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity gcd_top is
    Port ( Clk    : in  STD_LOGIC;
           Reset  : in  STD_LOGIC;
           InputX : in  STD_LOGIC_VECTOR (7 downto 0);
           InputY : in  STD_LOGIC_VECTOR (7 downto 0);
           Output : out STD_LOGIC_VECTOR (7 downto 0));
```

```
end gcd_top;

architecture Structural of gcd_top is

    component gcd_datapath
        Port ( Clk      : in  STD_LOGIC;
               Reset    : in  STD_LOGIC;
               InputX   : in  STD_LOGIC_VECTOR (7 downto 0);
               InputY   : in  STD_LOGIC_VECTOR (7 downto 0);
               In_X     : in  STD_LOGIC;
               In_Y     : in  STD_LOGIC;
               XLoad    : in  STD_LOGIC;
               YLoad    : in  STD_LOGIC;
               XY       : in  STD_LOGIC;
               Out_En   : in  STD_LOGIC;
               XeqY     : out STD_LOGIC;
               XgtY     : out STD_LOGIC;
               OutputData : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component gcd_control
        Port ( Clk      : in  STD_LOGIC;
               Reset    : in  STD_LOGIC;
               XeqY     : in  STD_LOGIC;
               XgtY     : in  STD_LOGIC;
               In_X     : out STD_LOGIC;
               In_Y     : out STD_LOGIC;
               XLoad    : out STD_LOGIC;
               YLoad    : out STD_LOGIC;
               XY       : out STD_LOGIC;
               Out_En   : out STD_LOGIC);
    end component;

    -- Internal Signals
    signal In_X, In_Y, XLoad, YLoad, XY, Out_En : STD_LOGIC;
    signal XeqY, XgtY : STD_LOGIC;

begin

    -- Instantiate Datapath
    Datapath: gcd_datapath port map (
        Clk        => Clk,
        Reset      => Reset,
        InputX     => InputX,
        InputY     => InputY,
        In_X       => In_X,
        In_Y       => In_Y,
        XLoad      => XLoad,
        YLoad      => YLoad,
        XY         => XY,
        Out_En     => Out_En,
        XeqY       => XeqY,
        XgtY       => XgtY,
        OutputData => Output
    );

    -- Instantiate Control Unit
    Control: gcd_control port map (
        Clk => Clk,
```

```
        Reset => Reset,
        XeqY => XeqY,
        XgtY => XgtY,
        In_X => In_X,
        In_Y => In_Y,
        XLoad => XLoad,
        YLoad => YLoad,
        XY => XY,
        Out_En => Out_En
    );

end Structural;
```

**Listing C.3.** GCD processor: top level connecting control unit and datapath.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gcd_tb is
-- Testbench has no ports
end gcd_tb;

architecture Behavioral of gcd_tb is

    -- Component Declaration for the Unit Under Test (UUT)
    component gcd_top
        Port ( Clk    : in  STD_LOGIC;
               Reset  : in  STD_LOGIC;
               InputX : in  STD_LOGIC_VECTOR (7 downto 0);
               InputY : in  STD_LOGIC_VECTOR (7 downto 0);
               Output : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    -- Inputs
    signal Clk    : STD_LOGIC := '0';
    signal Reset  : STD_LOGIC := '0';
    signal InputX : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal InputY : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');

    -- Outputs
    signal Output : STD_LOGIC_VECTOR(7 downto 0);

    -- Clock period definitions
    constant Clk_period : time := 10 ns;

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: gcd_top PORT MAP (
        Clk    => Clk,
        Reset  => Reset,
        InputX => InputX,
        InputY => InputY,
        Output => Output
    );

    -- Clock process definitions
    Clk_process :process
```

```
    begin
        Clk <= '0';
        wait for Clk_period/2;
        Clk <= '1';
        wait for Clk_period/2;
    end process;

    -- Stimulus process
    stim_proc: process
    begin
        -- hold reset state for 100 ns.
        Reset <= '1';

        -- Test Case 1: GCD(15, 10) = 5
        -- 15 = 00001111
        -- 10 = 00001010
        InputX <= std_logic_vector(to_unsigned(15, 8));
        InputY <= std_logic_vector(to_unsigned(10, 8));

        wait for 20 ns;
        Reset <= '0';
        wait for 20 ns;

        wait for 500 ns; -- Wait for computation to finish

        -- Expect Output = 5
        report "Test Case 1: GCD(15, 10)";

        -- Test Case 2: GCD(50, 20) = 10
        Reset <= '1';
        wait for 20 ns;
        Reset <= '0';
        InputX <= std_logic_vector(to_unsigned(50, 8));
        InputY <= std_logic_vector(to_unsigned(20, 8));

        wait for 500 ns;
        report "Test Case 2: GCD(50, 20)";

        wait;
    end process;

end Behavioral;
```

**Listing C.4.** GCD processor: testbench.

## C.2 EC-1 microprocessor (Section 10.3)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package EC1_Components is
    component d_ff
        port(clk, reset, d : in std_logic; q : out std_logic);
    end component;
    component Reg8
        port(clk, reset, load: in std_logic; D: in std_logic_vector(7 downto 0); Q: out
     std_logic_vector(7 downto 0));
```

```
    end component;
    component Reg4
        port(clk, reset, load: in std_logic; D: in std_logic_vector(3 downto 0); Q: out
     std_logic_vector(3 downto 0));
    end component;
    component Mux2_8
        port(sel: in std_logic; I0, I1: in std_logic_vector(7 downto 0); Y: out
     std_logic_vector(7 downto 0));
    end component;
    component Mux2_4
        port(sel: in std_logic; I0, I1: in std_logic_vector(3 downto 0); Y: out
     std_logic_vector(3 downto 0));
    end component;
    component Inc4
        port(A: in std_logic_vector(3 downto 0); Y: out std_logic_vector(3 downto 0));
    end component;
    component Dec8
        port(A: in std_logic_vector(7 downto 0); Y: out std_logic_vector(7 downto 0));
    end component;
    component ZeroCheck8
        port(A: in std_logic_vector(7 downto 0); is_not_zero: out std_logic);
    end component;
    component TriBuf8
        port(en: in std_logic; D: in std_logic_vector(7 downto 0); Q: out std_logic_vector(7
     downto 0));
    end component;
    component rom32x8
        port(
            address : in std_logic_vector(4 downto 0);
            clock : in std_logic;
            q : out std_logic_vector(7 downto 0)
        );
    end component;
end EC1_Components;

package body EC1_Components is
end EC1_Components;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- D Flip-Flop
entity d_ff is
    port(clk, reset, d : in std_logic; q : out std_logic);
end d_ff;
architecture Behavioral of d_ff is begin
    process(clk, reset) begin
        if reset='1' then q <= '0';
        elsif rising_edge(clk) then q <= d; end if;
    end process;
end Behavioral;

-- 8-bit Register
entity Reg8 is
    port(clk, reset, load: in std_logic; D: in std_logic_vector(7 downto 0); Q: out
     std_logic_vector(7 downto 0));
end Reg8;
architecture Behavioral of Reg8 is begin
```

```
    process(clk, reset) begin
        if reset='1' then Q <= (others=>'0');
        elsif rising_edge(clk) then if load='1' then Q <= D; end if; end if;
    end process;
end Behavioral;

-- 4-bit Register
entity Reg4 is
    port(clk, reset, load: in std_logic; D: in std_logic_vector(3 downto 0); Q: out
     std_logic_vector(3 downto 0));
end Reg4;
architecture Behavioral of Reg4 is begin
    process(clk, reset) begin
        if reset='1' then Q <= (others=>'0');
        elsif rising_edge(clk) then if load='1' then Q <= D; end if; end if;
    end process;
end Behavioral;

-- 2-to-1 Mux (8-bit)
entity Mux2_8 is
    port(sel: in std_logic; I0, I1: in std_logic_vector(7 downto 0); Y: out std_logic_vector
     (7 downto 0));
end Mux2_8;
architecture Behavioral of Mux2_8 is begin
    Y <= I1 when sel='1' else I0;
end Behavioral;

-- 2-to-1 Mux (4-bit)
entity Mux2_4 is
    port(sel: in std_logic; I0, I1: in std_logic_vector(3 downto 0); Y: out std_logic_vector
     (3 downto 0));
end Mux2_4;
architecture Behavioral of Mux2_4 is begin
    Y <= I1 when sel='1' else I0;
end Behavioral;

-- 4-bit Incrementer
entity Inc4 is
    port(A: in std_logic_vector(3 downto 0); Y: out std_logic_vector(3 downto 0));
end Inc4;
architecture Behavioral of Inc4 is begin
    Y <= A + 1;
end Behavioral;

-- 8-bit Decrementer
entity Dec8 is
    port(A: in std_logic_vector(7 downto 0); Y: out std_logic_vector(7 downto 0));
end Dec8;
architecture Behavioral of Dec8 is begin
    Y <= A - 1;
end Behavioral;

-- Zero Check
entity ZeroCheck8 is
    port(A: in std_logic_vector(7 downto 0); is_not_zero: out std_logic);
end ZeroCheck8;
architecture Behavioral of ZeroCheck8 is begin
    is_not_zero <= '1' when A /= "00000000" else '0';
end Behavioral;
```

```
120
121 -- Tri-state Buffer
122 entity TriBuf8 is
123     port(en: in std_logic; D: in std_logic_vector(7 downto 0); Q: out std_logic_vector(7
     downto 0));
124 end TriBuf8;
125 architecture Behavioral of TriBuf8 is begin
126     Q <= D when en='1' else (others=>'Z');
127 end Behavioral;
```

**Listing C.5.** EC-1: components package.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use work.EC1_Components.ALL;
4
5  entity ec1 is
6      Port ( clk : in STD_LOGIC;
7             reset : in STD_LOGIC;
8             input_bus : in STD_LOGIC_VECTOR (7 downto 0);
9             output_bus : out STD_LOGIC_VECTOR (7 downto 0);
10            halt : out STD_LOGIC;
11            dbg_opcode : out STD_LOGIC_VECTOR(2 downto 0);
12            dbg_A_neq_0 : out STD_LOGIC;
13            dbg_PC : out STD_LOGIC_VECTOR(3 downto 0);
14            dbg_IR : out STD_LOGIC_VECTOR(7 downto 0);
15            dbg_state : out STD_LOGIC_VECTOR(2 downto 0));
16 end ec1;
17
18 architecture Structural of ec1 is
19     component EC1_Control_Unit
20         Port ( clk : in STD_LOGIC;
21                reset : in STD_LOGIC;
22                opcode : in STD_LOGIC_VECTOR (2 downto 0);
23                A_neq_0 : in STD_LOGIC;
24                IRload : out STD_LOGIC;
25                PCload : out STD_LOGIC;
26                INmux : out STD_LOGIC;
27                Aload : out STD_LOGIC;
28                JNZmux : out STD_LOGIC;
29                OutE : out STD_LOGIC;
30                Halt : out STD_LOGIC;
31                dbg_state : out STD_LOGIC_VECTOR(2 downto 0));
32     end component;
33
34     component EC1_Datapath
35         Port ( clk, reset : in STD_LOGIC;
36                input_bus : in STD_LOGIC_VECTOR(7 downto 0);
37                IRload, PCload, INmux, Aload, JNZmux, OutE : in STD_LOGIC;
38                opcode : out STD_LOGIC_VECTOR(2 downto 0);
39                A_out_bus : out STD_LOGIC_VECTOR(7 downto 0);
40                A_neq_0 : out STD_LOGIC;
41                dbg_PC : out STD_LOGIC_VECTOR(3 downto 0);
42                dbg_IR : out STD_LOGIC_VECTOR(7 downto 0));
43     end component;
44
45     signal opcode : STD_LOGIC_VECTOR(2 downto 0);
46     signal A_neq_0 : STD_LOGIC;
47     signal IRload, PCload, INmux, Aload, JNZmux, OutE : STD_LOGIC;
```

```
48
49  begin
50
51      CU_Inst: EC1_Control_Unit port map(
52          clk => clk,
53          reset => reset,
54          opcode => opcode,
55          A_neq_0 => A_neq_0,
56          IRload => IRload,
57          PCload => PCload,
58          INmux => INmux,
59          Aload => Aload,
60          JNZmux => JNZmux,
61          OutE => OutE,
62          Halt => halt,
63          dbg_state => dbg_state
64      );
65
66      DP_Inst: EC1_Datapath port map(
67          clk => clk,
68          reset => reset,
69          input_bus => input_bus,
70          IRload => IRload,
71          PCload => PCload,
72          INmux => INmux,
73          Aload => Aload,
74          JNZmux => JNZmux,
75          OutE => OutE,
76          opcode => opcode,
77          A_out_bus => output_bus,
78          A_neq_0 => A_neq_0,
79          dbg_PC => dbg_PC,
80          dbg_IR => dbg_IR
81      );
82
83      dbg_opcode <= opcode;
84      dbg_A_neq_0 <= A_neq_0;
85
86  end Structural;
```

**Listing C.6.** EC-1: top level.

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3   use work.EC1_Components.ALL;
4
5   entity ec1_tb is
6   end ec1_tb;
7
8   architecture Behavioral of ec1_tb is
9       component ec1
10          Port ( clk : in STD_LOGIC;
11                 reset : in STD_LOGIC;
12                 input_bus : in STD_LOGIC_VECTOR (7 downto 0);
13                 output_bus : out STD_LOGIC_VECTOR (7 downto 0);
14                 halt : out STD_LOGIC;
15                 dbg_opcode : out STD_LOGIC_VECTOR(2 downto 0);
16                 dbg_A_neq_0 : out STD_LOGIC;
17                 dbg_PC : out STD_LOGIC_VECTOR(3 downto 0);
```

```
18                dbg_IR : out STD_LOGIC_VECTOR(7 downto 0);
19                dbg_state : out STD_LOGIC_VECTOR(2 downto 0));
20    end component;
21
22    signal clk : STD_LOGIC := '0';
23    signal reset : STD_LOGIC := '0';
24    signal input_bus : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
25    signal output_bus : STD_LOGIC_VECTOR(7 downto 0);
26    signal halt : STD_LOGIC;
27    signal dbg_opcode : STD_LOGIC_VECTOR(2 downto 0);
28    signal dbg_A_neq_0 : STD_LOGIC;
29    signal dbg_PC : STD_LOGIC_VECTOR(3 downto 0);
30    signal dbg_IR : STD_LOGIC_VECTOR(7 downto 0);
31    signal dbg_state : STD_LOGIC_VECTOR(2 downto 0);
32
33    constant clk_period : time := 10 ns;
34
35 begin
36
37    uut: ec1 port map (
38        clk => clk,
39        reset => reset,
40        input_bus => input_bus,
41        output_bus => output_bus,
42        halt => halt,
43        dbg_opcode => dbg_opcode,
44        dbg_A_neq_0 => dbg_A_neq_0,
45        dbg_PC => dbg_PC,
46        dbg_IR => dbg_IR,
47        dbg_state => dbg_state
48    );
49
50    clk_process :process
51    begin
52        clk <= '0';
53        wait for clk_period/2;
54        clk <= '1';
55        wait for clk_period/2;
56    end process;
57
58    stim_proc: process
59    begin
60        -- Hold reset for 20 ns
61        reset <= '1';
62        wait for 20 ns;
63        reset <= '0';
64
65        -- Input data: 10 (0x0A)
66        input_bus <= "00001010";
67
68        -- Wait until halt
69        wait until halt = '1';
70
71        wait for 20 ns;
72        assert false report "Simulation Finished" severity failure;
73    end process;
74
75 end Behavioral;
```

**Listing C.7.** EC-1: testbench (input value 10).

## C.3 EC-2 microprocessor (Section 10.4)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.EC2_Components.ALL;

entity EC2_Control_Unit is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           opcode : in STD_LOGIC_VECTOR (2 downto 0);
           Zero_Flag : in STD_LOGIC;
           Pos_Flag : in STD_LOGIC;
           IRload : out STD_LOGIC;
           PCload : out STD_LOGIC;
           MemInst : out STD_LOGIC;
           MemWr : out STD_LOGIC;
           Asel : out STD_LOGIC_VECTOR(1 downto 0);
           Aload : out STD_LOGIC;
           Sub : out STD_LOGIC;
           JMPmux : out STD_LOGIC;
           Halt : out STD_LOGIC;
           dbg_state : out STD_LOGIC_VECTOR(3 downto 0));
end EC2_Control_Unit;

architecture Structural of EC2_Control_Unit is
    signal Q, D : std_logic_vector(3 downto 0);
    signal Q3_n, Q2_n, Q1_n, Q0_n : std_logic;

    -- States
    signal state_Fetch, state_Decode : std_logic;
    signal state_Load, state_Store, state_Add, state_Sub : std_logic;
    signal state_In, state_Jz, state_Jpos, state_Halt : std_logic;

begin
    -- State Registers
    FF3: d_ff port map(clk, reset, D(3), Q(3));
    FF2: d_ff port map(clk, reset, D(2), Q(2));
    FF1: d_ff port map(clk, reset, D(1), Q(1));
    FF0: d_ff port map(clk, reset, D(0), Q(0));

    Q3_n <= not Q(3); Q2_n <= not Q(2); Q1_n <= not Q(1); Q0_n <= not Q(0);

    -- State Decoders (Binary Encoding)
    -- 0000 Fetch
    -- 0001 Decode
    -- 0010 Load
    -- 0011 Store
    -- 0100 Add
    -- 0101 Sub
    -- 0110 In
    -- 0111 Jz
    -- 1000 Jpos
    -- 1001 Halt

```

```
    state_Fetch  <= Q3_n and Q2_n and Q1_n and Q0_n; -- 0000
    state_Decode <= Q3_n and Q2_n and Q1_n and Q(0); -- 0001
    state_Load   <= Q3_n and Q2_n and Q(1) and Q0_n; -- 0010
    state_Store  <= Q3_n and Q2_n and Q(1) and Q(0); -- 0011
    state_Add    <= Q3_n and Q(2) and Q1_n and Q0_n; -- 0100
    state_Sub    <= Q3_n and Q(2) and Q1_n and Q(0); -- 0101
    state_In     <= Q3_n and Q(2) and Q(1) and Q0_n; -- 0110
    state_Jz     <= Q3_n and Q(2) and Q(1) and Q(0); -- 0111
    state_Jpos   <= Q(3) and Q2_n and Q1_n and Q0_n; -- 1000
    state_Halt   <= Q(3) and Q2_n and Q1_n and Q(0); -- 1001

    -- Next State Logic
    -- From Fetch (0000) -> Decode (0001)

    -- From Decode (0001) -> Specific State based on Opcode
    -- Op 000 (Load) -> 0010
    -- Op 001 (Store) -> 0011
    -- Op 010 (Add) -> 0100
    -- Op 011 (Sub) -> 0101
    -- Op 100 (In) -> 0110
    -- Op 101 (Jz) -> 0111
    -- Op 110 (Jpos) -> 1000
    -- Op 111 (Halt) -> 1001

    -- D3: 1 if (Decode & Op=JPOS) OR (Decode & Op=HALT) OR Halt (Loop)
    -- Op codes: JPOS(110), HALT(111). Common: Op(2) & Op(1)
    D(3) <= (state_Decode and opcode(2) and opcode(1)) or state_Halt;

    -- D2: 1 if (Decode & Op=Add/Sub/In/Jz) OR (Exec states returning to Fetch?? No, return is 0000)
    -- Wait, Exec states just go to 0000. So ONLY Decode sets D to high values.
    -- Ops: 010, 011, 100, 101. Common: Op(2)^Op(1)?
    -- Let's just sum the cases for D(2) logic from Decode
    -- Load(0010), Store(0011) -> D2=0
    -- Add(0100), Sub(0101), In(0110), Jz(0111) -> D2=1
    D(2) <= state_Decode and ( (not opcode(2) and opcode(1)) or (opcode(2) and not opcode(1)) );

    -- D1: 1 if (Decode & Op=Load/Store/In/Jz)
    -- Ops: 000, 001 (No), 010, 011 (No) -> Wait.
    -- Load(0010) - D1=1
    -- Store(0011) - D1=1
    -- Add(0100) - D1=0
    -- Sub(0101) - D1=0
    -- In(0110) - D1=1
    -- Jz(0111) - D1=1
    -- Jpos(1000) - D1=0
    -- D(1) <= state_Decode and ( (not opcode(2) and not opcode(1)) or (opcode(2) and not opcode(1) and opcode(0)) or (opcode(2) and opcode(1) and not opcode(0) and opcode(0)) );
    -- Simplification:
    -- D1 is high for 0010, 0011, 0110, 0111.
    -- Binary x010, x011, x110, x111 -> D1=1
    -- No, State is D3 D2 D1 D0.
    -- 0010 Load -> Op 000. Op(2)=0, Op(1)=0.
    -- 0011 Store -> Op 001. Op(2)=0, Op(1)=0.
    -- 0110 In -> Op 100. Op(2)=1, Op(1)=0.
    -- 0111 Jz -> Op 101. Op(2)=1, Op(1)=0.
    -- So logic: Decode and (Op(1)=0) ? No, store is 001.
    -- Let's stick to basics:
```

```
    -- D1 targets states: 0010, 0011, 0110, 0111.
    -- Corresponds to Opcodes: 000, 001, 100, 101.
    -- Common feature: Op(1)=0.
    D(1) <= state_Decode and (not opcode(1));

    -- D0: 1 if Fetch (to go to Decode 0001) OR (Decode & Op=Store/Sub/Jz/Halt)
    -- States with D0=1: Decode(0001), Store(0011), Sub(0101), Jz(0111), Halt(1001)
    -- Opcodes: 001, 011, 101, 111.
    -- Common feature: Op(0)=1.
    D(0) <= state_Fetch or (state_Decode and opcode(0)) or state_Halt;


    -- Output Logic

    IRload <= state_Fetch;

    -- PCload: Fetch OR (Jz AND Zero) OR (Jpos AND Pos)
    PCload <= state_Fetch or (state_Jz and Zero_Flag) or (state_Jpos and Pos_Flag);

    -- JMPmux: 1 for Jz/Jpos, 0 for Fetch (Inc)
    JMPmux <= state_Jz or state_Jpos;

    -- MemInst: 1 for Load/Store/Add/Sub
    MemInst <= state_Load or state_Store or state_Add or state_Sub;

    -- MemWr: 1 for Store
    MemWr <= state_Store;

    -- Asel: 00=Mem(Load), 01=ALU(Add/Sub), 10=In, 11=X
    -- Bit 1: In(0110)? No.
    -- Asel(1) = State_In
    -- Asel(0) = State_Add or State_Sub
    Asel(1) <= state_In;
    Asel(0) <= state_Add or state_Sub;

    -- Aload: Load, Add, Sub, In
    Aload <= state_Load or state_Add or state_Sub or state_In;

    -- Sub: 1 for Sub
    Sub <= state_Sub;

    -- Halt
    Halt <= state_Halt;

    dbg_state <= Q;

end Structural;
```

**Listing C.8.** EC-2: control unit.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.EC2_Components.ALL;

entity EC2_Datapath is
    Port ( clk, reset : in STD_LOGIC;
           input_bus : in STD_LOGIC_VECTOR(7 downto 0);
           IRload, PCload, MemInst, MemWr : in STD_LOGIC;
           Asel : in STD_LOGIC_VECTOR(1 downto 0);
```

```
            Aload, Sub, JMPmux : in STD_LOGIC;
            opcode : out STD_LOGIC_VECTOR(2 downto 0);
            Zero_Flag : out STD_LOGIC;
            Pos_Flag : out STD_LOGIC;
            A_out_bus : out STD_LOGIC_VECTOR(7 downto 0);
            dbg_PC : out STD_LOGIC_VECTOR(4 downto 0);
            dbg_IR : out STD_LOGIC_VECTOR(7 downto 0));
end EC2_Datapath;

architecture Structural of EC2_Datapath is
    signal IR_out, A_out, A_in : std_logic_vector(7 downto 0);
    signal RAM_out, ALU_out : std_logic_vector(7 downto 0);
    signal PC_out, PC_in, PC_inc : std_logic_vector(4 downto 0);
    signal Mem_Addr : std_logic_vector(4 downto 0);

begin
    -- Program Counter Logic
    PC_Inc_Unit: Inc5 port map(PC_out, PC_inc);
    PC_Mux: Mux2_5 port map(JMPmux, PC_inc, IR_out(4 downto 0), PC_in);
    PC_Reg: Reg5 port map(clk, reset, PCload, PC_in, PC_out);

    -- Memory Address Mux (PC vs IR address)
    Mem_Addr_Mux: Mux2_5 port map(MemInst, PC_out, IR_out(4 downto 0), Mem_Addr);

    -- RAM
    RAM_Unit: ram32x8 port map(
        clk => clk,
        we => MemWr,
        addr => Mem_Addr,
        din => A_out,
        dout => RAM_out
    );

    -- Instruction Register
    -- Loads from RAM output (which is Instruction during Fetch)
    IR_Reg: Reg8 port map(clk, reset, IRload, RAM_out, IR_out);
    opcode <= IR_out(7 downto 5);

    -- ALU
    ALU_Unit: Alu8 port map(A_out, RAM_out, Sub, ALU_out);

    -- Accumulator Logic
    -- Mux4_8: 00=RAM, 01=ALU, 10=Input, 11=X
    Acc_Mux: Mux4_8 port map(Asel, RAM_out, ALU_out, input_bus, "00000000", A_in);
    Acc_Reg: Reg8 port map(clk, reset, Aload, A_in, A_out);

    -- Flags
    ZC: ZeroCheck8 port map(A_out, Zero_Flag);
    PC_Check: PosCheck8 port map(A_out, Pos_Flag);

    -- Outputs
    A_out_bus <= A_out;
    dbg_PC <= PC_out;
    dbg_IR <= IR_out;

end Structural;
```

**Listing C.9.** EC-2: datapath.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package EC2_Components is
    component d_ff
        port(clk, reset, d : in std_logic; q : out std_logic);
    end component;
    component Reg8
        port(clk, reset, load: in std_logic; D: in std_logic_vector(7 downto 0); Q: out
     std_logic_vector(7 downto 0));
    end component;
    component Reg5
        port(clk, reset, load: in std_logic; D: in std_logic_vector(4 downto 0); Q: out
     std_logic_vector(4 downto 0));
    end component;
    component Mux2_5
        port(sel: in std_logic; I0, I1: in std_logic_vector(4 downto 0); Y: out
     std_logic_vector(4 downto 0));
    end component;
    component Mux2_8
        port(sel: in std_logic; I0, I1: in std_logic_vector(7 downto 0); Y: out
     std_logic_vector(7 downto 0));
    end component;
    component Mux4_8
        port(sel: in std_logic_vector(1 downto 0); I0, I1, I2, I3: in std_logic_vector(7
     downto 0); Y: out std_logic_vector(7 downto 0));
    end component;
    component Inc5
        port(A: in std_logic_vector(4 downto 0); Y: out std_logic_vector(4 downto 0));
    end component;
    component Alu8
        port(A, B: in std_logic_vector(7 downto 0); Sub: in std_logic; Y: out
     std_logic_vector(7 downto 0));
    end component;
    component ZeroCheck8
        port(A: in std_logic_vector(7 downto 0); is_zero: out std_logic);
    end component;
    component PosCheck8
        port(A: in std_logic_vector(7 downto 0); is_pos: out std_logic);
    end component;
    component ram32x8
        port(
            clk : in std_logic;
            we  : in std_logic;
            addr: in std_logic_vector(4 downto 0);
            din : in std_logic_vector(7 downto 0);
            dout: out std_logic_vector(7 downto 0)
        );
    end component;
end EC2_Components;

package body EC2_Components is
end EC2_Components;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```
-- D Flip-Flop
entity d_ff is
    port(clk, reset, d : in std_logic; q : out std_logic);
end d_ff;
architecture Behavioral of d_ff is begin
    process(clk, reset) begin
        if reset='1' then q <= '0';
        elsif rising_edge(clk) then q <= d; end if;
    end process;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- 8-bit Register
entity Reg8 is
    port(clk, reset, load: in std_logic; D: in std_logic_vector(7 downto 0); Q: out
     std_logic_vector(7 downto 0));
end Reg8;
architecture Behavioral of Reg8 is begin
    process(clk, reset) begin
        if reset='1' then Q <= (others=>'0');
        elsif rising_edge(clk) then if load='1' then Q <= D; end if; end if;
    end process;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- 5-bit Register
entity Reg5 is
    port(clk, reset, load: in std_logic; D: in std_logic_vector(4 downto 0); Q: out
     std_logic_vector(4 downto 0));
end Reg5;
architecture Behavioral of Reg5 is begin
    process(clk, reset) begin
        if reset='1' then Q <= (others=>'0');
        elsif rising_edge(clk) then if load='1' then Q <= D; end if; end if;
    end process;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- 2-to-1 Mux (5-bit)
entity Mux2_5 is
    port(sel: in std_logic; I0, I1: in std_logic_vector(4 downto 0); Y: out std_logic_vector
     (4 downto 0));
end Mux2_5;
architecture Behavioral of Mux2_5 is begin
    Y <= I1 when sel='1' else I0;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
109
110 -- 2-to-1 Mux (8-bit)
111 entity Mux2_8 is
112     port(sel: in std_logic; I0, I1: in std_logic_vector(7 downto 0); Y: out std_logic_vector
     (7 downto 0));
113 end Mux2_8;
114 architecture Behavioral of Mux2_8 is begin
115     Y <= I1 when sel='1' else I0;
116 end Behavioral;
117
118 library IEEE;
119 use IEEE.STD_LOGIC_1164.ALL;
120 use IEEE.STD_LOGIC_UNSIGNED.ALL;
121
122 -- 4-to-1 Mux (8-bit)
123 entity Mux4_8 is
124     port(sel: in std_logic_vector(1 downto 0); I0, I1, I2, I3: in std_logic_vector(7 downto
     0); Y: out std_logic_vector(7 downto 0));
125 end Mux4_8;
126 architecture Behavioral of Mux4_8 is begin
127     with sel select
128         Y <= I0 when "00",
129              I1 when "01",
130              I2 when "10",
131              I3 when "11",
132              (others => 'X') when others;
133 end Behavioral;
134
135 library IEEE;
136 use IEEE.STD_LOGIC_1164.ALL;
137 use IEEE.STD_LOGIC_UNSIGNED.ALL;
138
139 -- 5-bit Incrementer
140 entity Inc5 is
141     port(A: in std_logic_vector(4 downto 0); Y: out std_logic_vector(4 downto 0));
142 end Inc5;
143 architecture Behavioral of Inc5 is begin
144     Y <= A + 1;
145 end Behavioral;
146
147 library IEEE;
148 use IEEE.STD_LOGIC_1164.ALL;
149 use IEEE.STD_LOGIC_UNSIGNED.ALL;
150 -- ALU 8-bit (Add/Sub)
151 entity Alu8 is
152     port(A, B: in std_logic_vector(7 downto 0); Sub: in std_logic; Y: out std_logic_vector(7
     downto 0));
153 end Alu8;
154 architecture Behavioral of Alu8 is begin
155     process(A, B, Sub)
156     begin
157         if Sub = '1' then
158             Y <= A - B;
159         else
160             Y <= A + B;
161         end if;
162     end process;
163 end Behavioral;
164
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- Zero Check
entity ZeroCheck8 is
    port(A: in std_logic_vector(7 downto 0); is_zero: out std_logic);
end ZeroCheck8;
architecture Behavioral of ZeroCheck8 is begin
    is_zero <= '1' when A = "00000000" else '0';
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- Positive Check (MSB = 0)
entity PosCheck8 is
    port(A: in std_logic_vector(7 downto 0); is_pos: out std_logic);
end PosCheck8;
architecture Behavioral of PosCheck8 is begin
    is_pos <= not A(7); -- Positive if MSB is 0 (includes 0)
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- RAM 32x8
entity ram32x8 is
    port(
        clk : in std_logic;
        we  : in std_logic;
        addr: in std_logic_vector(4 downto 0);
        din : in std_logic_vector(7 downto 0);
        dout: out std_logic_vector(7 downto 0)
    );
end ram32x8;

architecture Behavioral of ram32x8 is
    type ram_type is array (0 to 31) of std_logic_vector(7 downto 0);
    signal RAM : ram_type := (
        0 => x"1E", 1 => x"7F", 2 => x"AA", 3 => x"C8", -- Input inputs at 30,31.
        4 => x"1F", 5 => x"7E", 6 => x"3F", 7 => x"C0",
        8 => x"3E", 9 => x"C0", 10 => x"E0",
        30 => x"0F", 31 => x"19", -- Inputs: 15 and 25
        others => x"00"
    );
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if we = '1' then
                RAM(conv_integer(addr)) <= din;
            end if;
        end if;
    end process;
    dout <= RAM(conv_integer(addr));
end Behavioral;
```

**Listing C.10.** EC-2: components package, including the RAM whose initial contents are the GCD program of Table 10.6.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.EC2_Components.ALL;

entity ec2 is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           input_bus : in STD_LOGIC_VECTOR (7 downto 0);
           output_bus : out STD_LOGIC_VECTOR (7 downto 0);
           halt : out STD_LOGIC;
           dbg_opcode : out STD_LOGIC_VECTOR(2 downto 0);
           dbg_PC : out STD_LOGIC_VECTOR(4 downto 0);
           dbg_IR : out STD_LOGIC_VECTOR(7 downto 0);
           dbg_state : out STD_LOGIC_VECTOR(3 downto 0));
end ec2;

architecture Structural of ec2 is
    component EC2_Control_Unit
        Port ( clk : in STD_LOGIC;
               reset : in STD_LOGIC;
               opcode : in STD_LOGIC_VECTOR (2 downto 0);
               Zero_Flag : in STD_LOGIC;
               Pos_Flag : in STD_LOGIC;
               IRload : out STD_LOGIC;
               PCload : out STD_LOGIC;
               MemInst : out STD_LOGIC;
               MemWr : out STD_LOGIC;
               Asel : out STD_LOGIC_VECTOR(1 downto 0);
               Aload : out STD_LOGIC;
               Sub : out STD_LOGIC;
               JMPmux : out STD_LOGIC;
               Halt : out STD_LOGIC;
               dbg_state : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    component EC2_Datapath
        Port ( clk, reset : in STD_LOGIC;
               input_bus : in STD_LOGIC_VECTOR(7 downto 0);
               IRload, PCload, MemInst, MemWr : in STD_LOGIC;
               Asel : in STD_LOGIC_VECTOR(1 downto 0);
               Aload, Sub, JMPmux : in STD_LOGIC;
               opcode : out STD_LOGIC_VECTOR(2 downto 0);
               Zero_Flag : out STD_LOGIC;
               Pos_Flag : out STD_LOGIC;
               A_out_bus : out STD_LOGIC_VECTOR(7 downto 0);
               dbg_PC : out STD_LOGIC_VECTOR(4 downto 0);
               dbg_IR : out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    signal opcode : STD_LOGIC_VECTOR(2 downto 0);
    signal Zero_Flag, Pos_Flag : STD_LOGIC;
    signal IRload, PCload, MemInst, MemWr, Aload, Sub, JMPmux, Halt_Sig : STD_LOGIC;
    signal Asel : STD_LOGIC_VECTOR(1 downto 0);
    signal dbg_state_sig : STD_LOGIC_VECTOR(3 downto 0);

begin

    CU_Inst: EC2_Control_Unit port map(
```

```
        clk => clk,
        reset => reset,
        opcode => opcode,
        Zero_Flag => Zero_Flag,
        Pos_Flag => Pos_Flag,
        IRload => IRload,
        PCload => PCload,
        MemInst => MemInst,
        MemWr => MemWr,
        Asel => Asel,
        Aload => Aload,
        Sub => Sub,
        JMPmux => JMPmux,
        Halt => Halt_Sig,
        dbg_state => dbg_state_sig
    );

    DP_Inst: EC2_Datapath port map(
        clk => clk,
        reset => reset,
        input_bus => input_bus,
        IRload => IRload,
        PCload => PCload,
        MemInst => MemInst,
        MemWr => MemWr,
        Asel => Asel,
        Aload => Aload,
        Sub => Sub,
        JMPmux => JMPmux,
        opcode => opcode,
        Zero_Flag => Zero_Flag,
        Pos_Flag => Pos_Flag,
        A_out_bus => output_bus,
        dbg_PC => dbg_PC,
        dbg_IR => dbg_IR
    );

    dbg_opcode <= opcode;
    dbg_state <= dbg_state_sig;
    halt <= Halt_Sig;

end Structural;
```

**Listing C.11.** EC-2: top level.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ec2_tb is
end ec2_tb;

architecture Behavioral of ec2_tb is
    component ec2
        Port ( clk, reset : in STD_LOGIC;
               input_bus : in STD_LOGIC_VECTOR (7 downto 0);
               output_bus : out STD_LOGIC_VECTOR (7 downto 0);
               halt : out STD_LOGIC;
               dbg_opcode : out STD_LOGIC_VECTOR(2 downto 0);
               dbg_PC : out STD_LOGIC_VECTOR(4 downto 0);
```

```
15                dbg_IR : out STD_LOGIC_VECTOR(7 downto 0);
16                dbg_state : out STD_LOGIC_VECTOR(3 downto 0));
17    end component;
18
19    signal clk : STD_LOGIC := '0';
20    signal reset : STD_LOGIC := '0';
21    signal input_bus : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
22    signal output_bus : STD_LOGIC_VECTOR(7 downto 0);
23    signal halt : STD_LOGIC;
24    signal dbg_opcode : STD_LOGIC_VECTOR(2 downto 0);
25    signal dbg_PC : STD_LOGIC_VECTOR(4 downto 0);
26    signal dbg_IR : STD_LOGIC_VECTOR(7 downto 0);
27    signal dbg_state : STD_LOGIC_VECTOR(3 downto 0);
28
29    constant clk_period : time := 10 ns;
30
31 begin
32
33    uut: ec2 port map (
34        clk => clk,
35        reset => reset,
36        input_bus => input_bus,
37        output_bus => output_bus,
38        halt => halt,
39        dbg_opcode => dbg_opcode,
40        dbg_PC => dbg_PC,
41        dbg_IR => dbg_IR,
42        dbg_state => dbg_state
43    );
44
45    clk_process :process
46    begin
47        clk <= '0';
48        wait for clk_period/2;
49        clk <= '1';
50        wait for clk_period/2;
51    end process;
52
53    stim_proc: process
54    begin
55        reset <= '1';
56        wait for 20 ns;
57        reset <= '0';
58
59        -- Inputs are pre-loaded in RAM (30=15, 31=25)
60        -- Program will run until Halt
61
62        wait until halt = '1';
63
64        -- GCD of 15 and 25 is 5.
65        -- Result should be in Accumulator (output_bus)
66        -- Check output_bus (assuming it reflects A)
67
68        wait for 20 ns;
69        assert output_bus = "00000101" report "GCD Test Failed" severity warning;
70
71        wait;
72    end process;
73
```

```
74 end Behavioral;
```

**Listing C.12.** EC-2: testbench.

# D

# Answers to Selected Exercises

Answers are given for most numerical and design exercises. For design exercises one correct solution is given; other correct solutions may exist. Verify your own answers by simulation before comparing.

## Chapter 1

**1.1** (a) analog; (b) digital; (c) digital (two states); (d) analog.

**1.2** (a) 11; (b) 50; (c) 129.

**1.3** (a) 100101; (b) 1100100; (c) 11111111.

**1.4** (a) 5 bits; (b) 4 bits; (c) 20 bits ($2^{20} = 1\,048\,576 \geq 10^6$).

**1.5** $Q = 3.3/256 = 12.9$ mV; maximum error 6.45 mV; $1.00/0.01289 = 77.6$, so the code is $77 =$ `01001101`.

**1.6** $500/0.1 = 5000$ levels, so 13 bits ($2^{12} = 4096 < 5000 \leq 8192$).

**1.8** Ratio $1.2 \times 10^6 / 275\,000 = 4.36$, $\log_2 = 2.12$ doublings in 4 years, i.e. about 1.9 years, a little faster than the long-run average of 2.3 years.

## Chapter 2

**2.1** (a) $f = 1$ for rows 3, 5, 6, 7; (b) $f = x'yz + xy'z + xyz' + xyz$; (c) $f = xy + xz + yz$.

**2.2** $A = S(D + W) = SD + SW$.

**2.3** $M_L = L\,(C \oplus R) = LCR' + LC'R$; $M_R = R\,(L \oplus C) = L'CR + LC'R$.

**2.4** Segment $a$ is lit for the digits 0, 2, 3, 5, 6, 7, 8, 9 (off for 1 and 4). With don't-cares for 10–15, $a_{\text{lit}} = b_3 + b_1 + b_2 b_0 + b_2' b_0'$; on a common-anode display the output is $\overline{a_{\text{lit}}}$.

**2.6** (a) $x(x'+y) = xy$; (b) $xy + xz = x(y+z)$; (c) $(x+y)(x'+z)(y+z) = (x+y)(x'+z)$.

**2.7** (a) $F' = x'y' + x'z$; (b) $F' = xy + x'y'$; (c) $F' = a'b' + c'd'$.

**2.8** (a) $F = x' + y$; (b) $F = xz$ (first $(x+y)(x+y') = x$); (c) $F = xy + x'z$ (consensus).

**2.9** (a) $f = x'y'z' + x'yz + xy'z + xyz'$, and $f = \prod M(1,2,4,7) = (x+y+z')(x+y'+z)(x'+y+z)(x'+y'+z')$; (b) $f = 1$ when the number of 1s among $x, y, z$ is even; (c) $f' = \sum m(1,2,4,7)$.

**2.10** $f = \sum m(3,5,6) = x_1 x_2 x_3' + x_1 x_2' x_3 + x_1' x_2 x_3$.

**2.11** $E = (A_1 \oplus B_1)'\,(A_0 \oplus B_0)'$.

**2.12** $2^{2^n}$: (a) 16; (b) 256.

**2.13** (a) $f = x_1 \oplus x_2 \oplus x_3 \oplus x_4$; (b) 8 minterms (the odd-weight inputs).

## Chapter 3

**3.1** $F = C' + A'B$.

**3.2** $F = x'y' + x'z' + xz$ (using $d(5)$ and $d(13)$ as 1s in the group $xz$).

**3.3** $F = (x' + z)(x + y' + z')$.

**3.4** Primes are 2, 3, 5, 7. $F = \sum m(2,3,5,7) + d(10\text{–}15)$ gives $F = B'C + BD$; as a NAND–NAND circuit, $F = \overline{\overline{B'C} \cdot \overline{BD}}$ (two 2-input NANDs, one inverter for $B$, one output NAND). $G = \sum m(0,3,5,6,9) + d(10\text{–}15)$ gives $G = (A'+D)(B'+C+D)(B+C'+D)(A+B+C+D')(B'+C'+D')$; as a NOR–NOR circuit, five first-level NOR gates form the complemented sum terms and one NOR gate combines them (plus inverters for the complemented literals).

**3.5** Prime implicants: $x'z'$, $x'y$, $xz$, $yz$. Essential: $x'z'$ and $xz$. Minimal SOPs: $x'z' + xz + x'y$ and $x'z' + xz + yz$.

**3.6** (a) $x'z'$; (b) $w'x' + wx$; (c) $x'z' + xz$.

**3.7** (a) $F = \overline{\overline{AB} \cdot \overline{CD'} \cdot \overline{A'C'}}$: 3 inverters ($A'$, $C'$, $D'$), three 2-input NANDs and one 3-input NAND = 7 gates. (b) NOR-only: 3 inverters ($A'$, $B'$, $C'$), three NORs ($AB = \text{NOR}(A', B')$, $CD' = \text{NOR}(C', D)$, $A'C' = \text{NOR}(A, C)$), a 3-input NOR and a final inverter = 8 gates.

**3.8** The odd function $A \oplus B \oplus C \oplus D$ has 8 ones in its K-map (a checkerboard).

**3.9** $P = \overline{w \oplus x \oplus y \oplus z}$, i.e. `p <= not (w xor x xor y xor z);`. For `0110`, $P = 1$; for `0111`, $P = 0$; for `1111`, $P = 1$.

**3.10** `1000001` has two 1s: (a) even parity bit 0; (b) odd parity bit 1. The received word has three 1s, so an error is detected, but the position of the error cannot be determined.

**3.11** $b_{\text{lit}} = b'_2 + b'_1 b'_0 + b_1 b_0$; $c_{\text{lit}} = b_2 + b'_1 + b_0$.

## Chapter 4

**4.1** (a) `1011 0101` = B5; (b) −75.

**4.2** $A = 105$, $B = -42$. (a) $A + B =$ `0011 1111` $= 63$, no overflow (different signs). (b) $-B =$ `0010 1010`; $A + (-B) =$ `1001 0011`, which reads as −109: overflow, since $105 + 42 = 147 > 127$.

**4.3** (a) about $16 \times 2t = 32t$. (b) $G_i = A_i B_i$, $P_i = A_i \oplus B_i$; they let every carry be computed directly from the inputs, $C_{i+1} = G_i + P_i C_i$.

**4.4** One internal signal, the carry between the two 4-bit adders: `signal c4 : std_logic;`, with `U0: ripple_adder_4bit port map (A(3 downto 0), B(3 downto 0), Cin, S(3 downto 0), c4);` and `U1: ripple_adder_4bit port map (A(7 downto 4), B(7 downto 4), c4, S(7 downto 4), Cout);`.

**4.5** (a) 182 = B6; (b) `1010 0111` = 167; (c) `111 010 101` = 469; (d) `0.1101`.

**4.6** +19 = `010011`, −19 = `101101`, −32 = `100000`; range −32 to +31.

**4.7** (a) 01001 + 00111 = 10000, overflow; (b) 10111 + 11000 = 1 01111 → 01111 = +15, overflow; (c) 01100 + 11011 = 1 00111 → 00111 = 7, no overflow.

**4.9** $C_{out} = AB + AC_{in} + BC_{in}$ (the majority function).

**4.10** $G = (G_3 G_2 G_1 G_0) = 0100$, $P = 1011$; $C_1 = C_2 = C_3 = C_4 = 1$; $S =$ `0010` with carry-out 1, i.e. 10010 = 18 = 11 + 6 + 1.

**4.11** Partial products 0, `10110`, `101100`, 0; the sum is 22 + 44 = 66 = `0100 0010`.

**4.12** (a) `1010.0101` = $A5_{16}$; (b) `0101.1100` = 5.75; (c) resolution $2^{-8} = 0.0039$; range 0 to 255.996.

**4.13** (a) 0.15625 = `1.01` $\times 2^{-3}$, $E = 124$, word `0 01111100 0100 ... 0` = 3E200000; (b) `0 10000010 1001 ...` $= 1.5625 \times 2^3 = 12.5$.

**4.14** (a) `0010 0000 0010 0110` (16 bits) versus `111 1110 1010` (11 bits). (b) 24 bits in BCD; 20 bits in binary.

**4.15** (a) 48 69 21; (b) subtract $30_{16}$ (or mask with $0F$); (c) set bit 5 (OR with $20_{16}$).

**4.16** The carry into the sign bit ($c_3$) must be available: add it as an extra output of `ripple\_adder\_4bit`; then $V = c_3 \oplus c_4$ (an XOR gate).

## Chapter 5

**5.1** Three 2-to-1 multiplexers: two in the first stage (select $S_0$), one in the second (select $S_1$).

**5.2** $I_0 = C, I_1 = C, I_2 = C, I_3 = 1$.

**5.4** $x_3 = 1, x_2 = 1, x_1 = 0, x_0 = 1$; $A > B = 1$, $A = B = 0$, $A < B = 0$.

**5.5** $F = (A \oplus B) + C$, which is 0 only for $ABC = 000$ and 110: $F = \sum m(1,2,3,4,5,7)$.

**5.6** $S = D_1 + D_2 + D_4 + D_7$, $C_{out} = D_3 + D_5 + D_6 + D_7$.

**5.7** $x = D_0' D_1' (D_2 + D_3)$, $y = D_0' (D_1 + D_2' D_3)$, $V = D_0 + D_1 + D_2 + D_3$.

**5.8** $I_0 = D'$, $I_1 = D'$, $I_2 = 0$, $I_3 = 1$, $I_4 = D'$, $I_5 = D'$, $I_6 = D$, $I_7 = D$.

**5.9** $x$ is the select input; $I_1 = y$, $I_0 = z$.

**5.10** Four 3-to-8 decoders (with enable) and one 2-to-4 decoder whose outputs drive the four enables.

**5.11** Seven.

**5.12** (a) `1100`, $> 9$, add `0110`: `1 0010`: carry 1, digit 2 (12). (b) `1 0011` ($K = 1$), add 6: `1 1001`: carry 1, digit 9 (19).

**5.13** $A > B = A_1 B_1' + (A_1 \odot B_1) A_0 B_0'$; $A = B = (A_1 \odot B_1)(A_0 \odot B_0)$; $A < B = A_1' B_1 + (A_1 \odot B_1) A_0' B_0$.

**5.14**
```
g(3) <= b(3); g(2) <= b(3) xor b(2); g(1) <= b(2) xor b(1); g(0) <= b(1) xor b(0);
```

**5.15**
```
bus_out <= reg_value when rd = '1' else (others => 'Z');
```

## Chapter 6

**6.1** $A(t+1) = Ax + Bx$, $B(t+1) = A'x$.

**6.2** $A(t+1) = T \oplus A = x$: from either state the next state is $x$.

**6.3** (a) $Q$ toggles at each clock edge; (b) two states with an arrow each way; (c) 500 kHz.

**6.4** Moore: the output depends only on the state (the counter value).

**6.5** $Q$ after each step: $0, 1, 1, 0, 0$.

**6.7** (a) $J = K = T$; (b) $J = D$, $K = D'$.

**6.8** In the K-map over $S, R, Q$ the 1s are at $SR'Q'$, $SR'Q$ and $S'R'Q$, and the squares with $SR = 11$ are don't-cares; grouping gives $Q(t+1) = S + R'Q$.

**6.9** Present state $AB$, input $x \to$ next state, $y$: $00,0 \to 00,0$; $00,1 \to 10,0$; $01,0 \to 10,0$; $01,1 \to 00,0$; $10,0 \to 01,1$; $10,1 \to 11,1$; $11,0 \to 11,0$; $11,1 \to 01,0$. The output

$y = AB'$ depends on the state only: Moore.

**6.10** States $S_0, S_1, S_2$ (no 1s, one 1, two or more 1s): $S_0 \xrightarrow{1} S_1$, $S_1 \xrightarrow{1} S_2$, $S_2 \xrightarrow{1} S_2$, $S_1 \xrightarrow{0} S_0$, $S_0 \xrightarrow{0} S_0$, $S_2 \xrightarrow{0/1} S_0$. With $S_0 = 00$, $S_1 = 01$, $S_2 = 10$: $D_A = x(A + B)$, $D_B = A'B'x$, $y = Ax'$.

**6.11** $J_i = K_i = T_i$: $J_0 = K_0 = 1, J_1 = K_1 = Q_0, J_2 = K_2 = Q_1Q_0$.

**6.12** (a) 3; (b) 4; (c) 9.

**6.13–6.14**

```
process (clk) begin                       -- 6.13: synchronous reset + enable
  if rising_edge(clk) then
    if reset = '1' then q <= '0';
    elsif en = '1' then q <= d; end if;
  end if;
end process;

process (clk, rstn) begin                 -- 6.14: 8-bit register, async active-low
    reset
  if rstn = '0' then q <= (others => '0');
  elsif rising_edge(clk) then q <= d; end if;
end process;
```

## Chapter 7

**7.1**

```
architecture behavioral of reg_inc is
  signal r : unsigned(3 downto 0) := (others => '0');
begin
  process (CLK) begin
    if rising_edge(CLK) then
      if    CLR = '1' then r <= (others => '0');
      elsif LD  = '1' then r <= unsigned(I);
      elsif INC = '1' then r <= r + 1;
      end if;
    end if;
  end process;
  Q <= std_logic_vector(r);
end behavioral;
```

**7.2** (a) `1101`; (b) `1010`; (c) `1010`; (d) `1100`.

**7.3** $J_0 = K_0 = 1$; $J_1 = Q_2'Q_0$, $K_1 = Q_0$; $J_2 = Q_1Q_0$, $K_2 = Q_0$. State 110 goes to 111 and 111 to 000: self-correcting.

**7.4** Each output has frequency $f_{clk}/4$ (duty cycle 25%). A Johnson counter has $2n = 8$ states, and each output $f_{clk}/8$.

**7.5** After four pulses $A = \texttt{1100}$ (restored) and $B = \texttt{1100}$.

**7.6** $T_1$: $A = 1011$, $B = 1010$, $S = 1$, $C = 0$; $T_2$: $A = 1101$, $B = 0101$, $S = 1$, $C = 0$; $T_3$: $A = 0110$, $B = 1010$, $S = 0$, $C = 1$; $T_4$: $A = 1011$, $B = 0101$, $S = 1$, $C = 0$. Sum $= \texttt{1011} = 11$.

**7.7** $f_{max} < 25$ MHz; the change 0111 → 1000 settles in 40 ns.

**7.8** Load condition $Q_3Q_0$ (it is 1 only for the count 9 among 0–9) with data inputs `0000`.

**7.9** $T_0 = 1$; $T_1 = Up\,Q_0 + Down\,Q_0'$; $T_2 = Up\,Q_1Q_0 + Down\,Q_1'Q_0'$.

**7.10** (a) 8 flip-flops; (b) 3 flip-flops and a 3-to-8 decoder (eight 3-input AND gates and three inverters).

**7.11** `0011` → `0110`; each shift left doubles the number ($\times 2^k$ for $k$ shifts, if no 1 is lost); shifting `0110` right gives `0011`.

**7.12–7.13**

```vhdl
-- 7.12  (entity: clk, en, sin : in std_logic; sout : out std_logic)
signal r : std_logic_vector(7 downto 0) := (others => '0');
process (clk) begin
  if rising_edge(clk) then
    if en = '1' then r <= sin & r(7 downto 1); end if;
  end if;
end process;
sout <= r(0);

-- 7.13  (signal cnt : unsigned(3 downto 0))
process (clk) begin
  if rising_edge(clk) then
    if rst = '1' or cnt = 9 then cnt <= (others => '0');
    else cnt <= cnt + 1; end if;
  end if;
end process;
```

## Chapter 8

**8.1** (A) Moore; (B) Mealy.

**8.2** $Q(t+1) = JQ' + K'Q = xQ'$. For $Q = 0$, $x = 1$: next state 1.

**8.3–8.4** Overlapping and non-overlapping 110 detectors are identical, because no proper suffix of 110 (namely 0 and 10) is a prefix (1, 11). States: $S_0$ (no progress), $S_1$ (seen 1), $S_2$ (seen 11); $S_0 \xrightarrow{0/0} S_0$, $S_0 \xrightarrow{1/0} S_1$, $S_1 \xrightarrow{0/0} S_0$, $S_1 \xrightarrow{1/0} S_2$, $S_2 \xrightarrow{1/0} S_2$, $S_2 \xrightarrow{0/1} S_0$. For 101 the overlapping machine goes $S_2 \xrightarrow{1/1} S_1$ and the non-overlapping one $S_2 \xrightarrow{1/1} S_0$.

**8.5** (a) 4; (b) 12.

**8.6** The reset must be in the sensitivity list: `process (clk, reset)`.

**8.7** 10.

**8.8** $D_0 = X$.

**8.9** (2); $f_{max} = 1/2.8\text{ ns} \approx 357$ MHz.

**8.10** $D_1 = Q_0$, $D_0 = Q_1'$.

**8.11** Four states: $S_0$, $S_1$, $S_2$ and $S_3$ (101 seen, output 1); $S_3 \xrightarrow{1} S_1$, $S_3 \xrightarrow{0} S_2$; the other transitions as in Figure 6.13 but with the output on the state.

## Chapter 9

**9.1** Registers $R_1$ and $R_2$, a subtractor with inputs $R_1$ and $R_2$ and its output to the input of $R_1$; control signals $R1Load$ (and the clock).

**9.2** $Mux1 = 0$ (select $B$), $Mux2 = 1$ (select 3), $ALoad = 1$.

**9.3** A magnitude comparator of $A$ with the constant 10 produces the status signal $AGT10$.

**9.4** $CW_1$: $Temp \leftarrow A$ ($TempLoad = 1$, source $A$); $CW_2$: $A \leftarrow B$ ($ALoad = 1$, source $B$); $CW_3$: $B \leftarrow Temp$ ($BLoad = 1$, source $Temp$).

**9.5** Six states: $S_1$ ($Sum = 0$), $S_2$ ($i = 1$), $S_3$ ($Sum = Sum + i$), $S_4$ ($i = i + 1$), $S_{test}$ (no operation; examine $i = 11$ after $i$ has been loaded), $S_5$ (output). From $S_{test}$: if $i \neq 11$ go to $S_3$, else to $S_5$.

**9.6** GCD(12,18)=6 in 7 cycles ($S_0, S_1, S_3, S_1, S_2, S_1, S_4$). GCD(48,18)=6 in 11 cycles.

**9.8** $D_2 = Q_2 + Q_1$, $D_1 = Q_1'Q_0$, $D_0 = Q_2'Q_1'(Q_0' + I)$. State 101 goes to 110 and then to 100; 110 and 111 go to 100: all unused states lead to $S_{output}$.

**9.12** FSM+D, because it gives full control of the datapath structure.

## Chapter 10

**10.1** `IN A; OUT A; DEC A; JNZ 01; OUT A; HALT` outputs $n, n-1, \ldots, 1, 0$. The EC-1 has no unconditional jump, so it cannot repeat the ramp; a `JMP` instruction would be needed.

**10.2** `00: IN A; 01: DEC A; 02: JNZ 04; 03: HALT; 04: DEC A; 05: JNZ 01; 06: HALT` = `60 A0 C4 E0 A0 C1 E0`. Halt at 03 means odd, at 06 even. For input 5 the program stops at 03 (odd); for input 6 at 06 (even).

**10.3** Data: $M[28] = 0$ (sum), $M[29] = 1$, $M[30] = a$, $M[31] = b$. Program (hex):

`00:1F 01:AA 02:1C 03:5E 04:3C 05:1F 06:7D 07:3F 08:AA 09:C2 10:E0`, i.e. `LOAD A,31; JZ 10; LOAD A,28; ADD A,30; STORE A,28; LOAD A,31; SUB A,29; STORE A,31; JZ 10; JPOS 02; HALT`. For $3 \times 4$ the sum becomes 3, 6, 9, 12 while $b$ counts 3, 2, 1, 0; the product is in $M[28]$.

**10.4** `LOAD A,30; SUB A,31; JPOS 05; LOAD A,31; SUB A,30; STORE A,31; HALT` = `1E 7F C5 1F 7E 3F E0`.

**10.5** (a) `60 A0 A0 80 E0`; (b) `05 46 27 E0`.

**10.6** `:050000006080A0C1E0DA`; end of file `:00000001FF`. (Checksum: $05 + 60 + 80 + A0 + C1 + E0 = 326_{16}$; $100_{16} - 26_{16} = \mathrm{DA}_{16}$.)

**10.7** States: Fetch, Decode, Input ($A = 2$); Fetch, Decode, Output (out 2); Fetch, Decode, Dec ($A = 1$); Fetch, Decode, JNZ (taken, $PC = 01$), Wait; Fetch, Decode, Output (out 1); Fetch, Decode, Dec ($A = 0$); Fetch, Decode, JNZ (not taken, $PC = 04$), Wait; Fetch, Decode, Halt.

**10.10** Three cycles per instruction (fetch, decode, execute): 9 cycles until the machine is in the Halt state.

**10.13** $M[30] = 6$; the instruction `SUB` is executed 6 times (the pairs $(48, 18)$, $(30, 18)$, $(12, 18)$ with two subtractions, $(12, 6)$ and $(6, 6)$).

# Glossary

**ALU (arithmetic logic unit)** A combinational circuit that performs a selectable set of arithmetic and logic operations on its operands (Section 5.8).

**Accumulator** A register that holds an operand of an instruction and receives its result (Section 10.3).

**ASIC** Application-specific integrated circuit: a chip customised for one application (Section 1.8).

**Asynchronous reset** A reset that acts immediately, independently of the clock (Section 6.4).

**Bit** A binary digit, 0 or 1.

**Boolean algebra** The algebra of two-valued variables with the operations AND, OR and NOT; the mathematical basis of logic design (Section 2.2).

**BCD (binary-coded decimal)** A code that represents each decimal digit by four bits (Section 4.8).

**Bus** A set of wires shared by several devices, only one of which drives it at a time (Section 5.9).

**Canonical form** A sum of minterms or a product of maxterms of a function (Section 2.5).

**Carry-lookahead adder** An adder that computes all carries in parallel from generate and propagate signals (Section 4.5).

**Characteristic equation** The equation that gives the next state of a flip-flop as a function of its inputs and present state (Section 6.4).

**Clock** A periodic signal that synchronises the state changes of a sequential circuit.

**Combinational circuit** A circuit whose outputs depend only on its present inputs (Chapter 5).

**Control unit** The FSM of a processor that generates control signals to sequence the datapath (Section 9.4).

**Control word** The vector of control-signal values that drives the datapath during one clock cycle (Section 9.6).

**Counter** A register that goes through a prescribed sequence of states (Section 7.3).

**Datapath** The part of a processor that performs data operations: registers, functional units and multiplexers (Section 9.4).

**Decoder** A circuit that activates one of $2^n$ output lines according to an $n$-bit input code (Section 5.5).

**Don't-care** An input combination whose output value is unspecified or irrelevant, used to simplify logic (Section 3.5).

**Duality** The principle that interchanging AND with OR and 0 with 1 in a true Boolean identity gives another true identity (Section 2.2).

**Edge-triggered** Responding only at a clock edge, as a flip-flop does (Section 6.4).

**Encoder** A circuit that produces the binary code of the active input line (Section 5.6).

**Excitation table** A table of the flip-flop inputs needed for each state transition (Section 6.7).

**FPGA** Field-programmable gate array: a programmable logic chip configured by the user (Section 1.8).

**Finite state machine (FSM)** A model with a finite set of states, transitions and outputs, used to design sequential circuits (Chapter 8).

**Flip-flop** An edge-triggered one-bit memory element (Section 6.4).

**FSMD** A style in which the datapath operations are written inside a behavioural FSM (Section 9.8).

**Gate** A circuit that implements a basic logic function (Section 2.4).

**Generate and propagate** Signals $G_i = A_i B_i$ and $P_i = A_i \oplus B_i$ used by carry-lookahead adders (Section 4.5).

**Gray code** A code in which successive values differ in one bit.

**Hardware description language (HDL)** A language, such as VHDL or Verilog, that describes digital hardware (Section 1.9).

**Hexadecimal** Base-16 notation, one digit per four bits.

**High impedance (Hi-Z)** The third state of a three-state output, in which the output is disconnected (Section 5.9).

**Instruction cycle** The sequence of fetch, decode and execute performed for every instruction (Section 10.2).

**Instruction set** The set of operations a processor can perform and their binary encoding (Section 10.2).

**Karnaugh map** A diagram that shows the minterms of a function so that adjacent squares differ in one variable (Chapter 3).

**Latch** A level-triggered one-bit memory element (Section 6.3).

**Logic synthesis** The generation of a logic circuit from a functional description (Section 2.5).

**Maxterm** A sum term that contains every variable once; it is 0 for exactly one input combination (Section 2.5).

**Mealy machine** An FSM whose outputs depend on the present state and the inputs (Section 8.2).

**Minterm** A product term that contains every variable once; it is 1 for exactly one input combination (Section 2.5).

**Moore machine** An FSM whose outputs depend only on the present state (Section 8.2).

**Moore's law** The observation that the number of transistors per chip doubles every one to two years (Section 1.7).

**Multiplexer** A circuit that selects one of several data inputs under the control of select lines (Section 5.7).

**One-hot encoding** A state encoding in which exactly one flip-flop is 1 in each state (Section 8.4).

**Opcode** The part of an instruction that specifies the operation.

**Overflow** A result that does not fit in the number of bits available (Section 4.3).

**Parity bit** An extra bit that makes the number of 1s in a word even or odd, used to detect errors (Section 3.10).

**Prime implicant** A product term that cannot be combined with another to form a larger group (Chapter 3).

**Program counter (PC)** The register that holds the address of the next instruction.

**Register** A group of flip-flops that stores one word (Section 7.1).

**Register-transfer level (RTL)** An abstraction that describes a circuit by data transfers between registers and the operations performed on them (Section 9.5).

**Ripple-carry adder** An adder in which the carry propagates through a chain of full adders (Section 4.4).

**Sampling** Measuring a signal at regular time intervals (Section 1.4).

**Shift register** A register that moves its contents one position per clock pulse (Section 7.2).

**Sign-magnitude** A signed-number representation with a sign bit and a magnitude (Section 4.2).

**State diagram** A graph with states as nodes and transitions as labelled arrows (Section 6.5).

**State table** A table of next states and outputs for each present state and input (Section 6.5).

**Status signal** A signal sent from the datapath to the control unit to report a condition (Section 9.4).

**Sum of products (SOP)** An OR of AND terms (Section 2.5).

**Product of sums (POS)** An AND of OR terms (Section 2.5).

**Synchronous circuit** A sequential circuit whose state changes only at clock edges (Chapter 6).

**Testbench** A VHDL entity that stimulates and observes a unit under test (Section 2.7).

**Three-state gate** A gate with outputs 0, 1 and high impedance (Section 5.9).

**Truth table** A table that gives the output of a function for every input combination (Section 2.3).

**Two's complement** The standard representation of signed integers, in which negation is inversion plus one (Section 4.2).

**Universal gate** A gate (NAND or NOR) from which any logic function can be built.

**VHDL** VHSIC Hardware Description Language, IEEE standard 1076 [10] (Chapter A).

**Von Neumann architecture** A computer organisation in which instructions and data share one memory (Section 9.1).

**Wait state** A state added to an FSM to give signals time to settle (Example 9.4).

# Bibliography


[1] *Coded Character Sets—7-Bit American National Standard Code for Information Interchange (7-Bit ASCII)*, American National Standards Institute, 1986, aNSI X3.4-1986.

[2] G. E. Moore, "Cramming more components onto integrated circuits," *Electronics*, vol. 38, no. 8, pp. 114–117, 1965.

[3] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3rd ed. New York: McGraw-Hill, 2009.

[4] E. O. Hwang, *Digital Logic and Microprocessor Design with Interfacing*, 2nd ed. Boston: Cengage Learning, 2018.

[5] G. Boole, *An Investigation of the Laws of Thought, on Which Are Founded the Mathematical Theories of Logic and Probabilities*. London: Walton and Maberly, 1854.

[6] C. E. Shannon, "A symbolic analysis of relay and switching circuits," *Transactions of the American Institute of Electrical Engineers*, vol. 57, no. 12, pp. 713–723, 1938.

[7] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. New York: Pearson, 2018.

[8] J. F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2006.

[9] P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed. Burlington, MA: Morgan Kaufmann, 2008.

[10] *IEEE Standard VHDL Language Reference Manual*, Institute of Electrical and Electronics Engineers, 2008, iEEE Std 1076-2008.

[11] M. Karnaugh, "The map method for synthesis of combinational logic circuits," *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics*, vol. 72, no. 5, pp. 593–599, 1953.

[12] E. J. McCluskey, "Minimization of Boolean functions," *Bell System Technical Journal*, vol. 35, no. 6, pp. 1417–1444, 1956.

[13] *IEEE Standard for Floating-Point Arithmetic*, Institute of Electrical and Electronics Engineers, 2019, iEEE Std 754-2019.

[14] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[15] E. O. Hwang, *Digital Logic and Microprocessor Design with VHDL*. Toronto: Thomson/CL Engineering, 2005.

[16] G. H. Mealy, "A method for synthesizing sequential circuits," *Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, 1955.

[17] E. F. Moore, "Gedanken-experiments on sequential machines," in *Automata Studies*, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton University Press, 1956, pp. 129–153.

[18] D. D. Gajski, *Principles of Digital Design*. Upper Saddle River, NJ: Prentice Hall, 1997.

[19] J. von Neumann, "First draft of a report on the EDVAC," Moore School of Electrical Engineering, University of Pennsylvania, Tech. Rep., 1945.

[20] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA: Morgan Kaufmann, 2012.

# Index


**Symbols**

74xx series, 9

**Numbers**

0, 3
1, 3

**A**

accumulator, 216
American Standard Code for Information Interchange, 77
analog signal, 2
analog-to-digital converter, 4
analysis, 82, 118, 156
AND, 16
application-specific integrated circuit, 10
arithmetic logic unit, 100
arithmetic overflow, 56
array multiplier, 69
asynchronous, 108, 117
axioms, 16

**B**

BCD, 75
BCD adder, 87
BCD counter, 140
binary, 3
binary logic circuit, 15
bit, 3, 53
Boolean algebra, 16
Boolean variable, 16
bootloader, 233
bus, 102
byte, 53

**C**

canonical product of sums, 21
canonical sum of products, 21
carry-lookahead adder, 65
characteristic equation, 114
characteristic table, 114
clock, 108
combinational, 15
combinational circuit, 81
concurrent signal assignment, 20
control signals, 191
control unit, 191
counter, 139
critical path, 65

**D**

D latch, 110
datapath, 190
De Morgan's theorem, 17
decoder, 89
dedicated microprocessor, 190
demultiplexer, 91
design, 83, 122
digital signal, 2
digital system, 2
don't-care conditions, 40
dual, 17

**E**

edge-triggered, 112
enable, 89
encoder, 91
ENIAC, 7
essential prime implicant, 38
even function, 47

even parity, 49
excess-3 code, 84
excitation table, 125
exclusive OR, 23
exclusive-NOR, 46
exclusive-OR, 46

**F**
field-programmable gate array, 9
finite state machine, 155
fixed-point, 73
flip-flop, 112
flip-flop input equations, 118
full adder, 57

**G**
gated SR latch, 110
general-purpose microprocessor, 215
generate, 65
Gray code, 35

**H**
hardware description language, 10
hidden bit, 75
high impedance, 102

**I**
ideal switch, 6
IEEE 754, 75
implicant, 38
instruction cycle, 216
instruction register, 216
integrated circuit, 7
Intel HEX, 217

**J**
JK flip-flop, 112
Johnson, 150

**K**
Karnaugh map, 33

**L**
latch, 108
level-triggered, 112
loading, 132
logic circuit, 15
logic gate, 19
logic network, 19
logic synthesis, 20

**M**
magnitude comparator, 102
maxterm, 21
Mealy model, 122
microprocessor, 7
minterm, 21
modulo-*N* counter, 147
Moore model, 122
Moore's law, 8
multiplexer, 96

**N**
NOT, 16

**O**
odd function, 47
odd parity, 49
opcode, 216
operands, 216
OR, 16
output equations, 118

**P**
parallel load, 132
parity bit, 49
positional number system, 3
prime implicant, 38
priority encoder, 93
program, 215
program counter, 216
programmable logic device, 9
propagate, 65

**Q**
quantisation, 4
Quine–McCluskey algorithm, 42

**R**
register, 131
register-transfer level, 192
relays, 6
ring counter, 150
ripple, 139
ripple counter, 139
ripple-carry adder, 59

**S**
sampling, 4
self-correcting, 147
sequential circuit, 81, 108
serial adder, 132
serial transfer, 132
Shannon's expansion theorem, 99
shift register, 132
sign bit, 54
sign-extend, 56
signal, 2
SR latch, 108
state, 108
state diagram, 118
state register, 155
state table, 118
status signals, 191
switching network, 15
synchronous, 108, 117, 139
synchronous counter, 142
synthesis, 10, 159

**T**
T flip-flop, 114
testbench, 27
three-state gate, 102
transistor, 7
truth table, 18
two-process method, 163

**U**
universal gates, 44
universal shift register, 135
up/down counter, 142

**V**
vacuum tube, 6